

Cadence[®] RTL-to-GDSII Flow

Course Version 7.0

Lecture Manual

Revision 1.0

© 1990-2025 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright, and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication, subject to the following conditions:

The publication may be used solely for personal, informational, and noncommercial purposes;

The publication may not be modified in any way.

Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make and expressly disclaims any representations or warranties as to the completeness, accuracy, or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

Cadence® RTL-to-GDSII Flow

Module 1	About This Course	2
Module 2	Design Specification and RTL Coding	10
Module 3	Design Simulation Using the Xcelium Simulator	32
	Lab 3-1 Simulating a Simple Counter Design	
Module 4	Code Coverage Using the Integrated Metrics Center	58
	Submodule Coverage Driven Verification Using the Integrated Metrics Center	60
	Submodule Metric Driven Verification Using Verisium Manager	95
	Lab 4-1 Code Coverage Flow for a Simple Counter Design	
Module 5	The Synthesis Stage	109
	Lab 5-1 Running the Basic Synthesis Flow	
	Lab 5-2 Running the Synthesis Flow with DFT	
Module 6	The Test Stage.....	161
	Lab 6-1 Running the Basic ATPG Flow in Modus Test	
Module 7	The Equivalency Checking Stage	175
	Lab 7-1 Running the Equivalence Checking Flow in Conformal	
	Lab 7-2 Creating a .v Format File from .lib Format	
Module 8	The Implementation Stage.....	189
	Lab 8-1 Running the Basic Implementation Flow	
Module 9	Timing Analysis and Debug	258
	Submodule Generating Reports	285
	Submodule Debugging Timing Analysis Results	294
	Submodule Global Timing Debug (GTD) Interface	300
	Submodule Manual ECOs	320
	Lab 9-1 Using the Global Timing Debug Interface to Debug Timing Results	

Module 10 Gate-Level Simulation.....331
Lab 10-1 Running Gate-Level Simulations on a Simple Counter Design

Module 11 Course Conclusions356

Module 12 Next Steps.....359



This page does not contain notes.



Module 1

About This Course

cādence[®]

This page does not contain notes.

Course Prerequisites

Before taking this course, you need to

- Have an understanding of device physics and the IC fabrication process
- Have knowledge of Verilog or any other hardware description language



This page does not contain notes.

Course Objectives

In this course, you:

- Implement the RTL of a design from its specification
- Use the Xcelium™ simulator to simulate the design
- Verify Code Coverage using the Integrated Metrics Center
- Synthesize the design from RTL to Gates using the Genus™ Synthesis Solution
- Insert test structures to be able to test the design using the Genus Synthesis Solution and verify the test coverage using the Encounter® test
- Compare the design against the RTL using the Conformal® Equivalence Checker
- Run the digital implementation flow with the Innovus™ Implementation System:
 - Create a floorplan
 - Implement power structures and clock trees
 - Place-and-route the design
 - Verify the design
- Run signoff checks to make sure that the design chip can be fabricated



This page does not contain notes.

Course Agenda

- Design Specification and RTL Coding
- Design Simulation Using the Xcelium Simulator
 - Simulating a Simple Counter Design
- Code Coverage Using the Integrated Metrics Center
 - Code Coverage Flow for a Simple Counter Design
- The Synthesis Stage
 - Running the Basic Synthesis Flow
 - Running the Synthesis Flow with DFT
- The Test Stage
 - Running the Basic ATPG Flow in Modus Test
- The Equivalency Checking Stage
 - Running the Equivalence Checking Flow in Conformal
 - Creating a .v Format File from .lib Format
- The Implementation Stage
 - Running the Basic Implementation Flow
- Timing Analysis and Debug
 - Using the Global Timing Debug Interface to Debug Timing Results
- Gate-Level Simulation
 - Running Gate-Level Simulations on a Simple Counter Design



This page does not contain notes.

Software and Licenses

For the software and licenses used in the labs for this course, go to:

https://www.cadence.com/content/cadence-www/global/en_US/home/training/all-courses/86136.html

If there is additional information regarding the specific software, it is detailed in a file called *csorc* and also the README file of the database provided with this course.



This page does not contain notes.

Become Cadence Certified by Earning a Digital Badge



Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
 - Expand career opportunities
- Professional credibility
 - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.



How do I register to take the exam?

- Log in to our [Learning Management System](#), click on the course in your transcript, and go to the Content tab to locate the exam.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- [Credly](#) validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

This page does not contain notes.

Icons Used in This Class



Best Practice



Language/
Command Syntax



Concept/
Glossary



Frequently Asked
Questions/
Quiz



Error Message

Throughout this class, we use icons to draw your attention to certain kinds of information. Here are the icons we use, and what they mean.



Problem & Solution



Quick Reference



GUI and Command



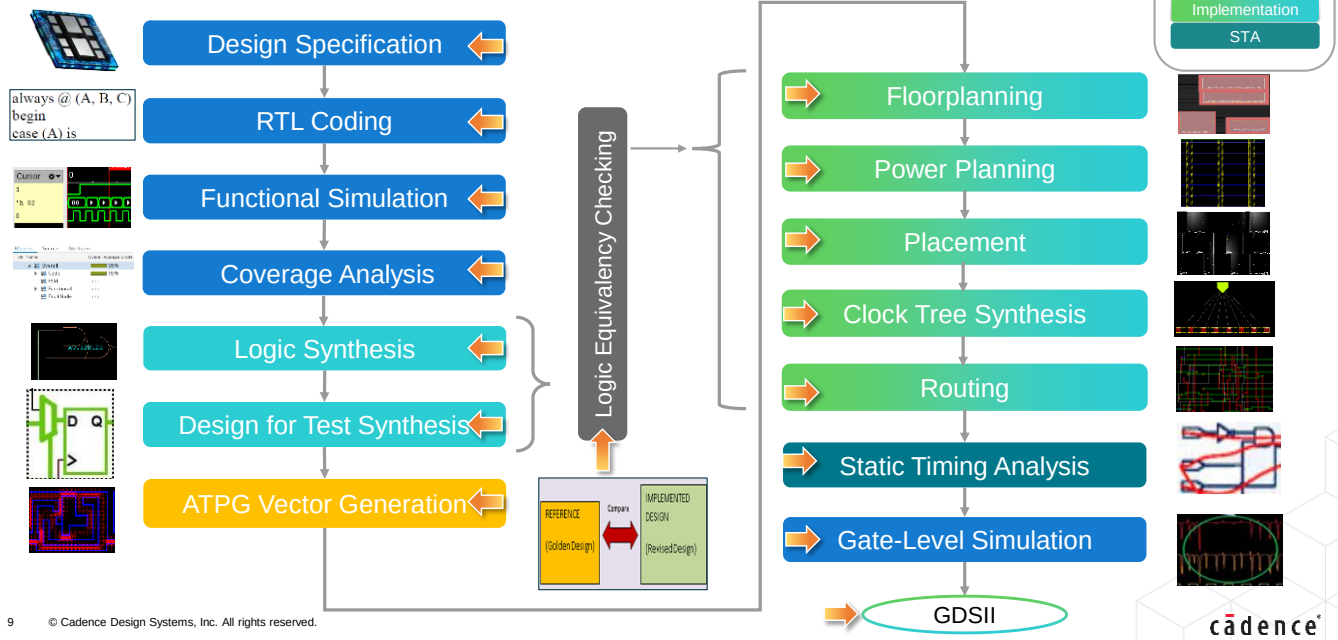
How To



Lab List

This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow



This page does not contain notes.



Module 2

Design Specification and RTL Coding

cā dence[®]

This page does not contain notes.

Module Objectives

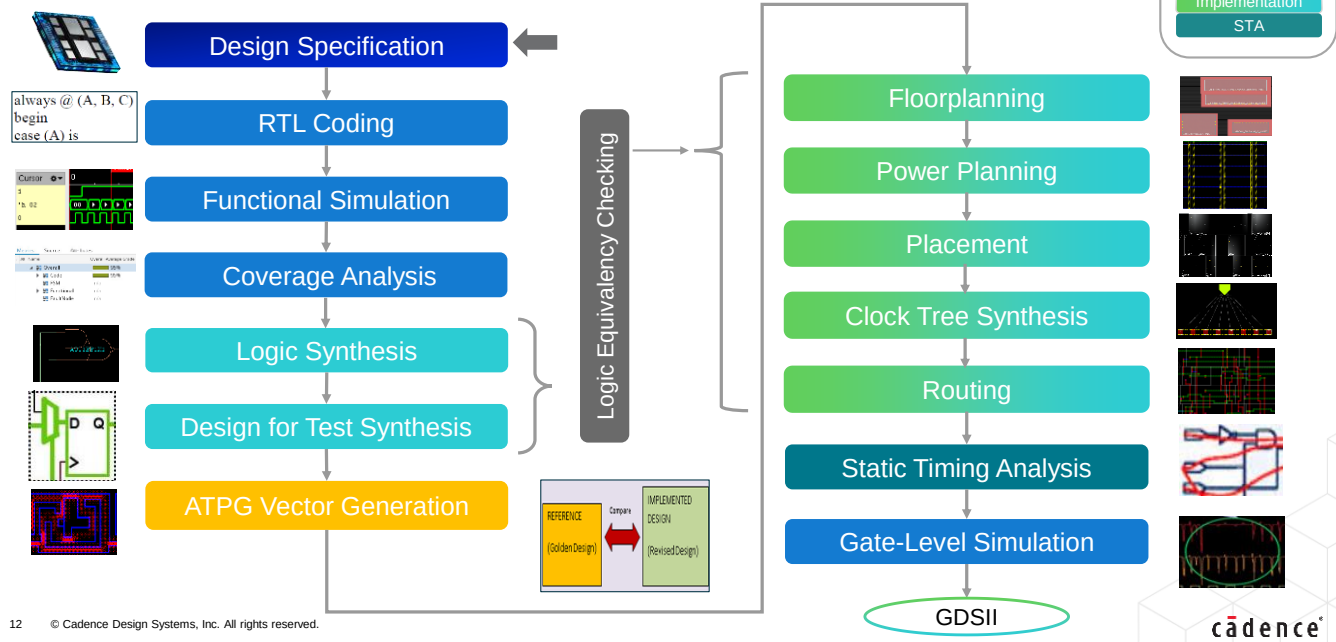
In this module, you

- Define Design Specification using an example of a simple counter-design spec
- Implement the specified design through the RTL coding process:
 - Identify what Hardware Description Languages, or HDLs, are
 - Implement a design spec for a given design criteria
 - Code a simple counter design given a design spec



This page does not contain notes.

Design Specification FlowChart



This page does not contain notes.

What Is a Design Specification?



A Design Specification is an explicit set of requirements to be satisfied by a material, product or service.

- An idea begins with a specification, which can be textual, graphical, or sometimes a software representation.
- It is a detailed document providing information about the characteristics of a project to set the criteria that design engineers will need to meet to code their designs.

APB / UART Project Specification

1 Introduction

The APB-UART design is a full duplex asynchronous receiver and transmitter that supports a wide range of software programmable baud rates and data formats, and can accommodate automatic parity generation and several error detection schemes. The interface has been developed for a modem to be connected, with both receive and transmit FIFO providing a buffer for the CPU. This interface is fully compatible with the AMBA APB bus.

This document defines the scope of the APB-UART block level verification and specifies its solution. The verification problem is quantified in a set of coverage models derived from architectural and implementation features. The solution is specified in the functional specification of the simulation verification environment in section 4, "Verification Environment Design".

The UART is an Advanced Microcontroller Bus Architecture (AMBA) compliant "System On Chip" block peripheral.

The key functions of this block are:

- Control and data access is provided by an Advanced Peripheral Bus (APB) bus interface
- APB access to Tx and Rx FIFO queues
- Interrupt Control
- Highly configurable Serial Interface
- Support for IrDA Modem

Further technical information on the UART design may be found in the UART functional specification.

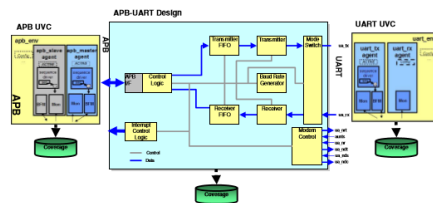


Figure 1 APB-UART design and verification environment.

This document refers in several sections to serial data frames, for the context of this document a frame shall be considered as a single data word transmission (start bit, 6,7,8 bit data payload, optional parity bit and stop bit(s)).

The APB interface shall be tested in conjunction with an AHB/APB bridge, requiring therefore verification at an AHB interface.

cadence

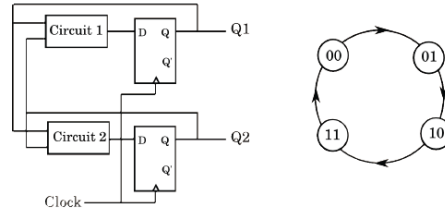
What Is a Design Specification?

A design specification serves as the foundation for any project, outlining the essential requirements that must be met. It can take various forms, including textual descriptions, graphical representations, or even software models. This document is crucial for design engineers as it sets clear criteria that guide their work, ensuring that the final product aligns with the initial vision. Here we have an example document titled "APB / UART Project Specification," which details the design and functionality of the APB-UART system. Additionally, the accompanying diagram illustrates the design and verification environment, highlighting how these specifications translate into practical applications. Understanding the importance of a design specification is key to successful project execution and meeting product expectations.

Example: Design Spec of a Two-Bit Synchronous Binary Counter

- The binary counter is an example of a simple synchronous digital circuit. It has no data inputs and no combinational output logic circuit.
- At each clock pulse, the counter takes up a new state and thus goes through a specific count sequence.
- We shall now write a design spec for a binary up-counter with two outputs, which go through the following sequence:
 - 00 -> 01 -> 10 -> 11 -> 00 -> etc.
- The counter should have the following inputs:
 - *rst*, which is synchronous, active low
 - *clk*
- The counter should have the following output:
 - count (1:0)

The block diagram, structure and state transition diagram of a two-bit binary counter (built with two D-type flip-flops) is of the following form:



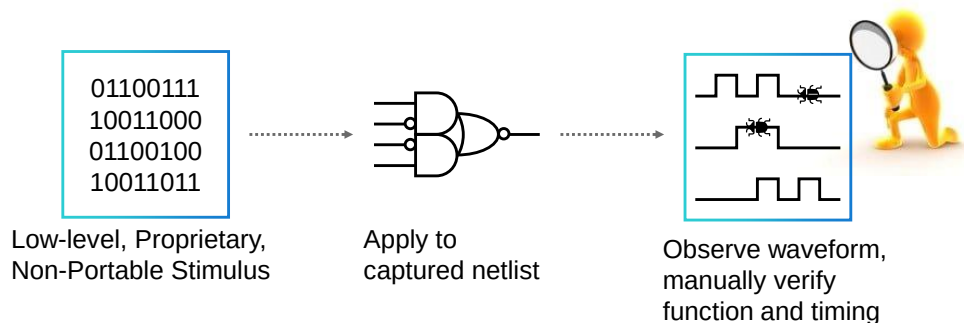
The truth table for the circuit is as follows:

Inputs t_n		Outputs t_{n+1}	
Q1	Q2	D1	D2
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

Here we have an elaborated design specification for a two-bit synchronous binary counter, which is a basic example of a digital circuit. The counter operates using two D-type flip-flops and updates its state with each clock pulse, cycling through a defined sequence: 00, 01, 10, 11, and back to 00. It's important to note that this counter does not have any data inputs or combinational output logic, simplifying its design. The inputs include a synchronous, active-low reset and a clock signal, while the output is a two-bit count. The accompanying block diagram, state transition diagram, and truth table provide a visual representation of how the counter functions, illustrating the relationship between the inputs and the resulting outputs.

This is how we analyze each scenario of specifications before moving on to the next steps in designing and verifying any digital design.

Legacy Schematic-Based Design



Let's see what an HDL is...

15 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the early days of digital design, engineers manually created binary input patterns to test circuits. These patterns were specific to the tools and platforms used, making them hard to share or adapt. This approach was time-consuming and error-prone. Once the binary inputs are ready, they are applied to the schematic netlist. This netlist represents the actual logic gates and connections in the circuit. Simulation tools then show how the inputs affect the outputs, mimicking real hardware behavior. Only 0 and 1 characters were used; we did not use the Z and X characters because those values do not exist in real hardware.

The final step is to look at the waveform outputs and verify that the circuit performs correctly. This manual process is tedious and requires a deep understanding of timing and logical behavior. Any errors must be traced back through the schematic, which can be very time-consuming.

This legacy approach highlights the challenges of schematic-based design: low reusability, high manual effort, and limited scalability. From here, we can compare the modern HDL-based design, which offers higher abstraction, automation, and portability.

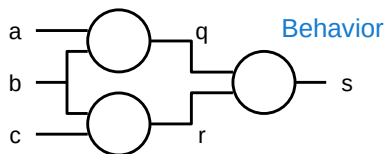
What Is a Hardware Description Language (HDL)?



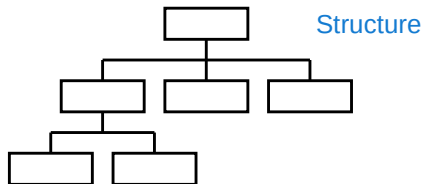
An HDL is a programming language with special constructs for modeling hardware concurrency and timing.

An HDL supports a design at multiple levels of abstraction:

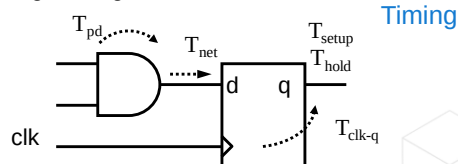
- Behavioral Level
 - Sequential behavior
 - Parallel behavior



- Structural Modeling
 - Hardware component hierarchy
 - Software subroutine hierarchy



- Time Modeling
 - An HDL has to model propagation delays, clock periods, and timing checks.
 - An HDL typically supports the simulation of estimated design timing.



16

© Cadence Design Systems, Inc. All rights reserved.

cadence

Hardware Descriptive Language (HDL) is similar to a procedural programming language but also contains constructs to describe digital electronic systems. An HDL contains features and constructs to support a description of the following:

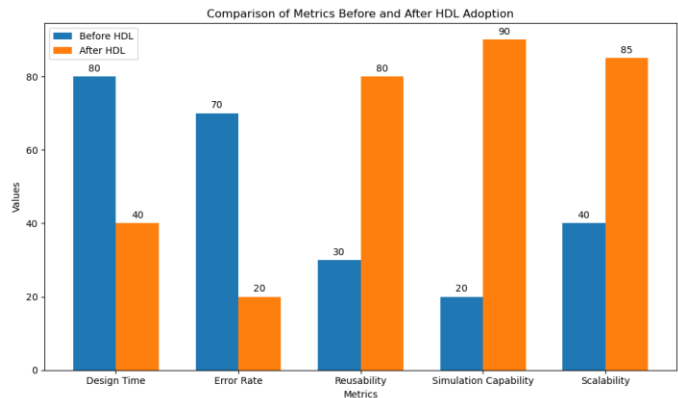
- Behavior – Both serial and parallel. In serial behavior, you pass the output of one functional block to the input of another, which is similar to the behavior of a conventional software language. However, in parallel behavior, you can pass a block output to the inputs of a number of blocks acting in parallel, where many separate events happen at the same moment in time.
- Structure – Both physical, such as hierarchical block diagrams and component netlists, and software, such as subroutines. This allows you to describe large, complex systems and manage their complexity.
- Time – Programming languages have no concept of time. An HDL has to model propagation delays, clock periods, and timing checks.

An HDL typically supports multiple abstraction levels. You can describe hardware behaviorally, without, and with sufficient detail for logic synthesis, and as a structured netlist of predefined components that can themselves be as simple as a transistor and as complex as another behavioral design.

Evolution of Hardware Description Languages (HDLs)

Key metrics before and after the adoption of Hardware Description Languages (HDLs):

- Design Time
 - Reduced significantly with HDL due to automation and abstraction.
- Error Rate
 - Lowered thanks to simulation and verification tools.
- Reusability
 - Significantly improved with modular HDL code.
- Simulation Capability
 - Enhanced with precise timing and concurrency modeling.
- Scalability
 - Easier to scale complex designs using hierarchical HDL structures.



Let's examine how Hardware Description Languages, HDLs, have changed the way we design digital circuits.

Before HDLs, designing hardware was slow, error-prone, and hard to scale. But with HDLs like Verilog and VHDL, things improved significantly.

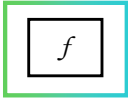
Let's look at five important areas:

- **Design Time:** HDLs reduce manual design effort by allowing engineers to describe hardware behavior using code. This speeds up the design process significantly..
- **Error Rate:** With HDLs, engineers can simulate and verify designs before fabrication, catching bugs early and reducing costly errors.
- **Reusability:** HDL code is modular and can be reused across projects, unlike traditional schematic-based designs.
- **Simulation:** HDLs support detailed simulation of timing and concurrency, helping engineers understand how the design behaves under different conditions.
- **Scalability:** It's easier to build big, complex systems. HDLs support hierarchical designs, making it easier to manage and scale complex systems.

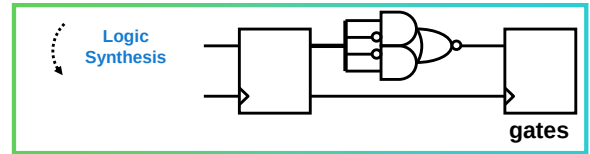
As you can see in the graph, all these areas improved a lot after HDLs were introduced. That's why HDLs are so important in modern electronics design!

An HDL Supports Multiple Levels of Abstraction

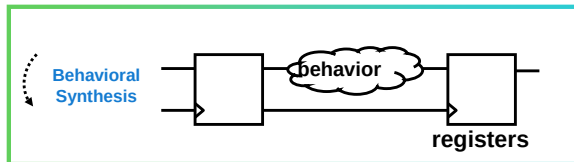
- Behavioral Level – Algorithms



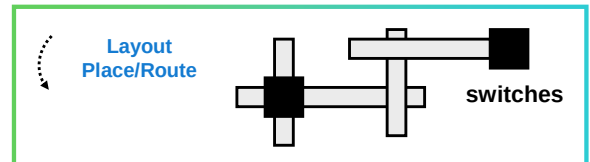
- Gate Level – Built-in and user-defined primitives



- Register Transfer Level (RTL) – Nets and Registers



- Switch Level – Built-in switch primitives



Here, we will discuss the various levels of abstraction supported by Hardware Description Languages (HDLs). It begins with the Behavioral Level, where high-level algorithms are defined, allowing designers to focus on functionality rather than implementation details. For example, we might describe a function `f` that performs some computation.

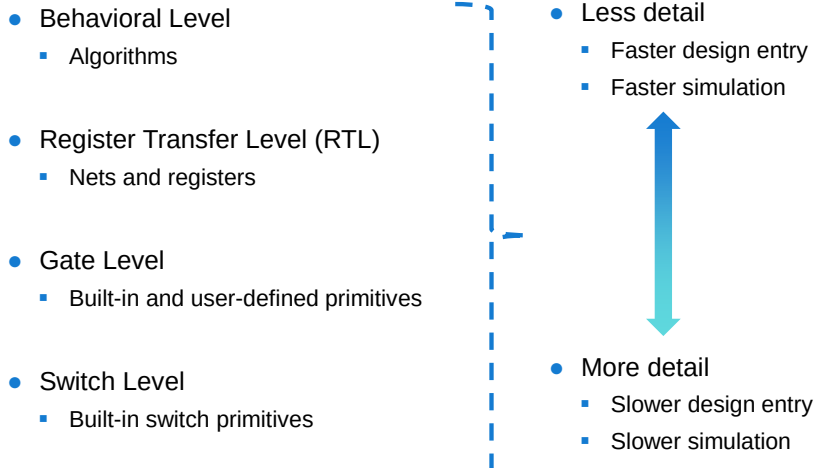
Next, we move to the Register Transfer Level (RTL), which describes the flow of data between registers and the operations performed on that data.

Moving to the Gate Level, we start thinking about how our algorithms can be implemented using basic logic gates like AND, OR, and NOT gates. Here, we perform logic synthesis to create gate-level representations of our designs.

Finally, at the Switch Level, we deal with physical implementation details such as layout, place-and-route using transistor-level or switch-level primitives to create detailed schematics for fabrication.

Abstraction Level Tradeoffs

Simulation effort is proportional to detail:



19 © Cadence Design Systems, Inc. All rights reserved.



In digital design, abstraction levels help us manage complexity. At the top, we have the behavioral level, where we describe what the system should do using algorithms. This is quick to write and simulates fast, but lacks hardware detail.

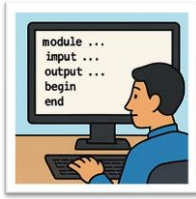
As we move down to RTL, we start specifying how data moves between registers and what logic controls it. This adds more detail and slightly slows down the simulation.

We define the actual logic gates used at the gate level. This is closer to the physical implementation and takes more time to simulate.

Finally, at the switch level, we model the circuit at the transistor level. This is the most detailed and slowest to simulate.

The key takeaway is that more detail means more simulation effort. So, we choose the level of abstraction based on the design phase and the accuracy we need.

Benefits of Hardware Description Language



Plain Text Coding

Write HDL in plain ASCII text-quick capture and easy edits



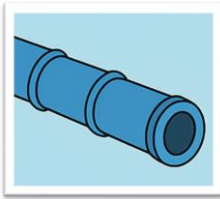
High-Level Abstraction

Design at a higher level-explore alternatives and catch issues early



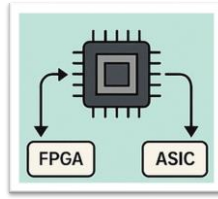
Design Reuse

Standard HDL enables reuse across teams and tools



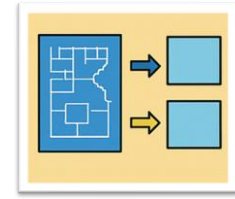
Implementation Independencies

Delay implementation choices-stay flexible



No Reformatting Needed

Standard HDL enables reuse across teams and tools



Future Adaptability

Easily adapt designs for future use

20

© Cadence Design Systems, Inc. All rights reserved.

cadence

Why should we use Hardware Description Language?

HDLs are written in plain ASCII text, making them easy to create, edit, and version control. This simplicity allows designers to quickly capture their ideas and make iterative changes without needing specialized tools.

HDLs support high-level abstraction, enabling designers to focus on functionality rather than low-level implementation. This allows for early detection of design flaws and exploration of multiple architectural alternatives before committing to hardware.

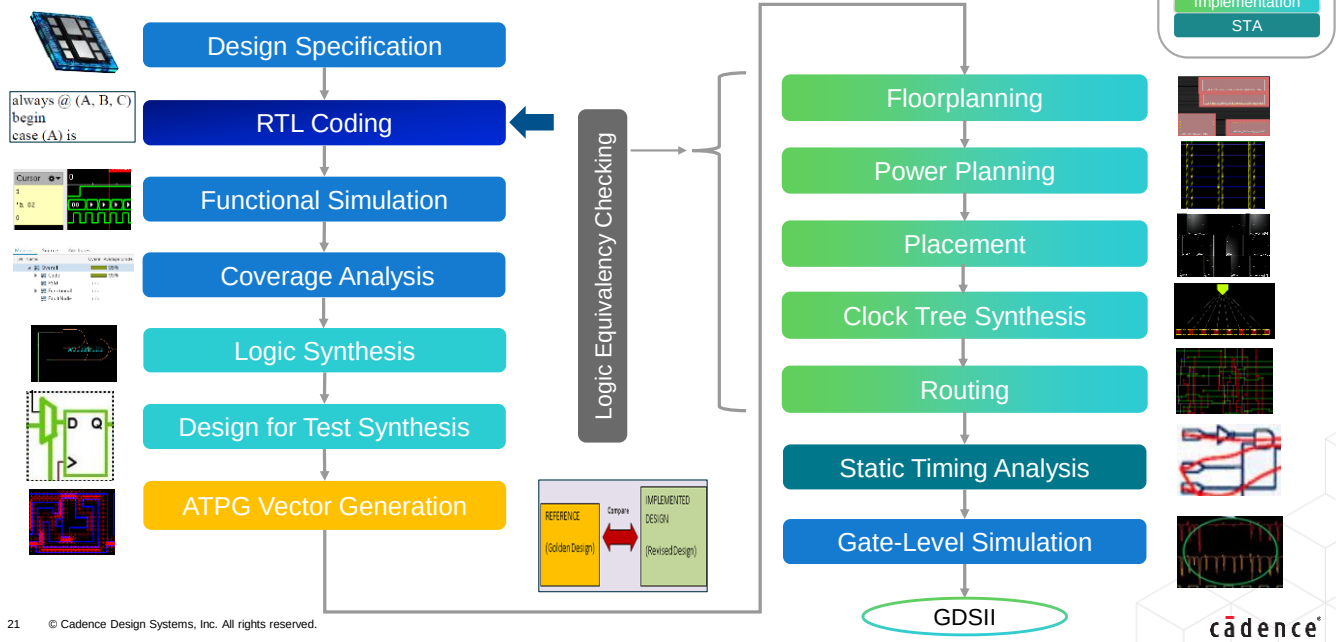
Using standardized HDL syntax and semantics promotes design reuse. Modules and components can be shared across teams and projects, improving productivity and consistency.

HDLs allow designers to describe functionality without tying it to a specific implementation. This flexibility enables late binding of implementation decisions, such as targeting FPGA or ASIC platforms.

Because HDLs follow standard formats, there is no need to reformat designs when moving between tools or teams. This streamlines collaboration and tool integration.

HDL-based designs are modular and scalable, making them easy to adapt for future requirements. This adaptability ensures long-term value and reduces redesign effort.

RTL Coding Flowchart



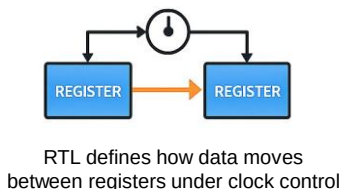
This page does not contain notes.

What Is Register Transfer Level (RTL) Coding?

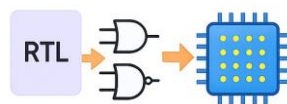


RTL coding is a method of describing synthesizable synchronous digital circuits using HDLs where the flow of data between hardware registers is controlled by clock signals.

Concept Overview



Synthesize to Hardware



RTL Code Example

```
always @(posedge clk,  
posedge rst)  
if (rst)  
  op <= 4'd0;  
else  
  op <= y;
```

RTL code written in Verilog/VHDL (any HDL) describes synchronous logic

Simulation and Verification

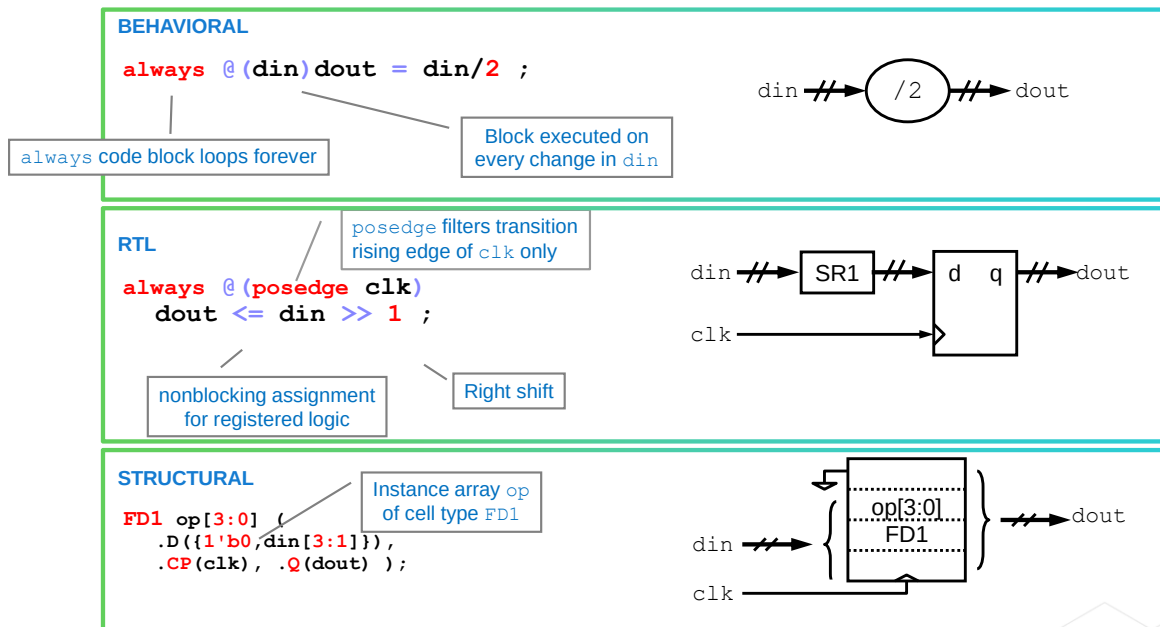


RTL coding is a method of describing synchronous digital circuits using HDLs where the flow of data between hardware registers is controlled by clock signals. This code can be synthesized into gate-level hardware and is simulated to verify logic correctness before physical implementation. Synthesis tools generate a gate level implementation from a higher abstraction level description called RTL (Register Transfer Level). Synthesis tools enforce very strict rules for RTL code, restricting RTL to a limited subset of HDL types, constructs, operators, statements, and even style and form. You must write your RTL code following a certain set of templates for it to be synthesizable.

Generally:

- You specifically identify the type and position of storage elements (registers) in your design using specific language templates.
- Combinational logic between registers can be defined using conditional, logical, and arithmetic operators.
- A synthesis tool infers registers from your RTL and maps your Combinational logic equations to basic Boolean gates (and, or, etc.)
- Hence, the name Register Transfer – you define logic to transfer data between register elements in your design.

Abstraction Level Example: Divide-by-2 Operation



23 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let's see how a simple divide-by-2 operation is represented at different abstraction levels in digital design.

These code fragments illustrate the three main levels of abstraction: Behavioral, RTL (Register Transfer Level), and Structural.

- The behavioral level of abstraction, we describe what the system should do, not how it does. We only describe the design behavior with no hint about how the operation is implemented. Here, we simply state that whenever the input `din` changes, the output `dout` should be half of `din`. This is like giving instructions without worrying about the underlying hardware.
- At the RTL level, we begin to describe how the operation is implemented in hardware. The divide-by-2 is done using a right shift, which is efficient in digital circuits. The operation is clocked, meaning it only happens on the rising edge of the clock signal. This is closer to how real hardware behaves.
- At the structural level, we describe the actual building blocks of the circuit. We instantiate and connect a predefined storage device from a macro library, not caring how the component is itself implemented. Here, we use a shift register to perform the divide-by-2 operation.
- This is how the RTL code is realized in physical hardware – using flip-flops and logic gates.

What Is Digital Design's Verification?



Digital Design's Verification verifies that the RTL design's functionality, logic, and quality meet the design specifications.



Definition

Verification ensures the design works as intended before hardware implementation



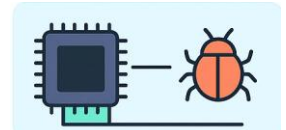
Simulation

Simulate HDL Code to test Logic and behavior



Formal Verification

Use math to prove correctness without test vectors



Coverage and Debugging

Measure test completeness and measure design bugs early

Verification is the process of checking that our design behaves as expected before hardware fabrication. It acts as a safety net to catch logical or functional errors early, saving time and cost in the long run.

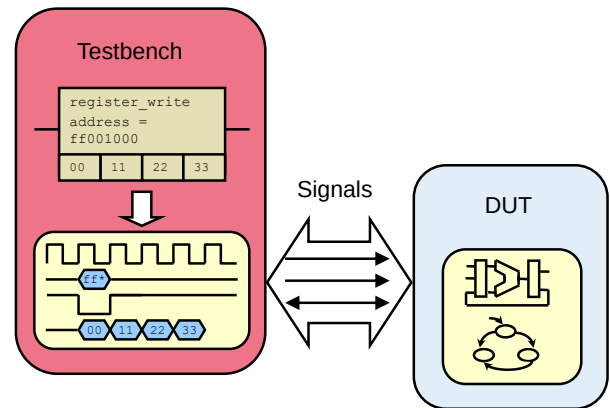
Simulation allows us to test our design in a virtual environment. We write testbenches and run them to see how the design reacts. This helps us validate the logic and functionality before moving to physical implementation.

Formal verification is like a mathematical audit of our design. Instead of testing with specific inputs, we use logic and proofs to ensure that the design will always behave correctly, no matter what inputs it receives.

Coverage tells us how much of our design has been tested. Debugging helps us find and fix issues early. Together, they ensure that our verification process is thorough and effective.

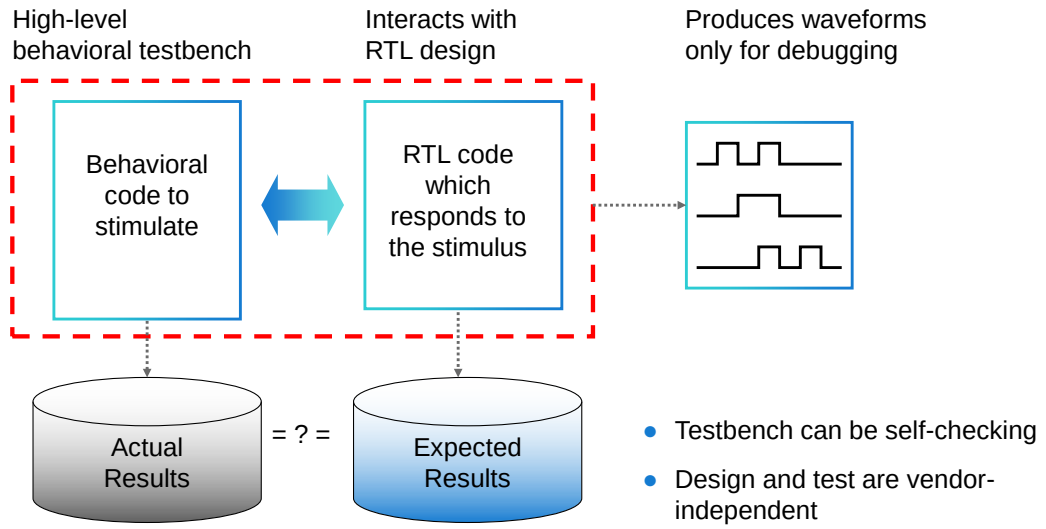
Why Is Design Verification Essential?

- Verifies that the code is syntactically correct.
- Ensures the design functionality and logical behavior are as intended.
- Verifies timing violations and power issues.
- Improves performance, efficiency and reliability.
- Minimizes design failures and bugs.
- Saves time and resources by reusing verified design components and modules.



Verification is essential to ensure that the design behaves as expected. We begin by checking for syntax errors, then move on to functional correctness. We also verify timing and power constraints to ensure the design meets performance requirements. A well-verified design is more reliable and efficient. The testbench-DUT setup shown in the slide is a standard approach to simulate and validate the design. The testbench sends inputs and observes outputs, mimicking real-world usage.

HDL-Based Simulation



The simulation process uses a hardware description language and focuses on the behavioral testbench, RTL interaction, waveform debugging, and result comparison.

We begin with the behavioral testbench, which acts as the stimulus generator. This code mimics real-world inputs and drives the RTL design under test. It is essential for verifying how the design responds to different scenarios.

Next, the RTL design receives the input stimuli. This is the core of the hardware design, written to reflect how data flows between registers and how logic gates process the inputs. We aim to validate this component through simulation.

As the simulation runs, it produces waveforms that graphically represent signal changes over time. These are crucial for debugging, allowing designers to trace and verify the internal operations of the design.

Finally, we compare the actual results from the simulation with the expected results. If they match, the design is verified. If not, we revisit the testbench or RTL code to identify issues. This step is critical in ensuring design correctness before hardware implementation.

Testbenches can be self-checking, automatically validating outputs.

HDL-based simulations are independent of vendors, ensuring portability across tools.

Who Uses Hardware Description Languages (HDLs)?



Digital Designers
Design digital systems like processors and controllers



Verification Engineers
Test and verify digital design for correctness



FPGA Developers
Implement HDL designs on reconfigurable hardware



ASIC Designers
Create custom chips using HDL for a specific application



Students and Researchers
Learn and explore digital design concepts using HDL



System Architects
Model and evaluate system-level behavior using HDLS

27 © Cadence Design Systems, Inc. All rights reserved.

cadence

- Digital designers are the architects of digital systems. They use HDLs to describe the behavior and structure of digital circuits. Their work includes designing components like ALUs, registers, and control units. They simulate and verify these components to ensure they function correctly before implementation.
- Verification engineers ensure that digital designs meet specifications and are free of bugs. They write test benches in HDLs to simulate various scenarios and validate the design's behavior. Their role is critical in catching errors early in the design cycle, saving time and cost.
- FPGA developers use HDLs to program Field-Programmable Gate Arrays. These are reconfigurable chips that can be programmed to perform specific tasks. Developers write HDL code to implement logic, test it in simulation, and then deploy it to the FPGA hardware for real-time applications.
- ASIC designers use HDLs to create Application-Specific Integrated Circuits. These are custom chips designed for specific tasks, offering high performance and efficiency. The HDL code is synthesized into a physical layout that is manufactured into silicon chips used in advanced technologies like AI and 5G.
- Students and researchers use HDLs as a learning tool to understand digital logic and system design. They experiment with writing and simulating HDL code to gain hands-on experience. Researchers also use HDLs to prototype and validate new digital architectures and algorithms.
- System architects use HDLs to model and design complex digital systems. They focus on the interaction between various subsystems and ensure that the overall architecture meets performance and functionality goals. HDLs help them simulate and validate system-level behavior before moving to detailed implementation.

Coding a Simple Generic Counter



Let us see how we can behaviorally model a generic 8-bit synchronous counter with an active low reset using the Verilog language.

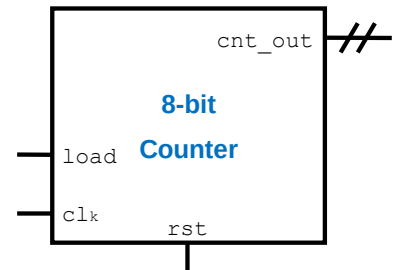
Creating Design File

Create a `design.v` file (or `.vhd` or `.sv` or with any other HDL Language of your choice).

Creating Testbench File

Create a `testbench.v` file (or `.vhd` or `.sv` or with any other HDL Language of your choice).

The following is the design block on which we are going to work.



We will learn how to code a simple generic 8-bit synchronous counter using the Verilog language. The process involves two main steps: creating the design file and creating the testbench file.

First, we need to create our design file. We are using Verilog; we can write our code in a `.v` file, but you can also use `.vhd` for VHDL, `.sv` for SystemVerilog, or any other HDL language you prefer. The purpose of this design file is to define the behavior and structure of our 8-bit counter.

Next, we need to create our testbench file. Similar to our design file, this can be written in `.v`, `.vhd`, `.sv`, or another HDL language. The testbench is crucial as it allows us to simulate and verify that our counter works correctly under different conditions before implementing it on hardware.

On the right side of the slide, you will see an illustration representing our 8-bit counter's design block. It includes four key signals:

- `cnt_out`: This represents the output count value.
- `load`: This signal is used to load an initial value into the counter.
- `clk`: The clock signal that drives the counter.
- `rst`: An active-low reset signal that initializes the counter.

This structured approach ensures that students understand both the design and verification aspects of digital circuit development.

Step 1: Creating the Design File `counter.v`

Creating a
design.v
File

Creating a
testbench.v
File

- We will use Verilog language constructs such as the `module` and the `always` (procedural block construct) to model the counter design.

```
module counter(clk,rst,count);  
input clk,rst;  
output reg [7:0] count;  
always @(posedge clk or  
negedge rst)  
begin  
    if(!rst)  
        count<=0;  
    else  
        count<=count+1;  
    end  
endmodule
```

Module named counter with inputs clk (clock) and rst (reset), and an 8-bit output count.

Declares count as an 8-bit register to hold the counter value.

This block is triggered on the rising edge of the clock or the falling edge of the reset signal.

Let's write the Verilog code for an 8-bit counter module. We will use Verilog constructs such as the module declaration and the `always` procedural block to model a simple counter.

Let's walk through the Verilog code for a simple 8-bit counter. We start by defining a module named 'counter' with inputs for clock and reset, and an 8-bit output named 'count'. Inside the `always` block, we specify that the logic should trigger on the rising edge of the clock or the falling edge of the reset signal. If the reset is active (low), we reset the counter to 0. Otherwise, we increment the counter by 1 on each clock cycle. This is a classic example of synchronous design with asynchronous reset, and it demonstrates how Verilog can be used to model sequential logic using procedural constructs.

Step 2: Creating the Testbench File `counter_test.v`

Creating a
design.v
File

Creating a
testbench.v
File

- A testbench is used to simulate and verify the behavior of a Verilog design – in this case, a counter module.
- It provides stimulus (like clock and reset signals) and observes the output to ensure the design works as expected.

```
module counter_test;
  reg clk, rst;
  wire [7:0] count;
  initial
  begin
    clk=0;
    rst=0;#10;
    rst=1;
  end
  counter counter1(clk, rst, count);
  always #5 clk=~clk;
endmodule
```

clk and rst are
inputs to the counter.

This block initializes
the clock and reset.

Instantiate the
counter module.

Toggle clock every
5 time units.

Now we will see how to create a Verilog testbench file.

- A testbench is a simulation environment used to verify the functionality of a Verilog design.
- It generates input signals (also known as stimuli) and monitors the design output under test (DUT).
- In this example, the DUT is a counter module.

Let's walk through the testbench for our counter module. We begin by declaring the clock and output signals. In the `initial` block, we set the clock to 0 and apply a reset pulse after 10 time units. This ensures the counter starts from a known state. We instantiate the counter module and connect it to our testbench signals. Finally, we use an `always` block to toggle the clock every 5 time units, simulating a regular clock signal. This setup allows us to observe how the counter behaves in response to clock and reset inputs.

Module Summary

In this module, you covered various aspects of digital design and verification.

- Started with the concept of a design specification, which is a detailed document outlining the requirements for a project.
- Defined Hardware Description Languages (HDLs), which are used to model hardware concurrency and timing at different levels of abstraction, including behavioral, RTL, gate, and switch levels.
- Defined RTL (Register Transfer Level) coding, which involves writing code using an HDL to describe data transfer between registers.
- Design verification is discussed, highlighting the importance of simulation, formal verification, and coverage analysis to ensure that the RTL design meets its specifications.



Let's summarize what we learned in this module.

Before we write a single line of code, we need to understand what we're building. The design specification acts like a blueprint – it defines what the system should do, how it should behave, and under what constraints. This ensures that everyone on the team is aligned and that the final product meets the intended goals.

HDLs are the language of hardware. Used to describe how hardware should behave. These languages allow us to model everything from high-level behavior to low-level transistor switching.

RTL is the most common abstraction level used in digital design. Here, we define how data moves through the system – what happens on each clock cycle, how registers are updated, and how logic is applied.

Verifying the design is critical. We use simulation to test different scenarios, formal methods to prove correctness, and coverage metrics to ensure we haven't missed anything. This step is vital to catch bugs before they become costly.



Module 3

Design Simulation Using the Xcelium™ Simulator

cā dence®

This page does not contain notes.

Module Objectives

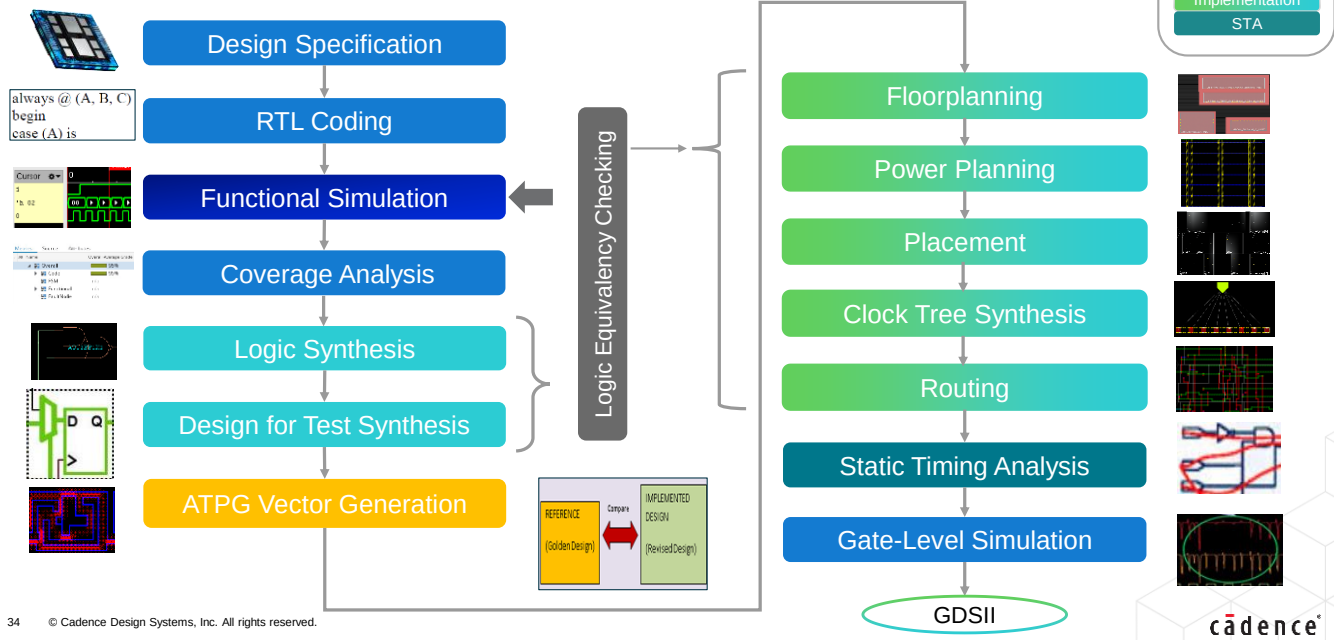
In this module, you

- Identify the process of simulation
 - What is compilation?
 - What is elaboration?
 - What is simulation?
- Execute the single-step *xrun* method of simulation
 - In both graphical and batch modes



This page does not contain notes.

Design Simulation Flow Chart



This page does not contain notes.

What Is Cadence Xcelium Simulator?

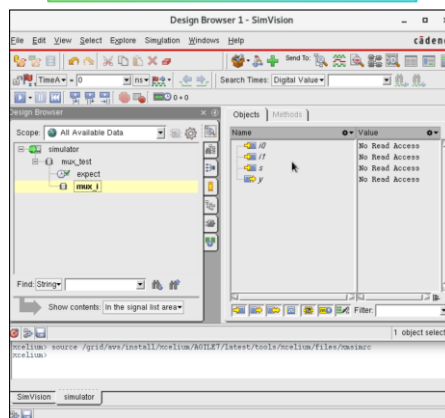


Cadence® Xcelium™ Simulator is a third-generation parallel simulator with the highest performance in simulation and debugging of digital design and verification environments.

Xcelium Simulator Key Features

- Single and multicore simulation technology.
- Smart `xrun` utilities for performing simulation with different input files and settings.
- Compiled code simulation with accuracy and flexibility.
- Debugging capabilities of event-driven simulation.
- Supports randomization, low-power simulation, mixed-signal verification, and functional safety.

Xcelium Graphical User Interface



This page does not contain notes.

How Does Xcelium Simulator Work?

The Xcelium Simulator run is separated into three major steps:

1. Compilation

- Checks syntax and semantics
- Creates design data objects
 - SystemC data (SCD) for SystemC,
 - Abstract Syntax Tree (AST) for VHDL, and
 - Verilog Syntax Tree (VST) for Verilog
- Creates SystemC and VHDL code object (.o,)

All three steps are achieved by a single command: **xrun**

2. Elaboration

- Constructs design hierarchy and connects signals
- Creates signature object (SIG) and Verilog code object (COD)
- Creates initial simulation snapshot object (SSS)

3. Simulation

- Executes simulation code



This page does not contain notes.

What Is the `xrun` Utility?



`xrun` is a Linux command that lets users run the Xcelium Simulator using the single-core or multi-core engine.

Users must specify all input files and command-line options on a single command line.

- The `xrun` utility is the default method used to run the Xcelium Simulator.
- The `xrun` utility is a simplified single-step simulation method.
- `xrun` takes input files from different simulation languages such as Verilog, SystemVerilog, VHDL, Verilog AMS, VHDL AMS, Specman[®] e, and many more.
- The `xrun` command uses explicit executables to parse, compile, elaborate, and simulate a design.
- Using the `xrun` command, users can also run simulations in multiple steps.



This page does not contain notes.

Language Supported for Xcelium Simulator

Language	File Extension	Option
Verilog	.v, .V, .vp, .VP, .vs, .VS	-vlog_ext
Verilog 1995	.v95, .V95, .v95p, .V95P	-vlog95_ext
System Verilog	.sv, .SV, .svp, .SVP, .svi, .svh, .vlib, .VLIB	-sysv_ext
VHDL	.vhd, .VHD, .vhdl, .VHDL, .vhdp, .VHDP, .vhdlp, .VHDLP	-vhdl_ext
VHDL configuration	.vhcfg, .vhcfgp	-vhcfg_ext
Specman e	.e, .E	-e_ext
Verilog-AMS	.vams, .VAMS	-amsvlog_ext
VHDL-AMS	.vha, .VHA, .vhams, .VHAMS, .vhms, .VHMS	-amsvhdl_ex

Note: Use `-default_ext <file_type>` to specify a default file type for unregistered extensions.

This page does not contain notes.

xrun Command Invocation

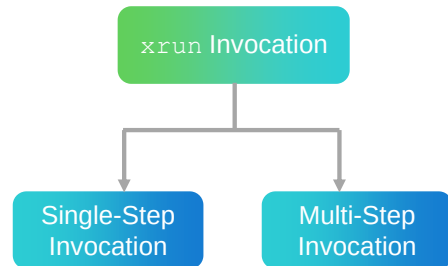


xrun **filename(s)** **[options]**

Simulation Source Files Command-line Options

The first run with `xrun` command:

- Creates a scratch directory called `xcelium.d`.
- Creates all required sub-directories with library.
- Invokes the appropriate compiler for each file specified on the command line.
- Invokes the elaborator (`xmelab`) to elaborate the design and generate a simulation snapshot.
- Invokes the simulator (`xmsim`) to simulate the snapshot.



This page does not contain notes.

Single-Step `xrun` Invocation



```
xrun [options] <source_files>
```

or

```
xrun <source_files> [options] -top [lib.]cell[.view]
```

- Cadence recommends using the single-step method of `xrun` for simulating the designs.
- The compilation, elaboration, and simulation processes are completed automatically with the single `xrun` command.
- Automatically desired compiler gets selected by looking at the source file's extension.
- Handles multiple libraries easily (with `cds.lib` and `hdl.var` files).



This page does not contain notes.

Example: Single-Step `xrun` Invocation

Consider the mixed-language example below,

```
>xrun -ieee1364 -v93 -access +r -gui verify.e top.v middle.vhd sub.v
```

- The files `top.v` and `sub.v` are compiled by the Verilog parser.
- The file `middle.vhd` is compiled by the VHDL parser `xmvhdl`. The `-v93` option is passed to the `xmvhdl` compiler.
- The file `verify.e` is compiled using `sn_compile.sh`.
- After compiling the files, `xrun` calls `xmelab` to elaborate the design. The `-access` option is passed to the elaborator to provide read access to simulation objects.
- After the elaborator has generated a simulation snapshot, `xmsim` is invoked for simulation.



This page does not contain notes.

Multi-Step xrun Invocation



```
xrun -compile [compilation_options] <source_files>  
xrun -elaborate -top [lib.]cell[:view] [elaboration_options]  
xrun -R
```

- Multiple-step `xrun` modes simulate the designs in two or three steps.
- The compilation, elaboration, and simulation processes are executed separately.
- Automatically desired compiler gets selected by looking at the source file's extension.
- Handles multiple libraries easily (with `cds.lib` and `hdl.var` files).
- The `xrun` supports using legacy code/pre-compiled libraries with the `-v <library_file>` and the `-y <library_directory>` options.



This page does not contain notes.

Example: Multi-Step `xrun` Invocation

In the following example, three HDL source files `buf.v`, `and2.v`, and `top.vhd` are considered.

```
% xrun -compile buf.v and2.v -v93 top.vhd
```

- The files `buf.v` and `and2.v` are compiled by the Verilog parser, `top.vhd` is compiled by the VHDL parser.

```
% xrun -elaborate -elabonly -top top:a
```

- Elaboration is done using the above command for which library cell view information is needed.
- Option `-elabonly` used to elaborate; without this option, both compilation and elaboration are performed.

```
% xrun -R
```

- Option `-R` takes no argument and simulates the last snapshot generated by `xrun`.
- Option `-r` takes a snapshot name as an argument and simulates the snapshot specified.



This page does not contain notes.

Xcelium Library and Directory Structure

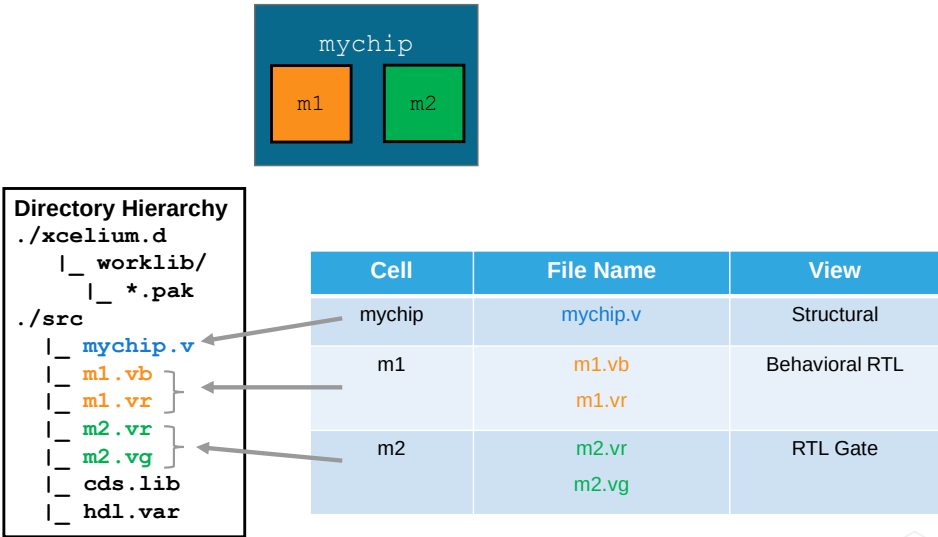
- The Xcelium library structure is organized according to `Library.Cell:View` approach.
- Xcelium automatically creates a work library called `worklib` in the directory called `xcelium.d`. The work library contains one `.pak` file.
- All the source files reside in the `src` subdirectory inside `xcelium.d`.
- The work library is saved in the `xcelium.d` directory.
- The `*.lib` directories are present inside the `src` directory.
- The user can modify the location of the `work` directory using the command-line option.



This page does not contain notes.

Example: Xcelium Library and Directory Structure

Consider a Verilog design where a module `mychip` instantiates two other modules, `m1` and `m2`.



This page does not contain notes.



Frequently Used `xrun` Options

Command-Line Option	What Does It Do?
<code>-f / -F <file></code>	Reads options/arguments from the specified file: <code>-f</code> scans the argument/arg file for files relative to <code>xrun</code> invocation dir. <code>-F</code> first scans for files relative to the location of the arg file and then re-scans relative <code>xrun</code> to the invocation directory.
<code>-sv</code>	Forces SystemVerilog compilation.
<code>-access +<rw></code>	Turns on Read, Write or Connectivity access.
<code>-v93</code>	Enables VHDL 93 features.
<code>-smartorder</code>	Enables order-independent compilation of VHDL design files.
<code>-gui</code>	Starts the simulation with the Graphical User Interface.
<code>-linedebug</code>	Allows breakpoints on lines of source code. This also executes <code>-access +rwc</code> .

46 © Cadence Design Systems, Inc. All rights reserved.



Among a lot of other options, these are the more frequently used command-line options that are ever required!

```
% xrun -f xrun.args // Scans for files relative to the xrun invocation directory
% xrun -F ./args/xrun.args // First scans for files relative to the location of xrun.args
% xrun mux.v muxtest.v -access +rwc -linedebug -gui
```

The last command invokes `xrun` and compiles the design files:

- The `-access` command-line option is used to turn on read, write or connectivity.
- `-linedebug` is to find out errors in the command line.
- `-gui` is to enable the Graphical User Interface.

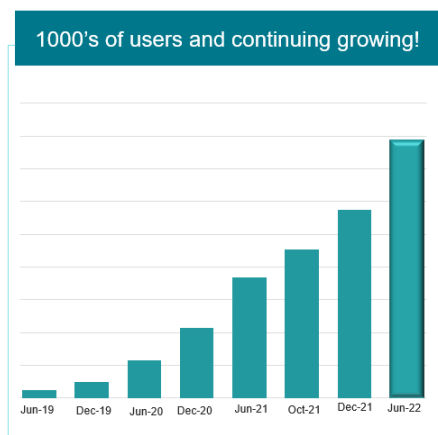
What Is Verisium Debug?



Verisium™ Debug is an advanced debugging tool that helps design engineers, integrators, and validation engineers explore, analyze, and debug complex designs and testbenches regardless of their size, language, or origin.

Why Choose Verisium Debug?

- Proven debug solution used by multiple significant customers worldwide
- Transparent compile environment with Xcelium
 - No additional compile required
 - Same compile result for simulation and debug
 - No risk of design mismatch
- 2X performance advantage over the competition
 - New VWDB waveform format
 - 2X faster in waveform dumping and reading
 - Smaller file size and memory footprint



47

© Cadence Design Systems, Inc. All rights reserved.

cadence

Nowadays, digital designs, like SoCs and IPs, are more complex, and sizes are increasing.

This brings challenges for design engineers, verification engineers, integrators, and validation engineers are also increasing to deliver a robust design. Debugging any design is the most time-consuming phase for any engineer. To make life a little easier for engineers, Cadence has Verisium Debug

Cadence Verisium Debug is an advanced debugging tool that helps design engineers, verification engineers, integrators, and validation engineers explore, analyze, and debug complex designs and test benches, regardless of their size and language.

Verisium Debug is one of the fastest-growing tools, used worldwide for debugging by leading semiconductor companies.

Users of Xcelium Simulator, can use Verisium Debug, as the same compile results are used for simulation and debugging. No risk of any mismatches between the designs.

Verisium Debug offers a substantial performance edge over competitors by leveraging a new VWDB waveform format. This enables twice as fast waveform dumping and reading capabilities. This efficiency translates into smaller file sizes and reduced memory footprints, making the tool highly efficient for large-scale designs.

Verisium Debug uses modern and innovative debug concepts like automated root cause analysis (RCA), smart log, advanced search, hierarchy browser, source code viewer, HDL Schematic browser, and advanced data exploration techniques. Using Verisium Debug, an Engineer can uncover bugs up to 50% faster than traditional, sample-based debug approaches.

The accompanying graph shows the exponential growth in user adoption. This trend highlights the increasing popularity and trust in Verisium Debug, with thousands of users joining the expanding community.

Advantages of Verisium Debug



SCALIBILITY

Quickly load large designs, easily display large waveforms



NAVIGABILITY

SmartLog, driver tracing, Root Cause Analysis (RCA)



SEARCHABILITY

Fast searching, hyperlinked search results



INTEGRABILITY

Cross-engine debugger



PORTABILITY

Save the complete debugging state for quick handoff

48 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let's see what advantages Verisium Debug offers over traditional Simulators.

Scalability and Performance – Verisium Debug quickly loads large designs, search efficiently in the databases, and is capable of displaying large waveform databases.

- **Navigability**

This Platform offers very efficient and fast navigation within the tool window, like Driver Tracing, Smart Log, and Root Cause Analysis.

- **Smart Log**

The Verisium Debug Smart Log is a dynamic, hyperlinked, and colorful log file message viewer. From the Smart Log, the user can locate the hierarchical object from where the message is printing and send messages to the waveform window. Errors are automatically bookmarked and are in red color. Many more user-friendly features are present in the smart log, which we will discuss further.

Root Cause Analysis (RCA) – With the Verisium Debug RCA tool, users can navigate through a complete failure scenario, including the testbench and RTL code, to diagnose the root cause of the failure.

- **Searchability**

Verisium Debug has a specialized, pre-indexed search database for fast searching throughout the design. It provides hyperlinked search results organized by source, value, type, and log.

- **Integrability**

The Verisium Debug provides a Cross-Engine Debugger that works with Xcelium™, Palladium®, and Embedded Software (ESW).

- **Portability**

Users can save the complete debugging state (like highlighted values, bookmarks, and notes) for quick handoff to another engineer who can continue the debugging process. With this feature, multiple teams can exchange debugging data efficiently.

To learn more about Verisium Debug, please take the Verisium Debug Course.

Quick Overview of SimVision™ Functions



The screenshot displays the Cadence SimVision interface with three main windows:

- Console - SimVision:** Shows simulation output including time steps (time=3 ns, time=4 ns), test results (TEST PASSED), and a message (Simulation is complete).
- Design Browser:** Displays the design hierarchy with a tree view showing 'simulator', 'mux_test', 'expect', and 'mux_1'. A red arrow points from the 'mux_test' node to the waveform window.
- Waveform 1 - SimVision:** Shows a digital waveform with signals 'i0', 'i1', 's', and 'y' over a time range from 0 to 4 ns. A red arrow points from the 'mux_test' node in the Design Browser to this window.

Annotations and callouts:

- Go through the design hierarchy:** A green arrow points from the Design Browser to the Console window.
- Simulate your design:** A green arrow points from the Console window to the Design Browser.
- Observe the waveform and debug:** A green arrow points from the Design Browser to the Waveform window.

At the bottom left, the text '49 © Cadence Design Systems, Inc. All rights reserved.' is visible. The Cadence logo is at the bottom right.

This page does not contain notes.

Quick Overview of SimVision Functions (continued)

The image displays two windows from the Cadence SimVision tool. The left window, titled 'Source Browser 1 - SimVision', shows the source code for a module named 'mux_test'. The code includes a task 'expect' that checks the output 'y' against expected values 's', 'i1', 'i0', and 'y'. A callout bubble points to the source code with the text 'Look at the source code'. The right window, titled 'Schematic Tracer 2 - SimVision', shows a schematic diagram of the 'mux' component. The diagram includes a multiplexer 'mux' with inputs 's=0 -> 1', 'i1=1 -> 0', and 'i0=0 -> 1'. It also shows a block 'Initial_0' with outputs 's=0 -> 1', 'i1=1 -> 0', and 'i0=0 -> 1'. A callout bubble points to the schematic with the text 'Trace the drivers of your design'. The Cadence logo is visible in the bottom right corner of the schematic window.

Look at the source code

Trace the drivers of your design

This page does not contain notes.

Executing xrun in GUI Mode



```
xrun <source files> -access +rwc -gui [other_options]
```

- The `-gui` option launches SimVision, a graphical tool for easier waveform viewing, signal tracing, and interactive debugging.
- GUI mode allows users to set breakpoints, step through simulation time, and inspect signal values interactively.
- Best for debugging, waveform analysis, and interactive exploration.

```
[pamulasa@jcvl-pamulasa simulation]$ ls
Cleanup dirs counter_test.v counter_test.v
[pamulasa@jcvl-pamulasa simulation]$ xrun counter.v counter_test.v -access +rwc -gui
TOOL:xrun(64): 25.03-s001: Started on Mar 10, 2024 at 02:44:38 PDT.
xrun: 23.09-s001: (c) Copyright 1995-2023 Cadence Design Systems, Inc.
file: counter.v
module worklib.counter.v
  errors: 0, warnings: 0
file: counter_test.v
module worklib.counter_test.v
  errors: 0, warnings: 0
  Caching library 'worklib' ..... Done
Elaborating the design hierarchy:
Top level design v
counter_test.v
Building instance
Generating native
worklib.cc
worklib.cc
Building instance
Loading native co
Design hierarchy s

Modules:
Registers:      3      3
Scalar wires:   2      -
Vectored wires: 1      -
Always blocks:  2      2
Initial blocks: 1      1
Pseudo assignments: 2    -

xmsim: *W,NOMTDGUI: Multi-Threaded Dumping is disabled for interactive debug mode.
Relinquished control to SimVision...
xcelium>
xcelium> source /grid/avs/install/xcelium/2309/23.09.001/tools/xcelium/files/xmsimrc
xcelium>
```

This page does not contain notes.

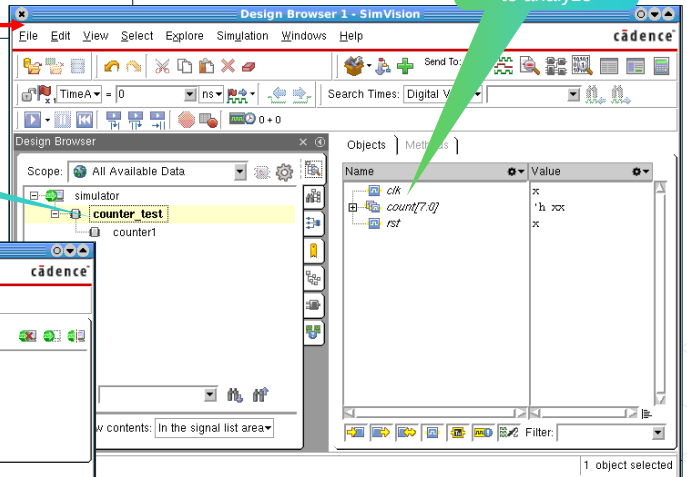
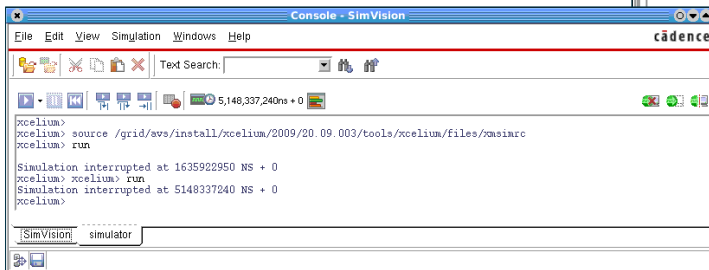
SimVision Invocation

```
Pseudo assignments: 2      2
Writing initial simulation snapshot: worklib.counter_test.v
-----
Relinquished control to SimVision...
xcelium>
xcelium> source /grid/avs/install/xcelium/2009/20.09.003/tools/xcelium/files/xmsimrc
xcelium>
```

Xcelium terminal

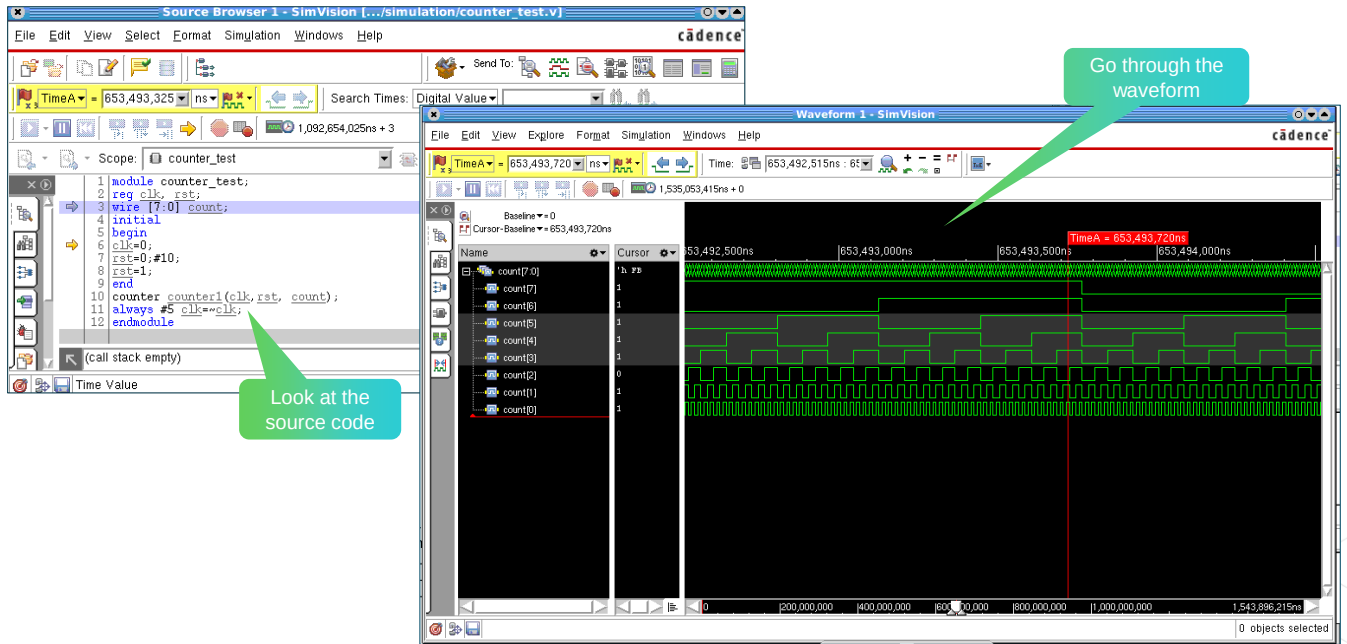
Go through the
design hierarchy

Select the
required signals
to analyze



This page does not contain notes.

Viewing the Source Code and the Waveform



This page does not contain notes.

Executing `xrun` in Batch Mode



```
xrun <source files> -access +rwc [other_options]
```

- When the `-gui` option is not provided, the command will run in the background.
- The Xcelium prompt can be used to issue other commands to view the result of the simulation.
- Best for automated runs, quick iterations, and resource-constrained setups.

```
[pamulasa@sjcvt-pamulasa simulation]$ ls
assertion_dkm.sv  counter_test.v  covfile.ccf  dkm_test.v  imc.log  mdv.log
cleanup_dirs     counter.v       cov_work     ida.db      lwddgen.log  simulation10704.die
[pamulasa@sjcvt-pamulasa simulation]$ xrun counter.v counter_test.v -access +rwc
TOOL: xrun 23.09-s001: Started on Mar 16, 2024 at 02:37:02 PDT
xrun: 23.09-s001: (c) Copyright 1995-2023 Cadence Design Systems, Inc.
      Caching library 'worklib' ..... Done
      Elaborating the design hierarchy:
      Top level design units:
          counter_test
      Building instance overlay tables: ..... Done
      Building instance specific data structures.
      Loading native compiled code: ..... Done
      Design hierarchy summary:
          Instances  Unique
      Modules:      2      2
      Registers:    3      3
      Scalar wires: 2      -
      Vectored wires: 1      -
      Always blocks: 2      2
      Initial blocks: 1      1
      Pseudo assignments: 2      -
      Writing initial simulation snapshot: worklib.counter_test:v
      Loading snapshot worklib.counter_test:v ..... Done
xcelium> source /grid/avs/install/xcelium/2309/23.09.001/tools/xcelium/files/xmsimrc
xcelium> run
```

This page does not contain notes.

Module Summary

In this module, you

- Identified the process of simulation
 - What is compilation?
 - What is elaboration?
 - What is simulation?
- Executed the single step *xrun* method of simulation
 - In both graphical and batch modes



This page does not contain notes.

References



Xcelium Training Bytes

- [Xcelium Training Byte References](#)

Xcelium One-Stop Page

- [Xcelium One-Stop Page Reference](#)

Xcelium Full Course Reference

- [Xcelium Simulator](#)

Xcelium User Manuals

- [Xcelium User Guide 25.09](#)
- [Xcelium Product Manuals](#)

Xcelium Articles Reference

- [Xcelium Articles and AppNotes](#)



This page does not contain notes.

Lab



Lab 3-1 Simulating a Simple Counter Design

- In this lab, you will execute the `xrun` command in GUI and batch modes to simulate a simple counter design with its testbench provided.



This page does not contain notes.



Module 4

Code Coverage Using the Integrated Metrics Center

cādence[®]

This page does not contain notes.

Module Objectives

In this module, you

- Define the concept of coverage in the verification process
- Understand the coverage flow through the Integrated Metrics Center and Verisium™ Manager



This page does not contain notes.



Submodule 4-1

Coverage Driven Verification Using the Integrated Metrics Center

cā dence®

This page does not contain notes.

Sub module Objectives

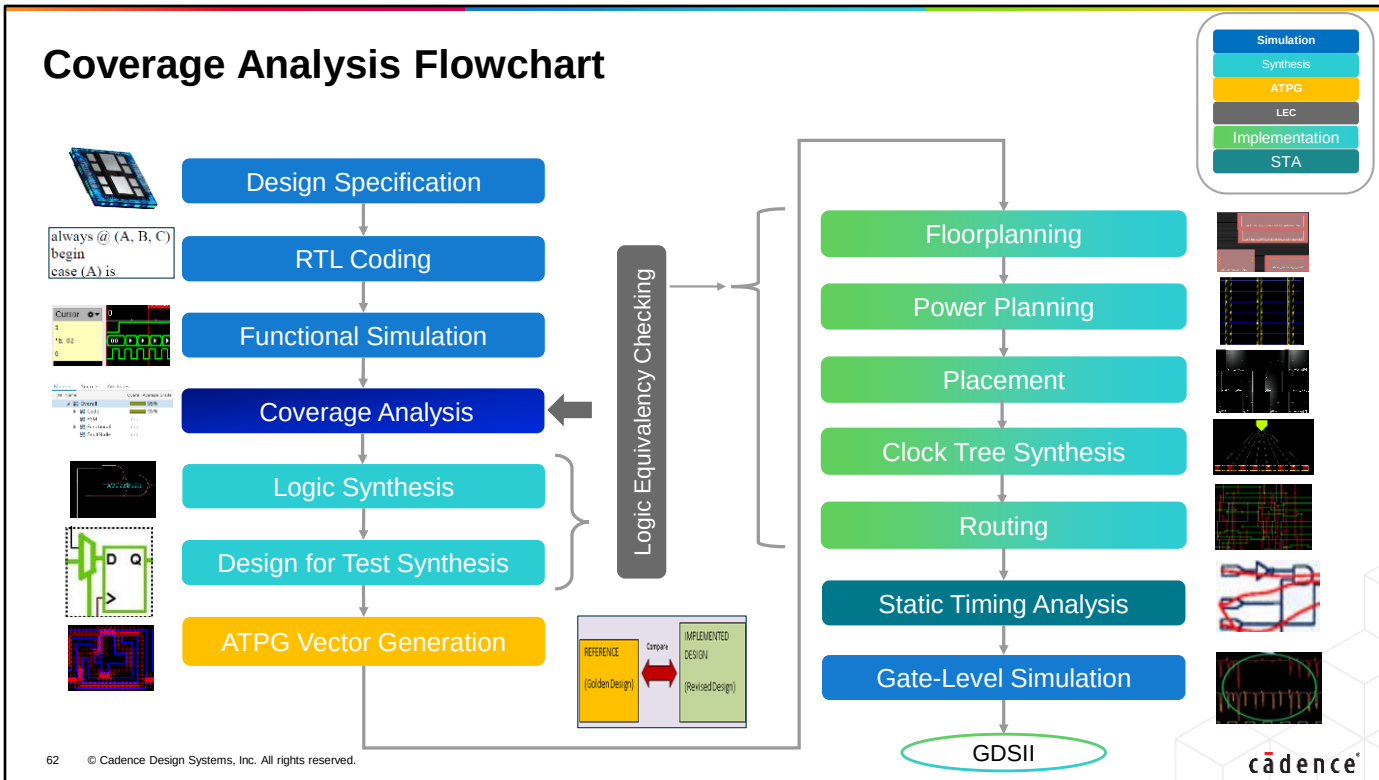
In this submodule, you

- Define the concept of coverage in the verification process
- Identify the different kinds of coverage
 - Functional
 - Code
 - Finite State Machine
- Identify the different kinds of code coverage
 - Block
 - Branch
 - Expression
 - Toggle
- Launch the Integrated Metrics Tool and go through the flow for code coverage



This page does not contain notes.

Coverage Analysis Flowchart



This page does not contain notes.

What Is Coverage?



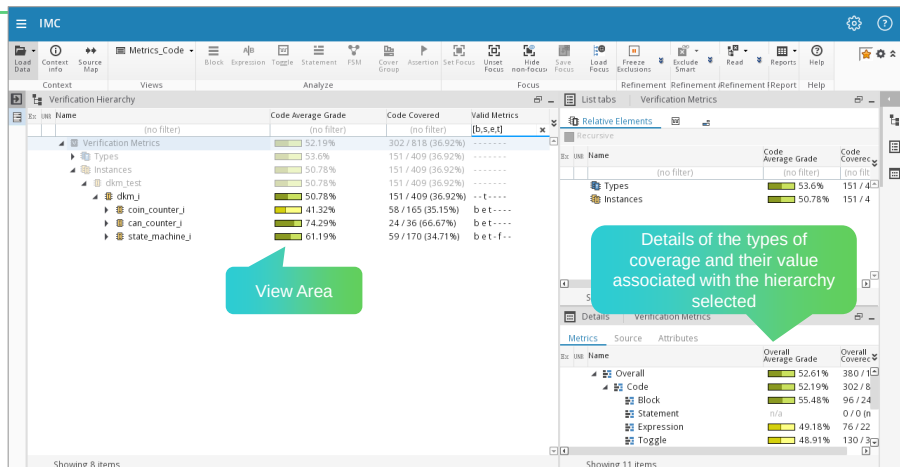
Coverage is the process of measuring how well the testbench verifies the design:

- Identify design areas in which to focus verification efforts.
- Estimate the remaining verification effort.

Coverage is the key for MDV and, hence, for Verisium™ Manager and IMC tools.

The process is to:

- Instrument the design
- Collect the coverage data
- Reduce the coverage data
- Analyze the coverage data



63 © Cadence Design Systems, Inc. All rights reserved.

cadence

- MDV is the acronym for Metric Driven Verification methodology.
- Coverage is about the *verification environment* (not about the design).
- Coverage identifies shortcomings of the verification environment that result in insufficient verification of the design.
- What matters most is detecting what coverage points are not covered, as these are holes in the verification process that must be addressed.
- By tracking verification progress over time and comparing it to that of previous projects, you can predict the additional effort required to reach the coverage target.

This section introduces the coverage process:

- In step 1, the tools automatically add coverage monitors to explicit or implicit coverage points, and the user manually adds coverage monitors for functional coverage.
- In step 2, the monitors count the coverage point hits throughout the simulation.
- In step 3, the tools reduce the coverage data to make it more manageable and record the data to coverage databases for later analysis.
- In step 4, the user analyzes the coverage data and diagnoses the reports to determine additional test requirements.



Coverage Types and Definition

Type	Definition
Code Types: Block, branch, expression, toggle	- Analyzes HDL code structure – which blocks of design code are executed. - Determines how a fully coded structure is exercised.
FSM Types: State, Transition, Arc	Interprets the synthesis semantics of the HDL design and monitors the coverage of the FSM representation of control logic blocks in the design.
Functional Types: Data oriented, control oriented	- Analyzes design functionality – which functions such as Add and Queue Filled are executed. - Determines how fully design behavior is exercised.



This page does not contain notes.

What Is Code Coverage?



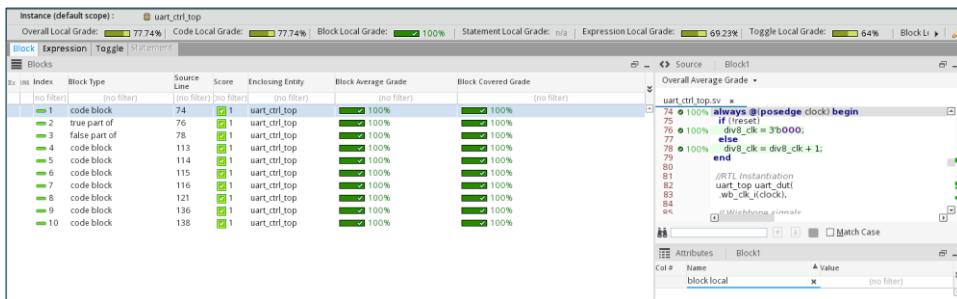
Code coverage measures how thoroughly a testbench exercises the lines of HDL code that describe a design.

- It is a method of assessing how well the test cases have exercised the design.

- The coverage results reveal areas of the design that have not been fully tested or did not meet the desired coverage criteria.

- Code coverage includes the following:

- Block coverage
- Expression coverage
- Toggle coverage
- FSM
 - State, Transition, Arc



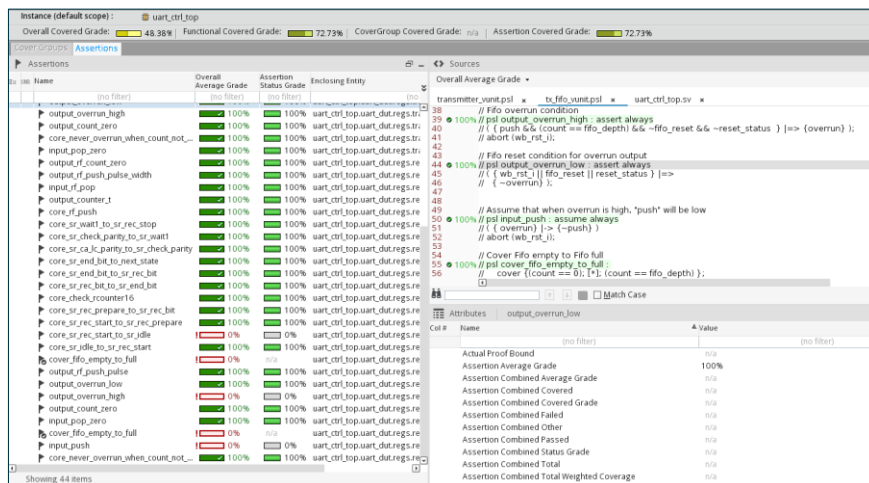
Analyzing code coverage. Code coverage measures how thoroughly a testbench exercises the lines of HDL code that describe a design. The coverage results reveal areas of the design that have not been fully tested or that did not meet the desired coverage criteria. Code coverage includes the following, which was described a couple of slides before: block coverage, expression coverage, toggle coverage, etc.

What Is Functional Coverage?



Functional coverage focuses on the functional aspect of a design and provides a very good insight into how the verification goals set by a test plan are being met.

- It is generated by inserting Property Specification Language (PSL), SystemVerilog Assertions (SVA), or SystemVerilog covergroup statements into the code and simulating the design.
- The functional coverage points specify scenarios, error cases, corner cases, and protocols to be covered and also specify analysis to be done on different values of a variable.
- It is of two types:
 - Assertion coverage
 - Covergroup coverage



66

© Cadence Design Systems, Inc. All rights reserved.

cadence

Now, let's look at the functional coverage definition as well as how it works. Functional coverage focuses on functional aspects of design and provides a very good insight on how the verification goals set by a test plan or verification plan are being met. It is generated by inserting Properties Specification Language or PSL or System Verilog Assertions SVA or the System Verilog cover group statements into the source code and then simulating the design. The functional coverage points specify scenarios, error cases, corner cases and protocols to be covered, and also specify analysis to be done on different values of a variable. It can be of two types. The assertion coverage, which is covered under SVA, and the cover group coverage, which are the cover group statements that are inserted into the code.

Comparing Code Coverage and Functional Coverage



Code coverage focuses on design code.

- Block execution
- Expression terms

Code coverage is:

- Instrumented by tool
 - Tool blindly focuses on individual items
- Easier to set up
- More difficult to analyze



Functional coverage focuses on design function.

- Data values
- Control sequences

Functional coverage is:

- Instrumented by user
 - User specifies scenarios, corner cases, protocols, etc.
- More difficult to set up
- Less difficult to analyze

Code coverage focuses on design code, such as blocks, expressions, and signals. For code coverage, the tool instruments the design, but the tool cannot understand the design functionality. You can relatively easily specify what code coverage to collect, but to understand the code coverage metrics is more difficult.

Functional coverage focuses on design function, such as data values and control sequences. For functional coverage, the tool cannot understand the design functionality, so the user instruments the design. To instrument the design is relatively more difficult, but to then understand the functional coverage metrics is less difficult.

You perform functional coverage on functional coverage points that you specify. You specify these coverage points using SystemVerilog covergroups to specify variable values and value transitions to cover, and SystemVerilog Assertions (SVA) or the Property Specification Language (PSL) to specify control scenarios, error cases, corner cases, and protocols. To implement functional coverage, you must be familiar with the design.

Functional coverage is further classified as either:

- Data-oriented functional coverage
- Control-oriented functional coverage

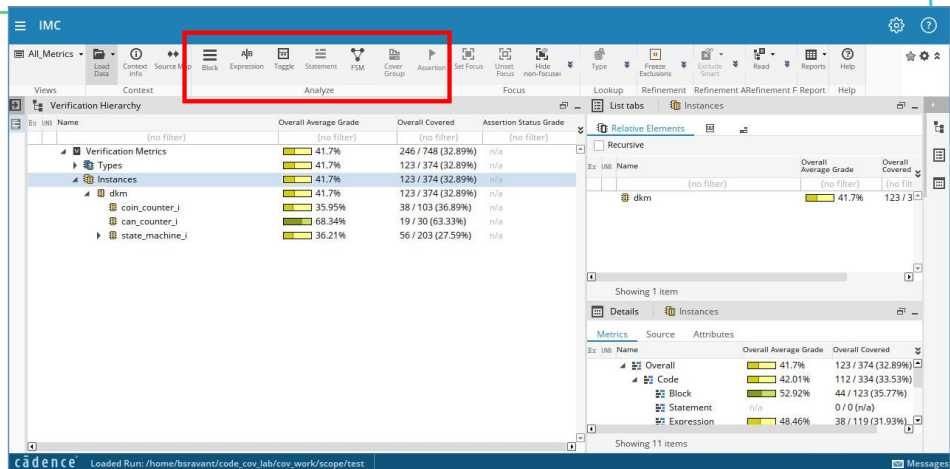
Different Types of Code Coverage and Their Analysis



Different types of code coverage include:

- Block, branch, expression, toggle and FSM

- Analyze HDL code structure to determine which blocks of design code are executed.
- Determine how fully the code structure is exercised.



This page does not contain notes.

What Is a Code Block?



A code block is a sequence of statements that always executes together.

- Either none or all of the statements in a block execute.
- A block can contain a single statement.
- Any construct that breaks execution flow creates a new block, for example:

Flow-Break Construct	Verilog	VHDL
begin (explicit or implicit)	begin, fork	begin
branch statements	if, else, case item	if, elsif, else, case item
event and timing control	wait, @, #	wait
loop statements	for, forever, repeat, while, disable	for loop, while loop, next, exit
subroutine execution	Function call, task call	Function return, procedure return
What is a scored?	Lines within a function, procedural block, or task	Lines within a function, process, or procedure
With configuration: set_assign_scoring	Continuous assignment	Concurrent signal assignment
With configuration: set_branch_scoring	Conditional expression	Conditional signal assignment selected signal assignment

69 © Cadence Design Systems, Inc. All rights reserved.



A block is a contiguous set of statements that always execute together. Any construct that breaks execution flow creates a new block. The Incisive Comprehensive Coverage does not necessarily start the new block at the flow break construct. For example, the Verilog at token (@) and hash token (#) create a new block starting with the *following* statement.

For Verilog:

- Blocks are lines of Verilog code within a function, procedural block, or task.
- The tool does not, by default, score Verilog continuous assignments. To score a continuous assignment, include the **set_assign_scoring** command in the coverage configuration file. This scores it as one block. To score the *selections* of a conditional expression, also include the **set_branch_scoring** command.
- The ICC does not score Verilog primitives, and no option exists to change this behavior.

For VHDL:

- Blocks are lines of VHDL code within a function, process, or procedure.
- The tool does not score VHDL concurrent signal assignment statements as the **set_assign_scoring** command does not apply to VHDL. To score the *selections* of a concurrent signal assignment, include the **set_branch_scoring** command.
- The ICC does not score VHDL (VITAL) primitives, and no option exists to change this behavior.

Example: Code Blocks

```
// SystemVerilog

begin                // BLOCK
    statement_or_null
end

if (a==b)
    statement_or_null // BLOCK
else
    statement_or_null // BLOCK

case (x)
    0 :
        statement_or_null // BLOCK
    default :
        statement_or_null // BLOCK
endcase

while (a==b)
    statement        // BLOCK

# 10
statement            // BLOCK
```

```
-- VHDL

begin
    sequential_statement -- BLOCK
end {process, procedure, task...};

if a=b then
    sequential_statement -- BLOCK
else
    sequential_statement -- BLOCK
end if ;

case x is
    when 0 =>
        sequential_statement -- BLOCK
    when others =>
        sequential_statement -- BLOCK
end case ;

while a=b loop
    sequential_statement -- BLOCK
end loop ;

wait for 10 ns ;
sequential_statement -- BLOCK
```

70

© Cadence Design Systems, Inc. All rights reserved.

cadence

Any construct that breaks execution flow creates a new block. This illustrates for some typical SystemVerilog and VHDL code, where the tool considers a new block to begin.

Here are the three rules for what is a block:

- All statements between the matching keywords “**begin**” and “**end**”, and do not contain a flow break statement, or a single statement that *could* have the keywords “**begin**” and “**end**” added around it, are a block.
- All statements from the keyword “**begin**”, up to and including completion of the next flow break statement, are a block.
- All statements from *after* completion of a flow break statement, up to and including the next flow break statement, or until the final keyword “**end**”, are a block.

Question: How Many Code Blocks Exist in This Code?

```
// SystemVerilog
10 always_ff
11 ☒ begin
12 ☒ @ ( posedge clk );
13     variable1 <= value1;
14     if ( condition1 )
15 ☒     variable2 <= value2;
16     else
17 ☒     begin
18         variable2 <= value2;
19         variable4 <= value4;
20     end
21 ☒ variable5 <= value5;
22     if ( condition2 )
23 ☒     variable6 <= value6;
24 ☒     // implicit else
25 ☒     variable7 <= value7;
26 end

// 8 blocks
```

```
-- VHDL
10 process
11     begin
12 ☒     wait until rising_edge(clk);
13 ☒     signal1 <= value1;
14     if condition1 = '1' then
15 ☒         signal2 <= value2;
16     else
17 ☒         signal2 <= value2;
18         signal4 <= value4;
19     end if;
20 ☒     signal5 <= value5;
21     if condition2 = '1' then
22 ☒         signal6 <= value6;
23 ☒     -- implicit else
24 ☒     end if;
25 ☒     signal7 <= value7;
26 end process;

-- 8 blocks
```

A block is a statement or sequence of statements that execute with no branches or delays. Either none or all of the statements in a block execute. The Incisive Comprehensive Coverage considers these procedural statements as a block:

- All statements between a matching *begin...end* pair do not contain a flowbreak statement or a single statement that could have *begin* and *end* statements added around it. A flowbreak statement is one that can alter the normal execution sequence of procedural statements at a given time.
- All statements from the beginning up to and including the completion of the next flowbreak statement.
- All statements from after the completion of a flowbreak statement up to and including the next flowbreak statement or until the final end statement.

For these processes:

- The first statement of the process starts a new block. Depending upon the language, the tool somewhat differently marks the first block of a process. This is because the first SystemVerilog “**begin**” keyword is a statement explicitly introducing a block, but the first VHDL “**begin**” keyword is a syntax requirement.
- A synchronization statement starts a new block on the statement after the synchronization.
- The true and false parts of each “**if**” statement start new blocks.
- The statement after each “**if**” statement starts a new block.

What Is a Code Branch?



A code branch is a statement that conditionally executes the following.

- Branch coverage (requires and) extends block coverage.
 - For example, branches of **if** and **case** statements.
- Most branches are also blocks, thus may overweight coverage metric.
- Here are some examples of branches that are *not* also blocks.

```
// SystemVerilog continuous  
  
assign y = c ?  
    a : // Branch  
    b ; // Branch
```

```
-- VHDL concurrent assignment  
  
y <=  
    a when c = '1' -- Branch  
    else b ;      -- Branch  
  
with c select z <=  
    a when '1',    -- Branch  
    b when others ; -- Branch
```

Branch coverage requires and extends block coverage.

It separately covers the branches of **if** and **case** statements. These branches are already covered as blocks, with the exception of multiple Verilog **case** branches that execute the same one block.

It covers the branches of the Verilog ternary assignment, both as a procedural assignment and as a continuous assignment.

It covers the branches of the VHDL conditional and selected signal assignments, which, until the year 2008 update, could appear only as concurrent statements. The simulator Version 14.2 does not support sequential conditional and selected signal assignments.

Block Coverage



A block is a statement or sequence of statements in Verilog/VHDL that executes without branches or delays.

Either none or all of the statements in a block are executed.

- Identifies the lines of code that are executed during a simulation run.
- It helps you determine what areas of the DUT design are not exercised by the testbenches and provides feedback for users to add more testcases.

```
always @ (in )
begin
    $display("TRUE");
    if ( in[1] == 1'b1 )
        $display("IF1 TRUE");
    else
        begin
            $display("IF1 FALSE");
            $display("IF1 FALSE");
        end;
    $display("TRUE");
    if ( in[1] == 1'b1 )
        $display("IF2 TRUE");
    $display("TRUE");
end
```

BLK1

BLK2

BLK3

BLK4

BLK5

BLK6

Any construct that breaks execution flow creates a new block, for example:
begin, if, else, case, wait, #, @, for, forever, repeat, while, disable

Block coverage is an essential first step in the overall verification process.

This page does not contain notes.

Block Coverage Analysis

The screenshot displays the Cadence IMC (Integrated Model Checker) interface for Block Coverage Analysis. The main window shows a list of blocks with columns for Block Type, Source Line, Score, and Enclosing Entity. A table of 'Blocks with Branches' is visible, listing various block types and their corresponding source lines. The 'Source Code' pane on the right shows the code for 'dmm_test.v'. The 'Attributes' pane at the bottom right lists various coverage metrics.

Annotations with red arrows point to the following elements:

- Block Page/Tab**: Points to the 'Block' tab in the top-left corner.
- Selected Instance Name**: Points to the 'Instance (default): dmm' dropdown menu.
- Line no. in the source code**: Points to the 'Source Line' column in the 'Blocks with Branches' table.
- Source Code**: Points to the 'Source Code' pane on the right.
- Block Name**: Points to the 'Block Type' column in the 'Blocks with Branches' table.
- Block ID**: Points to the 'Index' column in the 'Blocks with Branches' table.
- Covered or uncovered status**: Points to the 'Score' column in the 'Blocks with Branches' table.
- Attributes**: Points to the 'Attributes' pane at the bottom right.

Verification Hierarchy Table:

UN	Name	Overall Average Grade	Overall Covered	Assertion Status Grade
(no filter)	(no filter)	(no filter)	(no filter)	(no filter)
Types	Types	41.7%	246 / 748 (32.89%)	n/a
		41.7%	123 / 374 (32.89%)	n/a
			123 / 374 (32.89%)	n/a
			38 / 103 (36.89%)	n/a
			19 / 30 (63.33%)	n/a
			56 / 203 (27.59%)	n/a

Attributes Table:

Col #	Name	Value
	Block Active	100%
	Block Average Grade	100%
	Block Combined Average Grade	100%
	Block Combined Covered	1.0
	Block Combined Covered Grade	100%
	Block Combined Total	1.0
	Block Combined Total Weighted Coverage	1.0
	Block Combined Uncovered	0.0

This page does not contain notes.

Expression Coverage



Expression coverage measures how thoroughly the testbench exercises expressions in assignments and procedural control constructs (if/case conditions). It identifies each input condition that makes the expression true or false and whether that condition happened in the simulation.

- Expression coverage provides information on why a conditional piece of code was executed.
- It provides statistics for all expressions in the HDL code.
- You can analyze expression coverage using the Expression page.

SOP Method of Analysis – Reduces expressions to a minimum set of expression inputs that make the expression both true and false, inherently first-level.

		b	&	(c		d)
		<1>		<2>		<3>
		Expression Inputs <1>: b <2>: c <3>: d				
Simulation Scoring Count	hit	rval	<1>	<2>	<3>	
	1	1	1	-	1	
	1	1	1	1	-	
Coverage Hole	0	0	-	0	0	
	1	0	0	-	-	
		The resulting value of the expression (either zero or non-zero)				

Before scoring expression coverage, make sure you have high-block coverage in your regression.

Expression coverage. This provides information on why a conditional piece of code was executed. It provides statistics for all expressions in the HDL code. You can analyze expression coverage using the expression page of the Integrated Metrics Center GUI or the vManager GUI. To launch the expression page from the metrics page, where you see the verification metrics hierarchy, which was discussed in the previous slide, while discussing the block analysis. You select the instance or type or the level at which you want to analyze expression coverage and just click the expression icon or the button in the Analyze toolbar. Alternatively, you can **right-click** the instance or type that you have selected in the hierarchy and go down the menu that appears and select the expression analysis button.

Expression Analysis

Selected Instance

Source code, the selected expression

Expression Tab/Page

The screenshot displays the Expression Analysis tool interface. The top bar shows the 'Expression' tab selected. The main area is divided into several panes:

- Top Level Expressions:** A table listing expressions with columns for Index, Overall Average Grade, Overall Covered, Source Line, and Enclosing Entity. One expression is selected.
- Source:** A pane showing the source code for the selected expression, including a coverage equality table.
- Coverage Tables:** A pane showing a coverage table for the selected expression, with columns for Index, Overall Average Grade, and Overall Covered.
- Attributes:** A pane showing attributes of the selected expression, including Code Average Grade, Code Combined Average Grade, and Code Combined Covered.
- Sum of Products (SOP) table:** A pane showing the sum of products (SOP) table for the selected expression, with columns for Index, Name, and Value.

In the Expression page, you can:

1. Identify covered and uncovered expressions.
2. View the SOP table for the selected expression.
3. View source code, expression hierarchy, and expression.
4. View attributes of the selected expression.

Expression Analysis (continued)

The screenshot displays the Cadence Design Systems' Expression Analysis tool. The interface is divided into several panes:

- Top Bar:** Contains various tool icons and tabs like 'Expression', 'Context', 'Source Map', 'Locality', 'Refinement', 'Refinement Advanced', 'Refinement Files', 'Report', and 'Help'.
- Instance (default):** Shows overall coverage statistics: Overall Covered Grade: 32.89%, Code Covered Grade: 33.53%, Block Covered Grade: 35.77%, Statement Covered Grade: n/a, Expression Covered Grade: 31.93%, Toggle Covered Grade: 32.6.
- Block Expression Toggle:** A tabbed interface with 'Block', 'Expression', and 'Statement' tabs. The 'Expression' tab is currently selected.
- Top Level Expressions:** A table listing expressions with columns for Source, Index, Overall Average Grade, Overall Covered, Source Line, and Enclosing Entity. The first expression, 'rst == 1', is highlighted.
- Source:** A pane showing the source code for the selected expression. It contains a snippet of Verilog code:


```

rst == 1
reg [7:0] dime_count ;
if (rst == 1)
begin
nickel_count = 0;
dime_count = 0;
quarter_count = 0;
use_exact <= 1;
end

```
- Expression Hierarchy:** A pane showing the hierarchy of the selected expression. It contains a snippet of Verilog code:


```

rst == 1

```
- Full Expression:** A pane showing the full expression of the selected expression. It contains a snippet of Verilog code:


```

rst == 1

```
- Coverage Equality Table:** A table comparing two expressions, T1 and T2. It has columns for 'Term', 'Index', 'Overall Average Grade', and 'Overall Covered'. The table shows that T1 and T2 are equivalent.

Term	Index	Overall Average Grade	Overall Covered
T1	n/a	n/a	0 / 0 (n/a)
T2	n/a	n/a	0 / 0 (n/a)

Continuing the explanation of the expression analysis page. As on any page and any window, you can toggle floating and toggle auto hide options by clicking on these two buttons here, and the expression hierarchy you can see in here. I have highlighted that and presented the snapshot of the window, and the full expression of the one we have selected here will be seen here in full expression info.

Toggle Coverage



Toggle coverage provides information about the change of signals and ports during a simulation run.

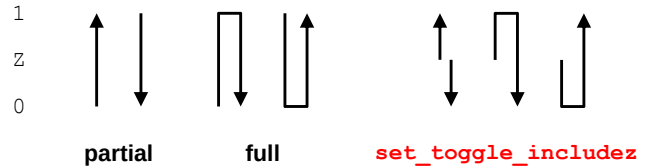
It basically monitors, collects, and reports signal toggle activity.

What does toggle coverage do?

- Monitor, collect and report design signal toggle activity.

What is a design signal toggle?

- A binary transition (and return after a finite delay) of a DUT signal.
- Signals may transition through (but not terminate at) an unknown state.



Why bother with toggle coverage?

- It is the only “code” coverage available for a gate-level netlist.
- It verifies that the design interconnect is connected and “wiggles.”

78 © Cadence Design Systems, Inc. All rights reserved.



- Toggle coverage monitors, collects, and reports signal toggle activity. Toggle coverage does not apply to variables such as integers or real numbers. You can also optionally exclude module ports. Integrated Comprehensive Coverage collects, and reports signal activity in the VHDL and Verilog portions of the design.
- The simulator normally records 0-to-1 and 1-to-0 transitions, and the reporting tool reports as full toggles the 0-to-1-to-0 and 1-to-0-to-1 transitions that pass the glitch filter and reports other transitions as partial toggles. With the *set-toggle-include-x* configuration command (**set_toggle_includex**), the simulator also records X-to-1 and X-to-0 transitions, and with the *set-toggle-include-z* configuration command (**set_toggle_includez**), the simulator also records Z-to-1 and Z-to-0 transitions. Signals must endure for at least the strobe interval. The strobe interval is, by default, the simulation time precision. To filter wider glitches, you can use a coverage configuration command to set a wider strobe interval.
- Toggle coverage is difficult to interpret but is the only code coverage available for a gate-level netlist. It verifies that the design interconnect is driven and exercised.

Toggle Coverage Analysis

The screenshot displays the Cadence IMC Toggle Coverage Analysis interface. The top menu bar includes options like Toggle, Load Data, Context Info, Source Maps, Show Local, Freeze Exclusions, Exclude, Exclude Local, Exclude Covered, Exclude Uncovered, Exclude UNR, User Comment, Un-Exclude, Exclude Smart, Exclude Type, Un-Exclude Type, Exclude Residue, Review, Un-Review, Read, Save, Reload, Unload, Report, and Help. The main area is divided into several panels:

- Instance (default):** dkm
- Overall Covered Grade:** 32.89% | **Code Covered Grade:** 33.53% | **Block Covered Grade:** 35.77% | **Statement Covered Grade:** n/a | **Expression Covered Grade:** 31.93%
- Block Expression Toggle Statement**
- Variables:** A table listing variables and their coverage. The 'nickel_out' variable is highlighted with a green checkmark and 100% coverage.
- Signals:** A table listing signals and their scores. The 'nickel_out' signal is highlighted with a green checkmark and a score of 1.
- Attributes:** A table showing attributes for the selected signal/variable. The 'nickel_out' attribute is highlighted.

The 'nickel_out' variable is selected, and its attributes are displayed in the 'Attributes' table:

Col #	Name	Value
(no filter)	(no filter)	(no filter)
	Code Average Grade	100%
	Code Combined Average Grade	100%
	Code Combined Covered	1.0
	Code Combined Covered Grade	100%
	Code Combined Total	1.0
	Code Combined Total Weighted Coverage	1.0
	Code Combined Total Weights	1.0

In the verification hierarchy page, you can click on any hierarchy, and if there is toggle coverage associated with it, then the toggle button will highlight. You can click on that and you could come to this toggle analysis page. The variables and the signals associated with the particular hierarchy are listed out here, and whatever is covered and uncovered are listed. The overall average grade and overall covered is listed here and on the right side. The source is also the source code can also be viewed by clicking on each of these variables or signals and for each of the clicked entity. For example, I have clicked on the address variable here, and you can see the associated toggle-related attributes of that particular entity. So, this is how you could perform a very basic toggle analysis and using this page, you can identify covered, uncovered signals and variables. You could also view source code and view attributes of the selected signal or variable.

What Toggle Constructs Are Covered?

	SystemVerilog	VHDL
Scopes	Module and interface nets and variables	Entity ports architecture signals
Types	tri, triand, trior, trireg, tri0, tri1, wire, wand, wor, bit, logic, reg Static array of the above Structure of the above	boolean, bit, std_[u]logic Vector of the above Record of the above

1. SystemVerilog static arrays are covered only with **set_toggle_scoring -sv_mda [<log2(max_bits)>]**.
2. Objects within generate scopes are not covered.
3. Type-parameterized objects are not covered.
4. Value-parameterized objects are covered only with **set_parameterized_module_coverage**.
5. You can provide a file of items to exclude from scoring.

80 © Cadence Design Systems, Inc. All rights reserved.



Toggle coverage by default scores 0-to-1 and 1-to-0 transitions of:

- Verilog nets and variables and ports of bit and logic types, including vectors and structures of those types.
- VHDL signals and ports of bit and logic types, including vectors and records of those types.

You can optionally elect to also score transitions originating in high-impedance or unknown states.

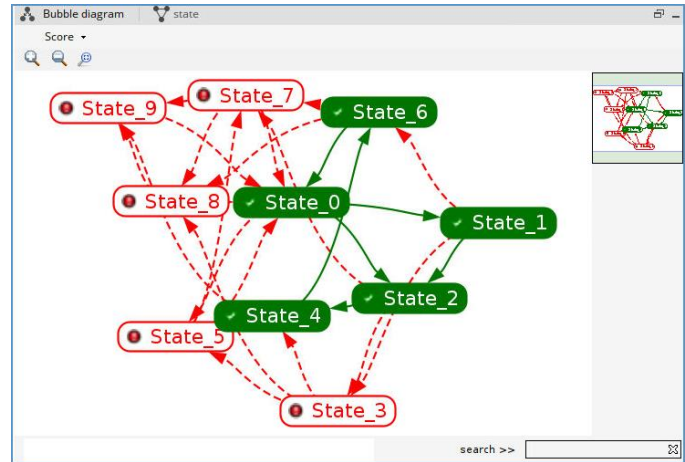
You can optionally elect to also score SystemVerilog static arrays of bit and logic types up to a maximum number of bits. The maximum is, by default, 2 raised to the 12th power (4096), and you can change that to between 2 raised to the 10th power and 2 raised to the 24th power.

FSM Coverage



Finite State Machine or FSM coverage interprets the synthesis semantics of the HDL design and monitors the coverage of the FSM representation of control logic blocks in the design.

- It answers the question, “Did I reach all of the states and cover all possible transitions or arcs in a given state machine?”
- To analyze FSM coverage:
 1. Navigate through the design hierarchy on the Analysis – Summary page and identify the overall coverage of the different state machines in the loaded run.
 2. Launch the FSM page to perform a detailed analysis of different state machines.



81 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the FSM Analysis view, there is first the list of states; the state or transaction/arc tabs will be populated with the states or transition/arcs of the selected state. When you select the *Transitions & Arcs* tab, a separate pane is displayed to show the arc source reference for the selected transition.

The *Bubble Diagram* pane of the *FSM* page displays a pictorial representation of the FSM state register. When you hover the mouse over a state or transition, the state/transition name and the count for that state/transition are shown at the bottom left corner of the diagram. One can zoom in and out of the bubble diagram. One can also search for a state via the search bar on the bottom right corner of the diagram.

In the details pane, there is a pane for the source code, arc tables, and attributes. The arc table shows the terms and values required for the selected arc.

FSM Coverage Analysis

The screenshot displays the Cadence FSM Coverage Analysis tool. The top menu bar includes options like Load Data, Context info, Source Map, Show Local, Freeze Exclusions, Exclude, Exclude Local, Exclude Covered, Exclude Uncovered, Exclude UNR, User Comment, Exclude Smart, Exclude Type, Un-Exclude Type, Un-Exclude Resilience, Review, Un-Review, Read, Save, Reload, Unload, Report, and Help. The main window is divided into several panes:

- Left Pane:** Shows a list of FSMs. A red arrow points to the 'List of FSMs' label. Below it, a table lists states and transitions.
- Center Pane:** Displays a state transition diagram (Bubble Diagram) with states 0 through 9. A red arrow points to the 'States and Transitions' label.
- Right Pane:** Shows details for the selected state (State_0), including arc conditions and attributes.

Annotations in the image include:

- List of FSMs:** Points to the table in the left pane.
- States and Transitions:** Points to the state transition diagram in the center pane.

State	Name	Encoding	Score	Is Reset State
State_0	0000	3	true	
State_1	0001	1	false	
State_2	0010	2	false	
State_3	0011	0	false	
State_4	0100	2	false	
State_5	0101	0	false	
State_6	0110	2	false	
State_7	0111	0	false	
State_8	1000	0	false	

To list all the state machines in the design for FSM Analysis:

1. Click a top-level instance in the verification hierarchy tree.
2. Select the FSMs tab in the right-hand pane and check the Recursive checkbox.

The FSMs tab page displays the list of FSMs along with the overall covered and uncovered grade of that FSM. From this list, you can identify the FSM that you want to analyze in detail and then improve its coverage. Similarly, you can select a specific instance in the verification hierarchy tree and list the FSMs in that instance.

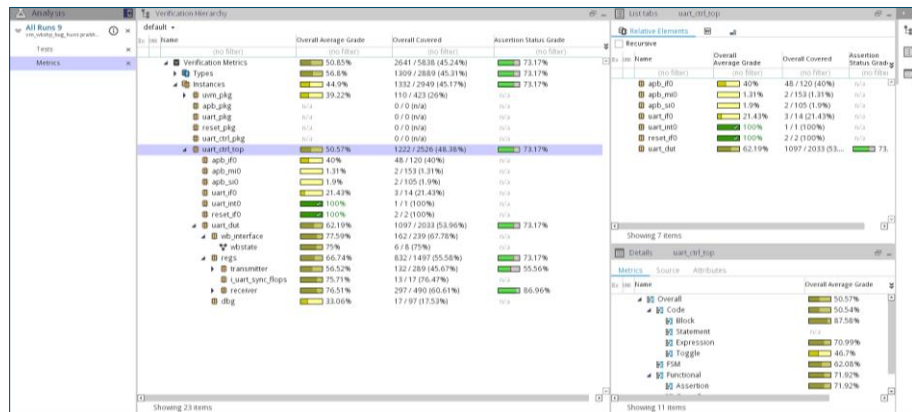
Different Types of Functional Coverage and Their Analysis



Different types of functional coverage include:

- Assertion coverage
- Covergroup coverage

- Functional coverage focuses on design functions, such as data values and control sequences.
- For functional coverage, the tool cannot understand the design functionality, so the user instruments the design.
- To instrument the design is relatively more difficult, but to then understand the functional coverage metrics is easier.



This page does not contain notes.

Assertion Coverage



Assertion coverage identifies interesting functions directly.

Assertion coverage points are specified using PSL or SVA **assert**, **assume**, and **cover** directives.

The coverage to be measured is directly specified using the PSL/SVA statements or is interpreted.

To analyze Assertion coverage:

1. Navigate through the design hierarchy on the Metrics page and identify the overall coverage of different assertions in the loaded run.
2. Launch the Assertion page by clicking the **Assertion** button from the toolbar to perform a detailed analysis of different assertions.

The screenshot shows the 'Sources' window in Cadence vManager. It displays a list of PSL assertions for a design. Two red boxes highlight specific lines of code:

- Line 36: `100% // psl core_smc_not_enabled : assert always`
- Line 48: `0% // psl core_s_idle_to_s_pop_byte : assert always`

A green callout box with the text "Assert, assume and cover directives in the testcases" points to these highlighted lines.

84 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertion coverage is an extension of assertion-based verification and identifies interesting functions directly. Assertion coverage points or specified using PSL or SVA assert. Assume and cover directives. The coverage to be measured is directly specified using the PSL or SVA statements or is interpreted from them. To analyze assertion coverage in the vManager tool, you need to navigate through the design hierarchy on the metrics page and identify overall coverage of different assertions in the loaded run. You could launch the assertion page to perform a detailed analysis of different assertions.

Assertion Coverage Analysis

The screenshot displays the Assertion Coverage Analysis tool interface. The top toolbar includes buttons for Metrics, Tests, vPlan, Reload, vPlan, New vPlan, Edit vPlan, Correlate, Rank, Item, Advanced, Show, Freeze, Exclude, Exclude Local, Exclude Covered, Exclude Uncovered, Exclude UMR, Comment, Un, Read, Save, Reload, Unload, Report, and Help. The main window is divided into several panes:

- Context Operations:** Shows the current context, 'Functional: uart_ctrl_top'.
- Assertions:** A list of assertions with columns for Name, Overall Average Grade, Assertion Status Grade, and Enclosing Entity. The list includes assertions like 'core_tf_count_zero', 'core_tf_pop_pulse', 'core_tf_pad_o_high', 'output_tf_pad_o_low', 'sugar_assert1', 'core_s_send_start_to_s_send_byte', 'core_tf_pop_low', 'core_tf_pop_high', 'core_s_pop_byte_to_s_send_start', 'core_s_idle_to_s_pop_byte', 'core_s_idle', 'core_smc_not_enabled', 'cover_fifo_empty_to_full', 'input_push', 'output_outrun_low', 'output_outrun_high', 'output_count_zero', 'core_never_outrun_when_count_not...', 'input_pop_zero', 'output_tf_count_zero', 'output_tf_push_pulse_width', 'input_tf_pop', 'output_counter_t', 'core_tf_push', 'core_sr_wait_to_sr_rec_stop', 'core_sr_check_parity_to_sr_wait', 'core_sr_cak_parity_to_sr_check_parity', and 'core_sr_end_bit_to_next_state'.
- Sources:** Displays the source code of the selected assertion, 'output_outrun_low'. The code includes comments and assertions like 'Fifo overrun condition', 'psl output_outrun_high: assert always', 'psl reset condition for overrun output', 'psl output_outrun_low: assert always', 'psl input_push: assume always', and 'psl cover_fifo_empty_to_full'.
- Attributes:** A table showing attributes for the selected assertion, 'output_outrun_low'. The attributes include Actual Proof Bound, Assertion Average Grade, Assertion Combined Average Grade, Assertion Combined Covered, Assertion Combined Covered Grade, Assertion Combined Failed, Assertion Combined Other, Assertion Combined Passed, Assertion Combined Status Grade, Assertion Combined Total, and Assertion Combined Total Weighted Coverage.

To list all the assertions in the design and identify an assertion for analysis:

1. Click the top-level instance in the hierarchy tree.
2. Select the **Assertions** tab in the right-hand pane and select the **Recursive** checkbox as shown above.

The Assertions page allows you to:

- View the list of assertions.
- View the underlying source code.
- View attributes of the selected assertion.

Note: The **Local** button on the Locality toolbar allows you to change the roll up type. Local is the default, which shows assertions in the selected instance. When you click **Local**, the button is named **Recursive**, which allows you to display assertions within all the children of the selected instance. If the button shows disabled, it indicates that you cannot change the roll-up type of the selected item.

CoverGroup Coverage



CoverGroup coverage focuses on tracking data values.

It includes coverage of variable values, binning, specification of sampling, and cross products.

It helps design engineers identify untested data values or subranges. CoverGroup coverage is specified using SystemVerilog constructs.

To analyze CoverGroup coverage:

1. Navigate through the design hierarchy on the Metrics page and identify the overall coverage of different covergroups in the loaded run.
2. Launch the CoverGroups page by clicking the **covergroup** button from the toolbar to perform a detailed analysis of different covergroups.

```

Details div_val
Attributes Source
Overall Average Grade
uart_ctrl_reg_model.sv
28 wr_cg.set_inst_name($formatf("%s.wcov", get_full_name()));
29 rd_cg.set_inst_name($formatf("%s.rcov", get_full_name()));
30 endfunction
31
32 100% covergroup wr_cg;
33   option.per_instance=1;
34 100% div_val : coverpoint div_val.value[7:0];
35 endgroup
36 17.2% covergroup rd_cg;
37   option.per_instance=1;
38   div_val : coverpoint div_val.value[7:0];
39 endgroup
40
41 virtual function void sample(uvm_reg_data_t data, byte_en, bit is_read, uvm_reg_map map
42   super.sample(data, byte_en, is_read, map);
43   if(!is_read) wr_cg.sample();
44   if(is_read) rd_cg.sample();
45 endfunction
    
```

86 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

CoverGroup Coverage Analysis

The screenshot displays the Cadence CoverGroup Coverage Analysis tool interface. The top bar contains various tool icons and tabs. The main window is divided into three panes:

- Left Pane:** Verification hierarchy tree showing the structure of the design.
- Center Pane:** List of Cover Groups. The table shows the following data:

Ex	Inst	Name	Overall Average Grade	Overall Covered	Enclosing Entity
	(no filter)	(no filter)	100%	64 / 64 (100%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_div_jatch0...	17.19%	11 / 64 (17%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_div_jatch1...	1.56%	1 / 64 (1%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_div_jatch1...	0%	0 / 64 (0%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jnt_jdr...	0%	0 / 8 (0%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jnt_jdr...	0%	0 / 8 (0%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jcr_wcov	53.57%	8 / 16 (50%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jcr_wcov	0%	0 / 16 (0%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jer_wcov	0%	0 / 8 (0%)	uvm_pkg
		reg_model_uart_ctrl_rf.ua_jer_wcov	0%	0 / 8 (0%)	uvm_pkg
		uart_trans_frame_cg	29.86%	6 / 31 (19%)	uvm_pkg

The right pane shows the detailed view of a selected Cover Group, including its attributes, source code, and a list of items. The source code snippet shows a virtual function void sample(uvm_reg_data_t data, byte_en, bit is_read, uvm_reg_map map); and a covergroup rd_cg with a coverpoint div_val.value[7:0].

To list all the state machines in the design for FSM analysis:

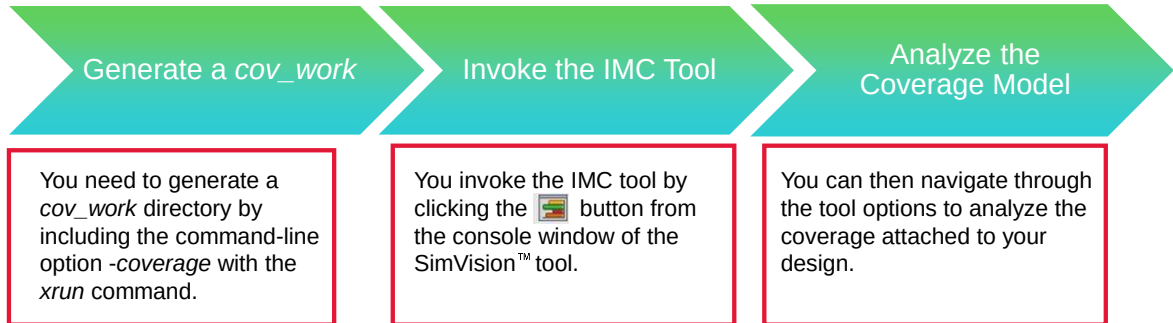
1. Click a top-level instance in the verification hierarchy tree.
2. Select the **FSMs** tab in the right-hand pane and check the **Recursive** checkbox.

The FSMs tab page displays the list of FSMs along with the overall covered and uncovered grade of that FSM. From this list, you can identify the FSM that you want to analyze in detail and then improve its coverage. Similarly, you can select a specific instance in the verification hierarchy tree and list the FSMs in that instance.

How to Perform Coverage Analysis Using the IMC Tool

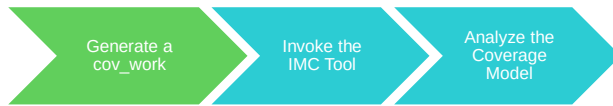


IMC (Integrated Metric Center) is a metrics analysis tool that provides an interactive way to evaluate both code as well as functional coverage.



Integrated Metrics Center Tool or IMC, is a metric analysis tool that provides you with an interactive way to evaluate both code as well as functional coverage. The way the flow goes for the IMC tool is that you need to first generate the `cov_work` directory, which contains all the coverage information by including the command-line option `-coverage` with the `xrun` command. Then you invoke the IMC tool by giving the command `imc` in your terminal or by clicking the coverage symbol button from the console window of the SimVision GUI tool. You then load the coverage model and coverage data generated in the `cov_work` directory within this directory into the IMC tool. You can then navigate through the tool options to analyze the coverage attached to your design.

Step 1: *cov_work* dir Creation with the *-coverage* Option



```

bsravant@nofcs1003:~$ xrun counter.v counter_test.v -access +rwc -coverage all -gui
TOOL: xrun(64): 25.03-s001: Started on Jan 24, 2025 at 15:16:01.131
xrun: 25.03-s001: (c) Copyright 1995-2025 Cadence Design Systems, Inc.
file: counter.v
  module worklib.counter.v
    errors: 0, warnings: 0
file: counter_test.v
  module worklib.counter_test.v
    errors: 0, warnings: 0
    Caching library 'worklib' ..... Done
  Elaborating the design hierarchy:
  Top level design units:
    counter_test

  Extracting FSHs for coverage:
    worklib.counter
    worklib.counter_test
  Total FSHs extracted = 0
  Building instance overlay tables: .....
  Enabling instrumentation for coverage types:
xmelab: *M,CVDCL: By default expression coverage is
ND, NDR, and NAND), and is scored only in condition
ts, use the "set_expr_coverable_operators" and "set_
elaboration.
  Generating native compiled code:
    worklib.counter.v <0x07852d33>
      streams: 2, words: 628
    worklib.counter_test.v <0x7ea503a5>
      streams: 4, words: 1703
  Building instance specific data structures.
  Loading native compiled code: ..... Done
  Design hierarchy summary:
    Modules:      Instances  Unique
               2          2
  
```

You need to generate a *cov_work* directory by including the command-line option *-coverage* with the *xrun* command.

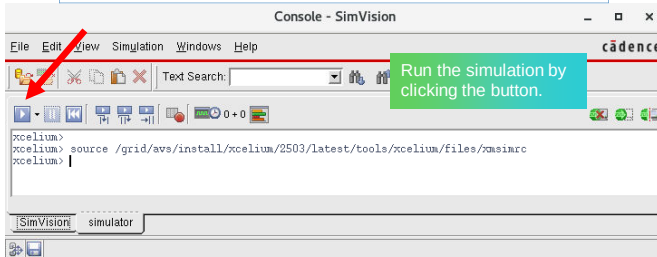
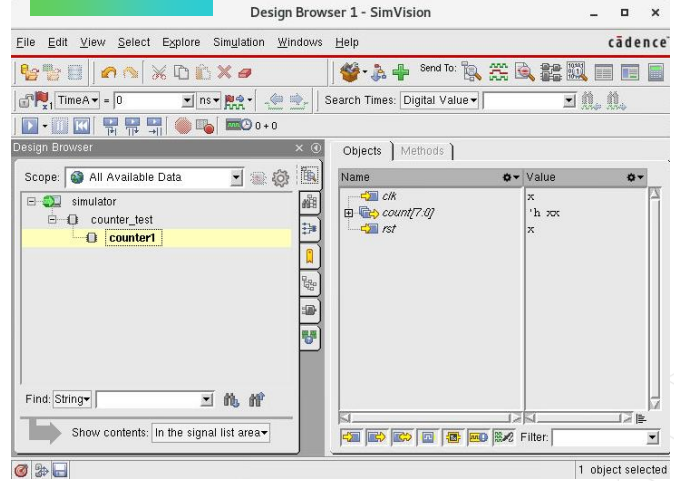
The Simulation window is shown in the next slide.

The first step is to create the *cov_work* directory, which contains all the coverage information. There is a simple counter example in the setup in this directory, and this is the command to give *xrun counter.v* and *counter_test.v*, which are the design files and the testbench files. They provide the *-access* option for read-write connectivity, and the most important option is the *-coverage all*. Then give the *-gui* option to invoke the SimVision and relinquish the control to the SimVision, and through the SimVision, we will invoke the IMC tool. So, you need to generate a *cov_work* directory by including the command-line option *-coverage*. That's the important step, the first step, and the SimVision GUI; Simulation Analysis Environment SimVision will open in this manner.

Simulation Using SimVision

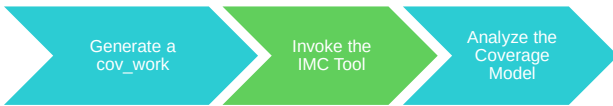
```
Generating native compiled code:
worklib.counter:v <0x07852d33>
  streams: 2, words: 628
worklib.counter_test:v <0x7ea503a5>
  streams: 4, words: 1703
Building instance specific data structures.
Loading native compiled code: ..... Done
Design hierarchy summary:
-----
Instances Unique
Modules:      2      2
Registers:    3      3
Scalar wires: 2      -
Vectored wires: 1      -
Always blocks: 2      2
Initial blocks: 1      1
Pseudo assignments: 2      -
Writing initial simulation snapshot: worklib.counter_test:v
xmsim: *W,NDHDDUI: Multi-Threaded Dumping is disabled for interactive debug mode.
-----
Relinquished control to SimVision...
xcellium>
xcellium> source /grid/avs/install/xcellium/2503/latest/tools/xcellium/files/xmsimrc
xcellium>
```

SimVision opens
and takes over the
control.



After the command given in the previous slide, the xrun command with the -gui option. After the initial simulation snapshot is created, the control is relinquished to the SimVision tool, as you can see here in the first screenshot. Then the Design Browser, as well as the console window of SimVision, opens. You can see the Design Browser, the simulator under that we have the counter_test, and the testbench under which the instantiation of counter1 and the signals that go into it are shown on the right. In the console window, you need to click on the run to run the simulation.

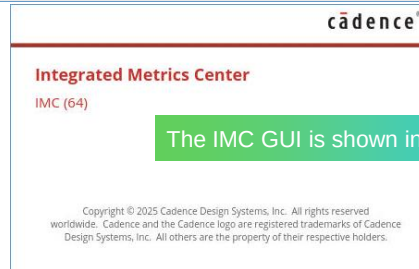
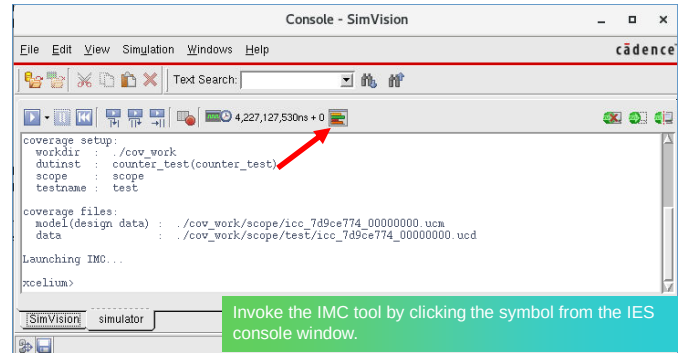
Step 2: Invoking the IMC Tool



```

bsravant@noivl-bsravant:simulation
File Edit View Search Terminal Help
[bsravant@noivl-bsravant simulation]$ ll
total 44
-rwxr-xr-x 1 bsravant cadence1 172 Mar 9 2024 cleanup_dirs
-rwxr-xr-x 1 bsravant cadence1 161 Mar 9 2024 counter_test.v
-rwxr-xr-x 1 bsravant cadence1 167 Mar 9 2024 counter.v
drwxr-xr-x 3 bsravant cadence1 4096 Jun 24 15:23 cov_work
-rw-r--r-- 1 bsravant cadence1 222 Jun 24 15:32 imc.log
-rw-r--r-- 1 bsravant cadence1 208 Jun 24 15:32 mdv.log
drwxr-xr-x 2 bsravant cadence1 4096 Jun 24 15:25 waves.shm
drwxr-xr-x 7 bsravant cadence1 4096 Jun 24 15:16 xcelium.d
-rw-r--r-- 1 bsravant cadence1 169 Jun 24 15:16 xrun.history
-rw-r--r-- 1 bsravant cadence1 189 Jun 24 15:32 xrun.key
-rw-r--r-- 1 bsravant cadence1 3008 Jun 24 15:32 xrun.log
[bsravant@noivl-bsravant simulation]$
  
```

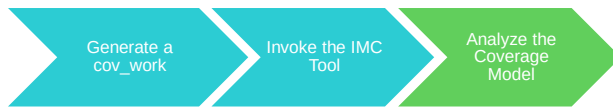
The coverage directory cov_work is created.



The IMC GUI is shown in the next slide.

After the xrun command is given and the SimVision tool takes over the simulation, you can let the counter run for a while and then check in your directory and go and list all the files and see if the cov_work directory is created. This is the basis for invoking the IMC tool and performing the Coverage Analysis. This contains the ucd and ucm directories which in turn contain all the coverage information of your design. Once confirmed that it is created, go to your Console window of the SimVision tool and click on this window for Coverage Analysis. The Integrated Metrics Center tool opens up. This flow shows how to invoke the IMC tool through the SimVision tool.

Step 3: Analyzing the Coverage Model: All Metrics View



IMC 64b | bsravant [Analysis Center] (on nofcs003)

IMC

Views: Context, Source Map, Block, Expression, Toggle, Statement, FSM, Cover Group, Assertion, Set Focus, Unset Focus, Hide non-focus, Type, Freeze Exclusions, Exclusions Smart, Read, Reports, Help

Verification Hierarchy

Name	Overall Average Grade	Overall Covered	Assertion
Verification Metrics	95%	48 / 52 (92.31%)	n/a
Types	95%	24 / 26 (92.31%)	n/a
counter_test	95%	12 / 13 (92.31%)	n/a
counter	95%	12 / 13 (92.31%)	n/a
Instances	95%	24 / 26 (92.31%)	n/a
counter_test	95%	24 / 26 (92.31%)	n/a
counter1	95%	12 / 13 (92.31%)	n/a

Relative Elements

Recursive

Name	Overall Average Grade	Overall Covered
counter1	95%	12 / 13 (92.31%)

Showing 1 item

Details

counter_test

Name	Overall Average Grade	Overall Covered
Overall	95%	24 / 26 (92.31%)
Code	95%	24 / 26 (92.31%)
Block	100%	6 / 6 (100%)
Statement	n/a	0 / 0 (n/a)
Expression	n/a	0 / 0 (n/a)
Toggle	90%	18 / 20 (90%)
FSM	n/a	0 / 0 (n/a)

Showing 11 items

cadence Loaded Run: ./cov_work/scope/test

92 © Cadence Design Systems, Inc. All rights reserved.

In the IMC tool GUI, we will open, by default, the All Metrics view. Here and you can see the Verification Metrics hierarchy. On the right side, you can see the Relative Elements, and on the bottom, the different metrics associated with whatever hierarchy we click here.

IMC Tool GUI: Block Coverage View

The screenshot displays the IMC Tool GUI with the following components:

- Top Bar:** IMC 64b | bsravant [Analysis Center] (on nofcs1003)
- Menu Bar:** IMC
- Toolbar:** Includes buttons for Block, Load Data, Context Info, Source Map, Show Recursive, Freeze Exclusions, Exclude, Exclude Local, Exclude Covered, Exclude Uncovered, Exclude UNR, User Comment, Un-Exclude, Un-Exclude Smart, Un-Exclude Type, Un-Exclude Resilience, Review, Un-Review, Read, Save, Reload, Unload, Report, Help, and a settings icon.
- Views Panel:** Shows 'Instance (default): counter_test > counter1'. Metrics include: Overall Covered Grade: 92.31%, Code Covered Grade: 100%, Statement Covered Grade: n/a, Expression Covered: n/a.
- Code Panel:** Displays the code for 'counter1' with a 'Block Coverage' tab selected. The code is:


```

always@(posedge clk or negedge rst)
begin
  if(rst)
    count<=0;
  else
    count<=count+1;
end
endmodule
      
```
- Attributes Panel:** Shows 'Block1' with the following values:

Attribute	Value
Block Active	true
Block Average Grade	100%
Block Combined Average Grade	100%
Block Combined Covered Grade	1.0
Block Combined Covered Grade	100%
Block Combined Total	1.0
Block Combined Total Weighted Coverage	1.0

A callout box with the text "This view shows the Block coverage reports." points to the 'Block Coverage' section of the 'Attributes' panel.

You can also select the Block Coverage view from the Views button. You can pull down and go to the Dynamic Views and select the Block Coverage View and you can see the code blocks and the related source code, as well as the different blocks associated on the right side.

IMC Tool GUI: Toggle Coverage View

IMC 64b | bsravant [Analysis Center] (on nofcs1003)

IMC

Toggle | Load Data | Context info | Source Map | Show Recursive | Freeze Exclusions | Exclude | Exclude Local | Exclude Covered | Exclude Uncovered | Exclude UNR | User Comment | Un-Exclude | Un-Exclude Smart | Un-Exclude Type | Un-Exclude Resilience | Review | Read | Save | Reload | Unload | Report | Help

Views | Context | Locality | Refinement | Refinement Advanced | Refinement Files | Report | Help

Instance (default): counter_test | counter1

Overall Covered Grade: 92.31% | Code Covered Grade: 92.31% | Block Covered Grade: 100% | Statement Covered Grade: n/a | Expression Covered: n/a

Block: Expression | **Toggle** | Statement

Variables

Ex	UNI	Name	Range	Overall Average Grade	Overall Covered	Enclosing En
		clk	(no filter)	100%	1 / 1 (100%)	counter_test
		rst		0%	0 / 1 (0%)	counter_test
		count	[7:0]	100%	8 / 8 (100%)	counter_test

Showing 3 items, 1 item selected

Signals: clk

Attributes: clk

Score: 1 | 1 | 1

Showing 1 item

Source: clk

```

counter.v * counter_test.v *
1 module counter(clk,rst,count);
2 input clk,rst;
3 output reg [7:0] count;
4 always@(posedge clk or negedge rst)
5 begin
6 if(rst)
7 count<=0;
8 else
9 count<=count+1;
10 end
11 endmodule
12
    
```

Code Average Grade: 100%
 Code Combined Average Grade: 100%
 Code Combined Covered: 1.0
 Code Combined Covered Grade: 100%
 Code Combined Total: 1.0
 Code Combined Total Weighted Coverage: 1.0
 Code Combined Total Weights: 1.0

cadence Loaded Run: /cov_work/scope/test

94 © Cadence Design Systems, Inc. All rights reserved.

In the same view, you can go there, and from the pull-down menu, select Dynamic views. Go to the Toggle Coverage view, and you can see the Toggle Coverage and the signals associated, the source code, and all the code blocks within it.



Submodule 4-2

Metric Driven Verification Using Verisium Manager

cā dence®

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Define the concept of Metric Driven Verification (MDV)
- Identify the process of MDV
 - Plan
 - Construct
 - Execute
 - Measure and Analyze
- Identify different centers of the Verisium Manager tool



This page does not contain notes.

Metric Driven Verification Using Verisium Manager



MDV is a powerful layer of methodology that sits above the verification testbench environment. It provides guidelines and tools for using metrics and automation to maximize the benefits of the verification testbench.

- It is a data-driven, decision-based flow that improves the Predictability, Productivity and Quality of the verification effort.
- MDV is a **closed-loop process** with the following stages:
 - **PLAN** an Executable Verification Plan.
 - **CONSTRUCT** a testbench and other verification infrastructure using Universal Verification Methodology (UVM).
 - **EXECUTE** the regression on various engines such as Simulation, Formal Verification, Emulation, etc.
 - **MEASURE AND ANALYZE** the results to see what else is left to be done to finish.
- Cadence introduces the new revolutionary tool – the Verisium Manager, which is built on MDV methodology.
- The Verisium Manager tool surrounds the whole MDV Methodology except the construction phase (which is pretty much the testbench and verification infrastructure creation). and helps us to go through each phase of the MDV process by automation.



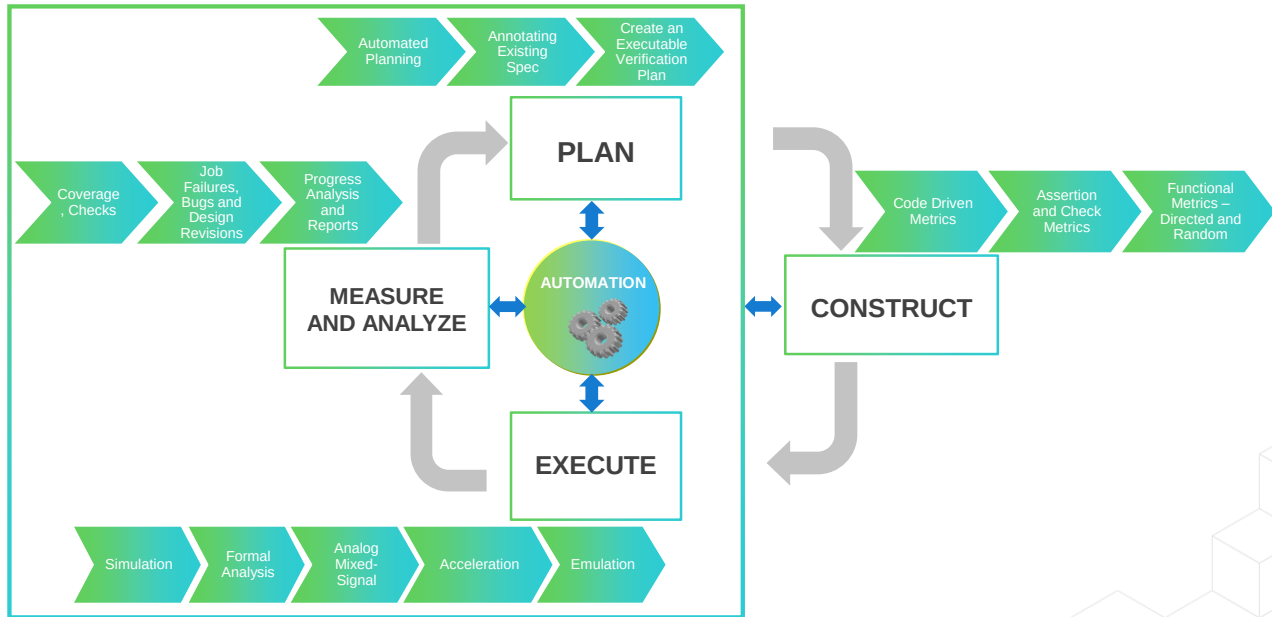
Regardless of stimulus type, MDV is about data-driven decision-making process providing clear metrics that are easily understood.

Metrics include the following:

- Spec driven:
 - Functional coverage
 - Checks and assertions
- Implementation driven:
 - Code coverage
(e.g., block, expression, toggle)
- Scenario driven:
 - Complex use-cases
(e.g., “Boot the OS”)
- Coverage contribution from:
Block → Chip → System

It is a closed-loop process, starting with creation of executable verification plans (vPlan), to testbench creation, to execution of regressions, and finishing with measurements and analysis to see what else is left to be done.

MDV Flow in Verisium Manager



98 © Cadence Design Systems, Inc. All rights reserved.



Cadence introduces the new revolutionary tool – the Verisium Manager – which is built on MDV methodology.

The Verisium Manager tool surrounds the whole MDV Methodology except the construction phase (which is pretty much the testbench and verification infrastructure creation) and helps us to go through each phase of the MDV process by automation.

Plan: The MDV solution utilizes the most powerful specific purpose verification plan authoring software, the vPlanner tool, which is an integral part of the Cadence® Verisium Manager solution. The MDV flow starts with automated planning, either annotating existing specifications with verification intent or creating an executable verification plan using team member inputs captured in a spreadsheet.

Construct: MDV spans from simple directed test methodologies, through to register-transfer level (RTL)-based coverage-driven and software-driven approaches. For RTL verification, the Universal Verification Methodology (UVM) standard has been specifically designed to provide powerful constrained random testing. The power of constrained random stimulus is that vectors are automatically generated without having verification engineers spell out each set of input vectors manually. With the constrained random stimulus comes constrained random changes in state. In this environment, it is imperative that the verification team is able to specify which states have been explored, and that for those explored states, the correct response to stimulus vectors is observed. For this reason, functional coverage is utilized. Functional coverage is defined for and captures scenarios that are important for the verification environment to observe and capture.

Execution: Verification simulation, formal analysis, analog mixed-signal, acceleration, and emulation execution engines are dispatched, and the convergence of results from multiple verification engines is managed within the Cadence Verisium Manager environment.

Measurement/Analysis: MDV is used to measure real-time progress with on-demand information, as well as to determine when high-quality verification closure is achieved. Coverage, checks, and assertions provide the verification-specific metrics used to determine closure. Verification job failures, bugs, and design revisions all provide insight into the actual status of a project. Progress analysis and reports help the verification team make adjustments to their resource allocation (people and tools) to reach closure more efficiently and measure closure more accurately.

How to Invoke the Verisium Manager (vManager) Client



You can launch a Verisium Manager Client using the Verisium Manager Profile you have created by using the host and port information.

1. Go to the directory where you want to launch the Verisium Manager tool and where you have created the server profile.
2. Use the command `vmanager -server host:port` to launch the client.

Example

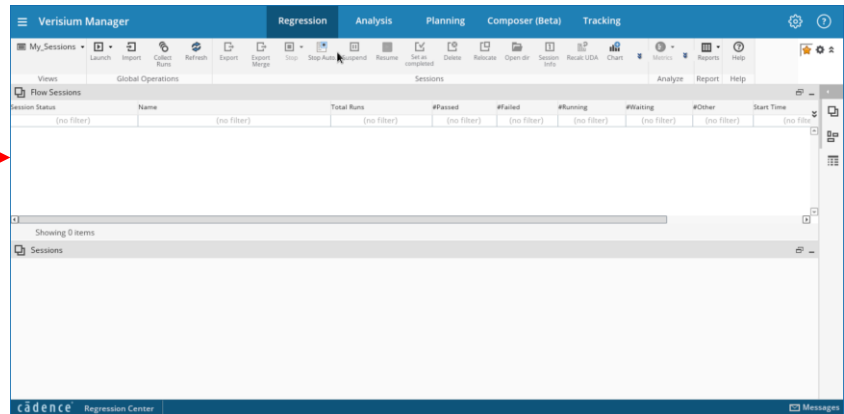
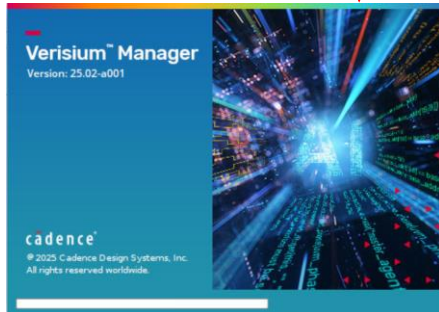
```
vmanager -server nofcn1037:8080
```



This page does not contain notes.

Verisium Manager Startup Connecting to the Server Using the Created Server Profile

```
vmmanager -server host:port  
vmmanager -server noivl-bsravant:8080
```



This page does not contain notes.

Quick Reference Guide

Verisium Manager Centers and Functionalities

Regression

Analysis

Tracking

Planning

Administration

Session Composer

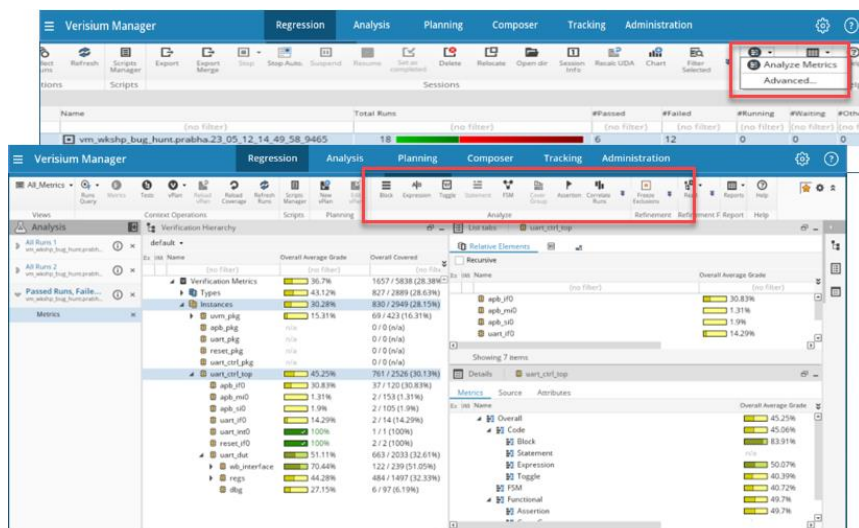


This page does not contain notes.

Analysis Center Metrics View

Clicking on the **Metrics** tab in the toolbar from the Regression center:

- Reveals the Verification Hierarchy
- Displays instances
- Provides metrics
- Displays source code



102 © Cadence Design Systems, Inc. All rights reserved.

cadence

The screenshot shows the Analysis center with the Metrics page.

To launch the metrics analysis from the Regression center:

1. In the Regression center, select the session whose metrics you want to analyze.
2. Select **Metrics** from the Analyze toolbar.

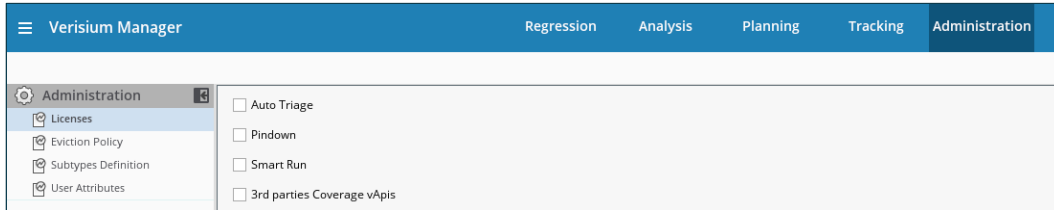
Note: Alternatively, you can **right-click** the session and select **Analyze Metrics** from the pop-up menu. You can also select **Analyze Metrics** from the Regression menu.

The Analysis center is invoked as shown in the screenshot.

Administration Center

`vmanager -admin -server <host:port>`

```
[prabha@noivl-prabha vmanager_xlm_workshop]$ vmanager -server noivl-prabha:8080 -admin &
```



103 © Cadence Design Systems, Inc. All rights reserved.

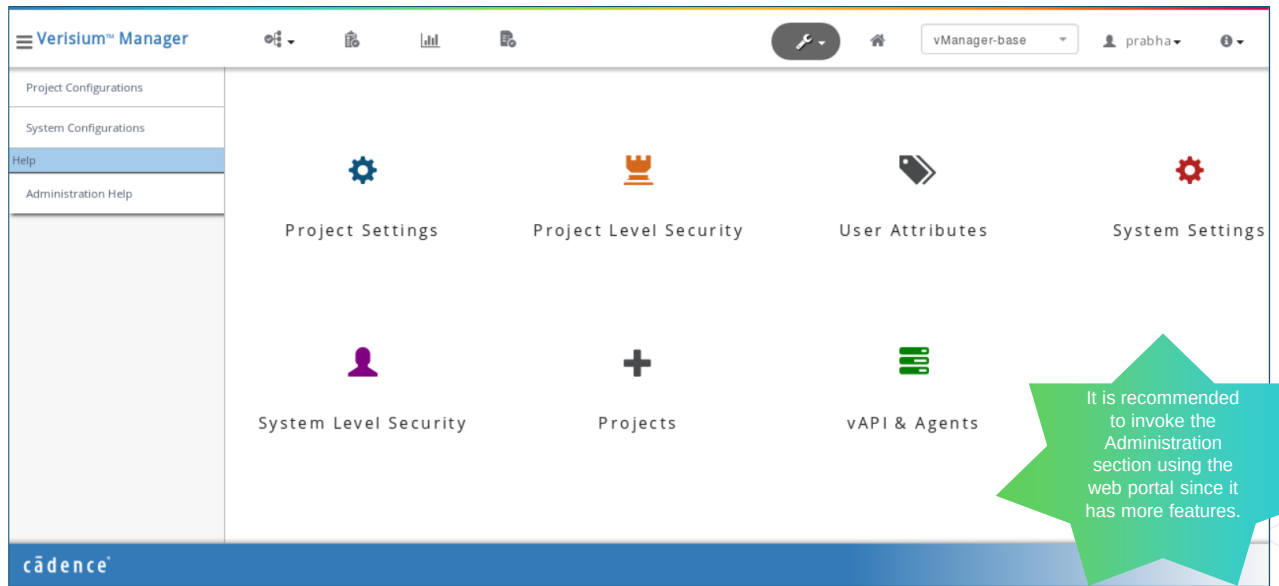


The Administration center is similar to the Profile Editor interface. It is invoked with the *vmanager – admin –server <host:port>* command.

It has following sections:

- Licenses – To manage licenses.
- Security – To manage security settings.
- vAPI Configuration – To configure the proxy manager of vAPI.
- Projects – To add additional projects.
- User attributes – To import and export user-defined attributes.

Administration Through the Web Portal



It is recommended to invoke the Administration section using the web portal since it has more features.

<https://host:port/vmgr>

104 © Cadence Design Systems, Inc. All rights reserved.

cadence

The Web Portal Administration center has the Project and System settings. This is not present in the Administration center invoked from the *vmanager -admin* option in GUI.

Module Summary

In this module, you

- Defined coverage in the verification process
- Understood the coverage flow through the Integrated Metrics Center and Verisium Manager.



This page does not contain notes.

Quiz



1. What is a code block?



2. What does expression coverage do?



3. *True or False?* A bus transitions multiple times from the high-impedance state to the same known value and then back to the high-impedance state. The bus bits are considered to have fully toggled.

106 © Cadence Design Systems, Inc. All rights reserved.



Pause here for a moment to consider these questions. Review this training module as needed. When you are ready, compare your answers to the solutions at the end of the module, and then continue on to learn about the lab and do it.

1. A block is a sequence of statements that always executes together. Either none or all of the statements in a block execute. A block can contain a single statement. Any construct that breaks execution flow creates a new block.
2. Monitors and counts operand states.
 - a. Calculates and reports term value coverage.
3. False. To have toggled each bit must visit both known states.

A code block is a sequence of statements that always executes together. Either none or all of the statements in a block execute. A block can contain a single statement. Any construct that breaks execution flow creates a new block.

The Integrated Metrics Coverage or IMC, by default, for expressions uses SOP single-bit change scoring.

To have toggled a signal must visit both known states. A bus that transitions multiple times from the high-impedance state to the same known value and then back to the high-impedance state has not fully toggled.

To enable extraction, an FSM must comply with logic synthesis standards, must utilize a single state vector, must test the current state in the outermost **if** or **case** branch, and must (generally) assign only constants to the entire state vector.

References



Integrated Metric Center Training Bytes

- Click on the [link](#) and go to [Learning > Training Bytes \(Videos\)](#)
- Use the search engine to bring up the IMC Training Bytes



One-Stop Page

- [IMC One-Stop Page Reference](#)
- [Verisium Manager One-Stop page](#)

Full Course Reference

- [Xcelium Integrated Coverage](#)
- [Verisium Manager](#)

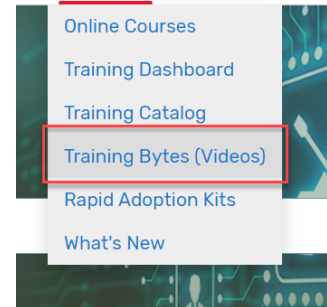
User Manuals

- [IMC User Guide 25.03](#)
- [Verisium Manager User Guide 25.03](#)
- [IMC Product Manuals](#)

Integrated Metric Center Articles Reference

- [Integrated Metric Center Articles and AppNotes](#)

as Learning Software My



This page does not contain notes.

Lab



Lab 4-1 Code Coverage Flow for a Simple Counter Design

- In this lab, you will execute the *xrun* command using the *-coverage* option to create the coverage data and model for the simple counter design and invoke the IMC tool to analyze the associated code coverage.



This page does not contain notes.



Module 5

The Synthesis Stage

cādence[®]

This page does not contain notes.

Module Objectives

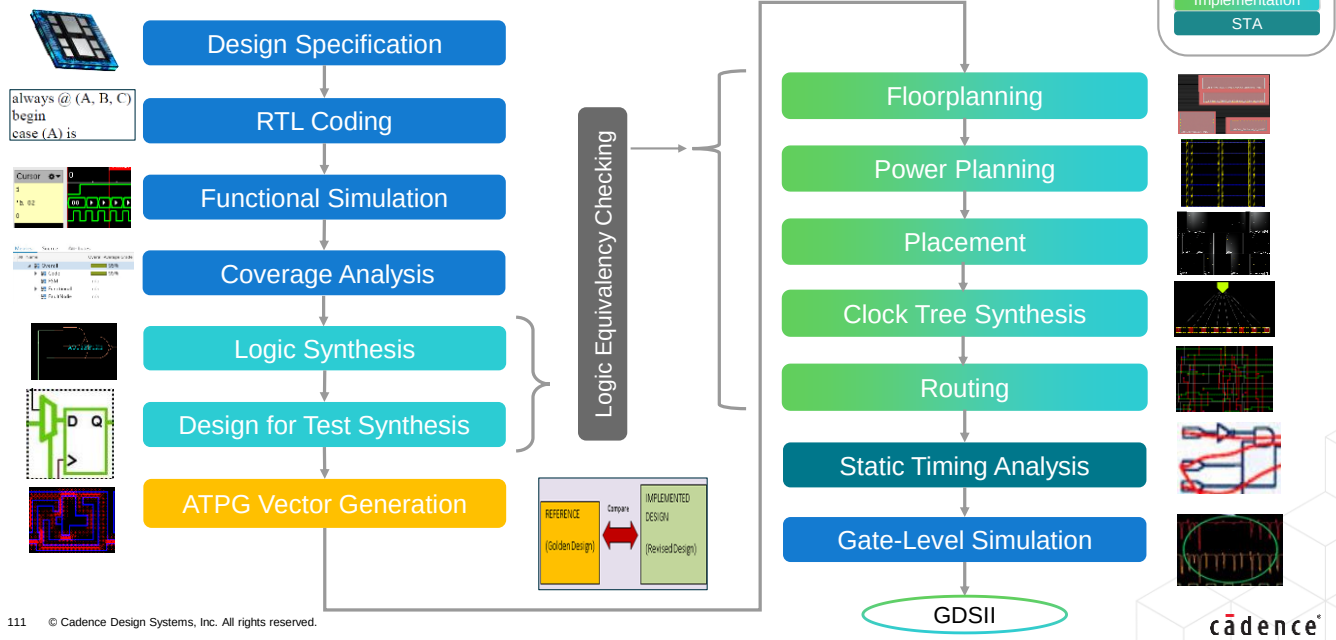
In this module, you

- Run the basic synthesis flow
- Write the synthesis outputs
- Set up Design For Test (DFT) during synthesis



This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow:



This page does not contain notes.

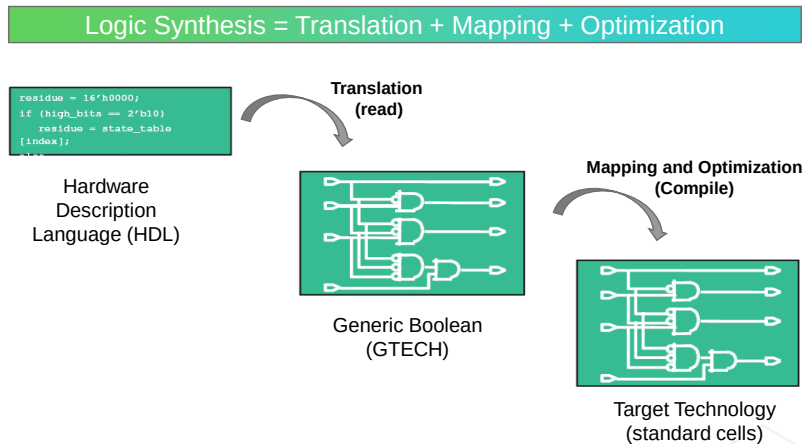
What Is Synthesis?



Logic Synthesis is the process of transforming Hardware Description Language (HDL) code into a logic circuit based on a compiled technology-specific library and user-specified optimization constraints.

Logic design is the representation of the functional design of a circuit in the form of logic operations, arithmetic operations, control flow, etc.

- Boolean AND, OR, XOR and NAND operations.



This page does not contain notes.

What Is the Stylus Common User Interface?



Stylus Common User Interface is an interface that is common across all tools: Genus™, Joules™, Modus™, Innovus™, Tempus™, and Voltus™.

Features of the Stylus Common User Interface:

- Common objects and commands subset across tools
- Unified Metrics for common Quality of Results (QoR) review
- Flowkit for common design flow deployment
- Increased productivity through a familiar interface across core implementation and signoff products
- Common and customizable reporting experience
- By using commands among the common command's subset, scripts can be run in different tools
- In Common UI, DB access is with `set_db` and `get_db`

Stylus Common UI is the *default* working mode of Genus Synthesis Solution.

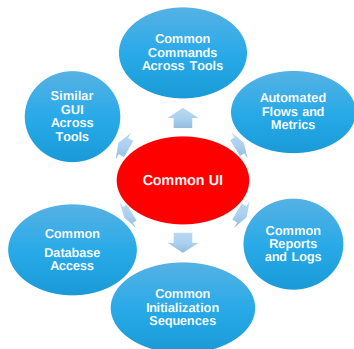
113 © Cadence Design Systems, Inc. All rights reserved.



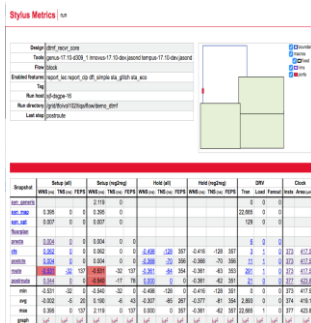
- The Stylus Common User Interface (CUI) has been designed to be used across the Genus™, Joules™, Modus™, Innovus™, Tempus™, and Voltus™ tools.
- By providing a common interface from RTL to signoff, the Common UI enhances your experience by making it easier to work with multiple Cadence® products.
- The Common UI simplifies command naming and aligns common implementation methods across Cadence digital and signoff tools.
- For example, the processes of design initialization, database access, command consistency, and metric collection have all been streamlined and simplified.
- In addition, updated and shared methods have been added to run, define, and deploy reference flows.

Three Components of the Stylus Common User Interface

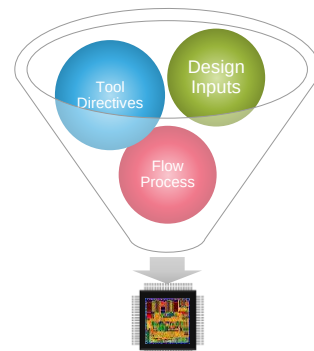
1 Common User Interface



Unified Metrics



Flowkits



Stylus is an infrastructure that offers three significant features:

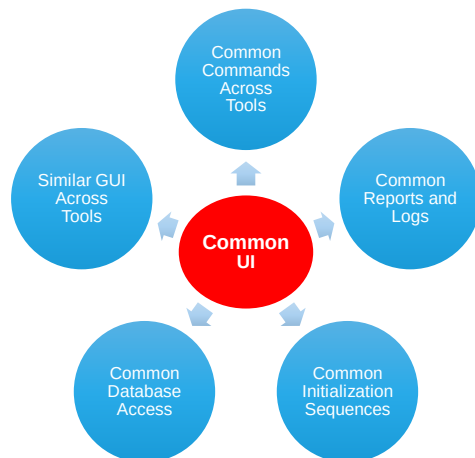
- Stylus Common User Interface offering consistent commands across the whole digital flow
- Stylus Unified Metrics for capturing and reporting
- Stylus Flowkit for design flow capture and deployment

What Is Common Interface?



Common interface is a common way to update and unify all aspects of the user experience.

- Accessing tool objects – *get_db/set_db*
- Setting up the tools – *init* and *MMMC*
- Controlling the tools – commands and attributes
- Driving the tools – startup files and reference flows
- Assessing the results – common report formats and styles



The Common UI provides an improved interface with a reduced number of commands and attribute cleanup. It also enhances usability with cleaned-up log files and improved messaging.

Command logging plays a vital role in the debug process. The Stylus Common UI provides improved and uniform logging across products by logging all commands in the log file, irrespective of whether they are issued interactively or through startup files and scripts.

- The *set_db* and *reset_db* are Common Database Access Methods in all tools.
- And are the companion commands to set values.
- Common Initialization Flow with a Common MMMC file:
 - The initialization flow is now the same for all the tools *supporting MMMC* and uses the same MMMC file.
 - A new *timing_condition* object has been added to the existing MMMC syntax in the MMMC file. This object is required by the Common UI and makes it possible to:
 - Remove all timing data from *power_intent* files, and
 - Bind *power_domains* to the MMMC timing data more efficiently.
- Similar GUI.
- Common Timing Reports:
 - In the Common UI, the *report_timing* command has been enhanced to:
 - Allow fast identification of issues related to clock definition, optimization, and constraints.
 - Facilitate analysis and debugging. (Enhancements include easier issue identification, the cut-and-paste option, similar reports for different tools, and some customization features.)
 - Produce a more efficient and consistent report format. (This includes aligning similar data from launch and capture paths and aggregating useful information that may not be visible in detailed paths.)

Common Database Access



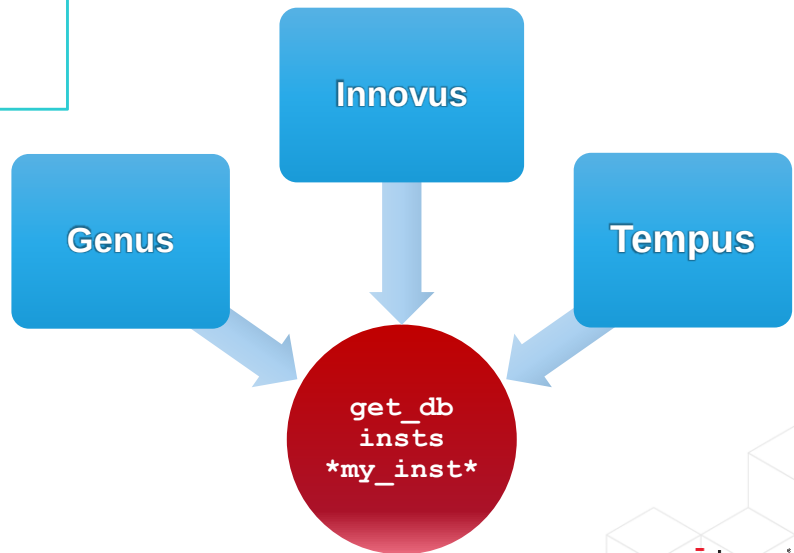
Unique user interfaces across tools:

- Create multiple learning curves
- Duplicate flow development needs

How do you resolve this?

Solution – Common *db* command for:

- Library data
- Logical connectivity
- Physical geometries
- Timing and power results



116 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

How to Start Up the Tool When Using the Common User Interface



In the Common UI mode, Genus uses the same startup scheme as the other Cadence tools that use the common UI (Innovus and Tempus).

Genus looks for the *genus.tcl* initialization file for setup information.

Genus will look for these files in four different directories using this search order:

1. The installation root directory:

`installation_dir/etc/synth/genus.tcl`

2. The `.cadence` directory in your home directory:

`~/ .cadence/genus/genus.tcl`

3. The `.cadence` directory in the current directory:

`./ .cadence/genus/genus_startup.tcl`

4. The *genus_startup.tcl* file in the current design directory (contains a project-specific setup)

If the GUI is enabled, the `.cadence` directory in your home directory:

`~/ .cadence/genus/gui.tcl`

```
Checking out license: Genus_Synthesis
Loading tool scripts...
Sourcing GUI preferences file /home/user/.cadence/genus/gui.tcl...
Finished loading tool scripts (11 seconds elapsed).
Sourcing /home/user/.cadence/genus/genus.tcl...
My name is Genus
```



The name of the startup file in the working directory is *genus_startup.tcl*. The name has been changed to prevent accidental removal of the startup file when using *genus.tcl* as the name of the run script or when issuing the *rm genus.** command.

If files exist at all the three places, then all encountered files will be read.

The four directories are listed below:

`installation_dir/etc/synth`

`~/ .cadence/genus`

`./ .cadence/genus`

`<current_directory>` from where we start Genus

Starting Genus Synthesis Solution



```
genus [-abort_on_error] [-batch]
      [-del_scale 10]
      [-disable_user_startup] [-
execute <string>]+ [-files
<string>]+
      [-help] [-legacy_ui] [-lic_stack
<integer>] [-lic_startup
<string>]
      [-lic_startup_options <string>]+
      [-log <prefix>] [-no_gui]
      [-legacy_gui] [-overwrite] [-
version] [-wait <integer>]
```

You can start Genus Synthesis Solution by writing **genus** on the UNIX/Linux shell.

Example

Linux> genus -f runTop.tcl -log genus_run

- The command file (.cmd) uses the same prefix as log or can be specified as a pair as well:

**Linux> genus -f runTop.tcl -log "runTop.log
runTop_test.cmdfile"**

- After Genus starts, you can run a script using source:
- You can set the search paths so that the software can locate the files that Genus uses.

To set the script search path, enter:

**genus@root:> set_db script_search_path {
path1 path2 ... } /**

Then, enter:

genus@root:> source script_file_name

Options to Start Genus

[-abort_on_error]	Exits on script error
[-batch]	Exits after processing files
[-del_scale 10]	For low frequency designs (below 5kHz)
[-disable_user_startup]	Reads only the master <i>init</i> file
[-execute <string>]+	Command string to execute before “-files” processing
[-files <string>]+	Executes command file
[-help]	Prints this message
[-legacy_gui]	Starts tool with Legacy GUI
[-legacy_ui]	Starts tool with Legacy UI
[-lic_stack <integer>]	Specifies the number of licenses for VDI
[-lic_startup <string>]	Specifies one of the following licenses: • Genus_Synthesis • Virtuoso® Digital Implem • Virtuoso_Digital_Implem_XL
[-lic_startup_options <string>]+	Checks out an option license at startup: • Genus_Low_Power_Opt • Genus_Physical_Opt • Vdixl_Capacity_Opt
[-log <string>]	Prefix for the .log and .cmd files. If two args are given like -log “a b”, then the two names are used without any extension added. If the .log file already exists, an integer is appended to all the names to make the files unique.
[-no_gui]	Starts with GUI disabled
[-overwrite]	Overwrites default logfiles
[-version]	Returns program version information
[-wait <integer>]	Queues wait timeout in minutes

Exiting Genus Synthesis Solution



`quit [return code] or exit [return code]`

Use *quit [return code]* or *exit [return code]* to close Genus.

The session data is lost unless you have written it to the disk.

```
genus@root:> exit
```

```
Normal exit.
```

```
linux:> echo $status
```

```
0
```

```
genus@root:> exit 12
```

```
Abnormal exit.
```

```
linux:> echo $status
```

```
12
```



This page does not contain notes.

Backward Compatibility



```
eval_legacy { source myScript.tcl }
```



```
is_common_ui_mode
```

To execute legacy Genus commands from Genus Stylus CUI, run:

```
genus@root> eval_legacy {source  
myScript.tcl}
```

Use `eval_common` to run CUI commands when in Legacy mode.

Detecting an Active Interface

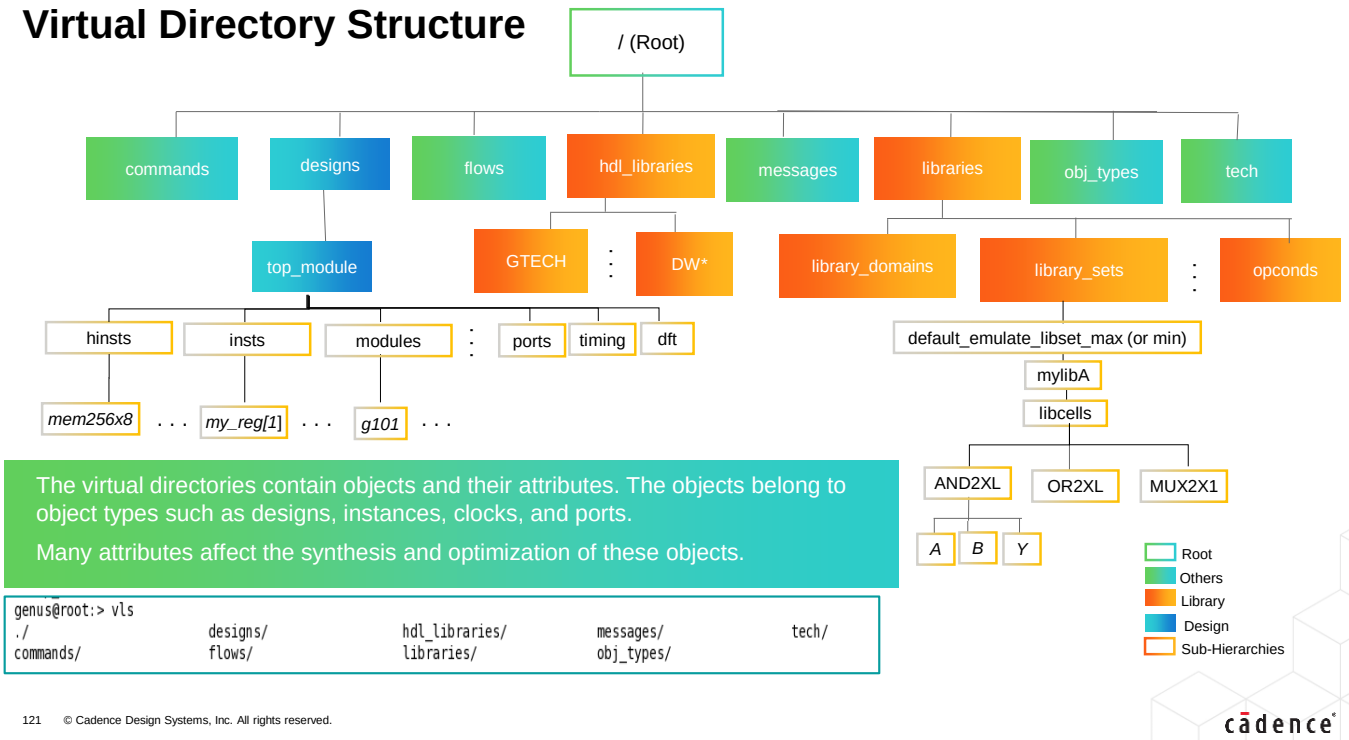
The `is_common_ui_mode` command can be used to programmatically determine an active interface:

```
genus@root:> is_common_ui_mode  
1  
genus@root:> eval_legacy {is_common_ui_mode}  
0
```



This page does not contain notes.

Virtual Directory Structure



Tcl commands help navigate the database and modify objects. You can set the values on some of these attributes or create your own attributes.



Navigating the Virtual Directory Structure

Genus uses the commands listed in the table to navigate the design hierarchy in memory.

Command	Details
vcd	Sets the current directory in the design hierarchy.
vls <-l> <-a>	Lists the objects in the current directory.
vpushd	Pushes the specified new target directory onto the directory stack and changes the current directory to that specified directory.
vpopd	Removes the topmost element of the directory stack, revealing a new top element and changing the directory to the new top element.
get_db	Finds an object (and query attributes) and passes it to other commands.
rename_obj	Changes the name of an object in the design hierarchy.
delete_obj	Removes an object (like a clock definition, a <i>flow_step</i> , etc.) from the database.

Examples

```
rename_obj test2 test3: Renames a design test2 as test3
delete_obj [get_db clocks *clock]: Removes clock objects
vls -l -a [get_db lib_cells]: Reports the libcells
```

Navigating the UNIX disk

```
pwd: Shows the current UNIX path
cd: Changes the current disk directory
ls: Lists the contents of the UNIX dir
```

Other navigation commands are *vpwd*, *vdirname*, *vdirs*, *vasename*, *vpushd*, *vpopd*.

Examples: vpushd

```
genus@root:> vls
./          designs/      hdl_libraries/  messages/      tech/
commands/   flows/          libraries/      obj_types/
```

```
genus@root:> vpwd
root:
```

```
genus@root:> vls
./          designs/      hdl_libraries/  messages/      tech/
commands/   flows/          libraries/      obj_types/
```

```
genus@root:> vpushd /dtmf_recvr_core/TDSP_CORE_INST_MPY_32_INST_csa_tree_SUB_TC_OP_groupi
hinst:dtmf_recvr_core/TDSP_CORE_INST_MPY_32_INST_csa_tree_SUB_TC_OP_groupi root:
```

```
genus@hinst:dtmf_recvr_core/TDSP_CORE_INST_MPY_32_INST_csa_tree_SUB_TC_OP_groupi> vpopd
root:
```

```
genus@root:> vls
./          designs/      hdl_libraries/  messages/      tech/
commands/   flows/          libraries/      obj_types/
```

```
genus@root:> vpwd
root:
```

Setting Attributes



```
set_db <object>
.<attribute_name> <value>
```

Examples

Root attribute:

```
set_db lp_insert_clock_gating true
```

Design attribute:

```
set_db design:top_mod
.lp_clock_gating_exclude true
```

<objectType>.<name>
What we call DPO notation (Dual Port Object)
It fully and clearly clarifies the object

Command `set_db` also returns a pair showing how many times it was executed and value assigned.

Example

```
genus@root:> set my_var 10
10
genus@root:> set_db -quiet [get_db messages] .max_print $my_var
4690 10
```

- In Genus, there are predefined attributes associated with objects in the database. Use the `set_db` command to assign values to the read-write attributes.
- Some of the attributes are read-only attributes and some are read-write.
- Attributes are dependent on the stage of the synthesis flow. In some cases, the object type of an attribute determines the stage in the synthesis flow at which the attribute can be set.
 - Example: After reading the design, you can set the attribute related to the design object.
- For root attributes, you can specify as:
`set_db / .<attr_name> <value>`
or abbreviate as:
`set_db <attr_name> <value>`

Genus might not issue any warnings or error messages when the attribute is set on the wrong design object.

For example, you defined the clock period on the `clk1` clock pin to 1000ps, but you needed to set it on the `clk2` clock pin.



Querying Attributes



```
get_db <object> .<attribute_name>
```

To retrieve the value of an attribute, use the **get_db** command.

This command works on a single object and with lists of objects.

Examples

```
get_db lp_insert_clock_gating
```

```
get_db "clock:map_design/functional/high_clk clock:map_design/functional/low_clk"  
.sources
```

```
pin:map_design/channel/ctrl/convert_high/clk_div_reg/Q
```

```
pin:map_design/channel/ctrl/convert_low/fopt/X
```

```
get_db $pin .propagated_clocks
```



This page does not contain notes.

DB Alignment Highlights



- The command **get_db/set_db** blends capabilities of the following:
 - find / filter / get_attribute / set_attribute (RC/Genus Legacy)
 - dbGet / get_property / user attributes / others (Innovus/Tempus)
- Chaining and integrated filter (-if) for better performance

Option	Details
-foreach	Iterates through each object allowing additional processing or early exit.
-invert	Complementary output.
-if	Conditional clauses.
-unique	Does not duplicate elements in output.
-regexp	Enables regular expressions in search pattern.
-match_hier	Wildcards do not resolve hierarchical separators '/' (as SDC get*).
-index	Specifies index values on indexable attributes (view, etc.).



This page does not contain notes.

Auto Completion in Tool Console

- You can abbreviate the commands as long as the abbreviations do not conflict with other commands. If you use a conflicting command, Genus gives an error message with a list of probable commands.

Example: The command *define_scan* returns the following:

```
@genus:root: 22> define_scan
define_scan_abstract_segment  define_scan_chain  define_scan_fixed_segment  define_scan_floating_segment  define_scan_preserved_segment
```

- When you are unsure of a command, enter the first few letters of the command and press the **Tab** key to display a list of commands that start with those letters. File completion also works from the command line.

Example

Press **Tab** to display a list of commands that starts with 'syn_'

```
genus@root:> set_db lp_in
lp_insert_clock_gating  lp_insert_clock_gating_incremental  lp_insert_discrete_clock_gating_logic
genus@root:> syn_
syn_generic  syn_map  syn_opt
genus@root:> source ../tcl/se
setup.lp.tcl  setup.opto.tcl  setup.tcl  setup.tcl.update  setup.dft.tcl
```



You can abbreviate the commands as long as the abbreviations do not conflict with other commands. If you use a conflicting command, Genus gives an error message with a list of probable commands.

Command Help



`help command_name`

- You can get help on all the commands by entering:

`help command_name`

- To view man pages from the UNIX shell, set your environment using:

`setenv MANPATH`

`$CDN_SYNTH_ROOT/share/synth/man_common`

- `$CDN_SYNTH_ROOT` points to the Cadence installation of Genus Synthesis Solution and must be included in your path.

Example

```
genus@root:> help read_sdc
Commands:
  read_sdc: reads in design constraints in SDC format

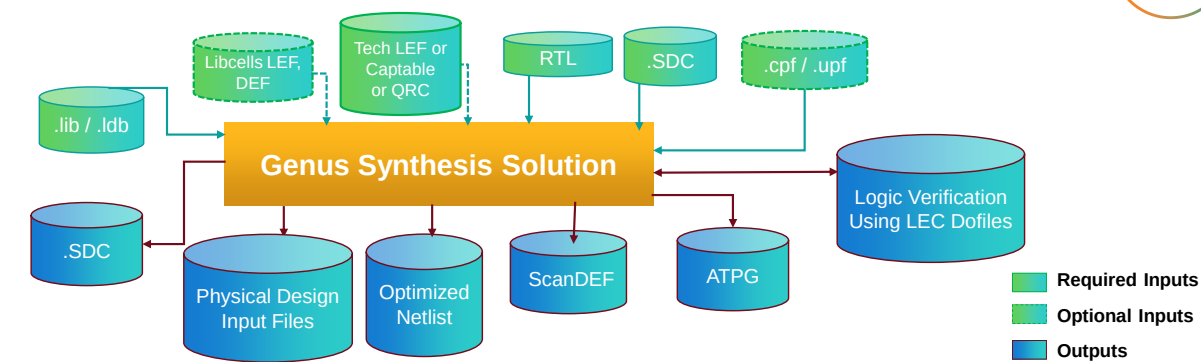
Usage: read_sdc [-stop_on_errors] [-no_compress] [-view <analysis_view>] [-echo] [-verbose] <string>+

[-stop_on_errors]:
  stops the script when errors encountered
[-no_compress]:
  turns off advanced compression
[-view <analysis_view>]:
  analysis view to read constraints
[-echo]:
  display the commands
[-verbose]:
  additional information about each command
<string>+:
  the sdc file names to be read
```



You can abbreviate the commands as long as the abbreviations do not conflict with other commands. If you use a conflicting command, Genus gives an error message with a list of probable commands.

Inputs and Outputs of Genus Synthesis Solution



Input	Output
RTL: Verilog, VHDL, directives, pragmas, SystemVerilog	Optimized netlist
Constraints: .sdc	LEC dofile
Library: .lib or .ldb	ATPG, ScanDEF, and others
Power Intent: CPF, UPF/IEEE 1801 (Optional)	Constraints: .sdc
Physical: Captable, QRC Technology file, LEF, DEF (Optional)	Physical design input files

This page does not contain notes.

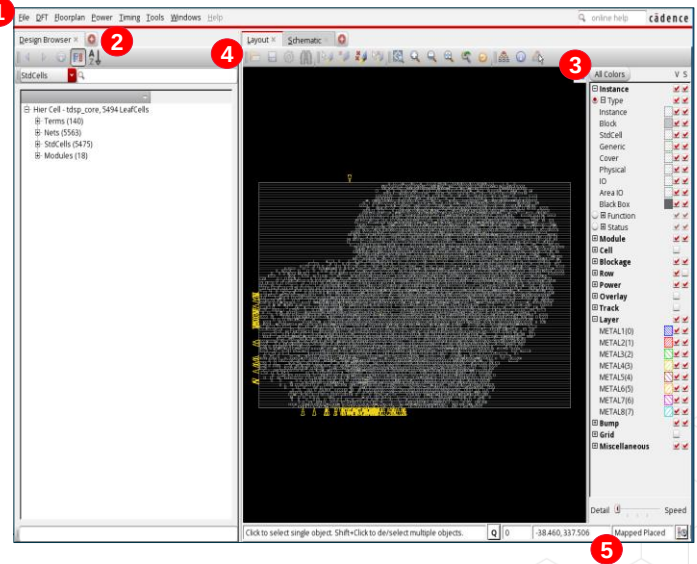
Accessing Genus Stylus Common UI GUI



gui_show

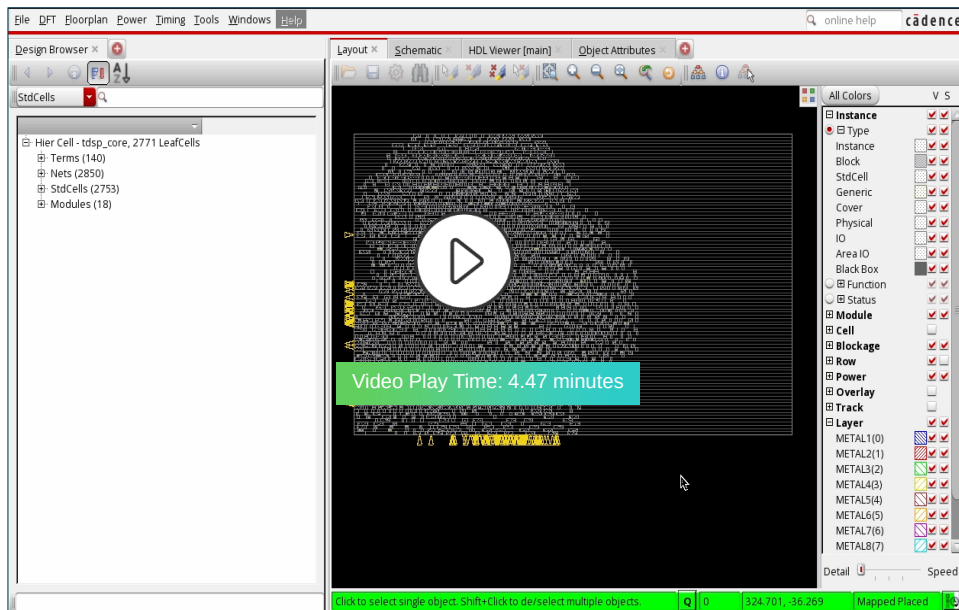
The main GUI window has the following components:

1. Menu bar
2. Viewers
 - Design Browser
 - Layout Viewer
 - Schematic Viewer
 - HDL Viewer
 - Object Attributes
3. Full Layer Control
4. Toolbar
5. Status bar



This page does not contain notes.

Demo: Exploring Stylus Common UI GUI



130 © Cadence Design Systems, Inc. All rights reserved.

cadence

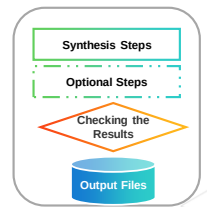
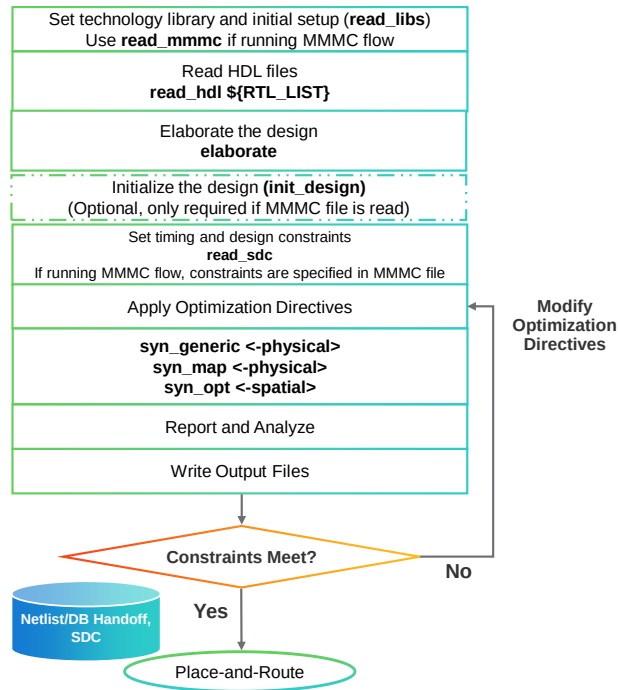
Video Play Time: 4.47 minutes

Click the play button to start the video.

Click the link [Exploring Genus Synthesis Solution Stylus Common UI Graphical User Interface](https://support.cadence.com) to play video on support.cadence.com

The video at COS runs in the channel player form where you can also find other Genus Stylus GUI related videos.

Basic Synthesis Flow



131 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Setting the Library Search Path



```
set_db init_lib_search_path <path>
```

Specifies a list of UNIX directories that Genus should search to locate the technology and LEF libraries.

Note: The “~” is supported.

Default: {./install_path/build/tools.lnx86/lib/tech}

```
genus@root:> set LIB_PATH ${LOCAL_DIR}/libraries/LEF/STDCELL
../libraries/LEF/STDCELL
genus@root:> set_db init_lib_search_path $LIB_PATH
Setting attribute of root '/': 'init_lib_search_path' = ../libraries/LEF/STDCELL
1 ../libraries/LEF/STDCELL
genus@root:> set_db init_lib_search_path ${LOCAL_DIR}/libraries/LEF/STDCELL
Setting attribute of root '/': 'init_lib_search_path' = ../libraries/LEF/STDCELL
1 ../libraries/LEF/STDCELL
genus@root:>
```



This page does not contain notes.

Reading Libraries



```
read_libs [-min_libs string] [-max_libs string] [-aocv_libs string]
[-socv string] [filenames...]
```

- The command will search for libraries with the specified name.
- If not found, it will search for the libraries in the library search path.
- If the same filename is available at different places in the search path, *only the first library encountered* will be loaded.

```
genus@root:> read_libs tutorial.lib

Message Summary for Library tutorial.lib:
*****
Could not find an attribute in the library. [LBR-436]: 52
Missing threshold parameters. [LBR-519]: 1
Missing library level attribute. [LBR-516]: 9
*****

Info      : Created nominal operating condition. [LBR-412]
           : Operating condition '_nominal_' was created for the PVT values (1.000000, 5.000000, 25.000000) in library 'tutorial.lib'.
           : The nominal operating condition represents either the nominal PVT values if specified in the library source, or the default PVT values (1.0, 1.0, 1.0).
timing_condition:default_emulate_timing_cond_min
```

133 © Cadence Design Systems, Inc. All rights reserved.

cadence

The *read_libs* command is preferred to the *set_db library* attribute for reading the library files.

The first library in the list ***is important***. The first library read in the list dictates the operating conditions for the library set.

Reading Libraries (continued)

Option	Description
<code>-aocv_libs string</code>	Specifies the list of AOCV libraries to be read for Advanced On-Chip-Variable support.
<code>-socv string</code>	Specifies the list of SOCV libraries to be read.
<code>filenames...</code>	Specifies the list of libraries. When this is used, the tool will populate the minimum and maximum with the same libraries.
<code>-max_libs string</code>	Specifies the list of libraries for maximum analysis to be read. If this option is used along with the <code>filenames</code> option, it will cause an error.
<code>-min_libs string</code>	Specifies a list of libraries for minimum analysis to be read. If this option is used along with the <code>filenames</code> option, it will cause an error.

Genus does not do hold timing analysis/optimization but may use the `min_libs` for checks of type data to data.



This page does not contain notes.

Flow When Using Multiple Corner Libraries

mmmc.tcl

Single Corner

Library setup
read_hdl / elaboration
Timing constraints
syn_generic
syn_map
syn_opt

MMMC

read_mmmc
read_hdl / elaboration
init_design
syn_generic
syn_map
syn_opt



```
create_library_set -name wcl_slow -timing { ...
}
create_library_set -name wcl_fast -timing { ...
}
create_library_set -name wcl_typical -timing { ...
}

create_opcond -name op_cond_wcl_slow -process 1 -voltage 1.08 -temperature 125
create_opcond -name op_cond_wcl_fast -process 1 -voltage 1.32 -temperature 125
create_opcond -name op_cond_wcl_typical -process 1 -voltage 1.25 -temperature 125

create_timing_condition -name timing_cond_wcl_slow -opcond slow -library_sets { wcl_slow }
create_timing_condition -name timing_cond_wcl_fast -opcond fast -library_sets { wcl_fast }
create_timing_condition -name timing_cond_wcl_typical -opcond typical -library_sets { wcl_typical }

create_rc_corner -name rc_corner -qrc_tech <path>/qrcTechFile

create_delay_corner -name delay_corner_wcl_slow -early_timing_condition timing_cond_wcl_slow \
-late_timing_condition timing_cond_wcl_slow -early_rc_corner rc_corner \
-late_rc_corner rc_corner

create_delay_corner -name delay_corner_wcl_fast -early_timing_condition timing_cond_wcl_fast \
-late_timing_condition timing_cond_wcl_fast -early_rc_corner rc_corner \
-late_rc_corner rc_corner

create_delay_corner -name delay_corner_wcl_typical -early_timing_condition timing_cond_wcl_typical \
-late_timing_condition timing_cond_wcl_typical -early_rc_corner rc_corner \
-late_rc_corner rc_corner

create_constraint_mode -name functional_wcl_slow -sdc_files { slow.sdc }

create_constraint_mode -name functional_wcl_fast -sdc_files { fast.sdc }

create_constraint_mode -name functional_wcl_typical -sdc_files { typical.sdc }

create_analysis_view -name view_wcl_slow -constraint_mode functional_wcl_slow -delay_corner
delay_corner_wcl_slow
create_analysis_view -name view_wcl_fast -constraint_mode functional_wcl_fast -delay_corner
delay_corner_wcl_fast
create_analysis_view -name view_wcl_typical -constraint_mode functional_wcl_typical -delay_corner
delay_corner_wcl_typical

set_analysis_view -setup { view_wcl_slow view_wcl_fast view_wcl_typical }
```

This page does not contain notes.

Reading the MMMC File



```
read_mmmc [-design string] filename
```

- Reads the MMMC view definition file and populates the MMMC-related attributes.
- MMMC refers to the full set of timing libraries, constraints, operating conditions, timing conditions, RC corners, Delay corners, and the associated analysis views.
- The *filename* can be specified with absolute paths in the MMMC or relative to the local directory or to the library search path attributes.
- When the *read_mmmc* command is run, the MMMC file is read and processed.

Example

```
@genus:root: 36> read_mmmc ../SCRIPTS/mmmc.tcl
Sourcing '../SCRIPTS/mmmc.tcl'
#@ Begin verbose source SCRIPTS/mmmc.tcl
@file(mmmc.tcl) 1: create_library_set -name wcl_slow -timing {
  ../LIB/gsclib045_v3.5/timing/slow.lib
  ../LIB/macro_libs/pllclk_slow.lib
  ../LIB/macro_libs/CDK_S128x16.lib
  ../LIB/macro_libs/CDK_S256x16.lib
  ../LIB/macro_libs/CDK_R512x16.lib
}
@file(mmmc.tcl) 8: create_library_set -name wcl_fast -timing {
  ../LIB/gsclib045_v3.5/timing/fast.lib
  ../LIB/macro_libs/pllclk_slow.lib
  ../LIB/macro_libs/CDK_S128x16.lib
  ../LIB/macro_libs/CDK_S256x16.lib
  ../LIB/macro_libs/CDK_R512x16.lib
}
@file(mmmc.tcl) 15: create_library_set -name wcl_typical -timing {
  ../LIB/gsclib045_v3.5/timing/typical.lib
  ../LIB/macro_libs/pllclk_slow.lib
  ../LIB/macro_libs/CDK_S128x16.lib
  ../LIB/macro_libs/CDK_S256x16.lib
  ../LIB/macro_libs/CDK_R512x16.lib
}
```

This page does not contain notes.

Reading Designs



`read_hdl/read_netlist`

- Use the `read_hdl` or `read_netlist` command to parse the HDL source.
- To set the HDL search path, enter the following:

```
set_db init_hdl_search_path "path" /
```

Example

- To read the RTL or mixed source (RTL/gate) design (parses only), use:
- To read-in the mapped netlist use:

```
read_hdl {design1.v subdes1.v subdes2.v}
```

```
read_netlist mapped_netlist.v
```

137 © Cadence Design Systems, Inc. All rights reserved.



`read_hdl` loads one or more HDL files in the order given into memory. Files containing macro definitions should be loaded before the macros are used. Otherwise, there are no ordering of constraints.

If you do not specify either the `-v1995`, `-v2001`, `-sv` or the `-vhdl` option, the default language format is then specified by the `hdl_language` attribute. The default value for the `hdl_language` attribute is `-v2001`.

The HDL files can contain structural code for combining lower-level modules, behavioral design specifications, or RTL implementations.

You can automatically read in or write out a compressed HDL file in *gzip* format.

When you load a parameterized Verilog module or VHDL architecture, each parameter in the module or architecture will be identified as an *hdl_parameter* object and located under `../architecture_name/parameters`. The default *hdl_parameter* attribute value for these parameters will be true.

Use the `genus -abort_on_error -f <your script>` command to specify that Genus automatically quits if a script error is detected when reading in HDL files instead of holding at the `genus@root:>` prompt.

Reading Designs (continued)

```
read_hdl file_list
    [-language {v2001 | v1995 | sv | vhdl }]
    [-netlist] [-f filename]
    [-define macro=value]... file_list.....
```

Option	Description
<i>-language</i>	Specifies the language of the HDL files. It can be any of these options: SV, v1995, v2001 (default), VHDL. The <i>hdl_vhdl_read_version</i> root attribute value specifies the standard to which the VHDL files conform (default: VHDL-1993).
<i>-define</i>	Defines Verilog macros.
<i>-netlist</i>	Reads a netlist.
<i>-f filename</i>	Specifies the name of the list file for reading files from the simulation environment. Note: This option is not supported for VHDL designs.
<i>file_list</i>	Specifies the name of the HDL files to load. If several files must be loaded, specify them in a string. Note: The files can be encrypted.

138 © Cadence Design Systems, Inc. All rights reserved.



read_hdl loads one or more HDL files in the order given into memory. Files containing macro definitions should be loaded before the macros are used. Otherwise, there is no ordering of constraints.

If you do not specify either the v1995, v2001, sv or the vhdl option, the default language format is then specified by the *hdl_language* attribute. The default value for the *hdl_language* attribute is -v2001.

The HDL files can contain structural code for combining lower-level modules, behavioral design specifications, or RTL implementations.

You can automatically read in or write out a compressed HDL file in *gzip* format.

When you load a parameterized Verilog module or VHDL architecture, each parameter in the module or architecture will be identified as an *hdl_parameter* object and located under *../architecture_name/parameters*. The default *hdl_parameter* attribute value for these parameters will be true.

Use the *genus -abort_on_error -f <your script>* command to specify that Genus automatically quits if a script error is detected when reading in HDL files instead of holding at the *genus@root:>* prompt.

Elaboration of Designs



```
elaborate [-h] [-parameters {} ] [<top_module_name>]
```

The elaborate command:

- Builds data structures and infers registers in the design.
- Performs high-level HDL optimization, such as dead code removal.
- Identifies clock-gating candidates.

If there are several top levels, only the top-level designs to elaborate are required on the elaborate command; Genus will automatically elaborate the submodules.

Example

```
elaborate -parameters {12 8 16} TOP
```

When compiling, these parameters are modified as follows:

- data_width1 = 12
- averg_period = 8
- data_width2 = 16

Best to write as shown below:

```
elaborate -parameters {{data_width1 12} {averg_period 8} {data_width2 16}}
```

```
module TOP ( data_in , data_out , averg_per_sel ) ;
    parameter data_width1 = 3;
    parameter averg_period = 2;
    parameter data_width2 = 4;

    input [data_width1-1:0] data_in ;
    ...
```

139 © Cadence Design Systems, Inc. All rights reserved.

cadence

-h – Help with the *elaborate* command

[-parameters <string>] – Specifies a list of parameter-value pairs for top-level Verilog module or VHDL entity

[-lib_path <string>] – Defines the Verilog library search path

[-lib_extension <string>] – Defines the Verilog library file extension

[<string>+] – The top-level Verilog modules or VHDL architectures to elaborate

Before elaborating a design:

- Read in all the designs.
- Set the general Genus environment.

After the elaboration finishes, the software reports:

- Unresolved references (that is, instances found with no corresponding module or library cell).
- Semantic problems, including:
 - Read before write.
 - Unused ports, inconsistent resets.

Initializing the Database



```
init_design [-top design]
```

Initializes the database and ensures that the tool is ready for full execution.

Example

```
@genus:root: 46> init_design
Info      : Missing library level attribute. [LBR-516]
Info      : slew derate_from library not specified in the library, using .lib default of 1. (File /MMMC_Flow/work_mmmc/./LIB/gsclib045
v3.5/timing/fast.lib, Line 8)
Info      : Missing a function attribute in the output pin definition. [LBR-518]
Info      : Functionality is missing at pin 'Y' for the cell 'HOLDX1'. (File /MMMC_Flow/work_mmmc/./LIB/gsclib045_v3.5/timing/fast.lib
, Line 32486)

Message Summary for Library all 5 libraries:
*****
Missing a function attribute in the output pin definition. [LBR-518]: 1
Missing library level attribute. [LBR-516]: 1
*****

Reading file '/MMMC_Flow/work_mmmc/./LIB/gsclib045_v3.5/timing/fast.lib'
Reading file '/MMMC_Flow/work_mmmc/./LIB/macro_libs/pllclk_slow.lib'
Warning : Libraries have inconsistent nominal operating conditions. In the Liberty library, there are attributes called nom_voltage,
nom_process and nom_temperature. Genus reports the message, if the respective values of the 2 given .libs differ. [LBR-38]
: The libraries are 'fast' and 'PLL worst'.
Reading file '/MMMC_Flow/work_mmmc/./LIB/macro_libs/CDK_S128x16.lib'
Warning : Libraries have inconsistent nominal operating conditions. In the Liberty library, there are attributes called nom_voltage,
nom_process and nom_temperature. Genus reports the message, if the respective values of the 2 given .libs differ. [LBR-38]
: The libraries are 'fast' and 'CDK_S128x16'.
```

This page does not contain notes.

Specifying Design Constraints for Simple Flow



```
read_sdc [-stop_on_errors] [-view <analysis_view>] \
[-no_compress] [-verbose] <sdcFileName>
```

You can specify the design constraints in either of these two ways:

1. SDC File (Preferred)

- You can read SDC directly into Genus after elaborating the top-level design.
- Always check for errors and failed commands when reading SDC constraints.
 - Look for failed commands using:


```
puts "$::dc::sdc_failed_commands > failed.sdc"
```
- Review the log entries and the summary table at the end.

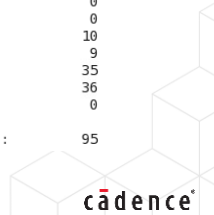
2. Genus Tcl Constraints

Command `read_sdc` is useful only for simple usage but not *MMMC*.

Always run `check_timing_intent -verbose` after reading the constraints to check for constraints consistency.

```
Lint summary
Unconnected/logic driven clocks                0
Sequential data pins driven by a clock signal    3
Sequential clock pins without clock waveform     2
Sequential clock pins with multiple clock waveforms 0
Generated clocks without clock waveform          0
Generated clocks with incompatible options        0
Generated clocks with multi-master clock         0
Paths constrained with different clocks           0
Loop-breaking cells for combinational feedback   0
Nets with multiple drivers                       0
Timing exceptions with no effect                  0
Suspicious multi_cycle exceptions                0
Pins/ports with conflicting case constants        0
Inputs without clocked external delays           10
Outputs without clocked external delays          9
Inputs without external driver/transition        35
Outputs without external load                    36
Exceptions with invalid timing start-/endpoints   0
```

Total: 95



When using an SDC file, the capacitance specified in picofarads (pF) (SDC unit) is converted to femtofarads (fF) (Genus unit) and time specified in *ns* is converted to *ps*.

You can use the SDC commands interactively. However, when mixing third-party constraints and Genus commands, be very careful with the units of capacitance and delay.

Specifying Design Constraints Through the MMMC File



`create_constraint_mode`

- In the MMMC flow, specify timing constraints in the MMMC file using the `create_constraint_mode` command for a specific mode of the design.
- If the constraints are in the MMMC file, they will be read automatically by the tool during the `init_design` command.

Example

```
create_constraint_mode -name functional_wcl_slow -sdc_files { \
../Constraints/mmmc/dtmf_recvr_core_gate_slow.sdc}

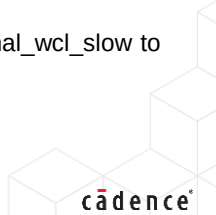
create_constraint_mode -name functional_wcl_fast -sdc_files { \
../Constraints/mmmc/dtmf_recvr_core_gate_fast.sdc}

create_constraint_mode -name functional_wcl_typical -sdc_files { \
../Constraints/mmmc/dtmf_recvr_core_gate_typical.sdc}
```

- To update or add timing constraints after reading the MMMC file, use `update_constraint_mode`.

Example: The following command changes the SDC files associated with the constraint mode `functional_wcl_slow` to `io.sdc`, `test-clks.sdc`, and `test-except.sdc`:

```
update_constraint_mode -name functional_wcl_slow \
-sdc_files {io.sdc test-clks.sdc test-except.sdc}
```



This page does not contain notes.

Controlling Optimization



Function	Attribute/Command	Description
Preserving Instances and Subdesigns	<code>set_db <object> .preserve {false true const_prop_delete_ok const_prop_size_delete_ok delete_ok map_size_ok size_ok size_delete_ok}</code>	Controls the optimization of the design. You can set the options available as per requirement.
Grouping and Ungrouping of Hierarchy	<code>group</code> <code>ungroup</code>	Creates hierarchy to partition your design with the group command. Manually dissolves an instance of the hierarchy with the ungroup command.
Optimizing Sequential Logic	<code>set_db delete_unloaded_seqs {true/false}</code> <code>set_db optimize_constant_0_flops {true/false}</code> <code>set_db optimize_constant_1_flops {true/false}</code>	By default, Genus removes flip-flops and logic that is not transitively driving an output port. (Dangling logic can be caused by boundary optimization.) To disable this feature globally, use these attributes to false (not recommended).
Merging Sequential Logic	<code>set_db optimize_merge_flops true/false</code> <code>set_db optimize_merge_latches true/false</code>	Sequential merging combines equivalent sequential cells, both flops and latches, in the same hierarchy. Default value is true.

This page does not contain notes.

How to Synthesize the Design



Synthesis is the process of transforming your RTL design (high-level description) into a gate-level netlist, given all the specified constraints and optimization settings.

Generic Optimization
**syn_generic <-
physical>**

Generic – Generic gate optimization.

```
wire n_66, n_67, n_68, n_69, n_70;
com_flop count_reg[0] (.clk (clk), .d (n_11), .sena (1'b1), .aclr
(n_53), .apre (1'b0), .srl (1'b0), .srd (1'b0), .q (count[0]));
```

A generic gate is used for the instance Count_reg[0].

Technology
Transformation
(Mapping)
**syn_map <-
physical>**

Maps the specified design(s) to the cells described in the *supplied* technology library and performs logic optimization.

```
count_reg[0] count_reg[0] (.rst (rst), .ck (clk), .d (n_0), .Q
(count[0]));
```

Maps the generic gate with std cell DFFRHQX1 for the instance Count_reg[0].

Incremental
Optimization
**syn_opt <-
spatial>**

Synthesizes the design to optimized gates.

```
NOR2BX1 g203 8246 (.AN (count[1]), .B (n_0), .Y (n_2));
DFFRX1 count_reg[0] (.RN (rst), .CK (clk), .D (n_0), .Q (count[0]),
.QN (n_0));
```

Optimize the cell DFFRHQX1 to DFFRX1 for the instance Count_reg[0].

Option **-physical** and **-spatial**

- Requires a Genus-Physical License option.
- Takes physical domain into consideration.
- Runs placement and performs physical optimization.

This page does not contain notes.

Setting Effort Levels Prior to Synthesis



You can change the default effort levels for synthesis by using attributes.

Effort Levels	Attributes
Generic Synthesis (syn_generic command) effort level	<code>set_db syn_generic_effort <low <u>medium</u> high none></code>
The optimization effort for the mapping stage (syn_map command)	<code>set_db syn_map_effort <low medium <u>high</u> none></code>
The optimization effort to use for incremental optimization (syn_opt command)	<code>set_db syn_opt_effort <none low medium <u>high</u> extreme></code>
To specify the global effort for all synthesis commands, use	<code>set_db syn_global_effort <<u>none</u> low medium high ></code>

- Default effort is:
 - *Medium* for generic.
 - *High* for mapping and optimization.
 - *None* for global.



This page does not contain notes.

Reporting



Command	Description
<code>report_area</code>	Prints an exhaustive hierarchical area report.
<code>report_dp</code>	Prints a datapath resources report (to be done before syn_map).
<code>report_design_rules</code>	Prints design rule violations.
<code>report_gates</code>	Reports libcells used, total area, and instance count summary.
<code>report_hierarchy</code>	Prints a hierarchy report.
<code>report_instance</code>	Generates a report on the specified instance.
<code>report_memory</code>	Prints a memory usage report.
<code>report_messages</code>	Prints a summary of the error messages that have been issued.
<code>report_power</code>	Prints a power report.
<code>report_qor</code>	Prints a quality-of-results report.
<code>report_timing</code>	Prints a timing report.
<code>report_summary</code>	Prints an area, timing, and design rules report.



This page does not contain notes.

Generating Timing Reports



```
report_timing [-fields column_list] [-path_type {full|summary|full_clock|endpoint}]
[-physical] [-nworst integer] [-from {inst|hinst|external_delay|clock|port|pin}...]
[-through {inst|hinst|port|pin}...] [-to
{inst|hinst|external_delay|clock|port|pin}...] [-exceptions exception...] [-
timing_bin bin] [-timing_path path] [-gui] [> file]
```

Generates a timing report of the current design.

- By default, the report gives a detailed view of the critical path of the current design.
- If the current session has multiple designs, use the **current_design <design:designName>** command to navigate to the desired design to generate the report.
- You can also generate a report on possible timing constraint problems (timing lint) or view slack at endpoints.

Examples

```
puts "WNS path/timing among all in2out
paths"
```

```
report_timing -from [all_inputs] -to
[all_outputs]
```

```
puts "WNS path/timing among all reg2reg
paths"
```

```
report_timing -from [all_registers] -to
[all_registers]
```

```
puts "WNS path/timing among paths from
$clk to $clk2 domains"
```

```
report_timing -from $clk -to $clk2
```

This page does not contain notes.

Reading Timing Reports

Generated by: Genus(TM) Synthesis Solution
Generated on:
Module: dtmf_recvr_core
Operating conditions: ss_lv08_125c
Interconnect mode: global
Area mode: physical library

The header shows operating condition and module information

Path 1: MET (0 ps) Setup Check with Pin TDSP_CORE_INST_EXECUTE_INST_acc_reg[30]/CK->D
Group: m_clk
Startpoint: (R) TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/CK
Clock: (R) m_clk
Endpoint: (F) TDSP_CORE_INST_EXECUTE_INST_acc_reg[30]/D
Clock: (R) m_clk

The body includes timing slack calculation

Capture Launch
Clock Edge: 8000 0
Src Latency: 0 0
Net Latency: 0 (I) 0 (I)
Arrival: 8000 0

Flags like
(C) : Congested area
(I) : Ideal net
(P) : Preserved, and so on

Setup: 592
Required Time: 7408
Launch Clock: 0
Data Path: 7407
Slack: 0

The table shows arrival time calculation

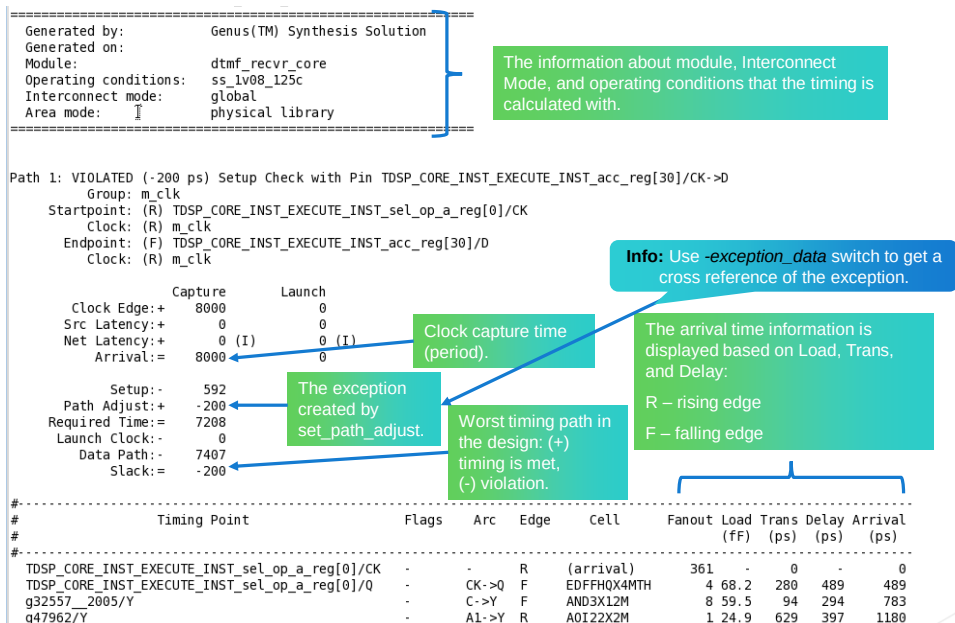
#-----
Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival
(fF) (ps) (ps) (ps)
#-----
TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/CK - - R (arrival) 361 - 0 - - 0
TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/Q - CK->Q F EDFFHQX4MTH 4 68.2 280 489 489
g32557_2005/Y - C->Y F AND3X12M 8 59.5 94 294 783
g47962/Y - A1->Y R A0I22X2M 1 24.9 629 397 1180
RAM_256x16_TEST_INST_RAM_256x16_INST/ADDRESS[5] <<< (P) - F CDK_S256x16 1 - - 6 15340

148 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Timing Report with the Path Adjust Constraint



After the constraints are normalized by removing the path adjusts, the timing report shows the actual slack.

In this case, the slack is positive/zero slack after normalizing the constraints.

Generating Outputs: Writing Database and Netlist



write_db

Use **write_db** to write the design to a database file.

write_db [-design design] db_file

```
@genus:root: 29> write_db -design dtmf_recvr_core design.db
Finished exporting design database to file 'design.db' for 'dtmf_recvr_core' (command execution time mm:ss cpu = 00:00, real = 00:00).
```



write_netlist

Use the **write_netlist** command to generate a gate-level netlist.

write_netlist > filename

```
@genus:root: 19> write_netlist > test.v
@genus:root: 20> write_netlist

// Generated by Cadence Genus(TM) Synthesis Solution <version>
// Generated on: <Date and Time>

// Verification Directory fv/dtmf_recvr_core

module bmux(ctl, in_0, in_1, z);
  input ctl;
  input [2:0] in_0, in_1;
  output [2:0] z;
  wire ctl;
  wire [2:0] in_0, in_1;
  wire [2:0] z;
```

150 © Cadence Design Systems, Inc. All rights reserved.



Use the **>** symbol to redirect the output to a file or **>>** to append it to the file.

The software uses a database for its operations. Therefore, making modifications to the database does not alter the design files on the hard disk.

You need to manually save the modifications to the hard disk by using the *write_netlist* command.

Use a naming convention when writing files so that you do not overwrite any existing files.

**write_db [db_file] [-all_root_attributes] [-no_root_attributes] [-script file]
[-design design] [-quiet] [-verbose]**

Writes the design and all its environment (timing, physical, flow, attributes ...) to a database file.

Root attributes with non-default settings can be included in the database or written in a script. By default, only root attributes affecting the QoR are written out.

Generating Outputs: Writing SDC



write_sdc

Use the **write_sdc** command to generate the SDC constraints file.

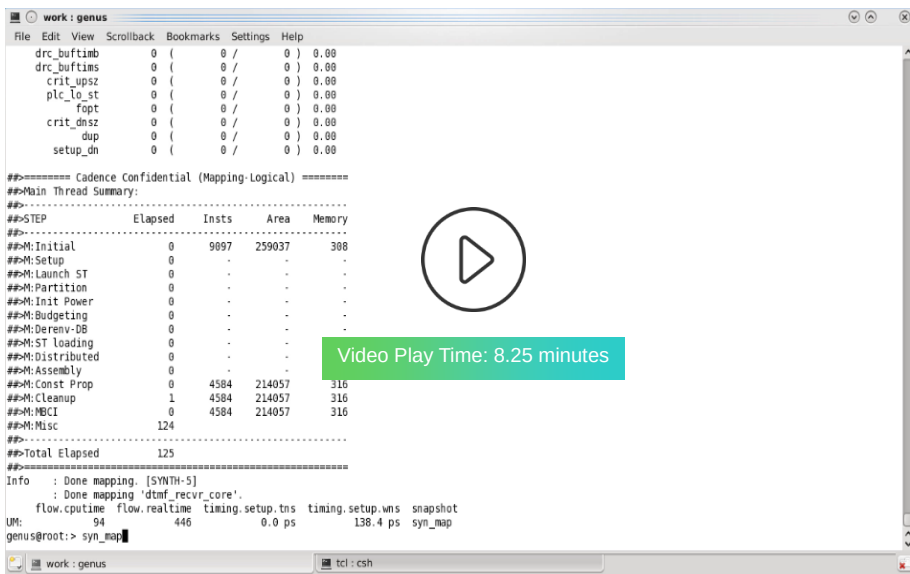
```
write_sdc [design] > [filename]
```

```
genus@root:> write_sdc dtmf_recvr_core > test.sdc  
Finished SDC export (command execution time mm:ss (real) = 00:01).
```



Use the > symbol to redirect the output to a file or >> to append it to the file.

Demo: Basic Synthesis Flow in Genus Synthesis Solution



The screenshot displays the Genus Synthesis Solution interface. At the top, a menu bar includes File, Edit, View, Scrollback, Bookmarks, Settings, and Help. Below the menu, a table lists various synthesis metrics with columns for name, value, and units. The table includes rows for drc_burftmb, drc_burftms, crit_upsz, plc_lo_st, fopt, crit_dnsz, dup, and setup_dn, all showing values of 0 or 0.00. Below this table, a section titled "Cadence Confidential (Mapping-Logical)" contains a "Main Thread Summary" table. This table has columns for STEP, Elapsed, Insts, Area, and Memory. It lists various synthesis steps such as Initial, Setup, Launch ST, Partition, Init Power, Budgeting, Derenv-DB, ST loading, Distributed, Assembly, Const Prop, Cleanup, MBICI, and Misc, along with their respective elapsed times, instance counts, areas, and memory usage. A large play button icon is overlaid on the right side of the terminal window. A green box with the text "Video Play Time: 8.25 minutes" is positioned over the play button. The bottom of the terminal window shows the command prompt "genus@root:> syn_map" and a status bar with "work : genus" and "tcl : csh".

STEP	Elapsed	Insts	Area	Memory
Initial	0	9097	259037	308
Setup	0	-	-	-
Launch ST	0	-	-	-
Partition	0	-	-	-
Init Power	0	-	-	-
Budgeting	0	-	-	-
Derenv-DB	0	-	-	-
ST loading	0	-	-	-
Distributed	0	-	-	-
Assembly	0	-	-	-
Const Prop	0	4584	214057	316
Cleanup	1	4584	214057	316
MBICI	0	4584	214057	316
Misc	124	-	-	-
Total Elapsed	125	-	-	-



Video Play Time: 8.25 Minutes

Click the play button to start the video.

Click the link [Basic Synthesis Flow in Genus Stylus CUI](#) to play video on support.cadence.com.

What Is DFT?



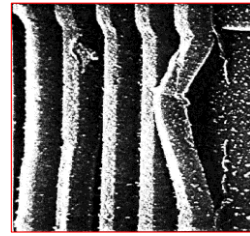
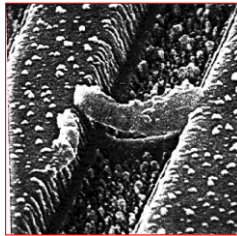
Design for Test (DFT) techniques provide measures to comprehensively test the manufactured device for quality and coverage.

Design for Testability (DFT) makes it possible to:

- Assure the detection of all faults in a circuit.
- Reduce the cost and time associated with test development.
- Reduce the execution time of performing a test on fabricated chips.

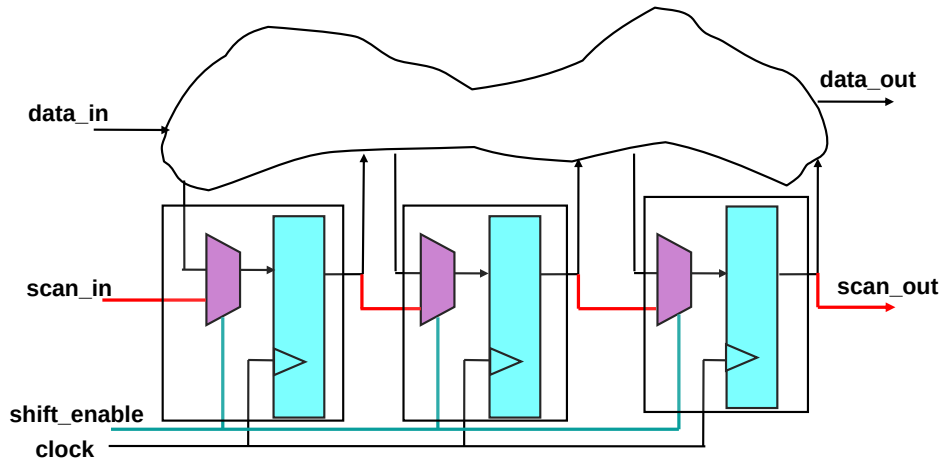
Different faults in DFT:

- Stuck-At faults
- Transition faults
- Stuck open and Shorts
- Bridging faults



This page does not contain notes.

Design with Test Circuit



154 © Cadence Design Systems, Inc. All rights reserved.



Any inferred register, or any instantiated edge-triggered registers that pass the DFT rule checks, are mapped to their scan equivalent cell when the scan connection engine runs.

Remapping of instantiated edge-triggered registers that pass the DFT rule checks to their scan equivalent cells occurs prior to placement only.

The test process refers to testing the ASIC for manufacturing defects on the automatic test equipment (ATE). It is the process of analyzing the logic on the chip to detect logic faults. The test process puts the circuit into one of the three test modes:

- *Capture mode*

This is the part of the test process that analyzes the combinational logic on the chip. The registers act first as pseudo-primary inputs (using ATPG-generated test data) and then as pseudo-primary outputs (capturing the output of the combinational logic).

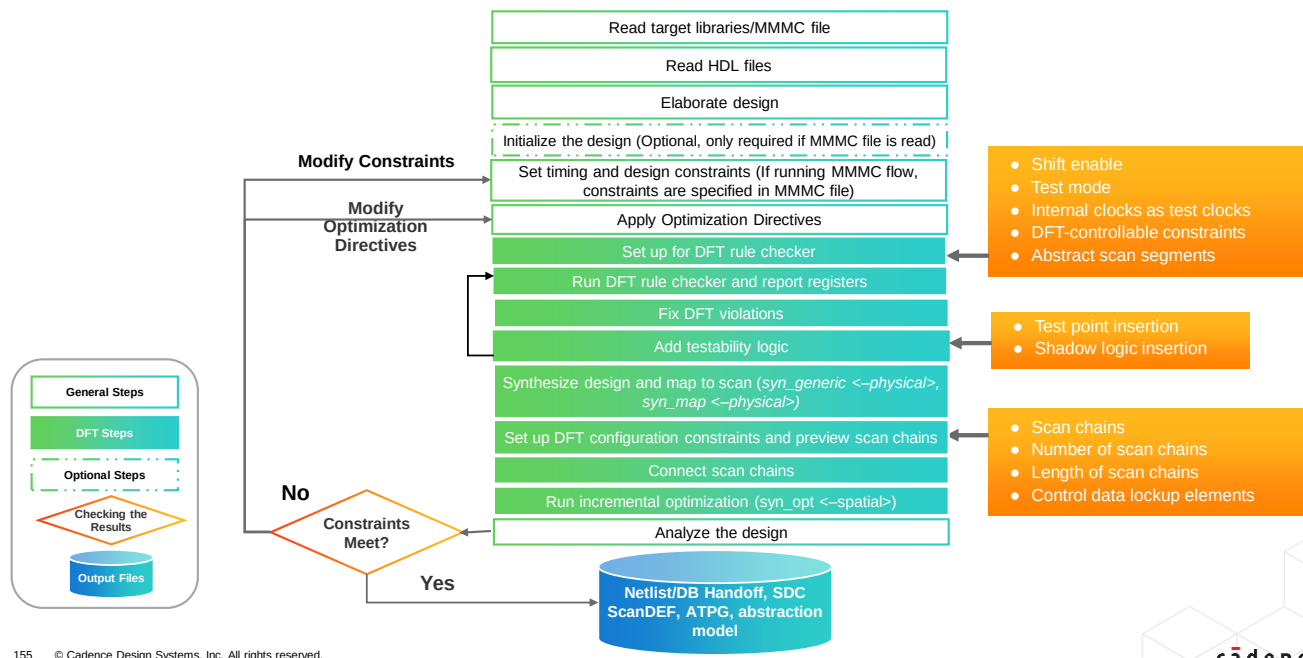
- *Scan-shift mode*

This is the part of the test process in which registers act as shift registers in a scan chain. Test vector data is shifted into the scan chain registers, and the captured data from capture mode are shifted out of the scan registers.

- *System mode*

This is the normal or intended operation of the circuit. Any logic dedicated for DFT purposes is *not* active in the system mode.

RTL Top-Down DFT Flow



The Cadence Genus Synthesis Solution provides a full-scan DFT solution including the following:

- DFT rule checking
- DFT rule violation fixing
- Scan mapping during optimization
- Scan chain configuration and connection

The main DFT techniques available today are given below:

- Scan insertion
- Programmable Memory BIST insertion
- Logic BIST insertion
- Boundary scan insertion
- Scan compression
- At-Speed Test using On-Product Clock Generation Logic (OPCG)

Scan insertion is one of the most used DFT techniques to detect stuck-at faults.

Scan insertion replaces the flip-flops in the design with special flops that contain built-in logic targeted for testability. Scan logic lets you control and observe the *sequential state* of the design through the test pins during the test mode. This helps in generating a high-quality and compact test pattern set for the design using an Automatic Test Pattern Generator (ATPG) tool.

Quiz: Optimization and Synthesis



Which of the following commands is used to set the library search path?

- A. `set_db init_lib_path`
- B. `set_db lib_search_path`
- C. `set_db init_lib_search_path`

C



Complete the synthesis breakdown:

`syn_generic`

`_____`
`syn_opt`



To specify a timing constraints in MMMC file, use command _____

`create_constraint_mode`

Answers

1. C
2. `syn_map`
3. `create_constraint_mode`

Quiz: Reading Timing Reports



In the given timing report:

- Where is the **operating condition** information?
- Where is the **timing slack** calculation?
- What is the **last column** in the table body?



```

Generated by: Genus(TM) Synthesis Solution
Generated on:
Module: dtmf_recvr_core
Operating conditions: ss_lv08_125c
Interconnect mode: global
Area mode: physical library

Path 1: MET (0 ps) Setup Check with Pin TDSP_CORE_INST_EXECUTE_INST_acc_reg[30]/CK->D
Group: m_clk
Startpoint: (R) TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/CK
Clock: (R) m_clk
Endpoint: (F) TDSP_CORE_INST_EXECUTE_INST_acc_reg[30]/D
Clock: (R) m_clk

Capture Launch
Clock Edge: + 8000 0
Src Latency: + 0 0
Net Latency: + 0 (I) 0 (I)
Arrival: 8000 0

Setup: - 592
Required Time: 7408
Launch Clock: - 0
Data Path: - 7407
Slack: 0

#-----#
# Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival
# (FF) (ps) (ps) (ps)
#-----#
TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/CK - R (arrival) 361 - 0 - 0
TDSP_CORE_INST_EXECUTE_INST_sel_op_a_reg[0]/Q - CK->Q F EDFFHQ4MTH 4 68.2 280 489 489
g32557_2005/Y - C->Y F AND3X12M 8 59.5 94 294 783
g47962/Y - A1->Y R A0122X2M 1 24.9 629 397 1180
    
```

The header shows operating condition and module information.

The body includes timing slack calculation.

It shows arrival time calculation.

157 © Cadence Design Systems, Inc. All rights reserved.

cadence

Answers

The header shows operating condition and module information.

The body includes timing slack calculation.

The last part of the table shows arrival time calculation.

Module Summary

In this module, you learned to

- Run the basic synthesis flow
- Write the synthesis outputs
- Set up Design For Test (DFT) during synthesis



This page does not contain notes.



References

Genus Training Bytes

- [Genus Training Byte References](#)

Genus One-Stop Page

- [Genus One-Stop Page Reference](#)

Genus Full Course Reference

- [Genus Synthesis Solution with Stylus Common UI](#)

Genus User Guide

- [Genus User Guide 25.1](#)
- [Genus Product Manual](#)

Genus Articles Reference

- [Genus Articles and Appnotes](#)



This page does not contain notes.

Labs

Lab 5-1 Running the Basic Synthesis Flow

Lab 5-2 Running the Synthesis Flow with DFT



This page does not contain notes.



Module 6

The Test Stage

cādence[®]

This page does not contain notes.

Module Objective

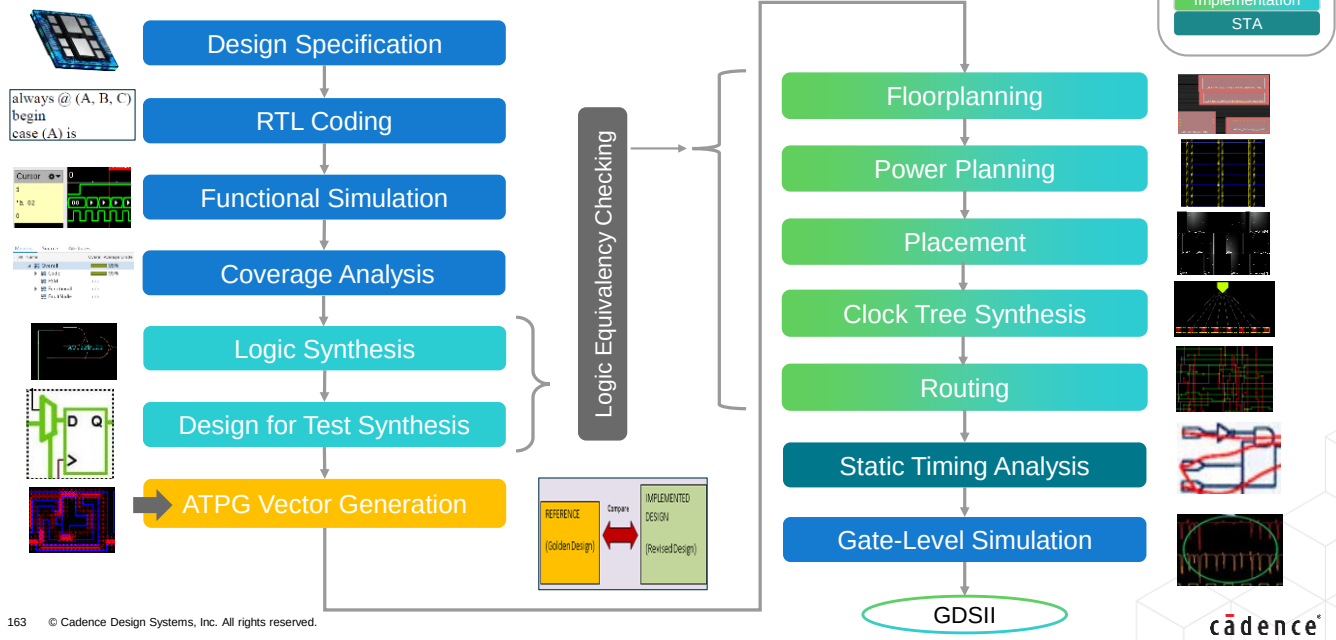
In this module, you

- Run the basic ATPG flow in Modus Test



This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow



This page does not contain notes.

Modus DFT Software Solution



Modus software is a fully integrated suite of tools designed from the beginning to interact and work with each other.

It is a complete tool suite supporting all four disciplines:

- **DFT Synthesis**
 - The process of inserting test logic into a design to help make it testable.
- **Test Verification**
 - The process of identifying the test logic and verifying that it has been inserted and integrated correctly. And identify any design characteristics that will create testing problems.
- **Pattern Generation**
 - The process of creating test vectors that are applied by the tester.
- **Failure Diagnostics**
 - The process of working backward from failure miscompares to the probable cause and helps in the production and yield management of semiconductor lines.

164 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

Modus DFT Software Solution (continued)

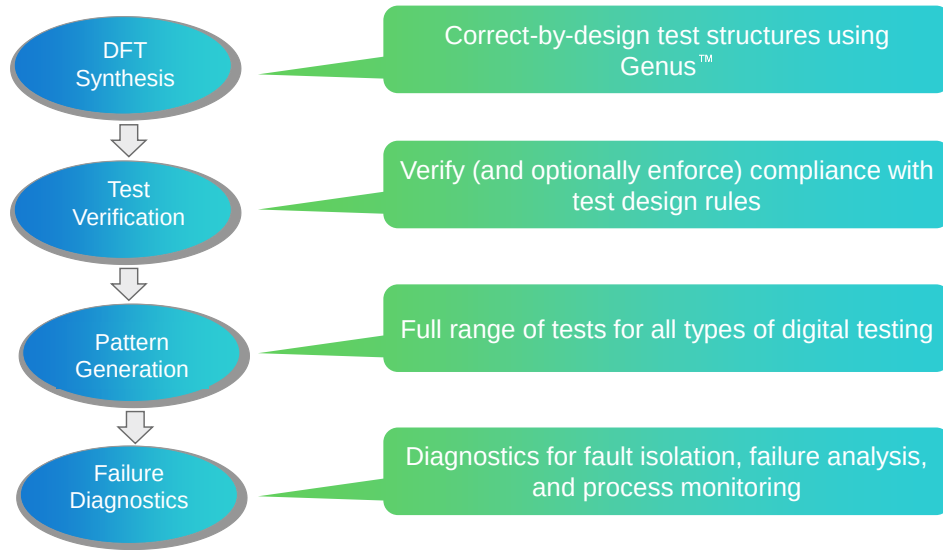
Modus DFT Software Solution provides:

- Comprehensive manufacturing test offering for logic and embedded memory IP.
- Single pass logic synthesis, test insertion, and pattern generation, that is, physically and power domain aware.
- Reduced cost of defect detection with higher quality test patterns and accurate silicon defect diagnostics.
- Applicable to all design sizes from small mixed-signal to very large SoCs.
- Test coverage analysis and augmentation.
- Common User Tcl Interface.



This page does not contain notes.

Modus DFT Software Disciplines



166 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

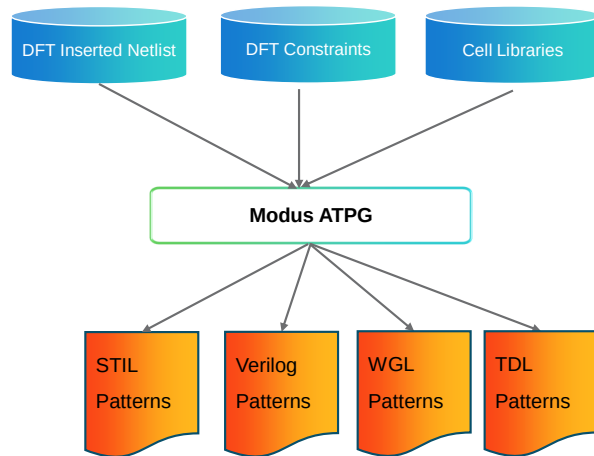
Modus ATPG: Inputs and Outputs

Inputs:

- DFT inserted netlist
- Test constraints
- Cell libraries

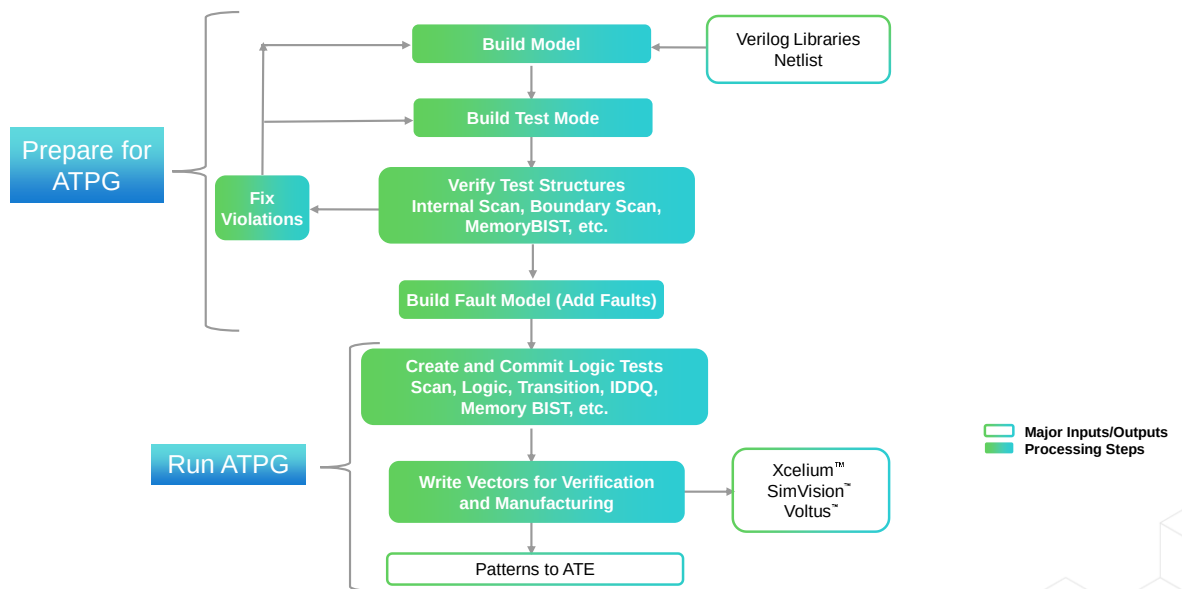
Outputs:

- STIL patterns
- Verilog patterns
- WGL patterns
- TDL patterns

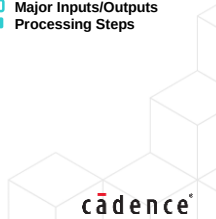


This page does not contain notes.

Modus Test ATPG Flow



168 © Cadence Design Systems, Inc. All rights reserved.



This is the basic recommended Modus Test flow.

Starting a Modus Test



- To bring up the Modus Test, enter:
`modus`
- To invoke Modus Test in Graphical User Interface, enter:
`modus -gui`
 - You can also invoke GUI by using `gui_open` command from the Modus Tcl Console.
- To execute a specific Modus binary shell command, enter:
`modus -e <Modus command or script>`
 - Run a command without starting a shell.
 - Run a command script like the *runet.atpg* scripts from Genus Synthesis Solution output.

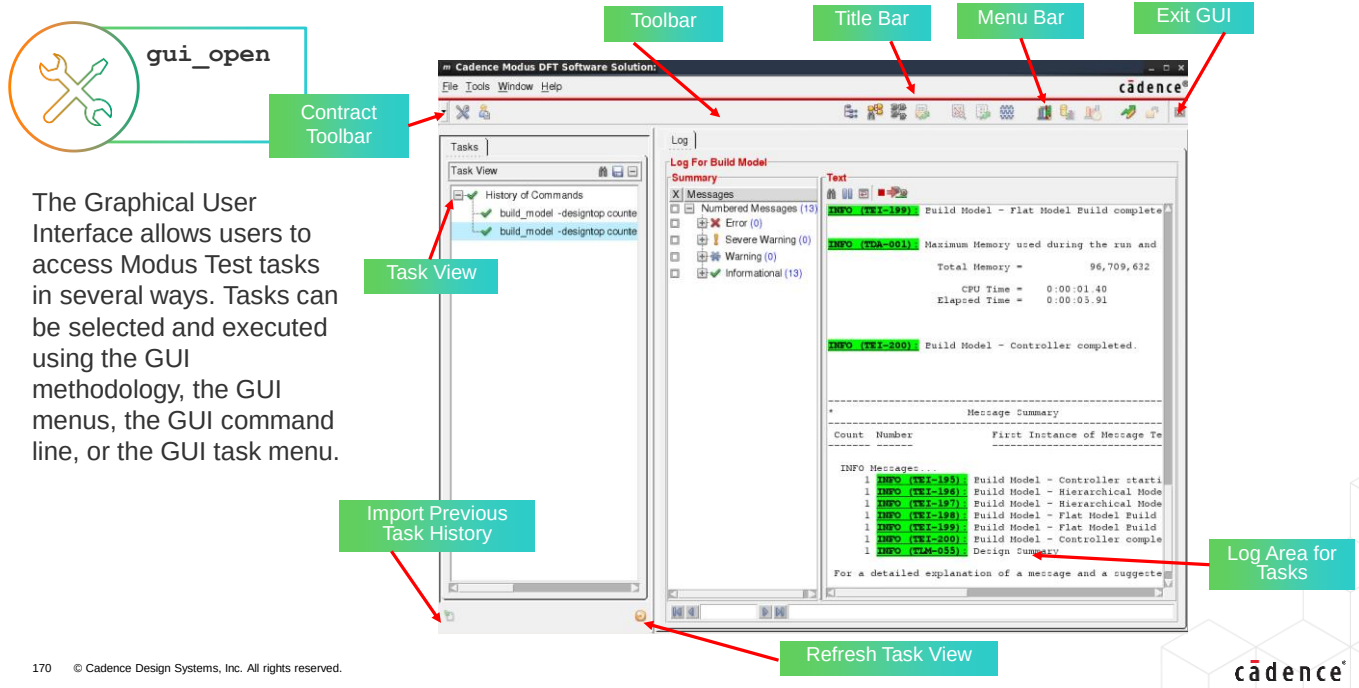


Cadence Modus Test inherently supports two interfaces to the applications. There is a command line, which can accept both commands and scripts and the Graphical User Interface (GUI).

The command-line interface is a UNIX or Tcl environment. Modus Test has its own commands to call our functions but also supports the interfaces to traditional UNIX commands.

Listed are the different ways to start Modus Test.

The Graphical User Interface



The Graphical User Interface allows users to access Modus Test tasks in several ways. Tasks can be selected and executed using the GUI methodology, the GUI menus, the GUI command line, or the GUI task menu.

- The methodology is displayed on the left-hand side when the **Methodology** tab is selected. This provides a hierarchical display of the processing tasks in order, and a task may be selected by clicking on it. Many users choose to create their own methodologies and flows to perform project-specific processing.
- In some cases, experts may prefer to use the GUI without a methodology. Direct GUI access to individual commands is available through the pull-down menus.
 - With either of the previous options, when a task is selected, a Form window appears, and design-specific information appears in many fields if defined in a setup file or saved from a previous step. The user can override any settings on the form.

The command line at the bottom of the Session Log window can display the command to be executed from the methodology or the menus, allowing the user to edit the command. It also allows the user to type a command from scratch. The results appear in the session log window. This is equivalent to the command-line mode but from within the GUI. Note that the command line allows entry of UNIX commands or scripts, so it can be used as you would use an *xterm*.

Whenever a command is executed from the GUI or from an *xterm* or script, a status record of the task is posted and displayed when the **Tasks** tab is selected from the left side of the window. When you select a task, the form for that task is displayed with all the options set exactly as when you originally ran the task. You can modify the settings and run the task from that form. When you run the command, a new task is displayed at the bottom of the task window.

Quiz: Starting the Modus Test



You can bring up the Modus Test Graphical User Interface by entering:



Invoke the tool using `modus -gui` OR run `gui_open` from the command console.



Answer

Invoke the tool using `modus -gui` OR run `gui_open` from the command console.

Module Summary

In this module, you

- Identified the fundamentals of the Modus Test ATPG flow



This page does not contain notes.



References

Modus Training Bytes

- [Modus Training Byte References](#)

Modus One-Stop Page

- [Modus One-Stop Page Reference](#)

Modus Full Course Reference

- [Modus DFT Software Solution](#)

Modus User Manuals

- [Modus User Guide 25.1](#)
- [Modus Product Manuals](#)

Modus Articles Reference

- [Modus Articles and AppNotes](#)



This page does not contain notes.

Lab



Lab 6-1 Running the Basic ATPG Flow in Modus Test

174 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.



Module 7

The Equivalency Checking Stage

cādence®

This page does not contain notes.

Module Objectives

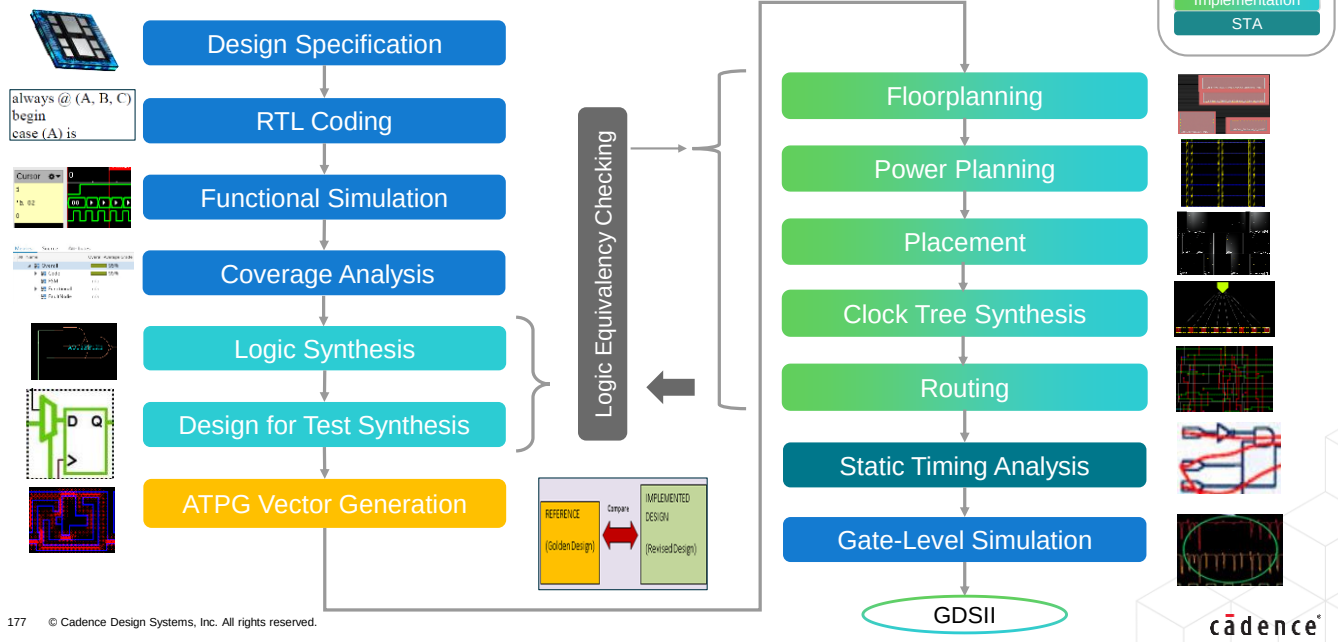
In this module, you

- Identify the basics of the equivalence checker software
- Set up a design for equivalence checking



This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow: The Equivalency Checking Stage



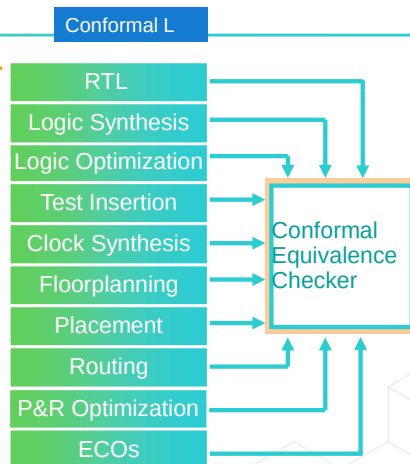
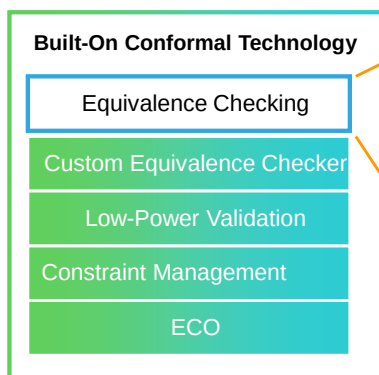
This page does not contain notes.

What Is Logic Equivalence Checking?



- Logic equivalence checking is a process to ensure that the synthesized netlist is functionally equivalent to the original RTL design. Since synthesis involves complex optimizations and transformations, it is recommended to perform functional verification after synthesis.
- The Conformal® Equivalence Checking solutions are logical equivalence-checking tools that verify RTL, gate, or transistor-level designs.

During development, a chip design undergoes numerous transformations and iterations prior to the final layout – and each step in this process has the potential to introduce logical bugs.



178 © Cadence Design Systems, Inc. All rights reserved.

cadence

ASIC (Application Specific Integrated Circuit) designs undergo many changes during synthesis and optimization at several stages. Conformal L verifies your RTL against the gate-level designs after each step.

FPGA (Field Programmable Gate Array) devices approaching ASIC complexity face similar verification challenges as ASIC devices:

- FPGA synthesis tools perform aggressive logical synthesis and optimization.
- Simulation and debugging are painful for FPGAs.

The Cadence Conformal Equivalence Checker (EC) provides verification support for synthesis tools from Synplify, Xilinx, and Altera.

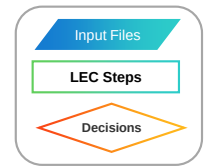
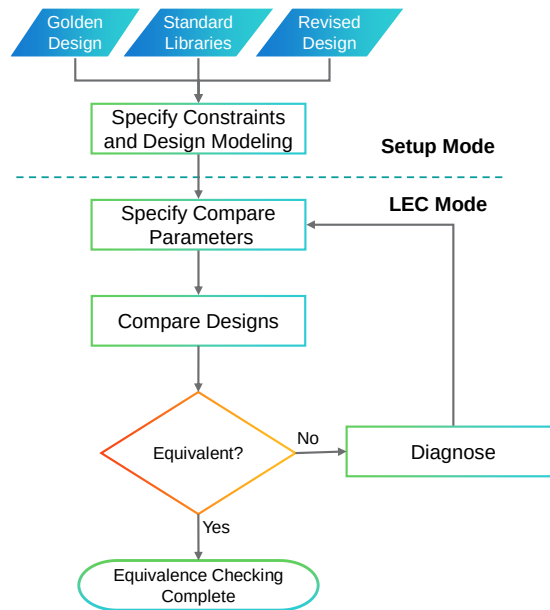
Supported FPGA tools include:

- SynplifyPro
- Xilinx ISE (Integrated Software Environment)
- Altera Quartus

In your design flow, you start with your RTL and then run synthesis. You then must compare your design between RTL and your synthesis results using Equivalence Checking. The basic "L" and "XL" licenses" can help you with that. If you have any custom logic, you can use the "Custom Equivalence Checker" or the GXL license.

As seen here, the various design stages where you would use the Conformal Equivalence Checker are RTL to logic synthesis, logic synthesis to logic optimization, before and after test insertion, before and after clock tree synthesis, and then before and after placement, and so on. The "XL" license is sufficient to verify most designs between each of these stages.

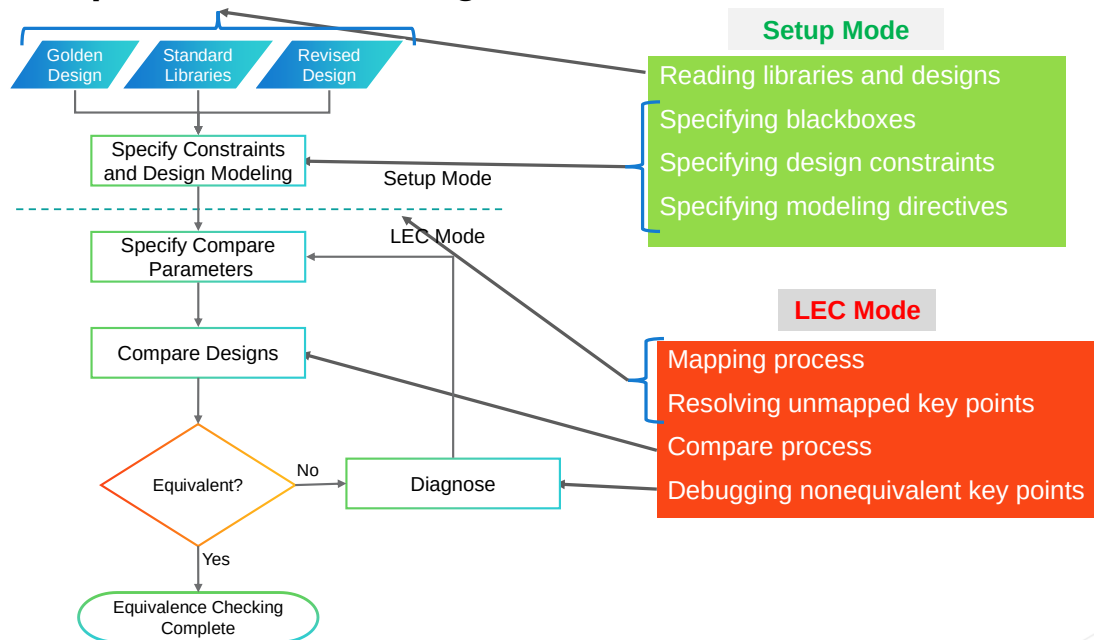
Logic Equivalence Checking Flow



Use the *Setup* mode for the setup of the design comparison, such as reading the designs, defining the design constraints, and specifying modeling options.

Use the *LEC* mode for the design comparison.

Logic Equivalence Checking Flow (continued)



180 © Cadence Design Systems, Inc. All rights reserved.



To compare your two designs, you'll be reading both designs. One design will be read as Golden, and the other design will be read as Revised.

The libraries for each will also be read for either or both designs. For some Verilog RTL, you might not have to read any libraries. For VHDL, you must map the libraries so that it will associate the VHDL entities with their architectures.

Then you can specify constraints and modeling directives. We will discuss these in detail in the upcoming slides.

After that, define any comparison parameters and run the comparison. If there are problems with comparison, you can diagnose and fix any problems.

The idea is to essentially prove the two designs are equivalent or not, with a degree of certainty.

How to Start and Exit Conformal LEC



To start the Equivalence Checker, use:
lec



To exit Conformal LEC, use:
exit [-force]

Starting Conformal LEC

- To get help on all the options, use:
lec -help
- Menu mode:
lec [-L | -XL | -GXL | -LPXL | -LPGXL | -ECO]
- Batch mode:
lec -dofile <batch_file> -nogui
- Interactive Command mode:
lec -nogui
//Then, in the tool, run the commands //
//dofile <command_file> //
- Switch between menu and command modes:
set gui [on | off]

LEC automatically closes all windows upon exit.

181 © Cadence Design Systems, Inc. All rights reserved.



L, XL, and GXL are the licensing flavors of the Conformal Equivalence Checker tool. You can also use the Conformal Low Power licenses, which are LPXL and LPGXL. The E-C-O license can also be used to start the tool.

Specify a license to use and start Conformal LEC in either the command-line mode or the graphical interface mode. By default, the tool opens the graphical interface. For batch mode, specify a do-file or the command file using the *-no-g-u-i* option. If you've already started Conformal and want to run other command files, you can run them using the *do-file* command.

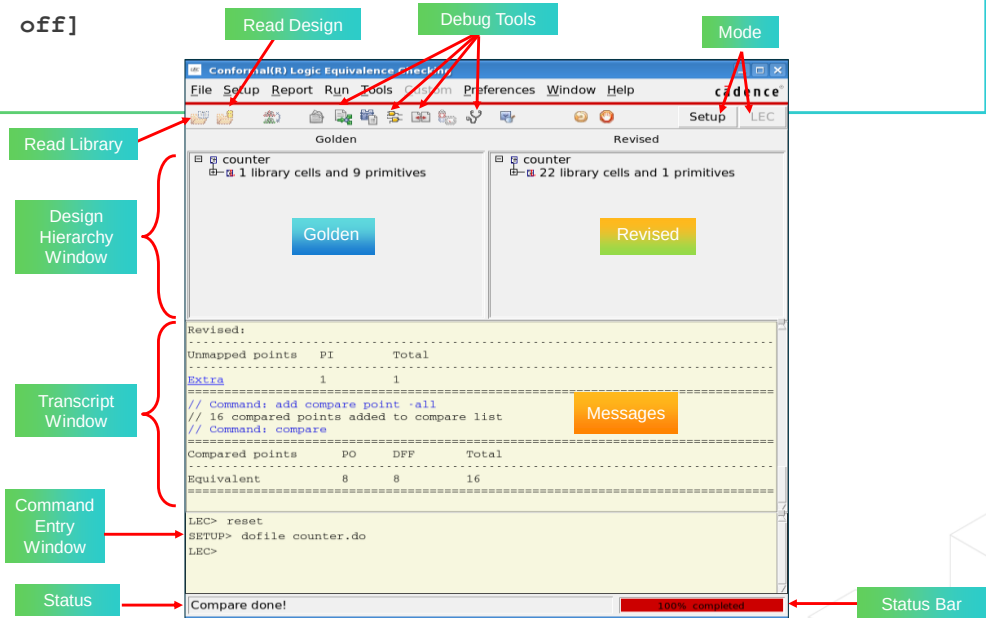
To exit the tool, use the *exit* command.

The Graphical User Interface (GUI)



`set gui [on | off]`

Use this command to switch back and forth between the graphical and non-graphical modes at any time during the session.



182 © Cadence Design Systems, Inc. All rights reserved.

cadence

The graphical interface has a menu bar at the top with File, Setup, Report and other menus. Below the menu bar is the icon bar with shortcuts to the debug tools. The Design hierarchy window shows the *Golden* design on the left and the *Revised* design on the right. The transcript window shows all the messages issued by the tool. The command entry window is where you enter all your commands. The bottom of the status bar indicates the state of your session.

Runtime for Graphical User Interface (GUI) and non-Graphical User Interface (non-GUI or command) mode might not be much different for small designs, but for large designs, the difference can be significant.

In general, you can run the design in non-GUI mode (command mode) to speed up the process, switch to the graphical interface mode to use the diagnosis windows for debugging, then switch back to command mode to rerun the software.

You can switch back and forth between the graphical and non-graphical modes at any time during the session by using this command:

`set gui [on | off]`

Example: Standard Dofile

```
set log file lec.log -replace
read library ...
read design -Golden ...
read design -revised ...
report design data
report black box
report module
<apply design constraints>
<apply modeling options>
set system mode lec
add compared points -all
compare
usage
```

Setup Mode

Reading libraries and designs
Specifying blackboxes
Specifying design constraints
Specifying modeling directives

LEC Mode

Mapping process
Resolving unmapped key points
Compare process
Debugging nonequivalent key points



This page does not contain notes.

Quiz: The Equivalency Checking Stage



What kinds of design can be compared based on the recommended use of the tool?



A) RTL – Synthesis Netlist

B) RTL – Full Netlist with DFT

C) Synthesis Netlist – P&R Netlist

D) RTL – P&R Netlist

E) Synthesis Netlist – Pre-CTS Netlist

F) P&R Netlist – GDS II

A) RTL – Synthesis Netlist

YES

B) RTL – Full Netlist with DFT

YES

C) Synthesis Netlist – P&R Netlist

YES

D) RTL – P&R Netlist

NO (we recommend you have intermediate steps)

E) Synthesis Netlist – Pre-CTS Netlist

YES

F) P&R Netlist – GDS II

NO (Not possible in this tool)



Successful mapping of the design is a required step for successful comparison.
(TRUE/FALSE)

TRUE

184 © Cadence Design Systems, Inc. All rights reserved.



Answers

1. A – Yes, B – Yes, C – Yes, D – No (we recommend you have intermediate steps), E – Yes, F – No (Not possible in this tool)
2. True

Quiz: The Equivalency Checking Stage (continued)



What is the flow of the equivalency checking (Conformal) tool?

A) Comparison

B) Diagnosis

C) Reading designs

D) Mapping

Reading designs

Mapping

Comparison

Diagnosis



What are the two main modes of the tool? What are each of the modes mainly used for?

1. Set up mode to read and set up designs
2. LEC mode to compare and diagnose designs

185 © Cadence Design Systems, Inc. All rights reserved.



Answers

1. C – D – A – B
2. Set up mode to read and set up designs, LEC mode to compare and diagnose designs

Module Summary

In this module, you learned to

- Identify the basics of the equivalence checker software
- Set up a design for equivalence checking



This page does not contain notes.



References

Conformal Training Bytes

- [Conformal Training Byte References](#)

Conformal One-Stop Page

- [Conformal One-Stop Page Reference](#)

Conformal Full Course Reference

- [Conformal Equivalence Checking](#)

Conformal User Guide

- [Conformal User Guide 25.1](#)
- [Conformal Product Manual](#)

Conformal Articles Reference

- [Conformal Articles and AppNotes](#)



This page does not contain notes.

Labs



Lab 7-1 Running the Equivalence Checking Flow in Conformal

Lab 7-2 Creating .v Format File from .lib Format



This page does not contain notes.



Module 8

The Implementation Stage

cādence[®]

This page does not contain notes.

Module Objectives

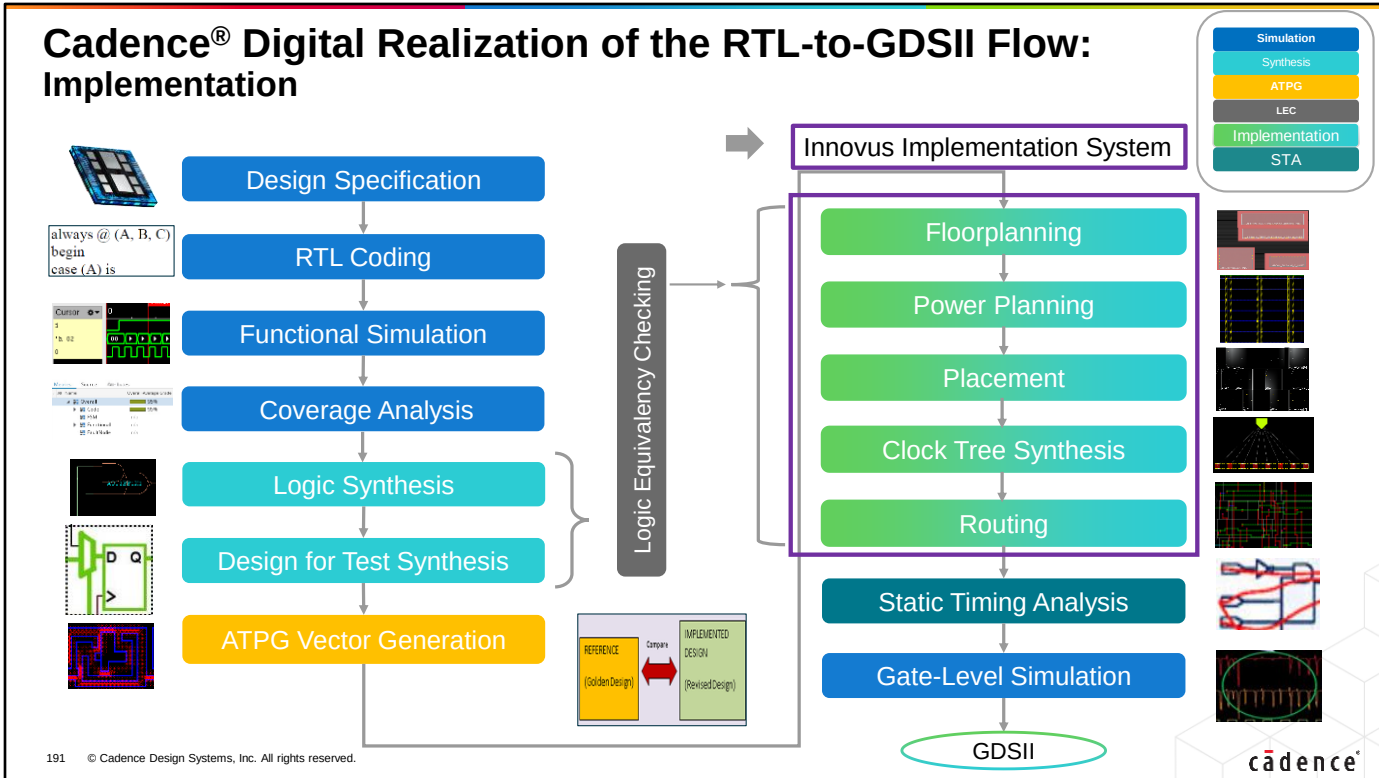
In this module, you

- Import and floorplan your design
- Place the standard cells in the design
- Run power planning and power routing
- Extract parasitics and generate timing reports
- Create clock trees
- Detail route of the design
- Output files for tapeout



This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow: Implementation



This page does not contain notes.

What Is Digital Implementation?



Digital Implementation is the process of importing a gate-level netlist as well as the technology libraries and generating a physical design that meets power, performance and area (PPA) goals.

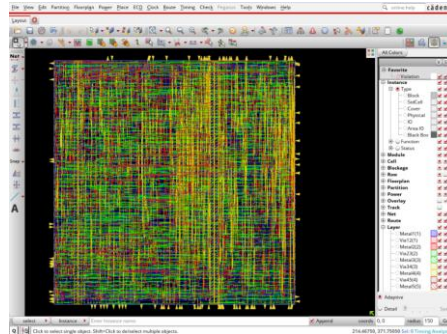
Gate-Level
Netlist



Innovus™
Implementation
System



```
module accu(n, acc, clk, reset);
  input [7:0] in;
  input clk, reset;
  output [7:0] acc;
  wire [7:0] in;
  wire clk, reset;
  wire [7:0] acc;
  wire n_0, n_1, n_2, n_3, n_4, n_5, n_6, n_7;
  wire n_8, n_9, n_10, n_11, n_12, n_13, n_14, n_15;
  wire n_16, n_17, n_18, n_19, n_20, n_21, n_22, n_23;
  wire n_24, n_25, n_26, n_27, n_28, n_29, n_30, n_31;
  DFFPOSX1 vacc_reg[0] (.CLK(clk), D(n_0), Q(acc[0]));
  DFFPOSX1 vacc_reg[1] (.CLK(clk), D(n_1), Q(acc[1]));
  DFFPOSX1 vacc_reg[2] (.CLK(clk), D(n_2), Q(acc[2]));
  DFFPOSX1 vacc_reg[3] (.CLK(clk), D(n_3), Q(acc[3]));
  DFFPOSX1 vacc_reg[4] (.CLK(clk), D(n_4), Q(acc[4]));
  DFFPOSX1 vacc_reg[5] (.CLK(clk), D(n_5), Q(acc[5]));
  DFFPOSX1 vacc_reg[6] (.CLK(clk), D(n_6), Q(acc[6]));
  DFFPOSX1 vacc_reg[7] (.CLK(clk), D(n_7), Q(acc[7]));
  NOR2X1 g146(A(n_30), B(reset), Y(n_31));
  XOR2X1 g148(A(n_26), B(n_3), Y(n_30));
  NOR2X1 g149(A(n_28), B(reset), Y(n_29));
  INVX1 g150(A(n_27), Y(n_28));
  FAX1 g151(A(acc[6]), B(in[6]), C(n_22), YC(n_26), YS(n_27));
  NOR2X1 g153(A(n_24), B(reset), Y(n_25));
  INVX1 g154(A(n_23), Y(n_24));
  FAX1 g155(A(acc[5]), B(in[5]), C(n_18), YC(n_22), YS(n_23));
  NOR2X1 g157(A(n_20), B(reset), Y(n_21));
  INVX1 g158(A(n_19), Y(n_20));
  FAX1 g159(A(acc[4]), B(in[4]), C(n_14), YC(n_18), YS(n_19));
  NOR2X1 g161(A(n_16), B(reset), Y(n_17));
  INVX1 g162(A(n_15), Y(n_16));
  FAX1 g163(A(acc[3]), B(in[3]), C(n_10), YC(n_14), YS(n_15));
  NOR2X1 g165(A(n_12), B(reset), Y(n_13));
  INVX1 g166(A(n_11), Y(n_12));
  FAX1 g167(A(acc[2]), B(in[2]), C(n_6), YC(n_10), YS(n_11));
  NOR2X1 g169(A(n_8), B(reset), Y(n_9));
  INVX1 g170(A(n_7), Y(n_8));
  FAX1 g171(A(acc[1]), B(in[1]), C(n_1), YC(n_6), YS(n_7));
  NOR2X1 g173(A(n_4), B(reset), Y(n_5));
  INVX1 g174(A(n_2), Y(n_4));
  FAX1 g175(A(in[0]), B(acc[0]), YC(n_1), YS(n_2));
  OAZ1X1 g176(A(in[7]), B(acc[7]), C(n_0), Y(n_3));
  NAND2X1 g177(A(acc[7]), B(in[7]), Y(n_0));
endmodule
```



This page does not contain notes.

Starting the Software



To start a session, in an xterm window:

```
lnx-cadence> innovus -stylus
```

- To display the command-line options available with the *innovus* command, enter the following:
`innovus -stylus -help`
- Examples of options include licenses to check out and display preferences.
- To check if the software is in Stylus mode or Legacy mode, run the following command from the Innovus prompt:
`is_common_ui_mode`
- The command will return a “1” if the mode is Stylus.

Note: The xterm window when you start the tool becomes the input window for Tcl commands, sourcing scripts after you start the software, and the console for displaying messages from the software.

Using the xterm window for operations other than to interface with the software suspends the session until the operation is completed.

Note: Running the Innovus tool without the `-stylus` option runs the tool in legacy mode, which has a different command structure and user interface. This course is based entirely on the Stylus Common UI, Flowkit and Metrics.



This page does not contain notes.

Log and Command Files

The Innovus Implementation System creates two types of files: log files and command files.

- Default log files: *innovus.log*<session#>, *innovus.logv*<session#>

The *.logv* file is a verbose version of the *.log* file

To suppress the verbose log file, enter the command:

```
innovus -stylus -nologv
```

- Default command filename: *innovus.cmd*<session#>
- Custom log filename: *innovus -stylus -log myfile*

Example Command File

Filename: *innovus.cmd*<session#>

```
read_floorplan dtmf.fp
create_scan_chain -name scanChain1 -start IOPADS_INST/Pscaninlip/C -stop
    IOPADS_INST/Pscanoutlop/I
place_opt_design
```

194 © Cadence Design Systems, Inc. All rights reserved.



The *innovus.cmd*<seq#> file contains commands that you have entered in the command window and the graphical interface. It can be used as a replay file.

The *.logv* file is a verbose version of the log file.

Software Installation Path



```
lnx-cadence> which innovus
```

- You can find the installation path, as long as the path is set, by typing the following in an xterm prompt:

```
lnx-cadence> which innovus
```

- The Innovus Implementation System binary is located in the following directory path:

```
<installation_directory>/INNOVUS231/<binary>/bin/innovus
```

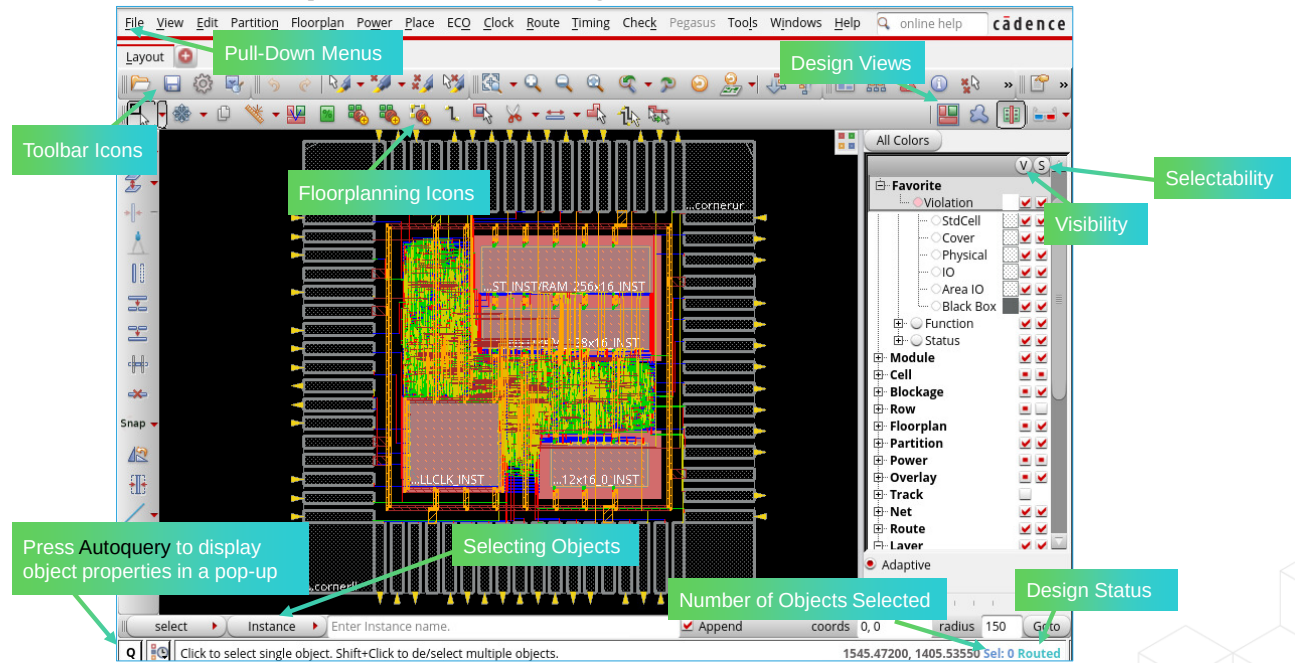
- The documentation is installed in the following location:

```
<installation_directory>/INNOVUS231/binary/doc
```



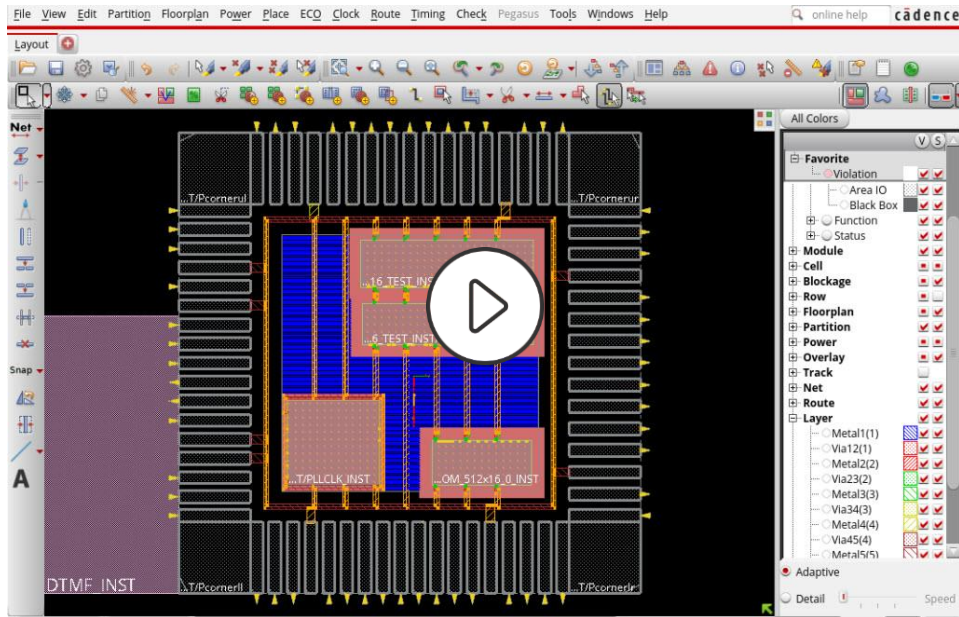
This page does not contain notes.

The Innovus Implementation System Window



This page does not contain notes.

Demo: The Innovus Graphical User Interface



197 © Cadence Design Systems, Inc. All rights reserved.

cadence

Video Duration: 06:34

Click the Play button to start the video.

Search for “The Innovus Graphical User Interface” on support.cadence.com to play the video online.

Running in Batch Mode



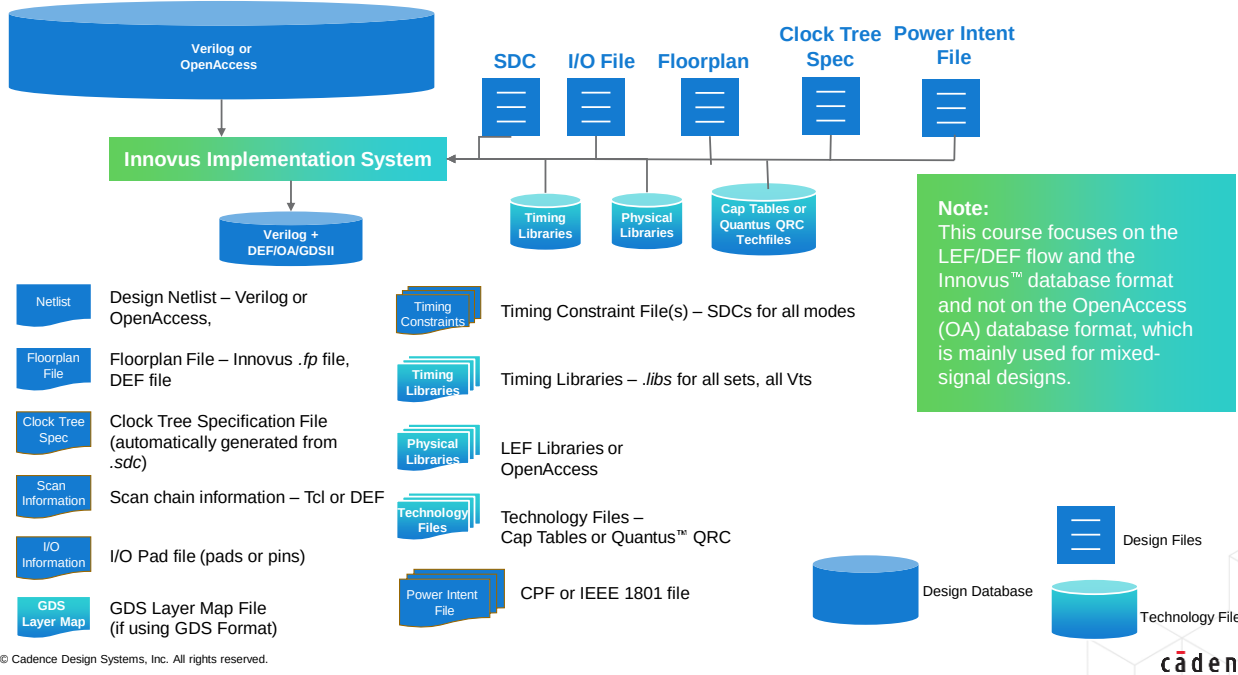
You can run the tool in batch mode, to source Tcl scripts or enter commands, in several ways if you do not want to display the graphical design window.

- To run the tool in non-graphical mode for the entire session by entering:
`innovus -stylus -no_gui`
- In the Innovus window, run a set of commands by entering:
`source timingReports.tcl`
- In the C-shell, run a job without displaying the Innovus Implementation System window by entering:
`innovus -stylus -files init.tcl`
 - To start the Innovus Implementation System graphical interface after running the tool in batch mode, enter:
`gui_show`
 - After you start the graphical interface, you can turn it off by entering:
`gui_hide`



This page does not contain notes.

Input Files for Implementation



This page does not contain notes.

Input Files Used for Implementation



Input Files	Definitions and Uses
Gate level Netlist (.v)	This file, which contains the logic connectivity of all the cells, is exported from the synthesis tool.
Design constraints (.sdc)	The .sdc file contains all the timing constraints that the tool must meet and fix, if there are violations in the design.
Liberty (.lib)	The .lib file is a timing library that contains logical information (setup time, hold time, cell delay, etc.) of all the standard cells/macros.
Library exchange format (.lef)	The LEF file is a physical library that contains physical information (cell/pin name, cell/pin dimensions, blockages, etc.) of the standard cells/macros. The LEF file is extracted from the abstract view of a cell.
Technology LEF file (.lef)	The technology LEF file contains the metal layers, vias, and their name and preferred directions for routing. It also has design rules for each metal layers.
Extraction Technology File (.qrctech)	The .qrctech file contains the values of capacitance and resistance per unit length of each metal layer. These RC parasitics are used to calculate net delays during extraction.

This page does not contain notes.

Reading and Writing Innovus System Design Database



`write_db <database>`

When you save a database, the command `write_db` creates a directory and saves the database files into that directory.

Example

`write_db routed.inn`

Write out the database to checkpoint the implementation flow.



`read_db <database>`

Example

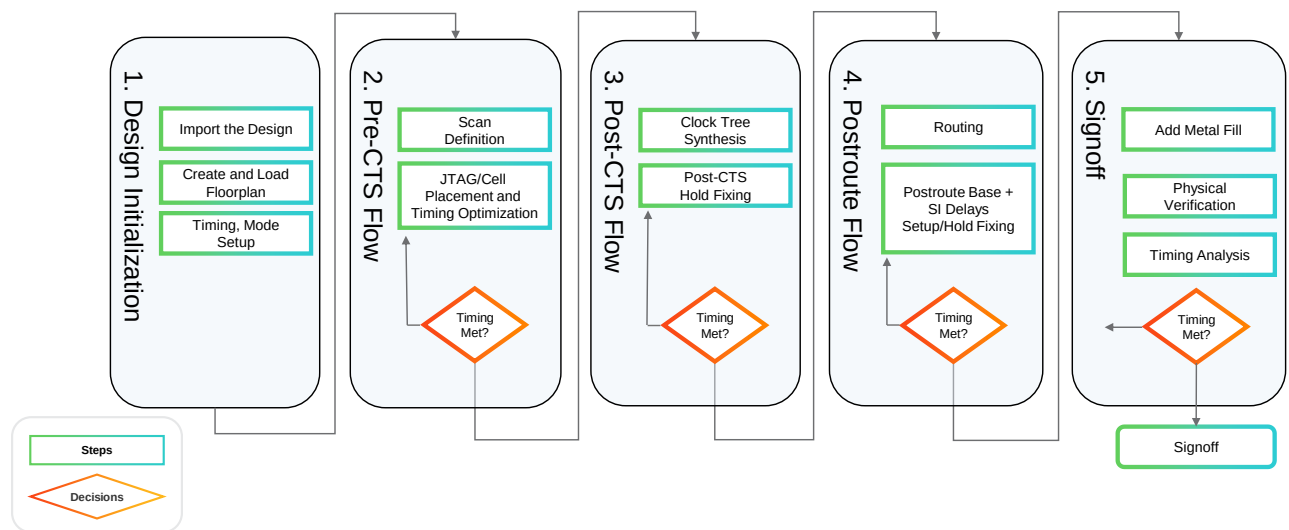
`read_db routed.inn`

Read a database that was previously saved, either in Legacy or Stylus mode to restore the design data.



This page does not contain notes.

Flowchart: The Digital Implementation Flow



202 © Cadence Design Systems, Inc. All rights reserved.

cadence

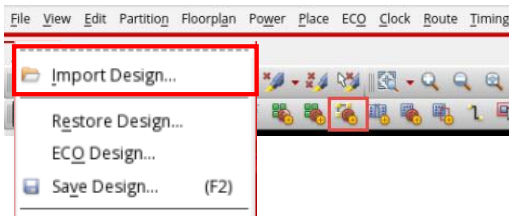
This page does not contain notes.

Importing the Design



File → Import Design

The Design Import menu selection brings up a form for importing the RTL netlist, or gate-level netlist, and the physical and timing libraries.



Use this form to import the design libraries (physical and timing) and the netlists.

The 'Design Import' dialog box is shown with the following sections and annotations:

- Netlist:** The 'Verilog' radio button is selected. The 'Files' field contains 'C:/verilog/dtmf_chip_ak.v'. A green callout points to this field with the text: 'Enter the Verilog Netlist files.'
- Technology/Physical Libraries:** The 'LEF Files' section has a text field containing 'J1ef/all.lef'. A green callout points to this field with the text: 'Specify the LEF file for place and route.'
- Floorplan:** The 'IO Assignment File' field is empty. A green callout points to this field with the text: 'Enter the name of the I/O file that contains information about where the I/Os are placed around the core area.'
- Power:** The 'Power Nets' field contains 'VDD' and the 'Ground Nets' field contains 'VSS'. A green callout points to these fields with the text: 'Specify the timing libraries and SDC constraints in the MMMC View definition file.'
- Analysis Configuration:** The 'MMMC View Definition File' field contains 'dtmf.mmmc'.

Buttons at the bottom: OK, Cancel, Help.

203 © Cadence Design Systems, Inc. All rights reserved.

cadence

Use the **Save Design** menu command to save your design often.

The *saveDesign* command lets you write out a hierarchical Verilog netlist after running placement (with timing-driven option), scan optimization, clock tree synthesis, and optimization.

The Innovus Implementation System supports the OpenAccess database format. You can use standard C++ as the interface (API) to access and manipulate database objects.

Commands to Import the Design

The following commands are equivalent to using the design import form to initialize the design.

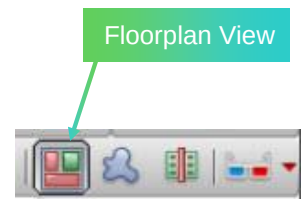
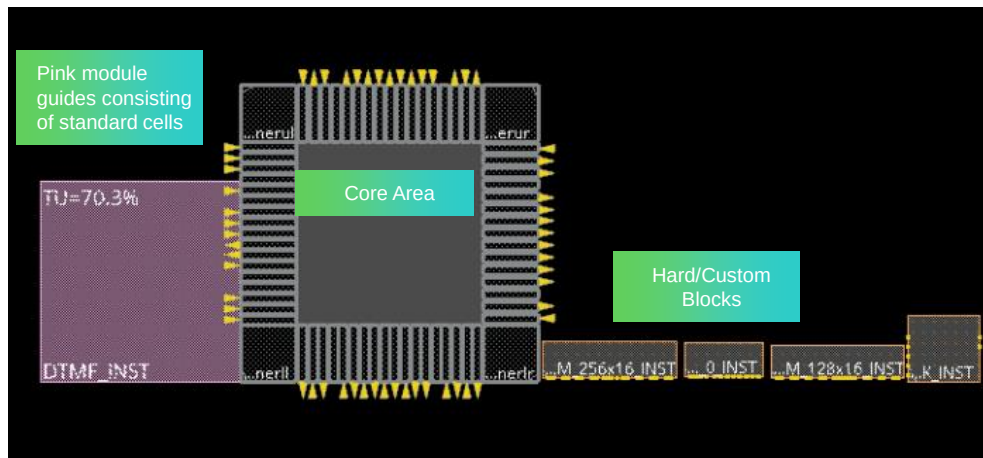
```
set_db init_read_netlist_files ../physical_design/counter_netlist.v
set_db init_lef_files {../lef/gsclib045_tech.lef ../lef/gsclib045_macro.lef}
set_db init_power_nets VDD
set_db init_ground_nets VSS
set_db init_mmmc_files counter.view
read_mmmc counter.view
read_physical -lefs {../lef/gsclib045_tech.lef ../lef/gsclib045_macro.lef}
read_netlist ../physical_design/counter_netlist.v -top counter
_init_design
```

You can source the Tcl script of commands instead of using the form.



This page does not contain notes.

Displaying the Design After Design Import



Example to resize the main window:

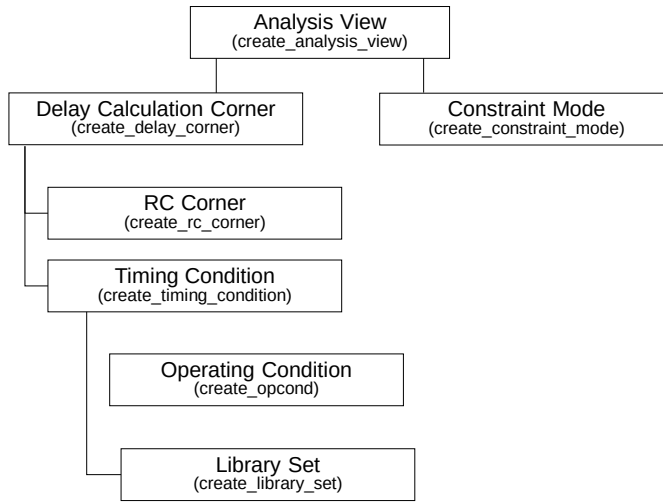
```
gui_set_ui main -geometry 1000x800
```

205 © Cadence Design Systems, Inc. All rights reserved.



The module guides follow the hierarchy in the netlist.

Flowchart: Setting Up Analysis Views



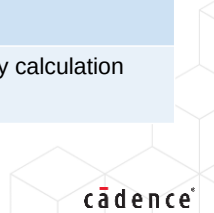
This page does not contain notes.

Setting Up Analysis Views



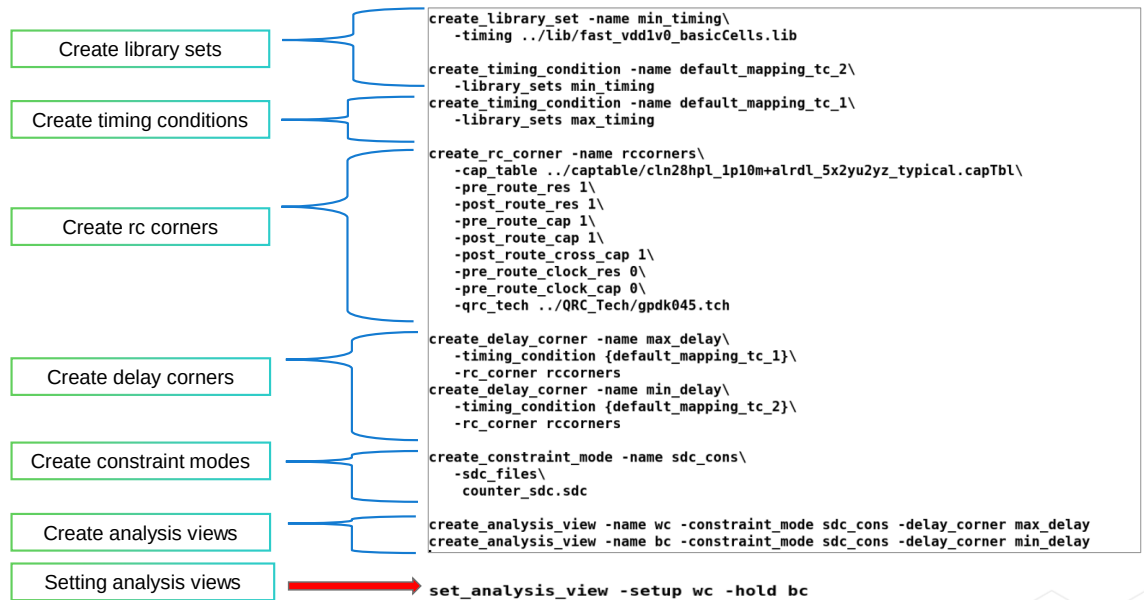
Innovus uses the commands listed in the table to set up analysis views.

Command	Details
<code>create_library_set</code>	This command creates a library set that associates timing libraries.
<code>create_timing_condition</code>	This command describes a specific timing condition.
<code>create_rc_corner</code>	The command creates an RC corner that uses the capacitance tables and derates the resistance values based on the temperature using a file QRC TECH.
<code>create_delay_corner</code>	This command creates a delay calculation corner using created library sets and RC corners.
<code>create_constraint_mode</code>	This command creates constraint modes using SDC files.
<code>create_analysis_view</code>	This command creates an analysis view object that associates a delay calculation corner with a constraint mode.



This page does not contain notes.

Example: Setting Up Analysis View



208 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

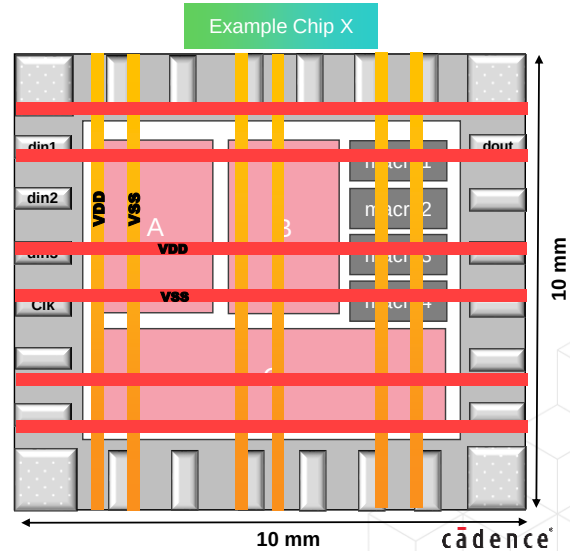
What Is Floorplanning?



Floorplanning is the process of deriving the die size, allocating space for soft blocks, planning power, and macro placement.

With a top-level netlist, you can start to floorplan the chip.

- Set die size to 10x10 mm²
- Place the I/Os: The *din*, *clk*, and *dout* I/Os are shown assigned to the perimeter
- For hierarchical implementation, create soft blocks for A, B, and C (partitioning)
 - Size the blocks A, B, and C
- (Optional) Place critical macros
- Perform power planning
- Perform macro placement
- Check for early routing congestion
- Check for early block utilization
- Power domain definition
- Flip-chip bump placement



209 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

Specifying the Floorplan

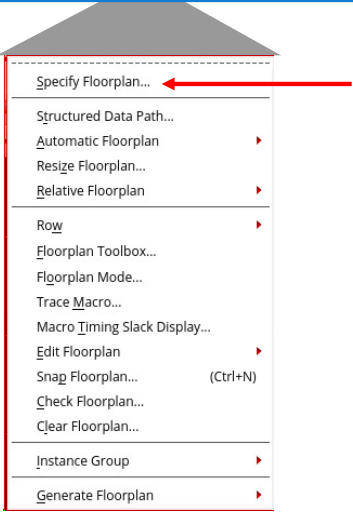


Floorplan → Specify Floorplan

With this form, you can modify the floorplanning parameters.

Command
`create_floorplan`

... Partition Floorplan Power Place...



This page does not contain notes.

Specifying the Floorplan: Basic Tab

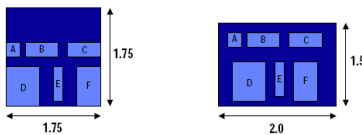


Floorplan → Specify Floorplan
(Basic Tab)

Command
create_floorplan

Usage Tip:

If there are more horizontal routing layers than there are vertical and a block requires a lot of routing resources, the block should be short and wide



211 © Cadence Design Systems, Inc. All rights reserved.

cadence

With this form, you can modify floorplanning parameters.

Die, I/O, and Core coordinates

Die Size by Width and Height

The Core size can be defined in two ways:

- Core size by Aspect Ratio or Core Size by Width and Height

Core Margins can be defined in two ways:

- Core to Die Boundary or Core to IO Boundary

Die Size Calculation – Max IO Height is selected with multiheight I/O pad instances.

Floorplan Origin at – Default is at Lower Left or change to Center

Assigning Pads and Pins



Pads and pins are assigned as part of Design Import by reading in an I/O assignment file or reading in a DEF file.

To assign pads and pins:

1. Create an I/O Assignment file.
2. Read it in during Design Import or by running the `read_io_file` command.

Or,

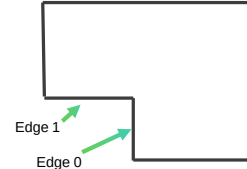
Create an assignment file in DEF format, and read it in with the following command:

```
read_def
```

The Innovus I/O file format supports:

- Top-level design with pads
- Rectilinear block level design
- I/O ordering, that is:
 - Clockwise
 - Counter-clockwise
 - Left to right, bottom to top (default)

In this diagram, Edge 0, the left-most edge at y=0, is the starting point for the I/O assignment.



To display the edge numbers, run the command:

```
gui_show_edge_number -objects  
<string>
```

This page does not contain notes.

I/O File Example for Pad Placement

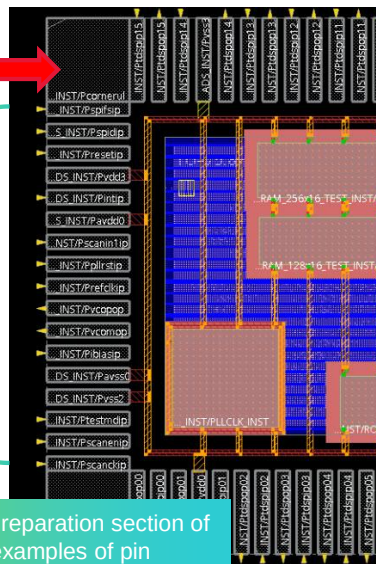
Example

```
(topleft
  (inst name="IOPADS_INST/Pcornerul" )
)
(left
  (inst name="IOPADS_INST/Pscanckip" offset=235.0000 )
  (inst name="IOPADS_INST/Pscanenip" offset=297.6300 )
  (inst name="IOPADS_INST/Ptestmdip" offset=360.2650 )
  (inst name="IOPADS_INST/Pvss2" offset=422.8950 )
  (inst name="IOPADS_INST/Pavss0" offset=485.5300 )
  (inst name="IOPADS_INST/Pibiasip" offset=548.1600 )
  (inst name="IOPADS_INST/Pvcomop" offset=610.7950 )
  (inst name="IOPADS_INST/Pvcomop" offset=673.4250 )
  (inst name="IOPADS_INST/Prefclkip" offset=736.0600 )
  (inst name="IOPADS_INST/Pplrstrtip" offset=798.6900 )
  (inst name="IOPADS_INST/Pscaninlip" offset=861.3250 )
  (inst name="IOPADS_INST/Pavdd0" offset=923.9550 )
  (inst name="IOPADS_INST/Pintip" offset=986.5900 )
  (inst name="IOPADS_INST/Pvdd3" offset=1049.2200 )
  (inst name="IOPADS_INST/Presetip" offset=1111.8550 )
  (inst name="IOPADS_INST/Pspidip" offset=1174.4850 )
  (inst name="IOPADS_INST/Pspifsip" offset=1237.1200 )
)
(bottomleft
  (inst name="IOPADS_INST/Pcornerll" )
)
(bottom
  (inst name="IOPADS_INST/Ptdsop00" offset=235.0000 )
)
```

top left

left

Refer to the Data Preparation section of the user guide for examples of pin placements for modules and area I/Os.



version = 3 #

io_order = clockwise is default #Order of the I/O pads and pins.

(inst

name = IOPADS_INST/pad1 W

offset = 235.0000 # Offset in ums. The offset of a pad is the offset from the die boundary, based on the order direction.

orientation = R0 # Orientation of the I/O.

place_status = fixed # Placement status of the I/O pad.

What Is Power Planning?



Power Planning is the process of designing a global power distribution network that supplies sufficient power and ground nets to all the instances of the design. Power planning includes sizing the power wires and choosing the metal layers necessary to deliver the required power to different parts of the chip.

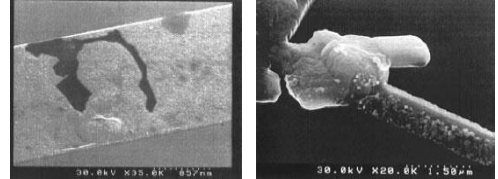
Inadequate power distribution can:

- Affect chip timing due to excessive rail voltage drop (IR drop) and ground bounce
- Lead to device failure due to electromigration effects

The effects of IR drop and other power-related issues can be limited by:

- Good power-grid design
- Sufficient VDD and VSS pads

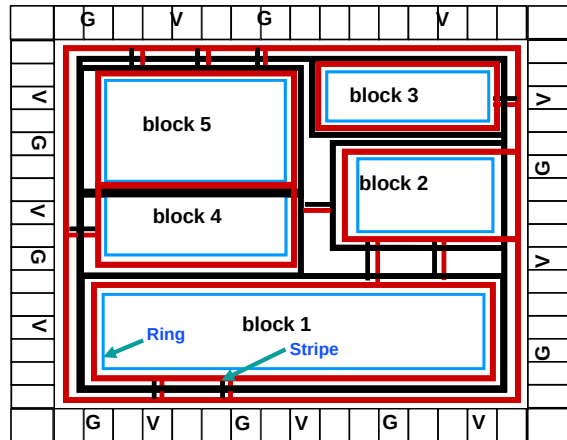
EM failures as seen through a scanning electron microscope (SEM)



This page does not contain notes.

Types of Power Planning: Rings and Stripes Methodology

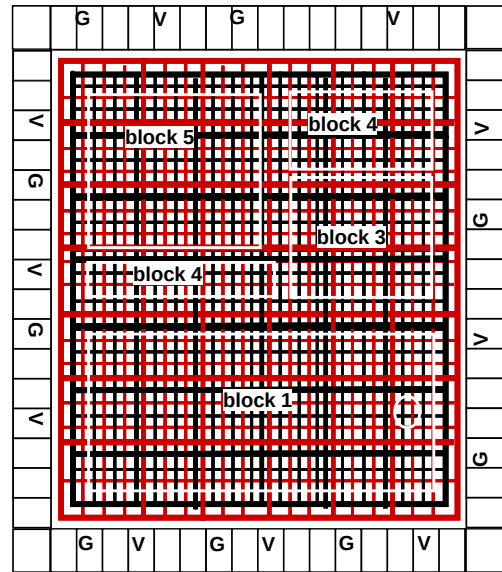
- Each block has its own ring structure
- Each block has stripes that connect the top level to the block
- Rings can be shared between abutted blocks
- Requires fewer routing resources
- Changes in design may require changes to the power structure



This page does not contain notes.

Types of Power Planning: Power Mesh Methodology

- Robust and redundant power network
- Seen in microprocessors and large ASICs
- Primary distribution is through upper metal layers
- Meshes of different blocks need to align with each other



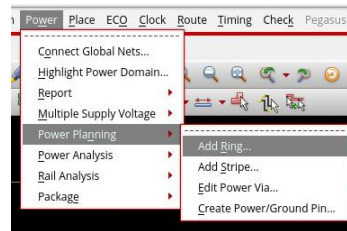
G = Ground
V = Power

This page does not contain notes.

Adding Rings: Basic Tab (Core Rings)



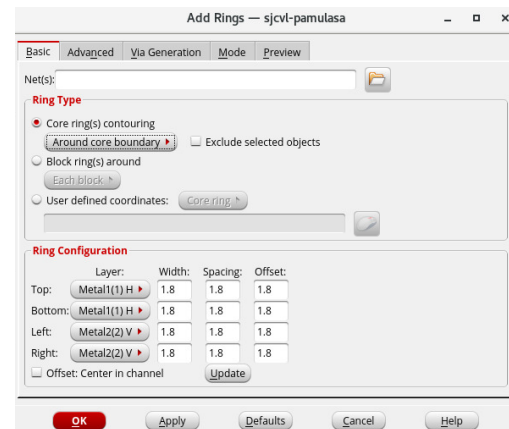
Power → Power Planning → Add Ring
→ Basic (Tab)



Core rings follow the contour of the core boundary or the I/O boundary.

- You can specify the layers, their widths, their spacing, and the offset.
- You can also exclude selected objects, such as blocks that typically have their own power structure.

Command
add_rings



217 © Cadence Design Systems, Inc. All rights reserved.

cadence

Select **Core ring(s) contouring** to create core rings that follow the contour of the core boundary or the contour around the I/O boundary.

- Select **Around core boundary** to create core rings that contour around blocks or rows in the core area of the design.

Use the options in the Ring Configuration panel to either center the ring in the channel between the core boundary and the I/O area, or offset each side of the ring by a specific distance from the core boundary.

- Select **Along I/O boundary** to create core rings that contour the I/O area. Use the options in the Ring Configuration panel to either center the ring in the channel between the core boundary and the I/O area, or offset each side of the ring by a specific distance from the I/O boundary.
- Optionally, select the **Exclude selected objects** option. This option creates a ring that does not include the selected objects.

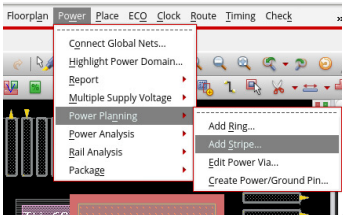
You can select **User defined coordinates**, and specify a set of coordinates. This creates a ring that has the same number of corners as the number of x and y coordinates that you specify.

- You can draw core or block rings anywhere in the core area.
- You need to specify the coordinates and the type of ring (either core or block).

Adding Stripes: Basic Tab



Power → Power Planning → Add Stripe
→ Basic (Tab)



You can create stripes for power and ground nets by choosing *Add Stripe*.

Set Configuration

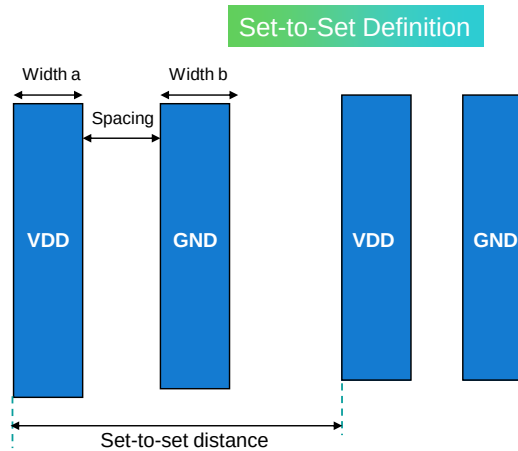
Command
`add_stripes`

Nets	Specifies the nets.
Layer	Specifies the layer to use.
Width	Specifies the width of the stripes that you want to create. If the number of widths specified is less than the number of nets specified, then the last value specified for width is used for the unmatched nets.
Spacing	Specifies the spacing between pair of the stripes.
Set Pattern	You can define the distance between each stripe set and the number of sets.
Stripe Boundary	You can specify the target of the stripe by selecting an object for the stripe to connect to, which enables relative power planning.



This page does not contain notes.

Add Stripes: Spacing Definitions



Boundary offsets are measured from core boundary edge.

You can improve routability and availability of routing tracks by selecting the width and spacing as it relates to the routing grid.

This page does not contain notes.

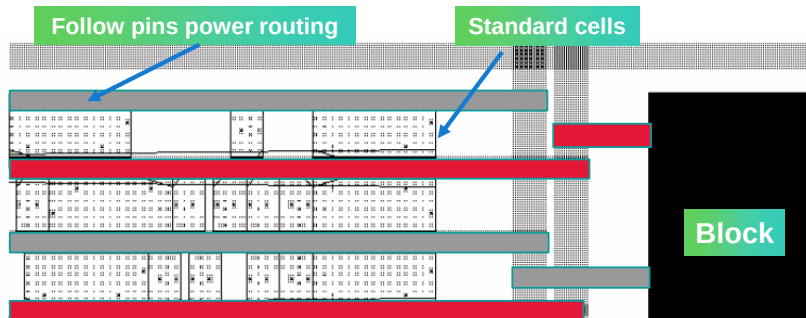
What Is Power Routing?



Power routing is the process of connecting the local power routes to the global power that was created during an earlier step of power planning.

During power routing (*route_special*), you create power routing to:

- Standard cells (follow pins power routing also called power rails)
- Blocks
- I/O pads



This page does not contain notes.

Running Special Route: Basic Tab



Route → Special Route → Basic (Tab)

Connect power pins to global power and also create power rails in the standard cell rows Special Route.

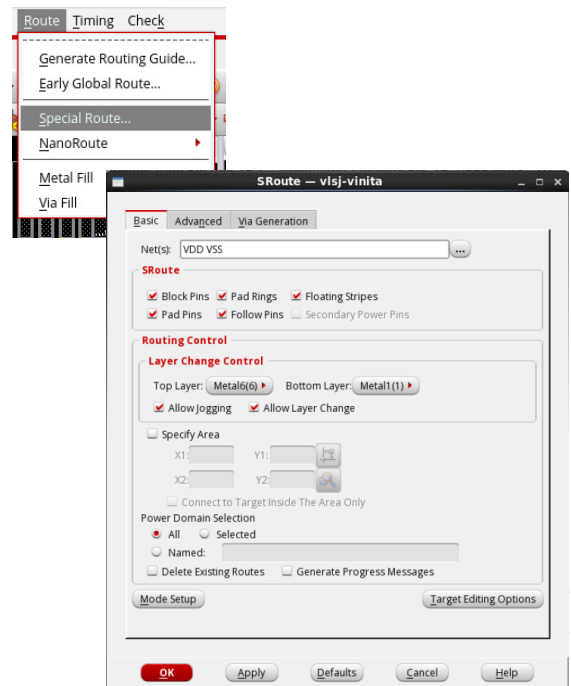
Modes of Operation

- Allow Jogging
- Allow Layer Change

Layer Change Control

- Controls the top/bottom layers used during Special Route.

Commands
`set_db route_special_*`
`route_special`



221 © Cadence Design Systems, Inc. All rights reserved.

cadence

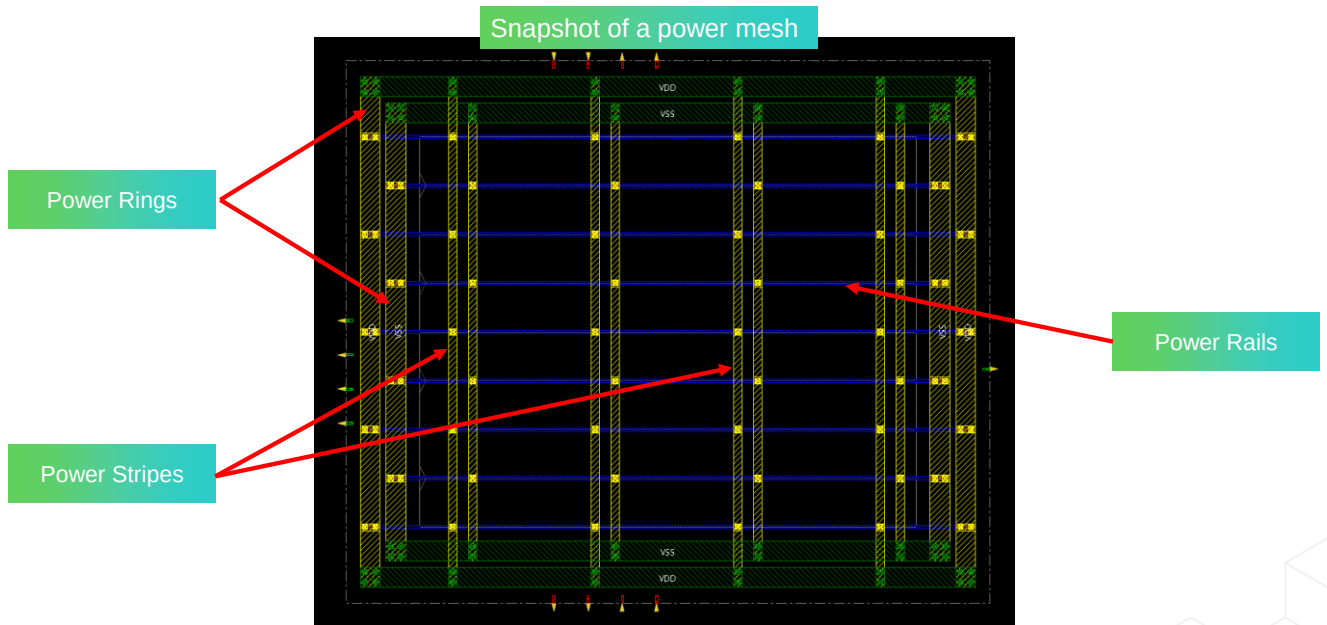
Block pins connect the power and ground pins of the blocks to rings and stripes.

Pad pins connect the power and ground pins of the power pads into the core power ring.

Pad rings create pad rings.

Follow pins connect the power and ground pins of the standard cells along the core rows. The end connections are based on the options you set using the *Advanced* tab of this form. By default, the end connections are at the first ring or stripe outside the row.

Example: Power Planning and Power Routing



222 © Cadence Design Systems, Inc. All rights reserved.



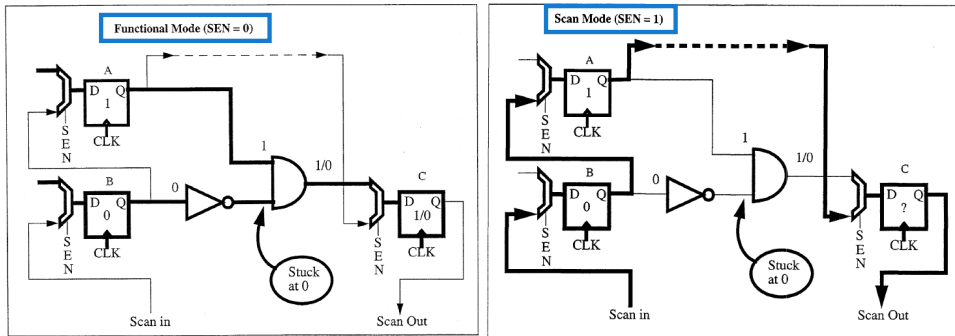
This page does not contain notes.

What Are Scan Chains?



Scan chains consist of a shift register of scan flip-flops, the purpose of which is to make the design controllable and observable.

When the SEN signal in the example below is asserted (set to 1), every flip-flop in the design becomes part of a long shift register, and the expected value that is shifted out is compared to the actual value.



223 © Cadence Design Systems, Inc. All rights reserved.

cadence

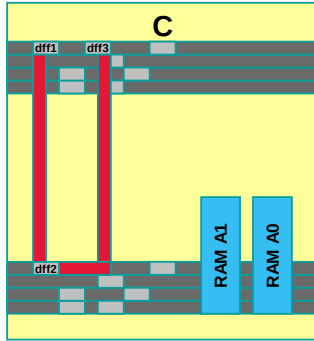
This page does not contain notes.

What Is Scan Reordering?

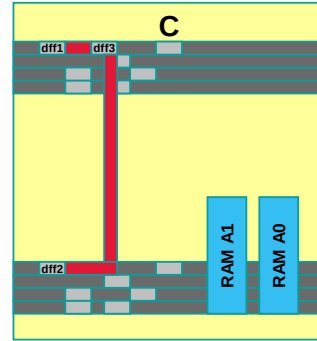


Scan reordering is the process of reordering scan chains to save routing resources.

Routing Before Scan Reorder



Routing After Scan Reorder



This page does not contain notes.

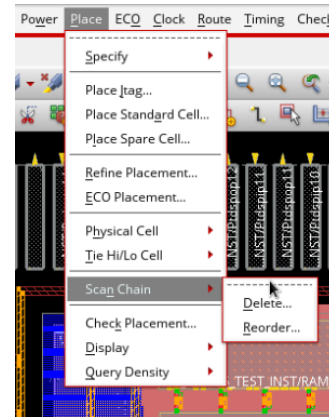
Deleting and Reordering Scan Chains



Place → Scan Chain → Delete

Place → Scan Chain → Reorder

- Scan reordering is the process of reconnecting the elements in scan chains to optimize their routing.
- This process improves timing and congestion.
- Scan insertion is performed in the synthesis tool, whereas scan reordering is performed during placement or as a separate step during implementation.



This page does not contain notes.

Loading Scan Chain Information



Scan chains are commonly loaded by reading in a *scanDEF* file.

Loading in a scan chain identifies the start and end points of a scan chain, which then allows you to run placement while ignoring scan chain connections within the scan chain, which in turn reduces routing congestion and wire lengths.

- Scan cells are identified in the timing library (*.lib*). If scan cells are not in the timing library, use this command to load information:

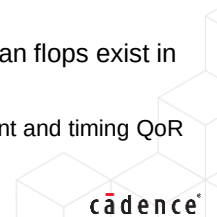
```
set_scan_cell <cellName> [-scan_in pinName] [-scan_out pinName]
[-scan_clock pinName] [-scan_enable pinName]
```

- If you do not have a *scanDEF* file, you can instead load the scan chain with a Tcl file containing scan chain information:

```
create_scan_chain -name <scanChainName> -start <startPinName>
-stop <stopPinName>
```

Scan chain definitions need to be read into Innovus either with a scan DEF or Tcl commands if scan flops exist in the netlist; otherwise, an error message is generated:

****ERROR: (IMPSP-9099):** Scan chains exist in this design but are not defined for xx.yy% flops. Placement and timing QoR can be severely impacted in this case!



This page does not contain notes.

Example: scanDEF File

```
...  
NAMECASESENSITIVE ON ;  
DIVIDERCHAR "/" ;  
BUSBITCHARS "[]" ;  
DESIGN test ;
```

```
SCANCHAINS 2 ;
```

```
- AutoChain_1_seg1_clk1_rising  
+ START PIN DFT_sdi_1  
+ FLOATING  
  out1_reg_0 ( IN SI ) ( OUT Q )  
  ...  
  out1_reg_74 ( IN SI ) ( OUT Q )  
+ STOP PIN DFT_sdo_1 ;
```

```
- AutoChain_2_seg1_clk1_falling  
+ START PIN DFT_sdi_2  
+ FLOATING  
  out2_reg[0] ( IN SI ) ( OUT QN )  
  ...  
  out2_reg[99] ( IN SI ) ( OUT QN )  
+ STOP PIN DFT_sdo_2 ;
```

Two scan chains with
START and STOP
keywords that denote the
starting and ending
points of the chain.

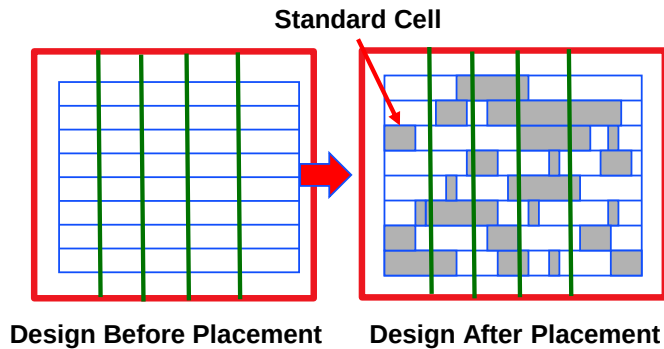
This page does not contain notes.

What Is Placement?



Placement is the process of placing the standard cells and blocks in a floorplanned design.

- When a design is read in, the tool creates rows for the standard cells to be placed into by the placer.
- A row is a multiple of a SITE that is defined in the LEF file.
- There can be several types of rows, for example, standard cell rows and I/O rows.



Sites in the LEF File

```
SITE coreSite
  SYMMETRY y ;
  CLASS core ;
  SIZE 0.660 BY 5.040 ;
END coreSite

SITE pad
  SYMMETRY x y r90 ;
  CLASS pad ;
  SIZE 0.100 BY 235.000 ;
END pad

SITE corner
  SYMMETRY x y r90 ;
  CLASS pad ;
  SIZE 235.000 BY 235.000 ;
END corner
```

This page does not contain notes.

What Is Optimization?



Optimization is the process of iterating through a design such that it meets timing, area, and power specifications.

- In general, optimization can be broken down into the following areas:
 - Timing
 - Signal integrity
 - Power
 - Area
- The design has to satisfy these multiple design objectives.



This page does not contain notes.

Optimization Operations

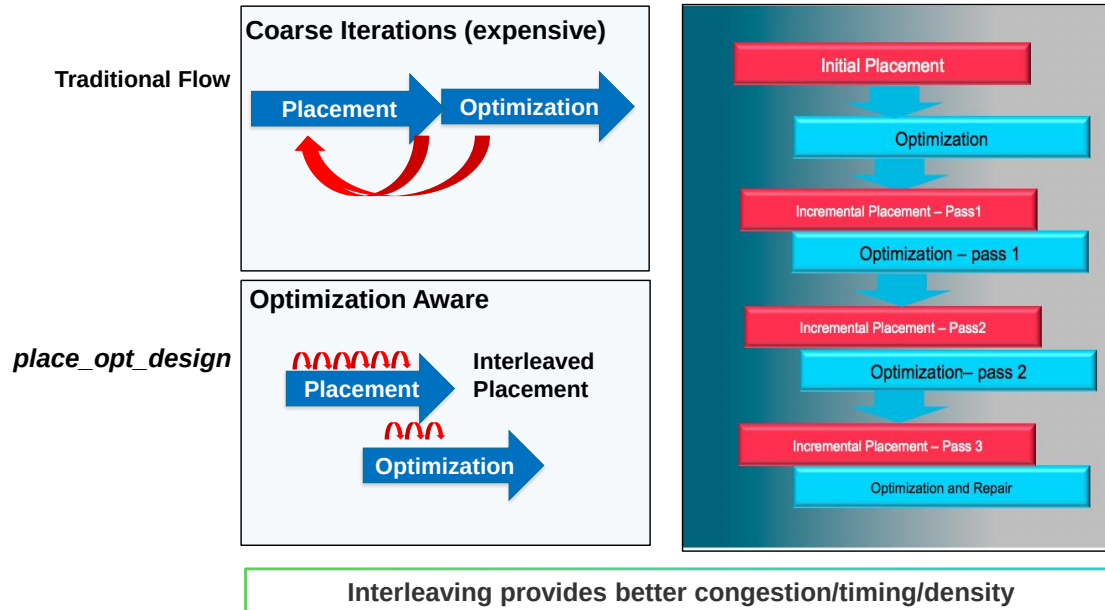
Depending on the stage of the design, optimization can include the following operations:

- Adding buffers
- Resizing gates
- Restructuring the netlist
- Remapping logic
- Swapping pins
- Deleting buffers
- Moving instances
- Applying useful skew
- Layer optimization
- Track optimization



This page does not contain notes.

Flowchart: Placement and Optimization Interleaving with *place_opt_design*



231 © Cadence Design Systems, Inc. All rights reserved.

cadence

GigaPlace™ ensures better interleaving between placement and optimization so that placement is aware of timing and congestion critical areas.

What Is Clock Tree Synthesis?



Clock Tree Synthesis is the process of inserting buffers in the clock path, with the goal of minimizing clock skew and latency to optimize timing.

Clock paths are one of the following:

1. Ideal
Equal delay from clock source to sinks.
2. Propagated
 - a. Traditional Skew Balanced
Delay from the clock source to sinks equals within some tolerance.
 - b. Paths with Useful Skew
 - Clock path delays adjusted to assist timing slack optimization.
 - Strict balancing is not required, and the skew can be exploited to save clock area and power

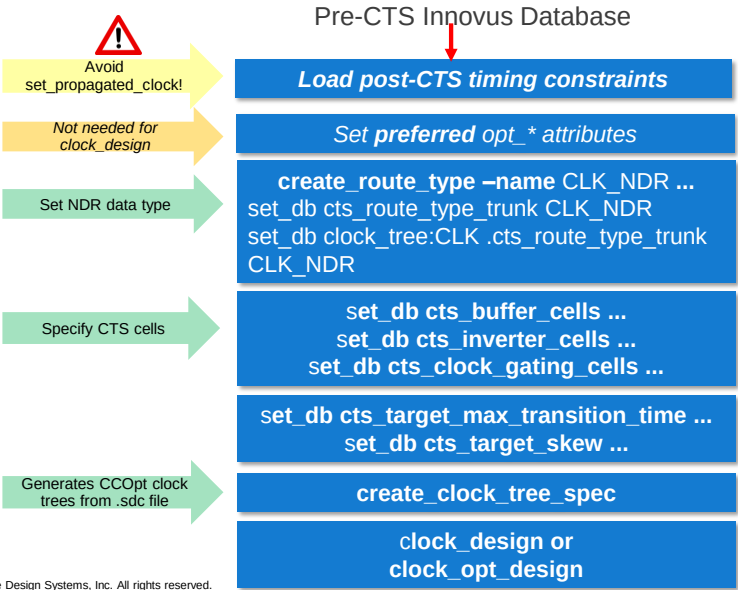


This page does not contain notes.

Setting Up CCOpt Properties and Running the Command Flow



The flow describes how to set properties and run commands for CCOpt.



Note

OCV or SOCV setup should be configured at the start of your flow.

Otherwise, latencies will not be calculated correctly by CCOpt.



This page does not contain notes.

Setting Up CCOpt Properties and Running the Command Flow (continued)

Attributes may be root attributes, clock attributes, lib_cell attributes, etc.

```
innovus> help cts_buffer_cells
      cts_buffer_cells(clock_tree) ← clock tree attribute
      # Description
      cts_buffer_cells(root) ← root attribute
      # Description
```

For CTS, a root attribute will be used unless a specific clock/skew group attribute is set.

Examples

Root Attributes	Clock/Skew Group Specific Attributes
set_db cts_route_type_trunk CLK_NDR	set_db clock_tree:CLK .cts_route_type_trunk CLK_NDR
set_db cts_inverter_cells {invA invB}	set_db clock_tree:CLK .cts_inverter_cells {invA invB}
set_db cts_target_max_transition_time 50	set_db clock_tree:CLK .cts_target_max_transition_time 50
set_db cts_target_skew 200	set_db skew_group:CLK/func .cts_target_skew 200



This page does not contain notes.

Setting Up CCOpt Properties and Running the Command Flow (continued)

As a part of the clock tree synthesis flow, the clock tree transitions from ideal to propagated.

Clock Tree Synthesis:

- Adds buffering and sizes/places clock cells for drive and delay

After the insertion of the clock tree, timing is with propagated clocks:

- SOCV/AOCV/derates impact timing from non-common clock path
- Clock gate enable timing is no longer ideal
- Interclock timing depends on achievable insertion delays
- Clock generator control logic timing is no longer ideal
- Single CTS unbufferable net can impact the entire design timing

The Clock Tree Synthesis Tool includes more steps than just buffering clock nets.

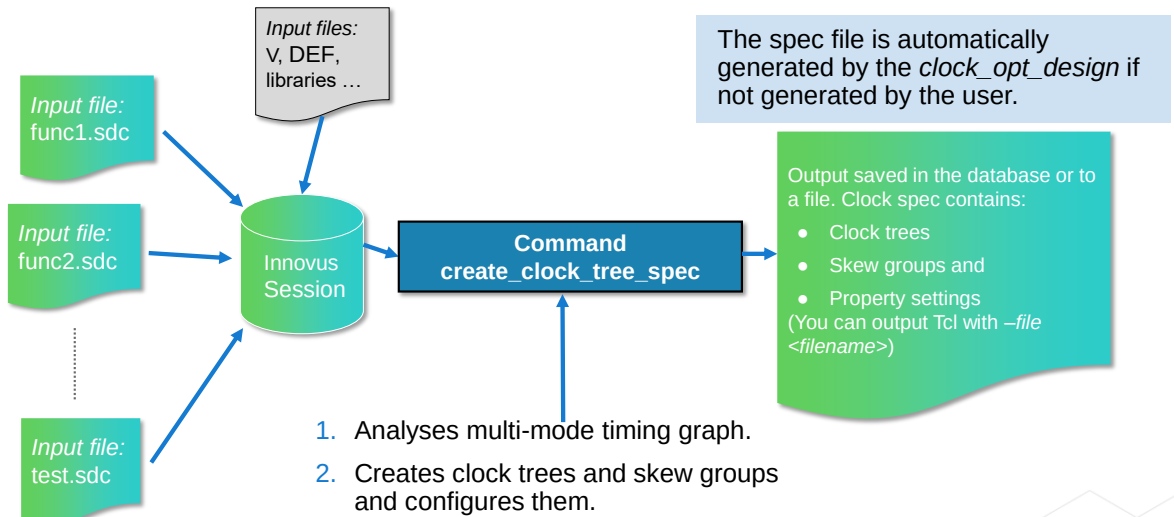


This page does not contain notes.

Automatically Generating a CTS Spec File



```
create_clock_tree_spec [-help] [{-out_file <filename>} ][ -views  
<analysis_view>+] [ -out_file <filename> ]...
```



236 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

Default CTS + post-CTS
optimization +
optional useful skew



© Cadence Design Systems, Inc. All rights reserved.

Debugging the Clock Tree with the Clock Tree Debugger

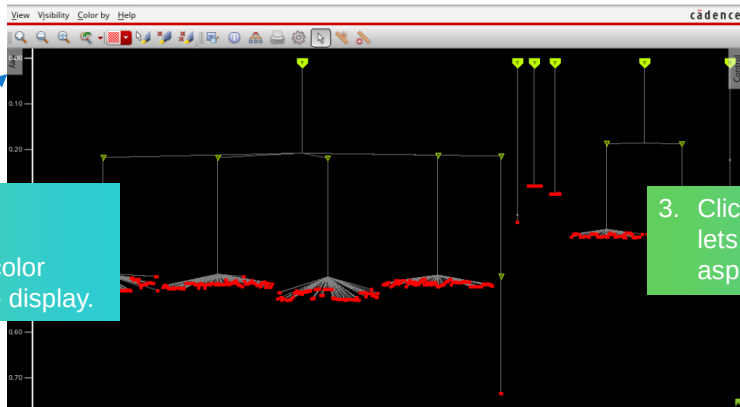


Bring up the clock tree debugger to view the clock trees and skew groups and to cross probe between the debugger window, the physical view, the timing report and the schematic.

1. Click Clock—CCOpt Clock Tree Debugger.

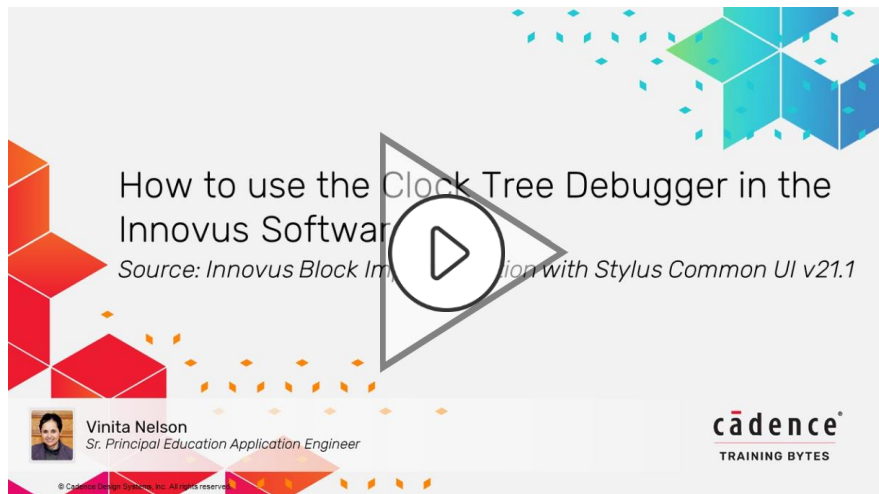
2. Click the Key panel that contains descriptions of the color coded objects in the display.

3. Clicking the Control panel lets you highlight various aspects of the clock tree.



This page does not contain notes.

Demo: Clock Tree Debugger Features



239 © Cadence Design Systems, Inc. All rights reserved.



Video Play Time: 3.46 minutes

Click the Play button to start the video.

What Is Routing?

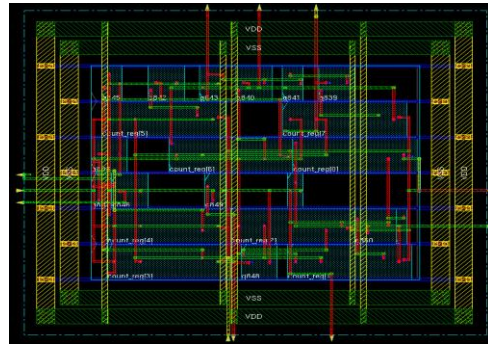


Detail routing is the process of connecting the cells and macros in the design on metal layers specified in the technology LEF file that is generally provided by the foundry so that the routes are DRC correct and timing and signal-integrity aware.

Example: Routing Rules defined in the LEF file

```
LAYER Metall  
  TYPE ROUTING ;  
  WIDTH 0.230 ;  
  SPACING 0.230 ;  
  SPACING 0.6 RANGE 10.0 100000.0 ;  
  PITCH 0.560 ;  
  DIRECTION HORIZONTAL ;
```

Routed Design

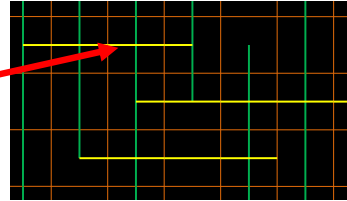
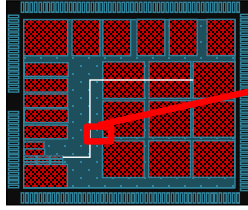


This page does not contain notes.

Flowchart: Routing Flow

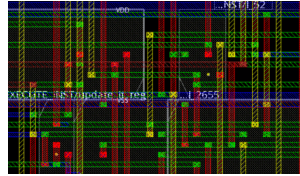
Global Route

Design divided into Gcells and routes
assigned to tracks
Congestion analyzed



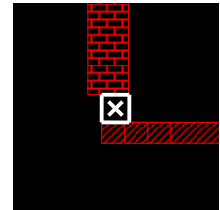
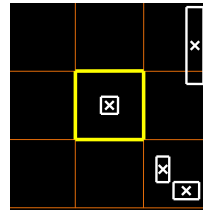
Detail Route

Creates the routes
Applies all DRC rules
Gives priority to critical nets



Search and Repair

Surgical fixes for individual vias and wires
Search area increases by 3x3, 5x5, and 7x7
in Gcell areas
Deletion and reroute as a last resort if no
other solution is found



This page does not contain notes.

Timing and SI-Driven Routing



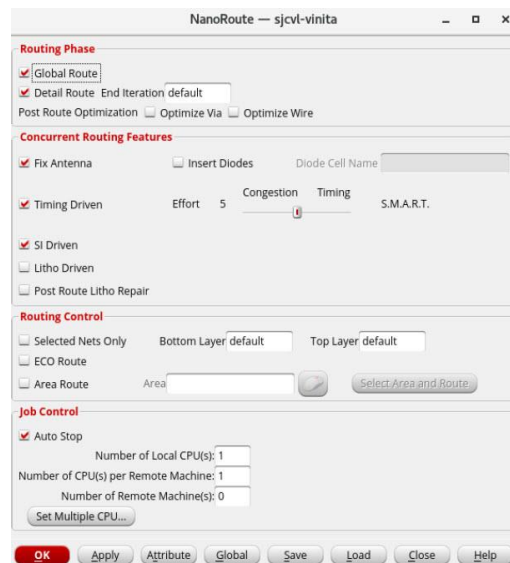
Set NanoRoute™ modes for timing and SI-driven routing guides the router to produce routes with the best Quality of Results (QoR).

To run timing- and SI-driven global and detailed routing set the route mode and run the `route_design` command:

```
set_db route_with_timing_driven true
set_db route_with_si_driven true
route_design or route_opt_design
```

To route selected nets, run the command:

```
set_db route_selected_net_only true
route_design or route_opt_design -route
```



242 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Adding Filler Cells



Place → Physical Cell → Add Filler → Basic (Tab)

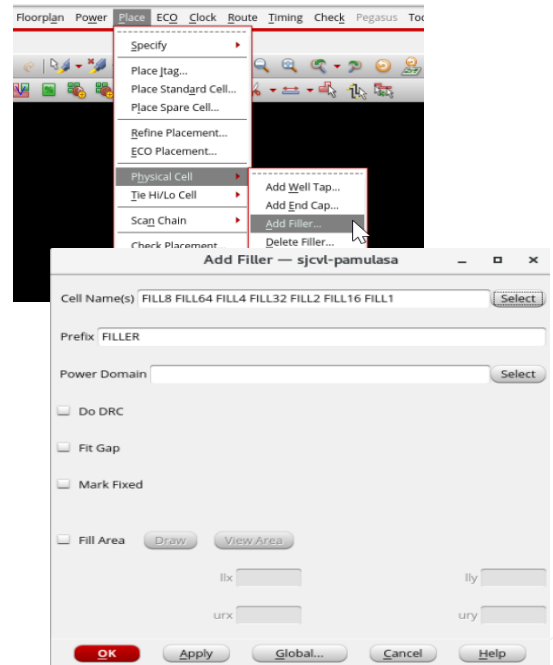
Filler cells are used to fill the gaps between standard cell instances to provide continuity for the power and ground rails and n-wells.

To place filler cells:

- Specify filler cell names
- Add prefix

This command can be used several times in the design flow.

Command
add_fillers



This page does not contain notes.

Extracting RC Data



Timing → Extract RC

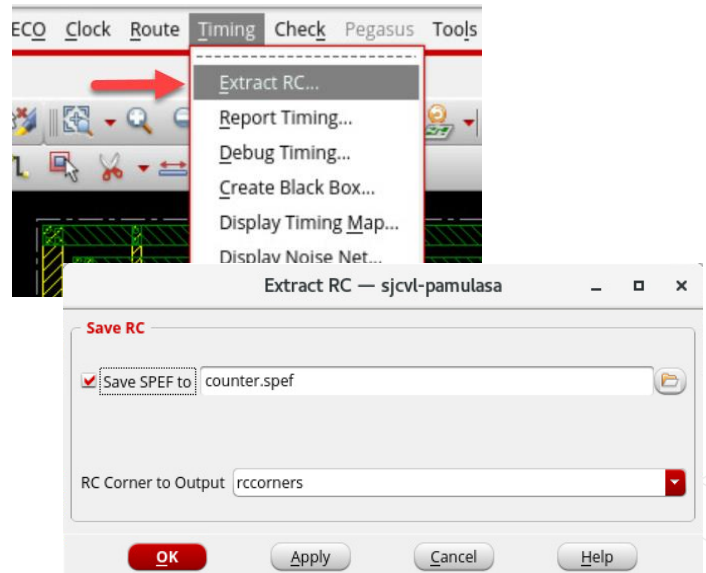
Extraction of Interconnect Data

- SPEF (standard parasitic exchange format)
- Select a corner for extraction

Equivalent Command: `extract_rc`

To delete the extracted parasitics but to maintain the RC extraction attributes that were set, run the command:

`reset_parasitics`



This page does not contain notes.

What Are DRC and LVS?



Design-rule checks (DRCs) are checks that are performed with respect to the rules provided by a foundry for a particular technology and process node.

Layout-vs.-schematic (LVS) violations occur when there is a mismatch in connectivity between what is in the placed and routed design and the Verilog netlist or shorted nets.

DRCs that are flagged as violations need to be fixed or waived before tapeout.

Examples of DRC violations

- Minimum width, minimum spacing, antenna violations.

Example of LVS violation

- An ECO performed on the physical design no longer matches the netlist that was read, resulting in a mismatch.

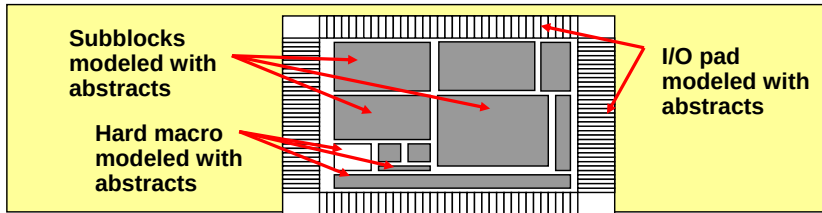


This page does not contain notes.

DRC Checks Using Place and Route Data

Description

All cells are modeled with a LEF (or abstract). No layout available.



Advantages

- Fast
- Small database
- Problems can be debugged and fixed fast

Disadvantages

- The checks are as accurate as the abstracts
- No checks at different hierarchy levels
- Connection to pins could have violations

246 © Cadence Design Systems, Inc. All rights reserved.



Design rule check in the place and route environment. The place-and-route environment is a flat environment, and it does not use a layout. It is mainly used to identify problems in the current level of hierarchy.

Accuracy

- It relies on the accuracy of the abstracts. Therefore, it is wildly inaccurate to identify problems between adjacent cells or routes and cell internal wires. For example, a process rule could impose the same net cut to cut minimum spacing. If the cuts are not modeled as being part of the pin or obstructions, then the router can connect to the pin by dropping a via, and a real violation is created. It can only be identified with the real layout, as the information is unavailable in the place and route environment.
- Normally, layers under *poly* and sometimes *metal1* are not part of the abstracts. Therefore, cell abutment problems can be generated on these layers and are not detected in the place and route environment.

Rule Availability

All process rules are not available in the place and route environment; they are only a subset of them. Therefore, a design can pass the DRC check in the place and route environment and fail in the layout environment. For example, a minimum area can be surrounded by metal (the area of the hole in a donut).

Checking Connectivity

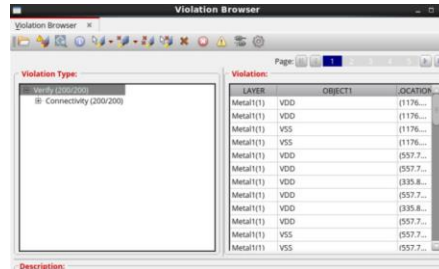
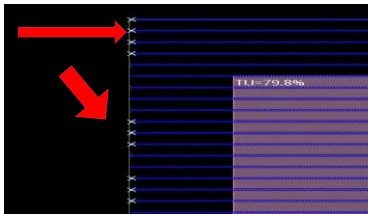


Check → Check Connectivity

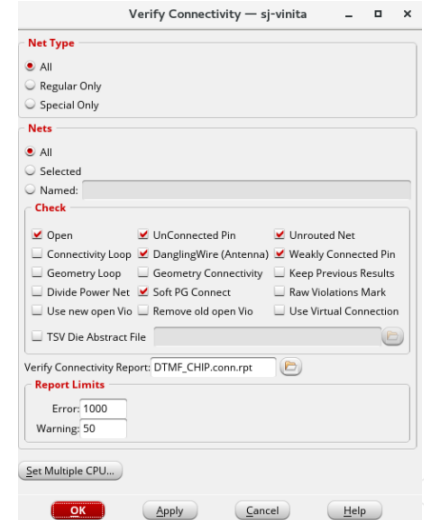
To report open nets, antennas, loops, and partial routing, for all nets or specified nets in your design.

Example

Violation markers generated for open/unconnected rails.



Command
`check_connectivity`



247 © Cadence Design Systems, Inc. All rights reserved.

cadence

Open – Partially routed

Unrouted Net – Not routed

Antenna – Dangling wires

Connectivity Loop – Loops in regular wires

Geometry Loop – If an external router does not use centerline loop and you want to check for that.

Geometry Connectivity – If any part of a wire touches another wire, they are considered connected.

Use Virtual Connection

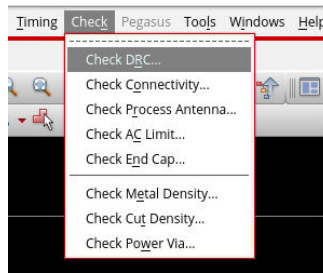
Implies a virtual connection for all bumps and external I/O pins of the same net. Select this option to override default behavior of Verify Connectivity in which bumps and external I/O pins of the same net are not considered to be interconnected. It is useful in flip-chip designs, where the power bumps are connected outside the chip.

Checking DRC

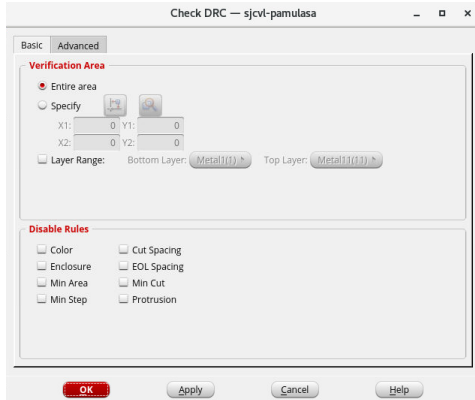


Check → Check DRC

To check for rule violations as specified in the *tech.lef*.

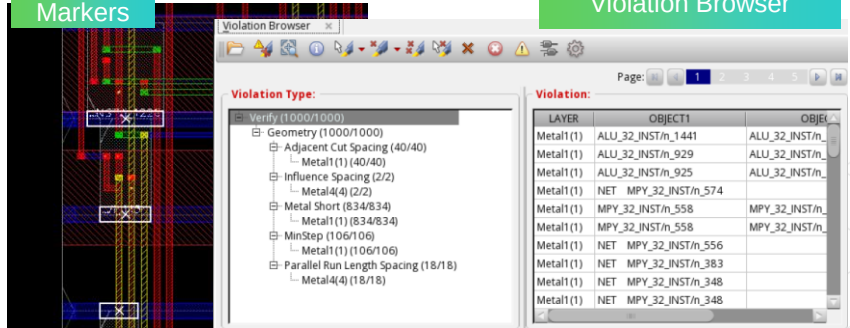


Command
`set_db check_drc*`
`check_drc`



DRC Error
Markers

Error Descriptions in the
Violation Browser



This page does not contain notes.

Setting Up and Running Power and Rail Analysis

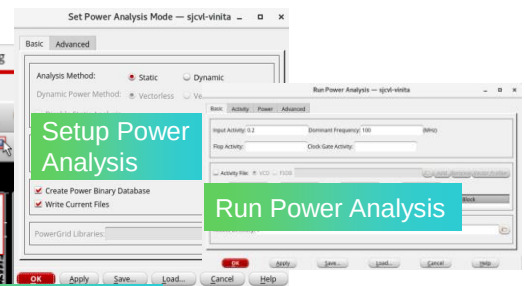
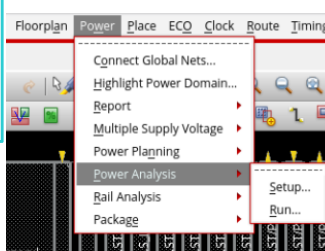


Power → Power Analysis

Power → Rail Analysis

- To generate power consumption reports, set up and run Power Analysis.
- Power Analysis is a prerequisite to Rail Analysis (IR drop analysis).
- To generate a graphical map of the IR drop, run Rail Analysis.

Power Analysis

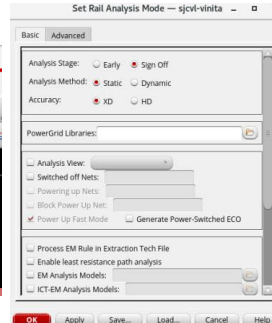
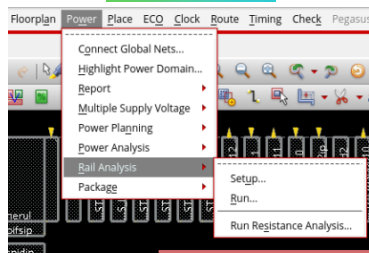


Setup Power Analysis

Run Power Analysis

Setup Rail Analysis

Rail Analysis



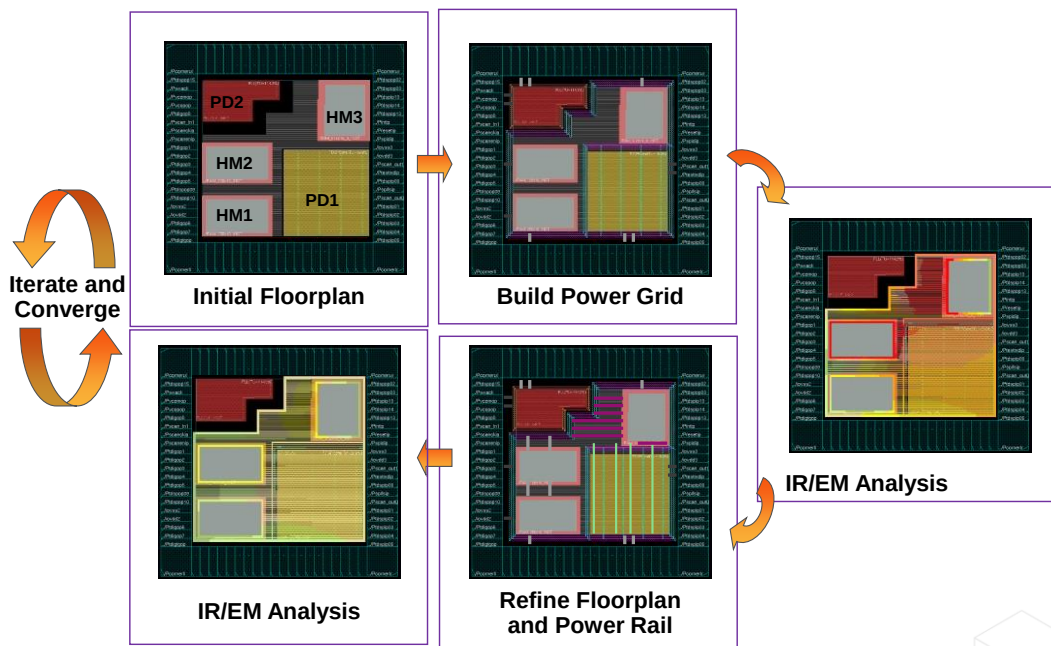
Run Rail Analysis

249 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Early Rail Analysis and Power Planning



250 © Cadence Design Systems, Inc. All rights reserved.

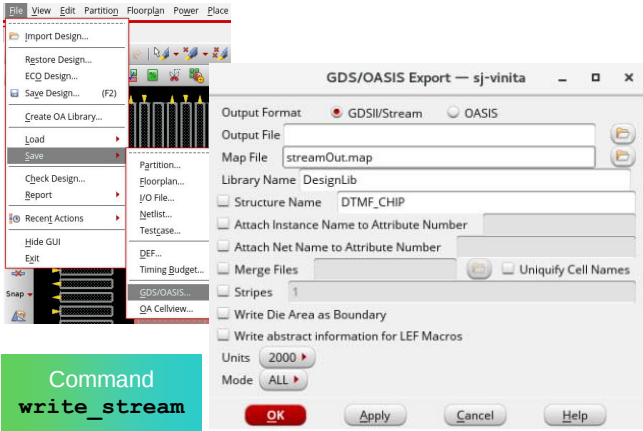


This page does not contain notes.

Saving a GDSII/OASIS File



File → Save → GDS/OASIS



- To write out a GDSII file, you need a GDSII map file that contains the layer name and equivalent layer number.
- The layer number is used by the custom tool (such as Virtuoso® Layout Editor) to interpret the layers in the same way as Innovus Implementation System software.
- In addition to writing out the design in the GDSII file, you can also include the GDSII library containing the cells and the macros in the design.
 - This feature is useful if you are not planning on using a custom tool for the chip-finishing step.

Example Layer Map File Format

Innovus Layer name	Layer Type	Layer Number	Data Type
Meta11	NET	1	0
Meta12	PIN	1	1
Meta13	NET	2	0
Meta14	PIN	2	1

Use the Output Stream File text field to specify the name of the GDSII output file. Click the file folder icon to find the directory and file you want.

Add the .gz extension to the filename to enable compression, such as *GDS_file.gds.gz*.

The Map File field specifies the file for layer mapping between the system and GDSII. Use the file folder icon to find the file you want. If a file is not specified, a default map file is created with the name *streamOut.map*.

If a map file is not specified, a default map file is created with the name *streamOut.map*.

The Library Name field specifies the library that you want to convert to GDSII format. The default name is *DesignLib*.

Quiz



Which of the following commands is used to create a floorplan in Innovus?

- A. Create floorplan
- B. Floorplan_size
- C. create_floorplan
- D. All the above

Answer: C



This page does not contain notes.

Quiz (continued)



During placement, cells and blocks are placed in rows derived from the SITE information in the LEF file.

- A. True
- B. False

Answer: True



This page does not contain notes.

Quiz (continued)



What are the two major goals of clock tree synthesis?

- A. Balance skew
- B. Placing of standard cells
- C. Balance Insertion delay
- D. DRC fixing

Answer: A & C



This page does not contain notes.

Quiz (continued)



SPEF stands for _____

- A. Standard Parasitic Exchange Format
- B. Standard Parameter Extraction File
- C. Standard Parasitic Extraction Format
- D. Standard Perimeter Evaluation Format

Answer: C



This page does not contain notes.

References



Innovus Training Bytes

- [Innovus Training Byte Videos](#)

Innovus One-Stop Page

- [Innovus One-Stop Page Reference](#)

Innovus Full Course Link

- [Innovus Block Implementation with Stylus Common UI](#)

Innovus User Manuals

- [Innovus User Guide 25.1](#)
- [Innovus Product Manuals](#)

Innovus Articles Reference

- [Innovus Articles and Application Notes](#)



This page does not contain notes.

Lab



Lab 8-1 Running the Basic Implementation Flow

257 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.



Module 9

Timing Analysis and Debug

cādence®

This page does not contain notes.

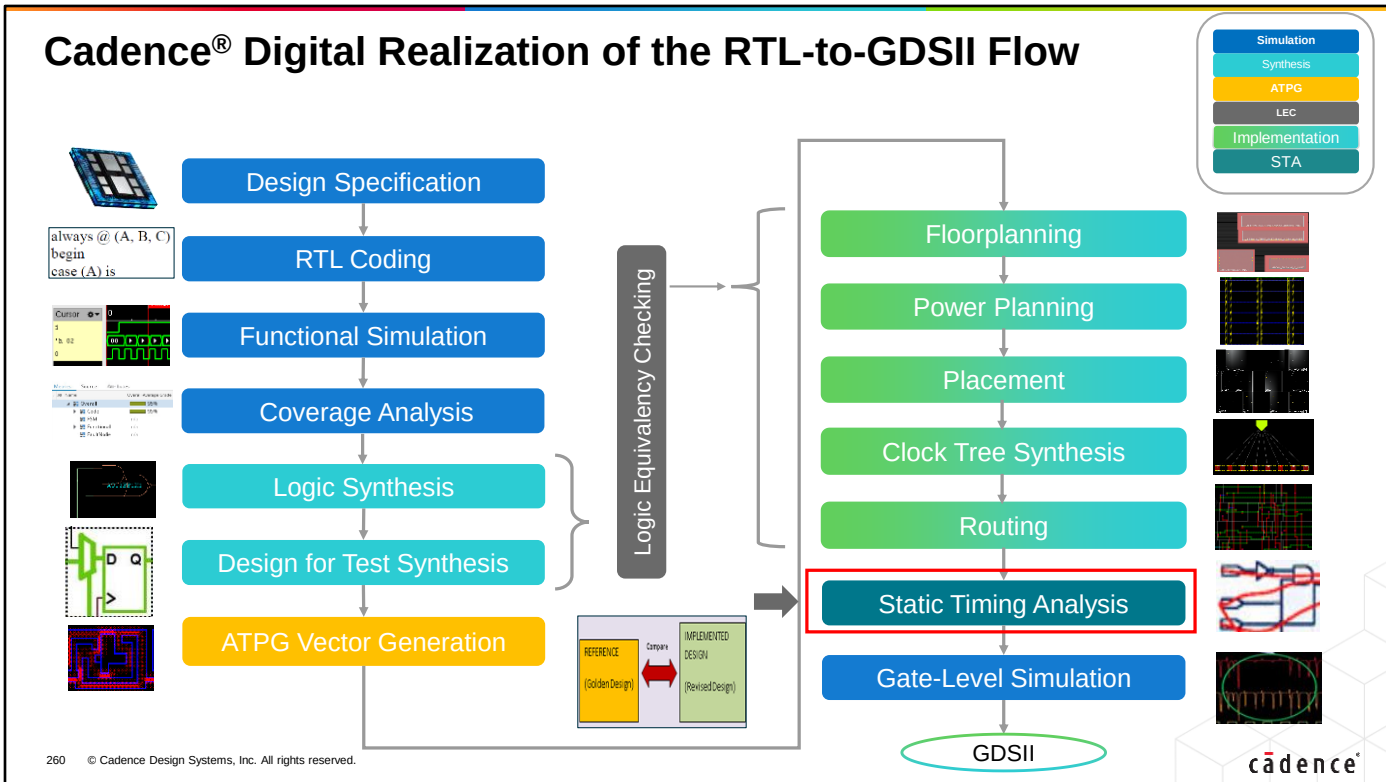
Module Objectives

In this module, you

- Identify where Static Timing Analysis (STA) fits into the flow
- Write parameters for timing information, such as
 - Timing libraries, timing arcs, cell delays and net delays, timing constraints and slew
- Generate timing reports
- Analyze different debugging strategies
- Execute manual timing ECOs



Engineering Change Order (ECO)



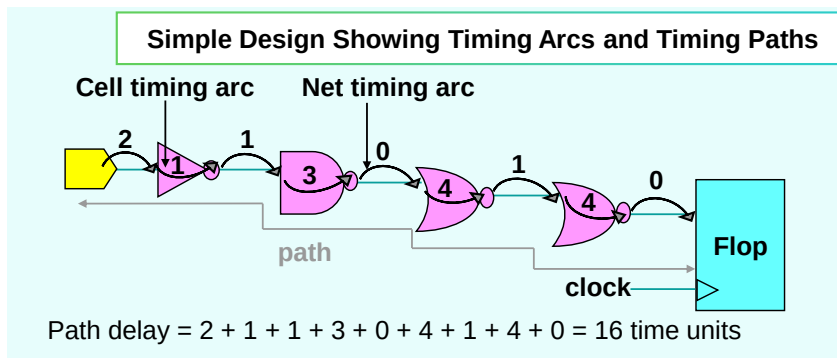
This page does not contain notes.

What Is Static Timing Analysis (STA)?



Static timing analysis is the process of adding the delays in the paths and verifying the path delay against the timing required for the design to work.

This illustration shows the calculation of the path delays.



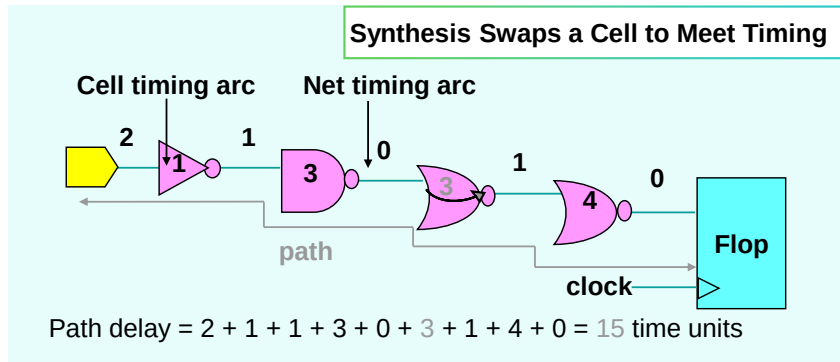
261 © Cadence Design Systems, Inc. All rights reserved.

cadence

- If you are the architect or design engineer, you run a simulation to test the functional aspect of the design. You also typically write the timing constraints for the static timing analysis.
- If you are a synthesis or place-and-route engineer, you run static timing analysis to verify timing and to verify whether the design works at the required speed.

What Is the Purpose of STA?

- STA calculates the path delays for optimization tools. Then, based on the path delays, the optimization tools choose cells from the timing library to create a circuit that meets your timing requirements.
- STA analyzes the timing of a circuit to verify that the circuit works at the specified frequency.



262 © Cadence Design Systems, Inc. All rights reserved.



In addition to timing arc data, timing tools require:

- Clock data

The goal of timing is to restrict the path delay to a single clock period.

- Timing check data

Timing checks define the requirements that the data has to meet before the clock can push it into a flop.

- Start point can be defined as all input ports or clock pins of a sequential element.
- End point can be defined as all output ports or D pin of a sequential element.

- Timing arcs in cells
- Timing arcs of nets



- Clock data

- Timing check data

Timing checks define the requirements that the data has to meet before the clock can push it into a flop.

What Are Timing Libraries?



Timing libraries provide the delays of the:

- Cells and
- Interconnects (nets)

The STA tool uses the delays of the nets and cells to calculate the path delays and verify the delays against the requirements.

The Liberty format (*.lib*) is the standard timing library in the industry.

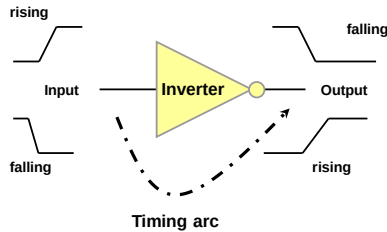


This page does not contain notes.

What Are Timing Arcs?



A timing arc is an imaginary arc that represents a single causal relationship. If a change in an input causes a change in the output, it is known as a causal relationship. Timing arcs provide a simple understanding of the structure and functionality of a gate.



Rising and falling timing arc delays across a gate is not always symmetric and are listed separately in a library.



Timing arcs are the building blocks of static timing analysis, and they provide a simple understanding of the structure and functionality of a cell or a net. Understanding timing arcs is critically important to determining the path delays correctly. Every timing arc has a causal relationship. A signal transition on an input causes a transition on the output.

Timing Arcs: Cell and Net Delay



Delays encountered in digital circuitry are composed of two main components: cell delay and net delay.

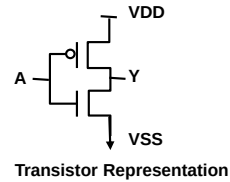
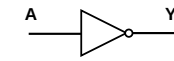
Each stage delay (Cell delay + Net delay) represents the time required to propagate a signal from the input of one gate to the input of the next.

Cell Delay

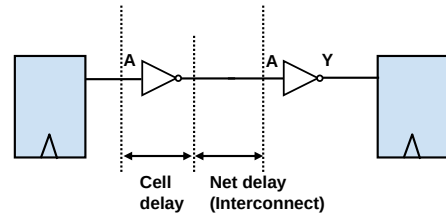
- Transistors within a cell take a certain amount of time to switch. Therefore, a change to the input of a cell takes time to cause a change to the output.

Net Delay

- Net delay is the delay between the time a signal is first applied to a net and the time it takes to reach other devices connected to that net.



Transistor Representation



Depending on the process technology, different physical elements have different levels of contribution.

Historically with process technologies above 90 nanometers, cell delay has been the major limiting factor in timing closure.

However, at process technologies below 90 nanometers, net delays dominate the cell delays.

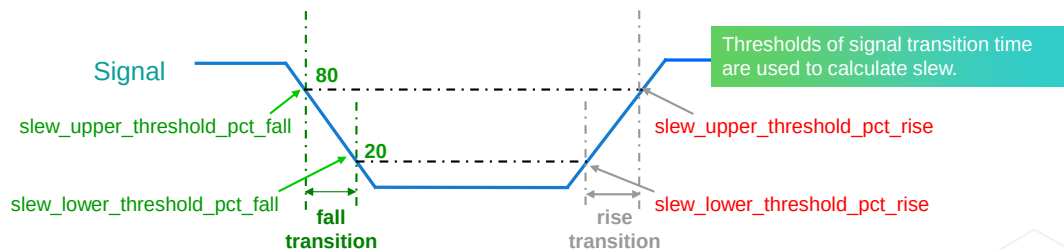
Signal Transition: Rise and Fall Transition Times



Transition time is the time that a signal takes to change states from low to high or high to low. STA tools use the slew threshold values from the library to calculate the transition times. Rise and fall transition times are the properties of a timing arc and are typically measured in nanoseconds.

STA tools calculate the rise time from the *slew_lower_threshold_pct_rise* of **20%** and the *slew_upper_threshold_pct_rise* of **80%**. Similarly, STA tools use the fall threshold values for fall time.

The graph shows **transition time** (slew) being measured from 80% to 20% of the falling signal, for the fall **transition time** (aka fall slew), and from 20% to 80% for the rise **transition time** (aka rise slew).



267 © Cadence Design Systems, Inc. All rights reserved.

cadence

The threshold values are used to calibrate the delays specified in the library. So, when the thresholds specified in the library do not match the thresholds used, then the STA tools scale the thresholds to calculate the delays and slews.

- Threshold scaling is done for parasitic-based calculations (Steiner and SPF), and changing thresholds affects slew/delay.

Threshold scaling is not done for wire-load-based calculations; delay and slew numbers are straight from the library.

All semiconductor devices take some time to switch between states. The transition time is the time it takes for a signal to rise or fall.

The upper threshold value determines the actual time a device turns on and stays on. The lower threshold value determines when the device turns off and stays off.

In the illustration, the lower thresholds are 20%, and the upper thresholds are 80%. The rise transition is measured from 20% to 80% of the signal, and the fall transition is measured from 80% to 20%.

What Are Timing Constraints?



Timing constraints provide specifications that the design must meet through optimization. STA tools also have their own style of writing constraints, which conform to Tcl syntax.

Typical examples of constraints are:

- Clock constraints
- External constraints
- Power constraints
- Net Delay constraints
- Environmental constraints
- Design rules for manufacturing

This example shows the clock creation SDC constraints:

```
create_clock -period 100 -waveform {0 50} clk
```



This page does not contain notes.

Common SDC Constraints



The Synopsys Design Constraints (SDC) format is the standard for writing constraints in the industry.

Operating Conditions

- `set_operating_conditions`

Environmental

- `set_driving_cell`
- `set_load`
- `set_fanout_load`
- `set_input_transition`
- `set_port_fanout_number`

Design Rules

- `set_max_capacitance`
- `set_max_fanout`
- `set_max_transition`

Timing

- `create_clock`
- `create_generated_clock`
- `set_clock_latency`
- `set_clock_transition`
- `set_disable_timing`
- `set_propagated_clock`
- `set_clock_uncertainty`
- `set_input_delay/set_output_delay`

Exceptions

- `set_false_path`
- `set_max_delay`
- `set_multicycle_path`

Power

- `set_max_dynamic_power`
- `set_max_leakage_power`

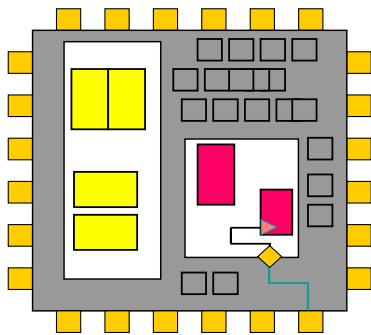


This page does not contain notes.

Design Objects

You apply certain constraints to design objects to affect different parts of the design.

The table shows several design objects and the commands to get a list of these objects.



Object	Command	Description
Design	current_design	Design is a container for cells or is the entire circuit.
Cell or Block	get_cells	Cell is an instance of a design or is a library component.
Port	get_ports all_inputs all_outputs	A port is a signal entry point or exit point to a design.
Pin	get_pins	A pin is a hierarchical port of a design, port of an instance, or a port of a library cell.
Clock	get_clocks all_clocks	A clock is a port or a pin that drives sequential cells.
Net	get_nets	A net is an interconnect between cell pins and design ports.



This page does not contain notes.

What Are the Inputs and Outputs for Tempus?



Signoff Minimum Mandatory Inputs

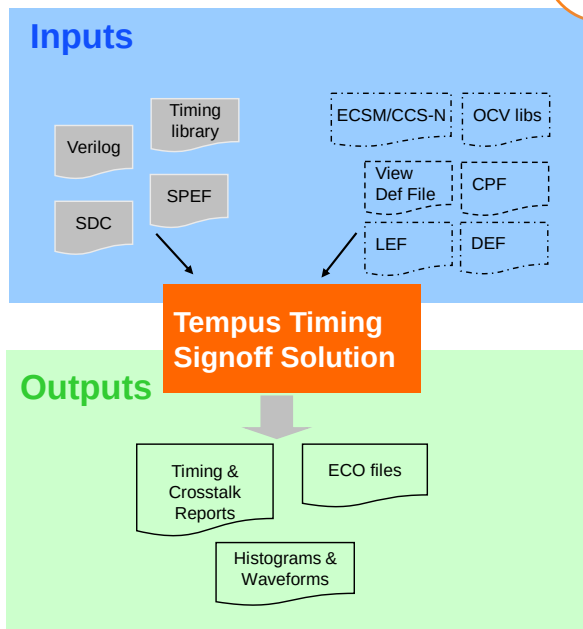
- Timing libraries (Liberty *.lib* with timing data)
- Verilog
- SDC and SPEF

Optional Inputs

- ECSM-Noise, CCS-Noise, or CDB for SI
- CPF/View Definition file for MMMC analysis
- AOCV libraries, LVF for on-chip variation effects
- Physical Data (LEF and DEF)

Outputs

- Timing and Crosstalk Reports
- Histograms and Waveforms
- ECO files



271 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Starting the Tempus Timing System



- Regular STA : `tempus -stylus <-file cmdfile>`
- Distributed STA : `tempus -stylus -distributed <-file cmdfile>`
- Tempus ECO : `tempus -stylus -eco <-file cmdfile>`

To start and stop the GUI, use these commands:

- **gui_show** and **gui_hide**

On starting a Tempus session:

- It checks out the base license as:
 - Tempus L or Tempus XL
- The software captures all the standard output in *tempus.log* and *tempus.logv*, which also shows the timestamps in a sequential manner.
- The commands executed at the command prompt are captured in *tempus.cmd*.

```
Cadence Tempus(TM) Timing Signoff Solution.
Copyright 2018 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:      v18.10-p001.1, built Fri May 25 10:41:01 PDT 2018
Options:      -stylus -file ../scripts/run.tcl
Date:         Fri Jul 6 16:34:05 2018
Host:         noi-sakshin (x86_64 w/Linux 2.6.32-431.11.2.el6.x86_64) (icore*icpu*Intel(R) Xeon(R) CPU E5-2699 v3 @ 2.30GHz 46080KB)
OS:           CentOS release 6.9 (Final)

License:
            tpsxl  Tempus Timing Signoff Solution XL  18.1  checkout succeeded

[07/06 16:32:23] 14s] Sourcing tcl/tk file '../scripts/run.tcl' ...
[07/06 16:32:23] 14s] @tempus > source ../scripts/run.tcl
[07/06 16:32:23] 14s] #@ Begin verbose source ../scripts/run.tcl
[07/06 16:32:23] 14s] @file(run.tcl) 10: set multi_cpu_usage -local cpu 1
[07/06 16:32:23] 14s] @file(run.tcl) 15: set table_style -no frame fix width -nosplit
[07/06 16:32:23] 14s] @file(run.tcl) 20: read_libs ../libs/Liberty/FreePDK45_lib_v1.0_worst.lib
[07/06 16:32:23] 14s] Scheduling timing library file(s) '../libs/Liberty/FreePDK45_lib_v1.0_worst.lib' to be loaded when the init_design command is issued

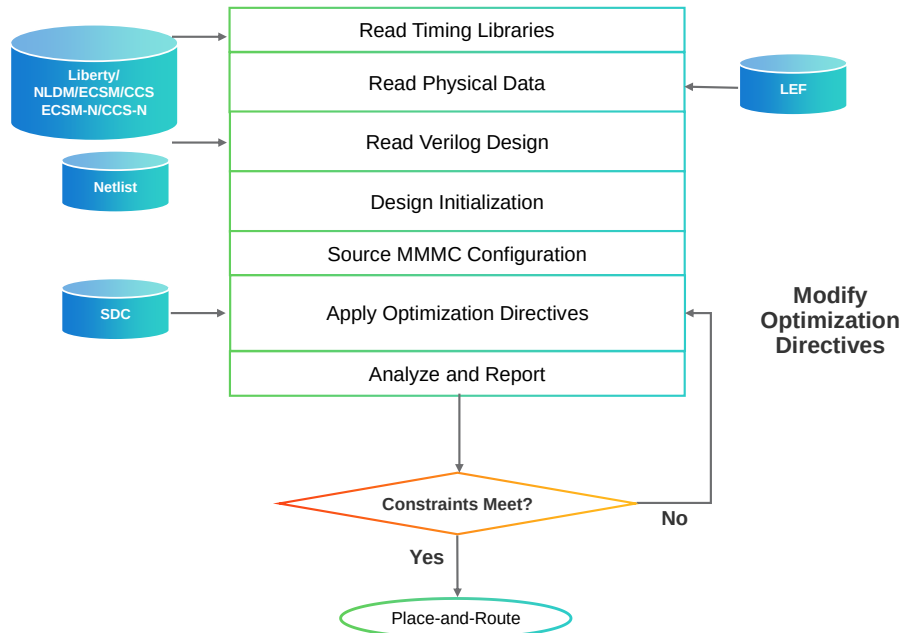
#####
# Tempus Timing Signoff Solution Command Logging File
# Created on Fri Jul 6 16:32:28 2018
#####
#@#(CDS: Tempus Timing Signoff Solution v18.10-p001.1 (64bit) 05/25/2018 10:41 (Linux 2.6.18-194.el5)
set multi_cpu_usage -local cpu 1
set table_style -no frame fix width -nosplit
read_libs ../libs/Liberty/FreePDK45_lib_v1.0_worst.lib
read_libs ../libs/MACRO/LIBERTY/pllclk.lib
read_libs ../libs/MACRO/LIBERTY/ram_256x16A.lib ../libs/MACRO/LIBERTY/ram_512x16A.lib
read_netlist ../design/ECO_INIT_11_optSetup.enc.dat/dtmf_recvr_core.v.gz -top dtmf_recvr_core
init_design
```

272 © Cadence Design Systems, Inc. All rights reserved.

cadence

- Running the Tempus tool without the *-stylus* option runs the tool in the Legacy mode, which has a different command structure. This course is based entirely on the Stylus Common UI, flow kit and metrics.
- To get help on a command, use **help command_name** and/or **man command_name**.
- Use the **Tab** key for command completion. Use the **↑↓** arrows to cycle through history.

Analysis Flow in Tempus Timing Signoff Solution



273 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

What Is Graph-Based Analysis (GBA)?



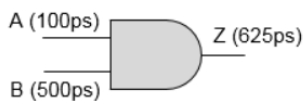
In static timing analysis (STA), Graph-Based Analysis is a pessimistic algorithm for timing which is based on the worst slew propagation (slew merging). It is the default mode of analysis in the implementation stage of the design.

In the GBA mode, the software considers both the worst arrival and the worst slew in a path during timing analysis, even if the worst slew corresponds to an input pin different than the relevant pin for the current path.

Example:

Here, it is assumed that for any input slew, the output slew is 25% more than the input slew. If the slew at B is 500ps, then the slew at Z is 625ps.

In GBA, for calculating delay and slew propagation through this AND gate, the worse input slew (through B) is always considered, irrespective of the fact that the path is through pin A or B.



This page does not contain notes.

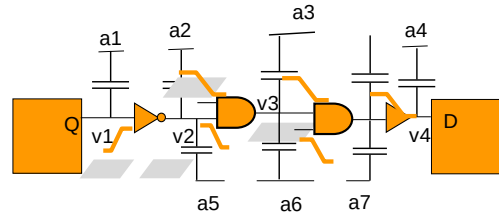
What Is Path-Based Analysis (PBA)?



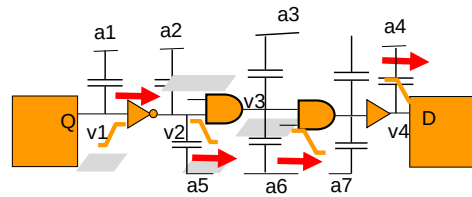
Path-Based Analysis (PBA) involves re-timing the components of a timing path based on the actual slew that is propagated in this path.

- It considers path-based slews and actual arrival times for both base and SI delay calculations.
- It removes the pessimism that is introduced due to slew merging at various nodes in the design when the graph-based analysis is run.
- It is recommended to use during the final stages of timing closure to ensure accuracy.
 - This reduces the area, power dissipation, and ECO cycles of your design but imposes a huge runtime hit.

GBA



PBA



Magnitude depicts the fast  or slow  slew.

This page does not contain notes.

PBA Report File: Example

```
# Command: report_timing -from TDSP_CORE_INST/DECODE_INST/ir_reg[9]/Q -to RESULTS_CONV_INST/r1477_reg[10]/D -retime path_slew_propagation -format {instance arc}
> pba
#####
Path 1: VIOLATED Setup Check with Pin RESULTS_CONV_INST/r1477_reg[10]/CK
Endpoint: RESULTS_CONV_INST/r1477_reg[10]/D (^) checked with trailing edge of 'm_rcc_clk'
Beginpoint: TDSP_CORE_INST/DECODE_INST/ir_reg[9]/Q (v) triggered by leading edge of 'm_clk'
Path Groups: {m_rcc_clk}
Analysis View: Scan Slow RC MAX
Retime Analysis { Path-Slew SI }
Other End Arrival Time      21.373
- Setup                    0.737
+ Phase Shift              0.000
- Uncertainty              0.400
= Required Time            20.235
- Arrival Time             20.351
= Slack Time               -0.116
= Slack Time(original)    -1.345
  Clock Rise Edge          0.000
+ Clock Network Latency (Prop) 5.824
= Beginpoint Arrival Time  5.824
```

Instance	Arc	Net	Cell	Delay	Retime Delay	Slew	Retime Slew
TDSP_CORE_INST/DECODE_INST/ir_reg[9]	CK ^	m_clk_L19 N6	-	-	-	0.145	0.145
TDSP_CORE_INST/DECODE_INST/ir_reg[9]	CK ^ -> Q v	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	SDFS X1	0.694	0.694	0.146	0.146
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	-	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	BUF X16	0.000	0.000	0.146	0.146
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	A v -> Z v	port_address[1]	BUF X16	0.525	0.525	0.475	0.475
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	-	port_address[1]	NOR3 X1	0.037	0.037	0.485	0.485
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	A3 v -> ZN ^	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	NOR3 X1	0.911	0.910	1.138	1.124
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	-	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	NAND2 X1	0.000	0.000	1.138	1.124
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	A2 ^ -> ZN v	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	NAND2 X1	0.365	0.362	0.660	0.383
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	-	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	NOR4 X2	0.000	0.000	0.660	0.383
TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	A4 v -> ZN ^	TDSP_CORE_INST/DECODE_INST/FE_OFN186_port_address_1	NOR4 X2	0.893	1.770	2.748	2.740

276 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

What Are On-Chip Variations and Their Sources?



Chip Variations are the intrinsic variability of semiconductors subjected to process variations.

Changes in physical parameters affect electrical parameters, in turn causing delay variations.

- Variation in length, width, and thickness.
- Doping variation.

OCV in STA

- OCV will affect wire and cell delays during STA.
- OCV also affects clock and data paths differently.
- OCV will increase the pessimism in the design.
- OCV is the reason for location and depth-based variations.

How to model on-chip variations!!!



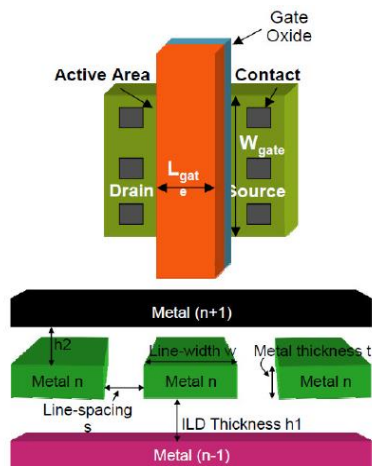
OCV, AOCV and SOCV !!!

Key Device parameters:

- L_{gate} – channel length
- V_T – threshold voltage
- T_{ox} – oxide thickness
- W_{gate} – Channel Width

Key Interconnect Parameters

- Metal thickness – t
- Dielectric thickness – h_1, h_2
- Metal line width (or line spacing) – w or s



277 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

What Is Advanced OCV (AOCV)?

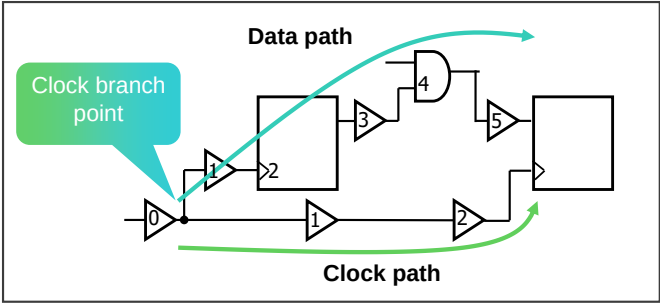


AOCV is a design approach where derates applied to the cells vary with path depth or distance or both.

The shortcoming of OCV is that timing closure gets difficult due to extra pessimism added with fixed derates for the cells.

“Some stages will be faster, others slower. So, the more stages you have, the more it averages out.”

Here, it shows the derate table for a buffer cell based on the depth along the timing graph.



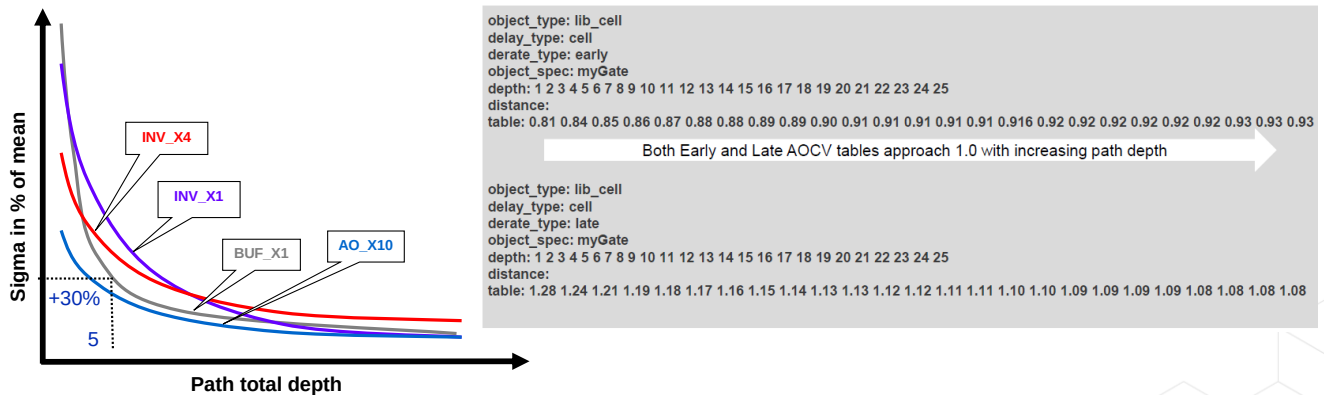
Depth	1	3	5	7	10	15	20	30
Late derate	1.6	1.4	1.3	1.25	1.21	1.17	1.12	1.1
Early derate	0.5	0.6	0.7	0.8	0.88	0.89	0.91	0.95



AOCV reduces the level of derating for each stage on the basis that each successive stage will cancel out the variation.

AOCV: Calculation

In AOCV mode, the software uses derating libraries where an AOCV factor is chosen so that when multiplied by the lib nominal delay value, you get close to the **mean $\pm$ 3 sigma** value for that many stages in the path. As the number of stages increases, the sigma value relative to the mean decreases on the order $1/\sqrt{n}$, and it decreases the AOCV derate value.



279 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

What Is Statistical OCV (SOCV)?



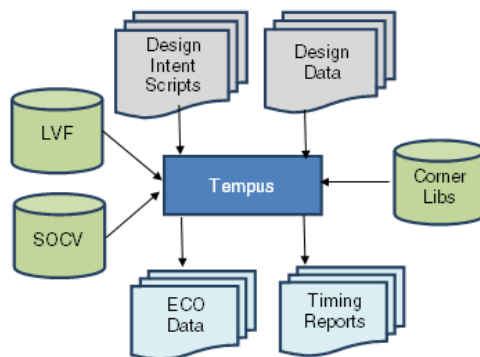
SOCV is a variation modeling technique that computes the impact of local process variations on the delay and slew of each instance in the design at a given global variation corner. SOCV propagates the sigma of arrival and required times through the timing graph and then computes statistical characteristics of slack at all timing pins.

SOCV solves shortcomings of the AOCV approach:

- Inefficiency at ultralow voltage operation ($V_{\text{operation}} \sim V_{\text{threshold}}$)
- Cell timing dependency on slews and loads

SOCV analysis requires variation libraries in the form of a Cadence® SOCV library or as a Liberty Variation Format (LVF) library.

- LVF includes arc-level absolute variations of cell delays, output transitions, and timing checks as functions of input slew and load.

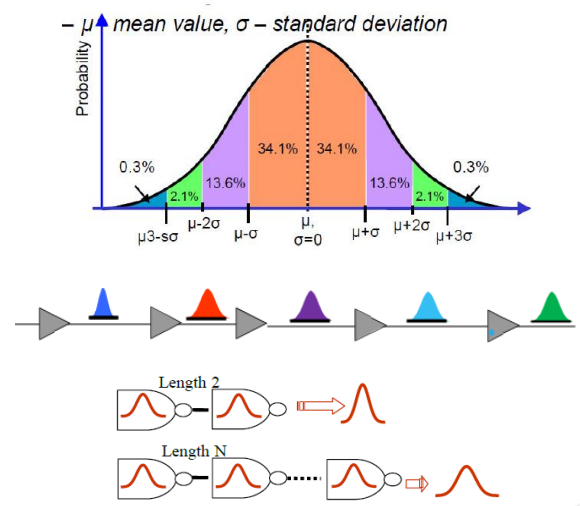


This page does not contain notes.

Liberty Variation Format (LVF)

```
cell (cell_name) {
  ocv_derate_distance_group: ocv_derate_group_name;
  ...
  pin | bus | bundle (name) {
    direction: input | output;
    timing() {
      ...
      ocv_sigma_cell_rise(delay_lu_template_name){
        sigma_type: early | late | early_and_late;
        index_1 ("float, ..., float");
        index_2 ("float, ..., float");
        values ( "float, ..., float", \
          ..., \
            "float, ..., float");
      }

      ocv_sigma_cell_fall(delay_lu_template_name){
        sigma_type: early | late | early_and_late;
        index_1 ("float, ..., float");
        index_2 ("float, ..., float");
        values ( "float, ..., float", \
          ..., \
            "float, ..., float");
      }
      ...
    } /* end of timing */
    ...
  } /* end of pin */
  ...
}
```



(Statistical calculations happen during path tracing.)

cadence

This page does not contain notes.

Quiz



For AOCV analysis, the setup check delay values are calculated from (select the best choice):

- A. A combination of the library delays based on variable derating factors which are a function of the distance of the cell and logic depth.
- B. Only the max library, if that is the only one read.
- C. A combination of the library delays based on fixed derating factors.
- D. Always the fast library.

Answer: A



This page does not contain notes.

Quiz (continued)



Which of the following command used to open the GUI in tempus?

- A. gui_open
- B. Show_gui
- C. gui_show
- D. gui show

Answer: C



This page does not contain notes.

Quiz (continued)



Graph-Based Analysis is a pessimistic algorithm for timing that is based on the worst slew propagation.

- A. True
- B. False

Answer : A



Path-Based Analysis (PBA) involves re-timing the components of a timing path based on the worst slew.

- A. True
- B. False

Answer: B

This page does not contain notes.



Submodule 9-1

Generating Reports

cādence®

This page does not contain notes.

Submodule Objective

In this submodule, you

- Generate and analyze timing reports



This page does not contain notes.

Report Timing



```
report_timing [-check_type {setup|hold|clock_gating_setup|..}] \
[-retime {aocv|ssta|path_slew_propagation|..}] \
[-late| -early] [-through pin_list] [-to pin_list] [-from pin_list]
```

It generates a timing report that provides information about the various paths in the design.

The timing report contains this information:

- The slack times of the arriving signal at the end node and the associated transitions.
- The start/end nodes of each path.
- The signal-required times and the actual signal arrival times.
- Summary of propagated vs. ideal status of launching and capturing clocks.
- Any phase shifts/CPPR values applied to timing check.

```
#####
# Generated by: Cadence Tempus 18.10-b016.1
# OS: Linux x86_64(Host ID noi-sakshin)
# Generated on: Thu May 10 12:50:10 2018
# Design: super_filter
# Command: report_timing
#####
AAE DB initialization (MEM=797.918 CPU=0:00:00.1 REAL=0:00:00.0)
#####
# Design Name: super_filter
# Design Mode: 65nm
# Analysis Mode: MMCM OCV
# Parasitics Mode: SPEF/RCDB
# Signoff Settings: SI Off
#####
Using master clock 'ref_clk' for generated clock 'pll_clk' in view 'AV_wc_on'
Start delay calculation (fullDC) (1 TI). (MEM=806.535)
End delay calculation. (MEM=870.844 CPU=0:00:00.8 REAL=0:00:00.0)
End delay calculation (fullDC). (MEM=862.844 CPU=0:00:01.1 REAL=0:00:02.0)
Path 1: MET Setup Check with Pin A01/SUM reg 3 15/CK
Endpoint: A01/SUM_reg_3_15/D (^) checked with leading edge of 'pll_clk'
Beginpoint: bypass (^) triggered by leading edge of 'io_clk'
Path Groups: {pll_clk}
Analysis View: AV_wc_on
Other End Arrival Time 0.000
- Setup 0.241
+ Phase Shift 8.000
= Required Time 7.759
- Arrival Time 5.803
= Slack Time 1.956
Clock Rise Edge 0.000
+ Input Delay 0.750
= Beginpoint Arrival Time 0.750
#####
```

287 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Sample Report: *report_timing*

```
#####
# Command: report_timing
#####

Path 1: VIOLATED Setup Check with Pin RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
Endpoint: RAM_256x16_TEST_INST/RAM_256x16_INST/D[15] (v) checked with leading edge of 'm_dsram_clk'
Beginpoint: SPI_INST/dout_reg[7]/QN (^) triggered by leading edge of 'm_clk'_
Path Groups: {m_dsram_clk}
Other End Arrival Time          0.000
- Setup                         0.292
+ Phase Shift                   4.000
= Required Time                 3.708
- Arrival Time                 4.667
= Slack Time                    -0.959
  Clock Rise Edge              0.000
  + Clock Network Latency (Prop) 0.337
  = Beginpoint Arrival Time     0.337
```

Instance	Arc	Cell	Delay	Arrival Time	Required Time
SPI_INST/dout_reg[7]	CK ^	-	-	0.337	-0.622
SPI_INST/dout_reg[7]	CK ^ -> QN ^	SDDFHQNX4MTR	0.290	0.627	-0.332
TDSP_CORE_INST0/buf_dummy_314	A ^ -> Y ^	BUFGX2MTR	1.058	1.685	0.726
ULAW_LIN_CONV_INST/FE_RC_162_0	A ^ -> Y v	INVXLMTR	1.770	3.455	2.496
ULAW_LIN_CONV_INST/FE_RC_161_0	A1N v -> Y v	OAI2B2X8MTR	0.350	3.805	2.846
DATA_SAMPLE_MUX_INST/g2	B v -> Y ^	NAND2BX2MTR	0.176	3.980	3.021
DATA_SAMPLE_MUX_INST/g279	B0 ^ -> Y v	OAI2B1X2MTR	0.470	4.451	3.491
RAM_256x16_TEST_INST/FE_RC_76_0	A v -> Y v	CLKBUFX16MTR	0.213	4.664	3.705
RAM_256x16_TEST_INST/RAM_256x16_INST	D[15] v	sram_sp_metro	0.003	4.667	3.708

This page does not contain notes.

Reporting Critical Instances



```
report_critical_insts [-cost_type <string>] [-estimate] [-
include_sequential] [-max_insts <integer>] [-max_slack <double>] [-out_file
<string>] [> <file>] [>> <file>]
```

It reports the timing critical instance (bottleneck) information.

The timing report includes the following:

- The total negative slack for each instance.
- The total count of connected start and end points.
- The total count of paths through an instance.

The following command reports the critical instance information, including sequential instances group for 5 worst cost instances:

```
@tempus 3> report_critical_insts -include_sequential -name_length 42 -max_insts 5
#####
# Generated by: Cadence Tempus 22.10-p001.1
# OS: Linux x86_64(host:10 s:jvl-sakshin)
# Generated on: Fri Apr 14 05:19:48 2023
# Design: dtmf_recvr_core
# Command: report_critical_insts -include_sequential -name_length 42 -max_insts 5
#####
Critical nets number = 217.
design dtmf_recvr_core has 9357 instances and 38645 terms.

# -----
# Report Critical Instance:
# Slack: max slack = 0.0
# (only count the paths and end points whose slack is worse than 0.0)
# Cost: worst slack.
# Date: Fri Apr 14 05:19:49 PDT 2023
# -----
# Inst:combinatorial Cell Cost
# -----
DATA_SAMPLE_MUX_INST/g279 OAI2B1X2MTR -1.376
RAM_256x16_TEST_INST/FE_RC_76_0 CLKINX1X1MTR -1.376
TDSP_CORE_MUX_INST/g497 AO22X2MTR -1.376
TDSP_CORE_MUX_INST/g505 INOX2MTR -1.376
TDSP_CORE_MUX_INST/g506 INOX2MTR -1.376
# -----
# Inst:sequential Cell Cost
# -----
RAM_256x16_TEST_INST/RAM_256x16_INST sram_sp_metro -1.376
SPI_INST/dout_reg[7] SDFFRQX2MTR -0.961
ARB_INST/tdsp_grant_reg SDFFRQX2MTR -0.775
ARB_INST/ana_grant_reg SDFFRQX2MTR -0.623
SPI_INST/dout_reg[4] SDFFRQX2MTR -0.470
# -----
report_critical_instance Done
```

This page does not contain notes.

Reporting Path Exceptions



```
report_path_exceptions [-early | -late] [ -ignored] \
[-view view_name ] [{> | >>} filename | -tcl_list]
```

It generates a report about path exceptions specified using the *set_false_path*, *set_min_delay*, *set_max_delay* and *set_multicycle_path* commands.

Multiple path exceptions that match a given path are prioritized and applied.

When the software is in Multi-Mode Multi-Corner Timing Analysis mode, the report generates path exception information for every active analysis view.

In this example, the multicycle exception is ignored due to the “vclk1” false path:

```
set_false_path -from [get_ports {reset}]
set_false_path -from [get_clocks vclk1]
set_multicycle_path 2 -start -setup -to\
[get_pins{DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/p_reg_31/D}]
```

From	To	Early	Late
reset	DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/p_reg_31/D	false	false
"vclk1"	"*"	false	false



This page does not contain notes.

Reporting Clocks



```
report_clocks [-adjustment_table] [-arrival_points] [-delay_adjustment_table]
[-description] [-generated] [-groups] [-hierarchy] [-insertion]
[-phase_shift_table] [-source_insertion]
```

Clock Descriptions								
Attributes								
Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated	
CLK1	CLK1	view1	4.000	0.0000	2.000	n	y	
GEN_DIV1	DIV1/Q	view1	4.000	0.0000	2.000	y	y	
GEN_DIV2	DIV2/Q	view1	8.000	0.0000	4.000	y	n	
GEN_DIV3	DIV3/Q	view1	12.000	0.0000	4.000	y	n	
CLK1_IO	-	view1	4.000	0.0000	2.000	n	n	
Generated-Clock Descriptions								
--								
Name	Generated	Master	Master-clock	View	Invert	Freq.	Duty-Cycle	Edges
EdgeShift		Source (pin)	Source (pin)			Multiplier		
--								
GEN_DIV1	DIV1/Q	CLK1	CLK1	view1	n	1/1	-	-
GEN_DIV2	DIV2/Q	CLK1	CLK1	view1	n	1/2	-	-
GEN_DIV3	DIV3/Q	CLK1	CLK1	view1	n	-	-	[1 3 7]

291 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Reporting Clock Timing



```
report_clock_timing -type [{skew interclock_skew jitter summary latency
cprpr_stage_count}] [-early|-late] [-clock clock_list ] \
[-from_clock clk1] [-to_clock clk2] [-view view_name] ...
```

It generates a clock skew report for the current design.

The software generates a separate report for each specified clock or pair of clocks.

You can specify:

- The clocks using *-clock* and clock pairs using *-from_clock* and *-to_clocks* parameters.
- The *-early* and *-late* parameters to specify the type of skew to be reported.
- The *-view* parameter to report clock timing for a user specified view in MMMC mode.

To remove common path pessimism from the reported skew so that the report contains a separate column for common path adjustment values, use:

```
set_db timing_analysis_check_type setup
set_db timing_analysis_cprpr setup
report_clock_timing
```

Clock: CLK_W_4			
Skew	Latency	CPPR	Clock Pin
0.180			r u99/CK
0.000	0.120	-0.060	r u55/CK
Clock: CLK_W_3			
Skew	Latency	CPPR	Clock Pin
0.818			r seg3/u9/CK
0.139	0.496	-0.183	r seg3/u14/CK



This page does not contain notes.

Sample Report: *report_clock_timing*

```
#####
# Command:          report_clock_timing -type skew
#####
Clock: m_spi_clk
Skew                Latency                Clock Pin
-----
0.002              0.132                r    SPI_INST/spi_sr_reg[2]/CK
              0.129                r    SPI_INST/spi_sr_reg[3]/CK

#####
# Command:          report_clock_timing -type jitter
#####
Clock: m_clk
Jitter              Latency (Late)           Latency (Early)           Clock Pin
-----
0.000              0.299                r              0.299                r    RESULTS_CONV_INST/high_reg[0]/CK

#####
# Command:          report_clock_timing -type interclock skew
#####
Clocks: m_clk -> m_dsram_clk

Skew                Latency                Clock Pin
-----
0.344              0.344                r    TEST_CONTROL_INST/m_clk
              0.000                r    DMA_INST/a_reg[5]/CK
              0.000                r    TEST_CONTROL_INST/m_dsram_clk
0.344              0.000                r    RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
```

This page does not contain notes.



Submodule 9-2

Debugging Timing Analysis Results

cā dence®

This page does not contain notes.

Submodule Objective

In this submodule, you

- Debug timing reports



This page does not contain notes.

Debugging Disabled Arcs



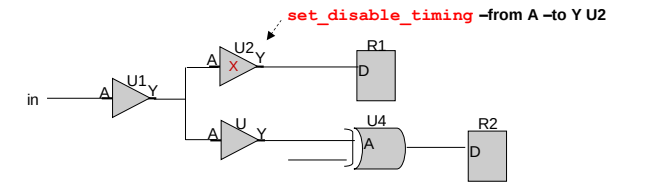
```
report_inactive_arcs [-help] [instance_port_obj] [-view view_name] \  
[-type {const snipped disable library_disable global_disable \  
conditional_disable library_mode}] [-delay_arcs_only | -check_arcs_only]
```

This command is useful for querying specific instances for the status of arcs.

It also reports about arcs disabled by constant propagation during timing analysis, snipped loop arcs, and arcs disabled in the timing library.

It is not very good for determining possible causes for the lack of timing data at a certain location:

- Use `report_fanout` or `report_fanin` with the `-trace_through` option to search for blockages.



```
tempus 2> report_fanout -from in -trace_through {all}
Fanout Network for Port: in
```

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc Disabled
in	U1/A	(net)	-	-
--	--	--	--	--
U1/A	U1/Y	BUFX1	positive_unate	-
--	--	--	--	--
U1/Y	U2/A	(net)	-	-
U1/Y	U3/A	(net)	-	-
--	--	--	--	--
U2/A	U2/Y	BUFX1	positive_unate	Y

In the timing analysis and debug, information about disabled timing and timing check arcs in the design is given by `report_inactive_arcs`. It reports all arcs disabled due to user-specified exceptions such as `set_disable_timing` or `set_case_analysis`.

Debugging Path Exceptions



```
report_path_exceptions
[-early | -late] [-ignored]
[-view view_name] [{> | >>} filename
| -tcl_list]
```

This command is:

- Useful for detecting problems with constraints (bad arguments) that were not reported immediately to the console.
- Particularly good for false path debugging since false paths have the highest priority among path exceptions.

By default, it reports activated path exceptions. To report only the path exceptions that have been ignored by timing analysis, specify the *-ignored* parameter.

The table shows the priorities for path exceptions applied to the same path.

Priority	Path Exception
1. (Highest)	set_false_path
2.	set_max_delay -from pin_list
3.	set_max_delay -to pin_list
4.	set_max_delay -through pin_list
5.	set_max_delay -rise <>
6.	set_max_delay -fall <>
7.	set_max_delay (most constraining adjustment has higher priority)
8.	set_multicycle_path -from <> -to <>
9.	set_multicycle_path -from <>
10.	set_multicycle_path -to <>
11.	set_multicycle_path -through <>
12.	set_multicycle_path -rise<>
13.	set_multicycle_path -fall <>
14. (Lowest)	set_multicycle_path (most constraining adjustment has higher priority)

297 © Cadence Design Systems, Inc. All rights reserved.

cadence

A path can have more than one path exception which can be queried with the command *report_path_exceptions*.

Debugging Path Exceptions (continued)

Apply Exceptions

```
<set_max_delay/set_multicycle_path/set_f
  else_path>
```

Check Exceptions `report_timing -` `path_exceptions`

In the example shown to the right, there is a mix of constraints applied against a register-to-register path from BLK/BR1 to BLK/BR2.

- As expected, the *set_false_path* constraint has precedence, causing the path to be initially blocked.
- By turning on the *report_timing -unconstrained* option with *-path_exceptions*, we can see all three exceptions reported along with their state.

The exceptions report format is similar to that provided by the global report from the *report_path_exceptions* command.

```
set_max_delay 1.0 -to BLK/BR2/D
set_multicycle_path 2 -from BLK/BR1
set_false_path -to CLK_5_7_15
```

```
tempus> report_timing -unconstrained -to BLK/BR2/D -path_exceptions all
Path 1:Endpoint: BLK/BR2/D (*) (unconstrained output)
Beginpoint: BLK/BR1/Q (*) triggered by leading edge of 'CLK_5_7_15'
+ Clock Network Latency (Prop) 0.534800
= Beginpoint Arrival Time 0.534800
```

Net	Instance	Arc	Cell	Delay	Arrival Time
BLK/bclkint	BLK/BR1	CK ^			0.534
BLK/net3	BLK/BR1	CK ^ -> Q ^	DFFHQ1	0.322	0.857
BLK/net4	BLK/BU3	A ^ -> Y ^	BUFX2	0.114	0.971
BLK/net5	BLK/BU4	A ^ -> Y ^	AND2K1	0.159	1.131
	BLK/BR2	D ^	DFFHQ1	0.000	1.131

```
Applied exceptions:
+-----+
| From | To | Late |
+-----+
| "*" | "CLK_5_7_15" | false |
+-----+
Ignored exceptions:
+-----+
| From | To | Late |
+-----+
| BLK/BR1/CK | BLK/BR2/D | cycles 2 |
| | | max delay 1 |
+-----+
```

298 © Cadence Design Systems, Inc. All rights reserved.

cadence

The *report_timing -path_exceptions* option is very useful for determining which exceptions were applied by the timer when there were overlapping types of constraint against a timing path.

Getting Details About the Timing Checks in a Design



```
report_analysis_coverage
-check_type [setup|hold|..]
-verbose -retime -max_paths -nworst
```

To quickly identify whether the endpoint is unconstrained, use `report_analysis_coverage`.

The `report_analysis_coverage` command provides details about the timing checks in the design.

- For each timing check, the command reports the number of checks that meet or violate constraints or are untested in SUMMARY.
- The DETAILS section includes data about signal/reference pins, check type, slack, and the reason for being untested.
- Use `-verbose` to get the reason for the endpoint being unconstrained.

```
#####
# Command: report_analysis_coverage
#####
```

TIMING CHECK COVERAGE SUMMARY				
Check Type	No. of Checks	Met	Violated	Untested
ExternalDelay (Early)	1	0 (0%)	0 (0%)	1 (100%)
ExternalDelay (Late)	1	0 (0%)	0 (0%)	1 (100%)
Hold	3	0 (0%)	0 (0%)	3 (100%)
Pulsewidth	6	4 (66%)	0 (0%)	2 (33%)
Setup	3	0 (0%)	0 (0%)	3 (100%)

```
#####
# Command: report_analysis_coverage -check_type setup -verbose untested
#####
```

TIMING CHECK COVERAGE SUMMARY				
Check Type	No. of Checks	Met	Violated	Untested
Setup	3	0 (0%)	0 (0%)	3 (100%)

TIMING CHECK COVERAGE DETAILS				
Pin	Reference Pin	Check Type	Slack	Reason
DFF1/D	DFF1/CK ^	Setup	UNTESTED	False Path
DFF2/D	DFF2/CK ^	Setup	UNTESTED	disable
DFF3/D	DFF3/CK ^	Setup	UNTESTED	const

This page does not contain notes.



Submodule 9-3

Global Timing Debug (GTD) Interface

cādence®

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Identify the features of the GTD Interface in Tempus™
- Generate and analyze the timing reports



This page does not contain notes.

Timing Debug and Fix: Problem and Solution



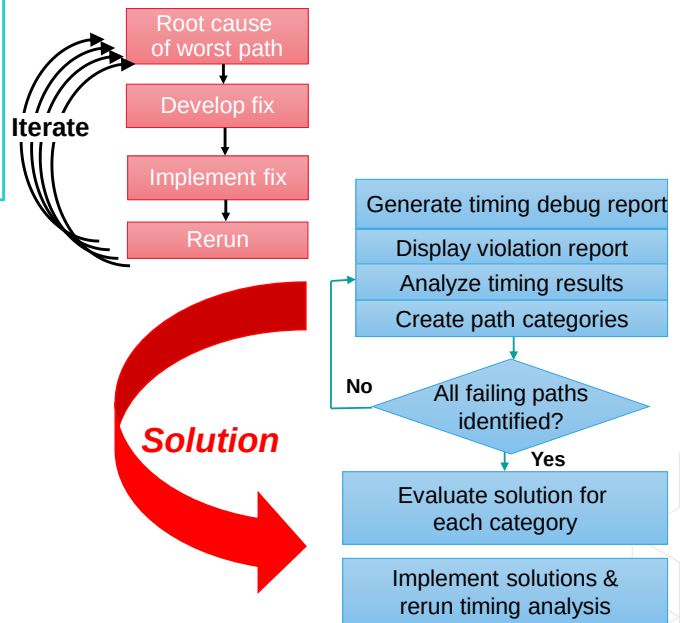
Today's designs are huge, with thousands of complex failing paths having high WNS. It takes several iterations to identify the root cause. How do you find a good debug tool?

Solution: *Global Timing Debug* Interface in Tempus

The features include:

- Unique path categorization capabilities
- Category visualization capabilities
- Enhanced path-centric debug features

The goal is to reduce thousands of failing paths to a small number of unique problems to solve. Then, rerun the software after implementing those fixes.



This page does not contain notes.

Displaying/Generating Timing Report



Timing & SI → Debug Timing

The Display/Generate Timing Report form can be used to specify the violation report to be read in for debugging timing results.

- In the Display/Generate Timing Reports tab, the Generate option creates a violation report file in machine-readable format (default *top.mtarpt*).
- Check Type can be selected for the type of Check: *Setup* or *Hold*.
- Retime specifies the advanced timing methods, i.e., aocv, ssta, path_slew_propagation and waveform propagation.

The Timing Debug form is shown in the next slide.

Timing & SI Verify Tools

MMMC Browser...

Extract RC...

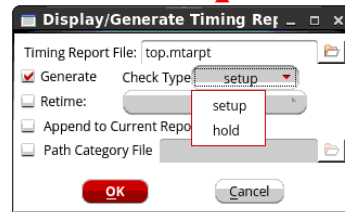
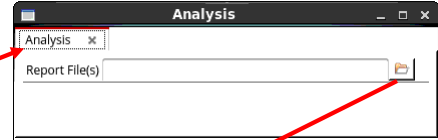
Timing...

Debug Timing...

Display Timing Map...

Crosstalk Analysis...

Display Noise Net...



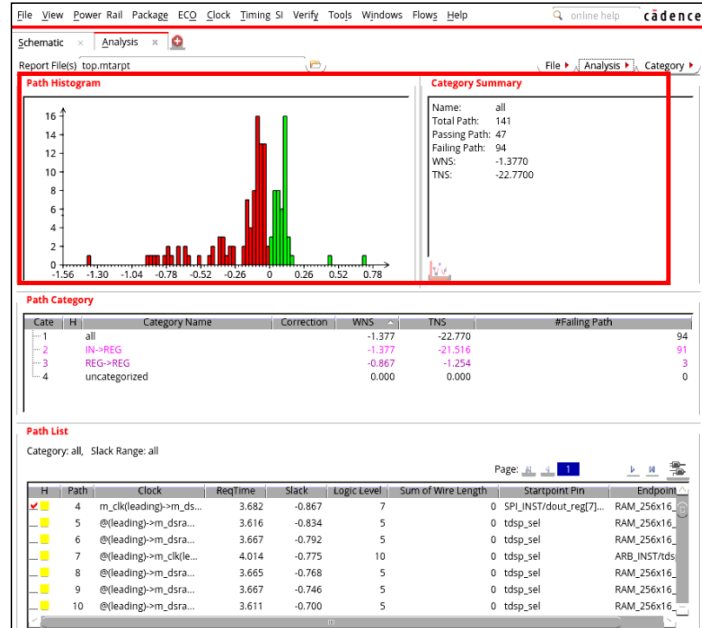
This page does not contain notes.

Displaying Timing Debug Window in Analysis



Timing SI → Debug Timing

- The Timing Debug form is used to view debugging timing results.
- This form is displayed in the Analysis tab once the reports are generated and loaded in the software.
- Use the path histogram to identify all categories. The idea is to start looking at the big picture (overall histogram).
- The Debug form shows information such as the summary of timing results, path list, and path categories and can be used to start debugging the results.



304 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

Using the Timing Path Analyzer

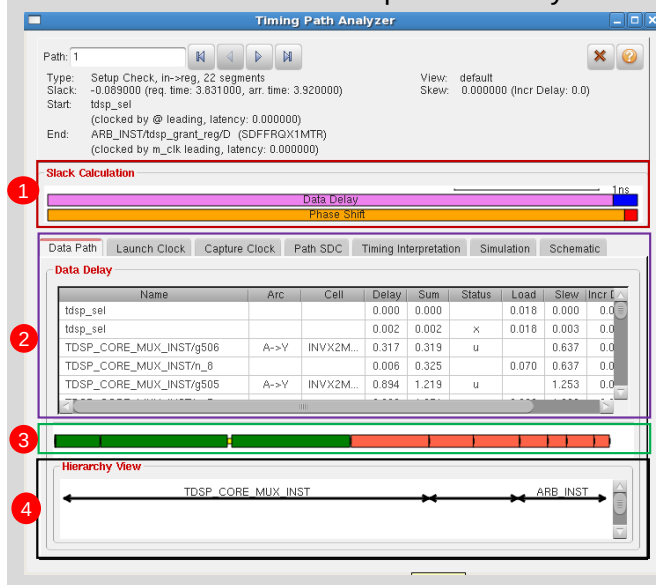


Analysis Tab → Timing Debug Form
→ Path List → double-click on a path
opens the Timing Path Analyzer.

The Timing Path Analyzer form is used to identify issues related to a path using slack calculation bars, timing bar, and hierarchy view.

1. The **Slack Calculation** column displays path arrival time and required time calculations in color bars.
2. The **Data Delay** column displays the details of the selected path in the violation reports.
3. **Timing Bar** displays the instance and net delay; the size of the bar indicates the delay associated.
4. **Hierarchy view** displays a path's traversal of logical hierarchy on a time axis.

The idea is to make obvious problems very visible.



305 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Debugging Through the Timing Path Analyzer

Slack calculation

- Clock skew issues
- Latency balancing issues
- Huge clock uncertainties

Detailed path tabs

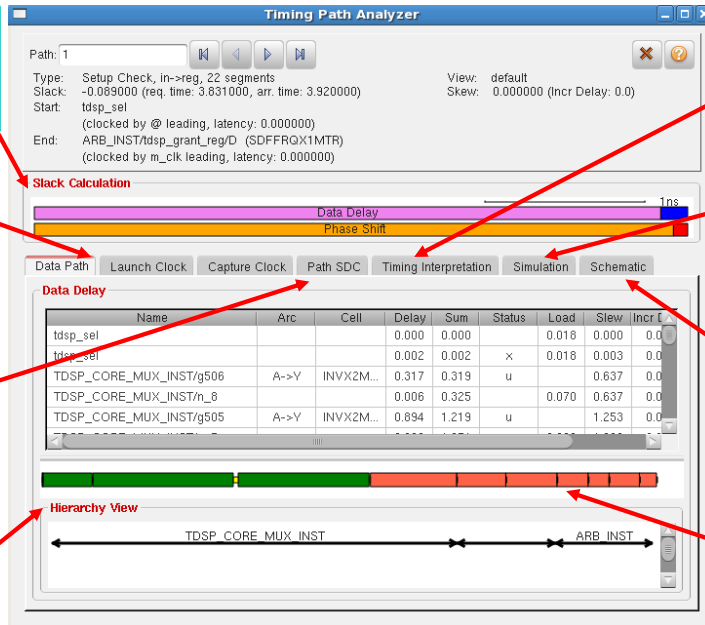
View data, launch, and capture clock segments of path, timing interpretation...

SDC cross-probing

See which SDC constraints affect this path

Time-based hierarchical view

Interpartition path problems



Timing Interpretation

Rule Based Path Analysis

Simulation tab

SPICE Deck

Schematic tab

Rule Based Path Analysis

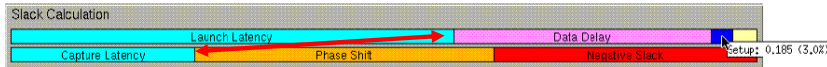
Path delay bar

- Large instance/net delays
- Repeater chains
- Paths with many buffers
- Large macro delays

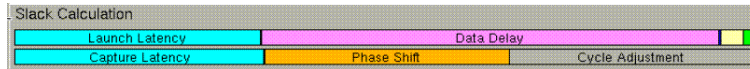
This page does not contain notes.

Debugging Through the Timing Path Analyzer (continued)

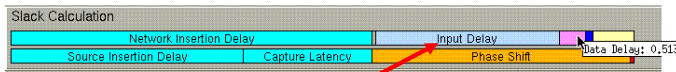
The Launch Latency and Capture Latency components are not aligned. Therefore, there can be a large clock latency mismatch in this path.



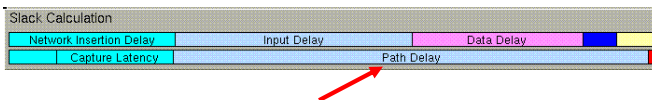
The Cycle Adjustment bar in the required time indicates the presence of a multicycle path.



A large input delay in an I/O path is represented by the light-blue bar in the arrival time.

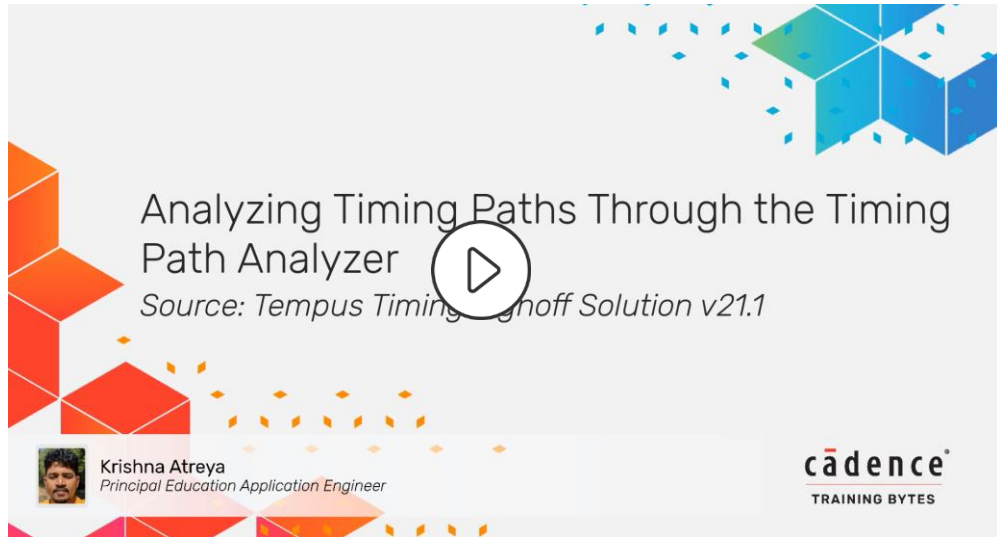


The Path Delay bar in the required time indicates a `set_max_delay` constraint.



This page does not contain notes.

Demo: Analyzing Timing Paths Through the Timing Path Analyzer



308 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

Creating Path Categories

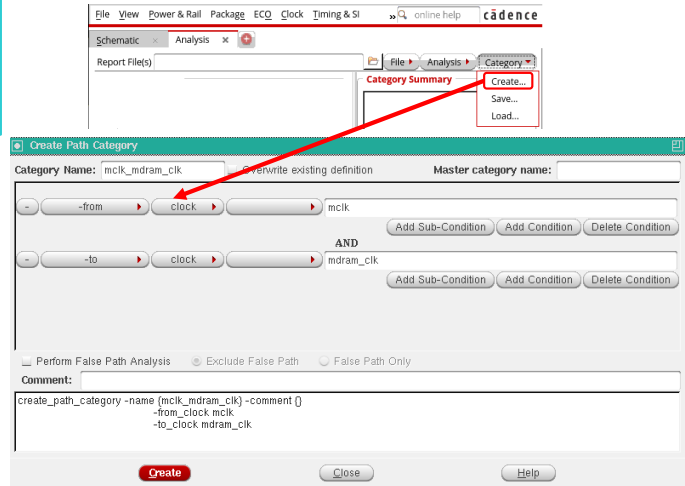


Analysis Tab → Category → Create Category

The Timing Path Analyzer form can be used to analyze the critical paths and identify possible problems with the design.

The Create Path Category form can then be used to define a category or categories to group all paths with the same problem:

- Creates standard path-group-like conditions.
- Defines many other conditions not supported in any path-group definition.



This page does not contain notes.

Creating Path Groups



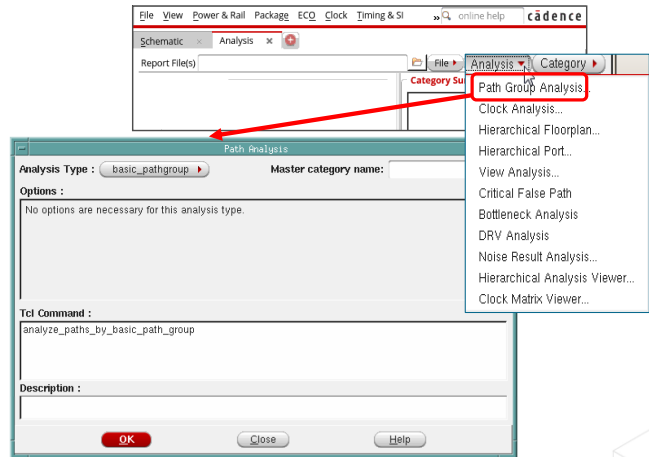
Analysis Tab → Analysis → Path Group Analysis

The Path Analysis form is used to create standard path categories according to basic path groups.

The basic path groups are:

- Register to register
- Input to register
- Register to output
- Input to output

Registers can be macros, latches, or sequential-cell types.



This page does not contain notes.

Creating Clock Analysis Categories

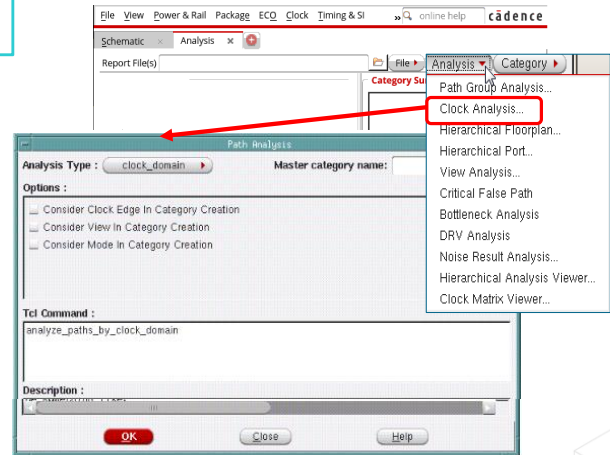


Analysis Tab → Analysis → Clock Analysis

The Path Analysis form is used to create categories according to the launch clock-capture clock combinations.

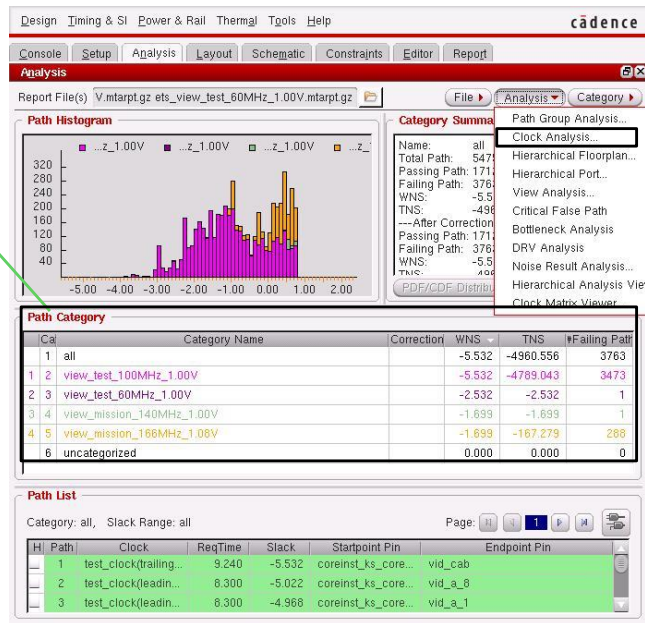
The following categories are created in clock analysis:

- Paths with clock fully contained in a single domain, clk1.
- Paths with clocks starting one clock domain and ending at another, clk1→clk2.



This page does not contain notes.

Creating Clock Analysis Categories (continued)



Paths from the view are sorted by "from" and "to" clock.

312 © Cadence Design Systems, Inc. All rights reserved.

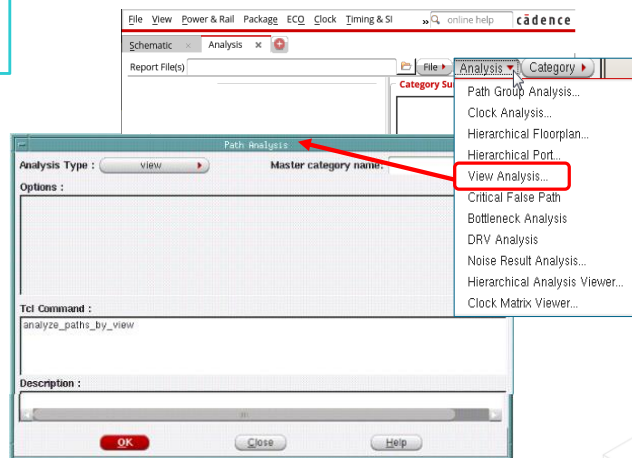
This page does not contain notes.

Creating View Analysis Categories



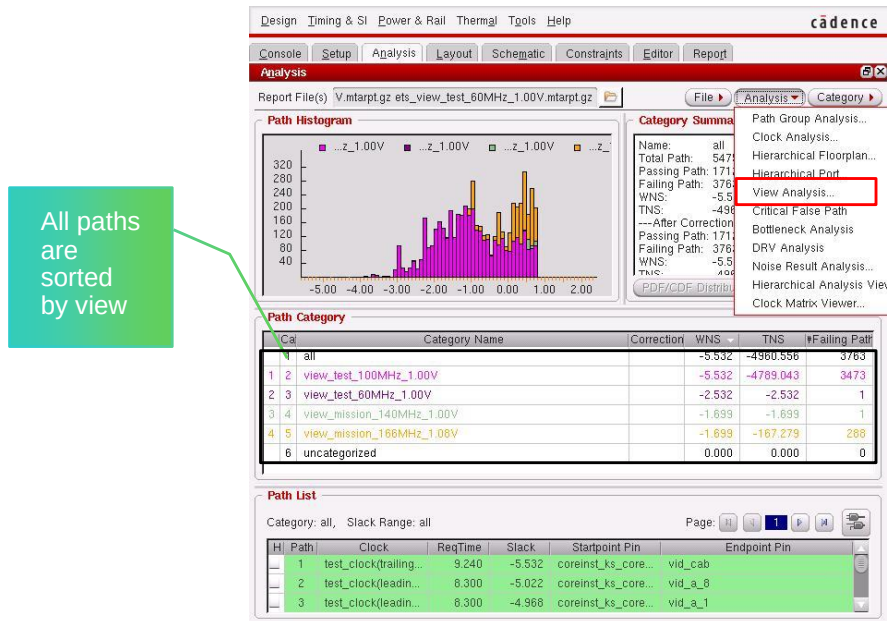
Analysis Tab → Analysis → View Analysis

- The Path Analysis form is used to create categories according to the view that each path is related to.
- The impact of each view on the performance can be analyzed using this category.
- In a single report (report containing all views together), each end point is reported once.
- When you load several reports that were generated for different views, each end point may appear in each report with a different slack value.



This page does not contain notes.

Creating View Analysis Categories (continued)



314 © Cadence Design Systems, Inc. All rights reserved.



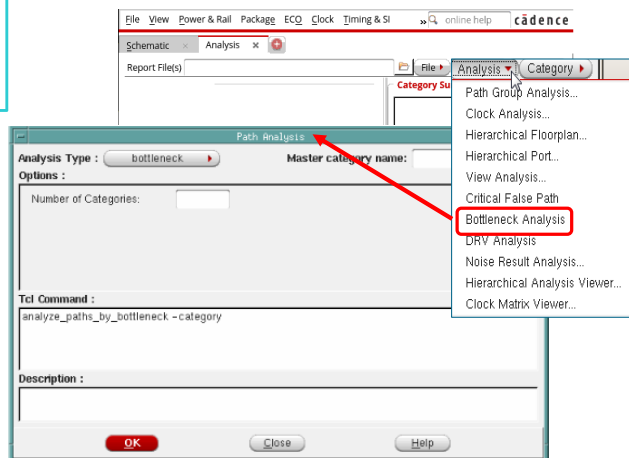
This page does not contain notes.

Creating Bottleneck Instances



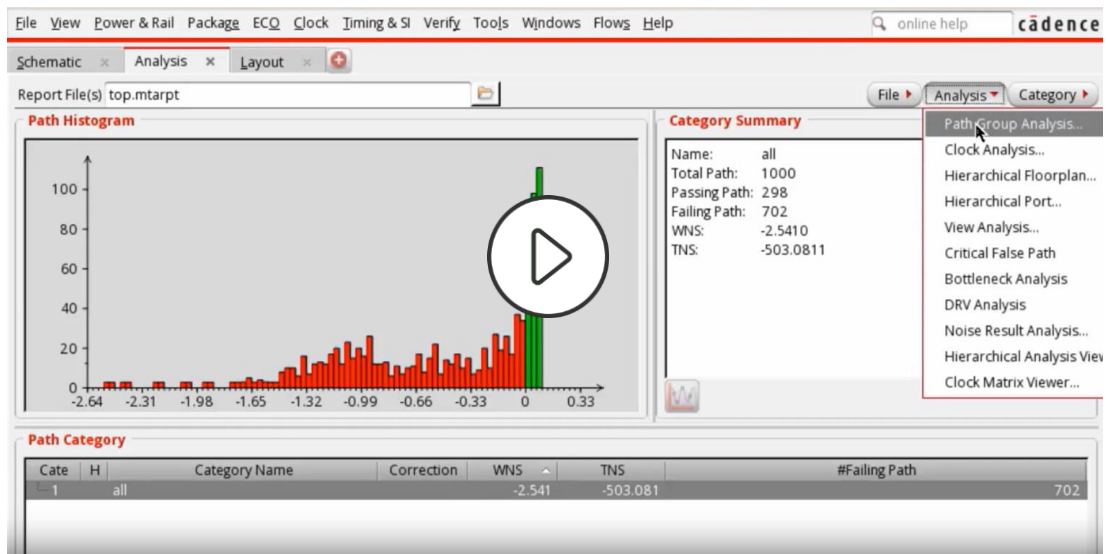
Analysis Tab → Analysis → Bottleneck Analysis

- Bottleneck Analysis detects the instances that are often involved in the critical paths in the design. Any change in timing for these instances impacts multiple critical paths in the design.
- You can use this information to change your design (floorplan, placement, constraints, etc.) around these instances.
- The bottleneck analysis computes the number of occurrences where an instance appears in the paths you display in the Timing Debug window.



This page does not contain notes.

Demo: Creating Predefined Path Categories



316 © Cadence Design Systems, Inc. All rights reserved.

To view the video online, click the following link:

[Creating Predefined Path Categories](#)

Quiz



Timing debugging is mainly done from which of these (select all that apply)?

- A. Schematics tab
- B. Timing Path Analyzer window
- C. Timing Interpretation tab
- D. Path Exceptions tab

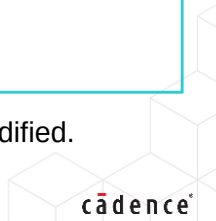
Answer: B. The rest are helpful in debugging, but the main one is the **Timing Path Analyzer**.



Rule-based analysis is done within which of the following (select all that apply)?

- A. Timing interpretation tab in Timing Path Analyzer
- B. Simulation tab in Timing Path Analyzer
- C. Path SDC tab in Timing Path Analyzer

Answer: A. Rules are defined in the **Timing Interpretation** tab, and they can also be modified.



This page does not contain notes.

Quiz (continued)



What issues can you identify from the slack calculation bar during timing analysis (select all that apply)?

- A. Huge clock uncertainties
- B. Latency Balancing issues
- C. Clock domain crossing issues
- D. Large I/O delays
- E. Repeater Chains

Answers: A & B. The others are not clearly identifiable in this view since they are merged with other items like data delay and such.



This page does not contain notes.

Quiz (continued)



What issues can you identify from the path delay bar (select all that apply)?

- A. Large instance or net delays
- B. Huge Clock Uncertainties
- C. Repeater Chains
- D. Latency Balancing issues

Answers: A & C. The others are not shown in this view.



This page does not contain notes.



Submodule 9-4

Manual ECOs

cādence®

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Identify the commands for manual ECOs
- Set up and run manual ECOs



This page does not contain notes.

Setting ECO Mode



Use `set_db eco_*` attributes such as:

```
[eco_honor_dont_touch {true/false}] [eco_honor_dont_use {}]
[eco_honor_fixed_wires{}] [eco_honor_fixed_status {}]
[eco_check_logical_equivalence {}][eco_si_effort {low/medium}]
```

Control timing updates during ECO changes using the following root attributes:

- `eco_honor_dont_use` checks for don't use on cells.
- `eco_honor_dont_touch` checks for don't touch on insts, cells and nets.
- `eco_check_logical_equivalence` allows swapping of non-equivalent cells.
- `eco_honor_fixed_wires` restricts the addition or deletion on a fixed net.
- `eco_honor_fixed_status` does not allow preplaced, fixed instances to be resized.
- `eco_honor_power_intent` performs MSV checks during ECO.

```
set_db eco_check_logical_equivalence true
set_db eco_si_effort medium

eco_add_repeater -cells <> [-net <> | -pins <>] [-new_net_name <>]
[-location {x1 y1}...]

eco_delete_repeater -insts <>

eco_update_cell {-cells <> | -down_size | -up_size } [-location {x
y}] {-insts <>}

connect_net net_name list_of_pins
disconnect_net -net <netName> [-pins <list of pins/ports>]
create_inst -cell <lib_cell_name> -inst <>
create_net -names <list_of_nets>
delete_nets <net>
read_eco eco_files
write_eco -format {tempus | innovus} -output <ecofile_name>
```



This page does not contain notes.

Adding a Repeater During Interactive ECOs in Tempus

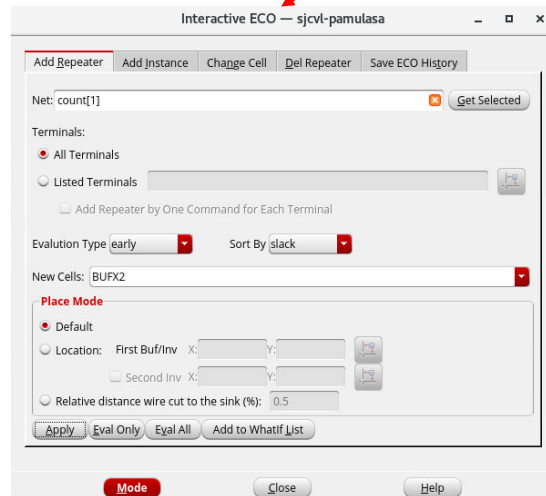
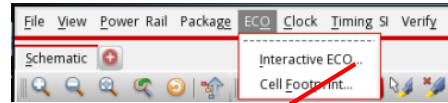


ECO → Interactive ECO → Add Repeater tab

The Add Repeater fields and options are listed below:

- **Net:** Enter the net name or click on a net in the Layout tab and click **Get Selected**.
- **New Cells:** Enter the cell type name of the buffer cell to add or click on the arrow to the right of the field and select a buffer from the list.
- **Add to Whatif List:** Allows you to add your specifications with ECO commands in series.
- **Eval Only:** Evaluates the effect on timing if add *New Cell*.

Equivalent Tcl command: *eco_add_repeater*



323 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Adding an Instance During Interactive ECOs in Tempus

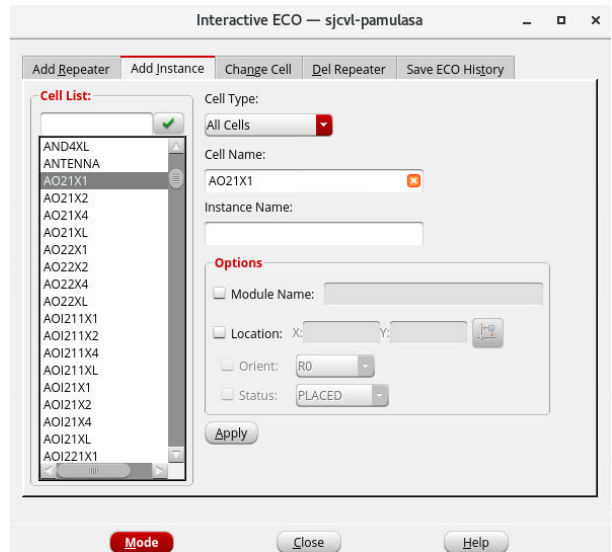


ECO → Interactive ECO → Add Instance tab

The Add Instance fields and options are:

- **Cell List:** Specifies the names of a collection of cells used as a reference for the new instances added. To search for a cell, enter the cell name in the text box and click the tick button.
- **Cell Type:** Specifies the type of cell used as a reference for the new instances added.
- **Cell Name:** Specifies the master of the instance.
- **Instance Name:** Specifies the name of the instance to add and place.

Equivalent Tcl command: *create_inst*



This page does not contain notes.

Changing a Cell During Interactive ECOs in Tempus

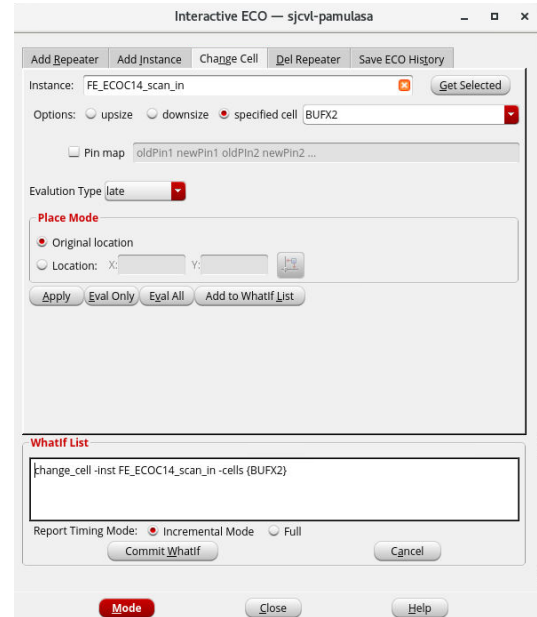


ECO → Interactive ECO → Change Cell tab

The Change Cell fields and options are:

- **Instance:** Enter the hierarchical instance name to be changed.
- **Get Selected:** Displays the name of the selected instance.
- **Options:** Select from the given options.
- **Add to Whatif List:** Allows you to add your specifications with ECO commands in series.
- **Eval Only:** Evaluates the effect on timing if you upsize or downsize a cell or change to a different cell. These values are reported but not applied.

Equivalent Tcl command: *eco_update_cell*



This page does not contain notes.

Deleting a Repeater During Interactive ECOs in Tempus

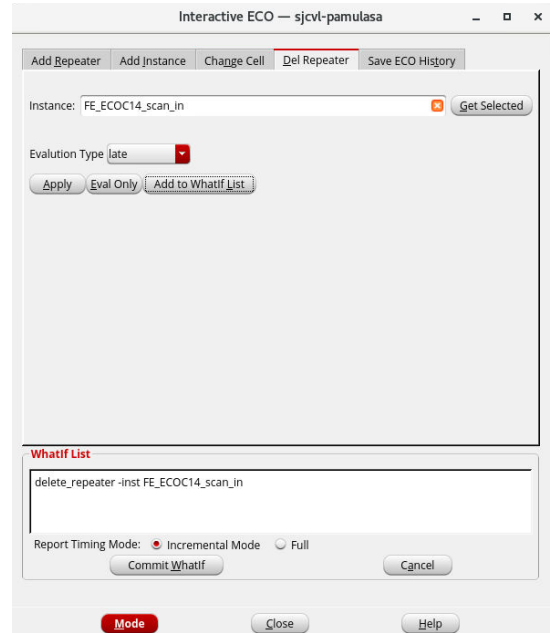


ECO → Interactive ECO → Del Repeater tab

The Del Repeater fields and options are listed below:

- **Instance:** Enter the hierarchical instance name to be removed.
- **Get Selected:** Displays the name of a selected instance.
- **Add to Whatif List:** Allows you to add your specifications with ECO commands in series.
- **Eval Only:** Evaluates the effect on timing if you *Del Cell*.

Equivalent Tcl command: *eco_delete_repeater*



This page does not contain notes.

Saving Interactive ECOs History in Tempus

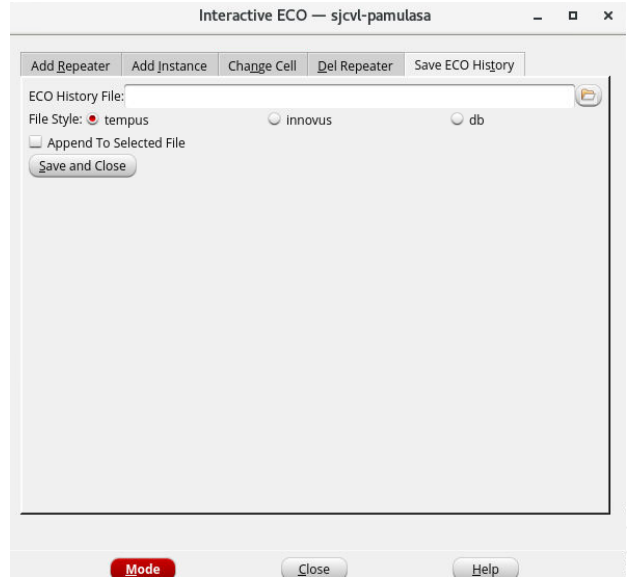


ECO → Interactive ECO → Save ECO History

The Save ECO History Fields and Options are listed below:

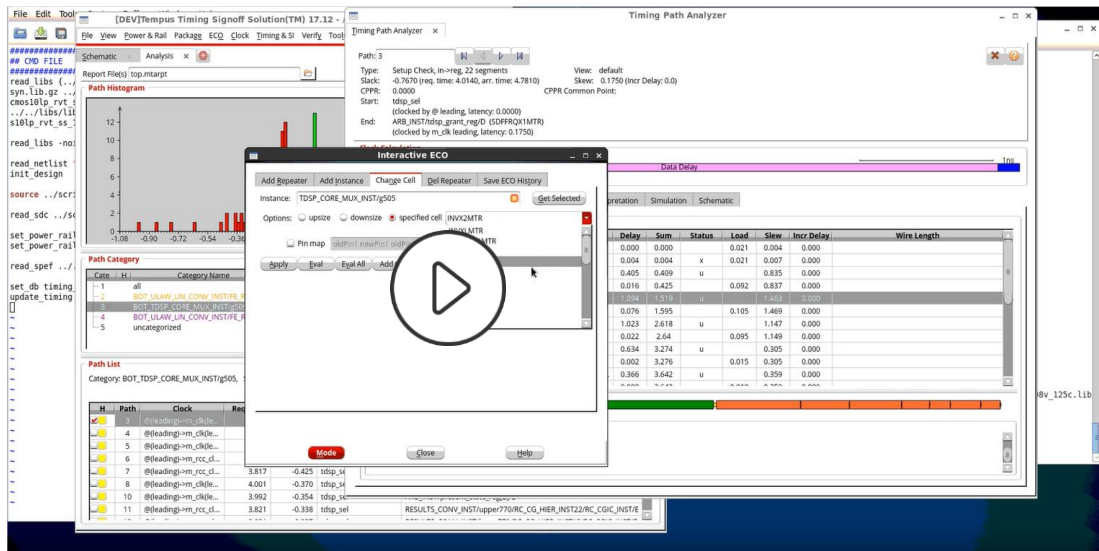
- **ECO History File:** Specifies the name of the file that this menu command produces.
- **File Style:** Generates an ECO file in the specified format.
- **Append To Selected File:** Appends changes to an existing ECO file.

Equivalent Tcl command: *write_eco*



This page does not contain notes.

Demo: Running Interactive ECO in Tempus Stylus



328 © Cadence Design Systems, Inc. All rights reserved.

cadence

To view the video online, click the following link:

[Running Interactive ECO in Tempus Stylus](#)

References



Tempus Training Bytes

- [Tempus Training Byte References](#)

Tempus One-Stop Page

- [Tempus One-Stop Page Reference](#)

Tempus Full Course Reference

- [Tempus Signoff Timing Analysis and Closure with Stylus Common UI](#)

Tempus User Manuals

- [Tempus User Guide 25.1](#)
- [Tempus Product Manuals](#)

Tempus Articles Reference

- [Tempus Articles and AppNotes](#)



This page does not contain notes.

Lab



Lab 9-1 Using the Global Timing Debug Interface to Debug Timing Results

330 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.



Module 10

Gate-Level Simulation

cādence[®]

This page does not contain notes.

Module Objectives

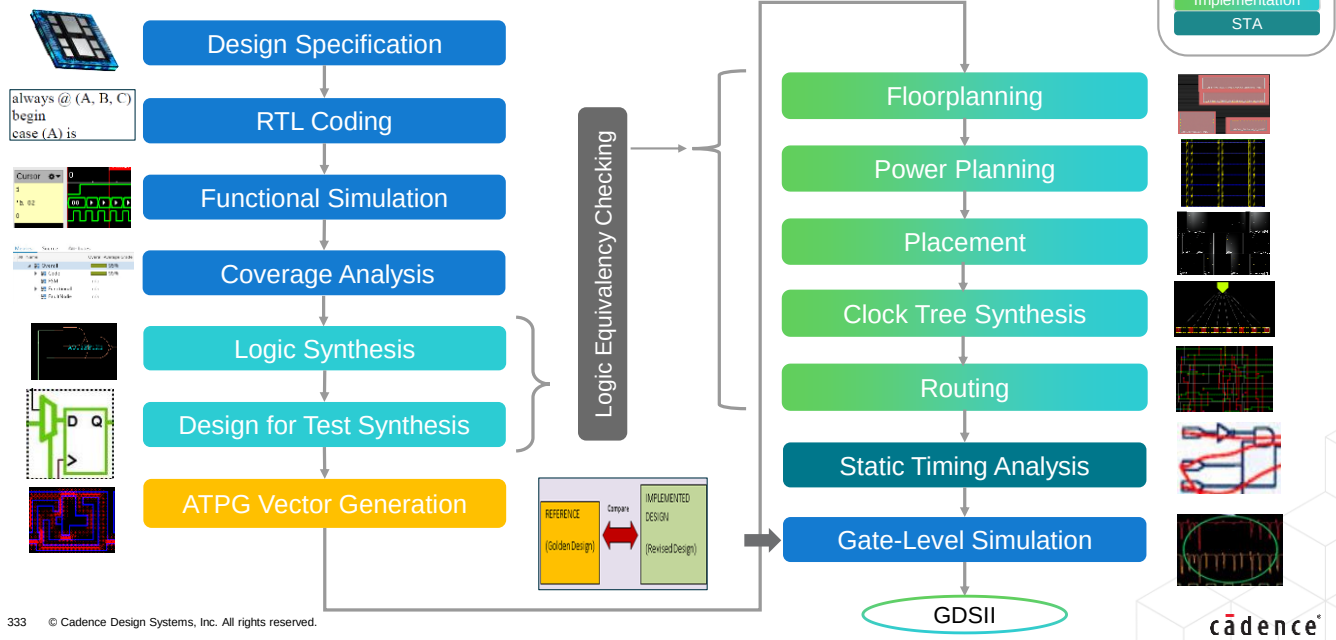
In this module, you

- Identify the process of gate-level simulation:
 - Describe the reason for performing it in the design flow
- Identify the concept of SDF Annotation
- Invoke the Xcelium™ simulator and simulate the gate-level netlist of a simple Counter Design



This page does not contain notes.

Cadence® Digital Realization of the RTL-to-GDSII Flow



This page does not contain notes.

What Is Gate-Level Simulation or GLS?



GLS is a step in the design flow to ensure that the design meets the functionality after synthesis or after placement and routing activities.

- Motivation for running GLS:
 - To give confidence in verifying low-power structures absent in RTL and added during synthesis.
 - It is a method that can catch multicycle* paths if tests exercising them are available.
- To verify DFT structures absent in RTL and added during or after synthesis:
 - Scan chains are generally inserted after the gate-level netlist has been created. Hence, gate-level simulations are often used to determine whether scan chains are correct.
 - GLS is also required to simulate ATPG* patterns.



*ATPG – Automatic Test Pattern Generation

*Multicycle Paths – Multicycle paths are paths in the design that takes more than a single clock cycle. The tools, by default, take all the paths to be a single-cycle operation.

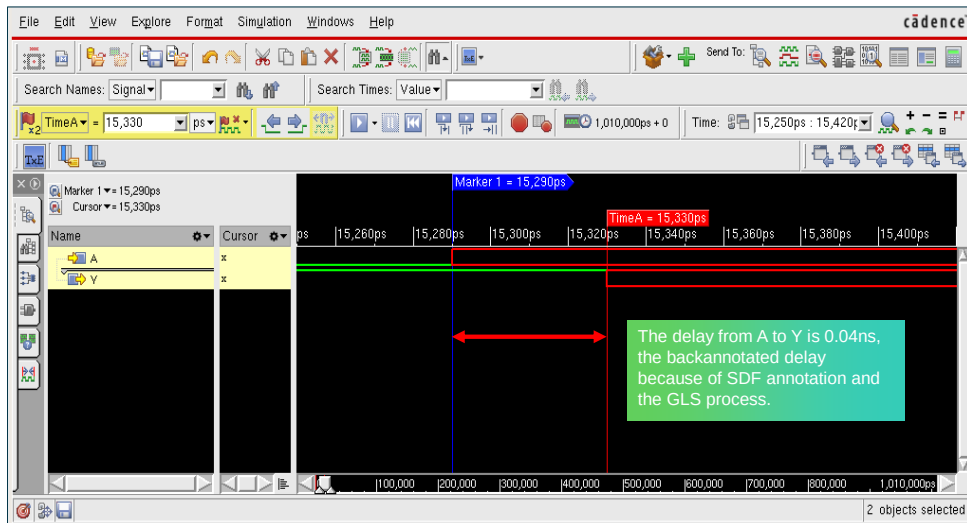
Simulations are an important part of the verification cycle in the process of hardware design. It can be performed at varying degrees of physical abstraction:

1. Transistor level
2. Gate level
3. Register Transfer Level (RTL)

In many companies, RTL simulation is the basic requirement to sign off the design cycle, but lately, there has been an increasing trend in the industry to run gate-level simulations (GLS) before going to the last stage of chip manufacturing. Improvements in static verification tools like Static Timing Analysis (STA) and Equivalence Checking (EC) have leveraged GLS to some extent, but so far, none of the tools has been able to completely remove it. GLS still remains a significant step of the verification cycle footprint. Here, we focus mainly on the steps to run GLS from a verification perspective and the challenges faced while running it.

Gate-Level Simulation (continued)

We need a synthesized/post-routed netlist, testbench, and the SDF (Standard Delay Format) file for running GLS.



335 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

What Is SDF Annotation?



Standard Delay Format (SDF) is an IEEE standard for the representation and interpretation of timing data for use at any stage of an electronic design process.

- This uses the SDF file for performing the annotation process.
- The SDF Annotator is the tool that uses the SDF file as input for the annotation process.
- The job of the annotator is to match data in the SDF file with the design description and the timing models.
- The Xcelium Simulator tool is used as the tool for SDF Annotation as well as for running GLS.

```
counter_test.v (~/rtl2gdsII_labs/counter_design_database_45nm)
File Edit Tools Syntax Buffers Window Help
[Icons]
`timescale 1ns/10ps
module counter_test;
  reg clk, rst;
  wire [7:0] count;
  wire scan_out;
  reg scan_in, SE;

  counter counter1(clk, rst, count, SE, scan_in, scan_out);

  `ifdef SDF_TEST
  initial
  begin
    $sdf_annotate("delays.sdf", counter_test.counter1, "sdf.log", "MAXIMUM");
  end
  `endif

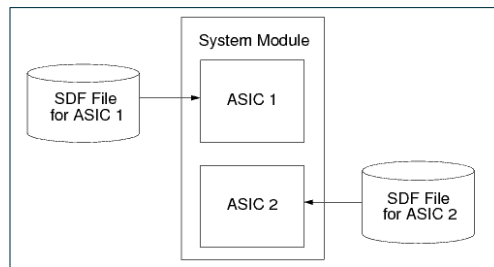
  initial
  begin
    clk=0;
    rst=0;
    #10 rst=1;
    #1000 $stop;
  end

  always #5 clk=~clk;
endmodule
```

This page does not contain notes.

The SDF File

- The contents of the file identify regions of the design and provide timing data that applies to the timing properties of that region.
- The SDF file supports hierarchical delay annotation.
- A design hierarchy might include several different application-specific integrated circuits (ASICs), including cells or blocks within ASICs.
- Each design hierarchy has its own SDF file.
- Throughout the design process, you can use several different SDF files.



This page does not contain notes.

The SDF File (continued)

The data in the SDF file is represented in a tool-independent way and can include:

- Delays: module path, interconnect path.
- Timing checks: setup, hold, recovery, removal, skew, width, period.
- Timing constraints: path, skew, period, sum, and diff.
- Timing environment: intended operating timing environment.
- Incremental and absolute delays.
- Conditional and unconditional module path delays and timing checks.
- Design/instance-specific or type/library-specific data.
- Scaling, environmental, and technology parameters.

Example SDF File Format

```
(DELAYFILE
// HEADER SECTION
.....
// CELL SECTION
// CELL 1
(CELL
  (CELLTYPE "AND2")
  (INSTANCE top/b/d)
  (DELAY
    // (RISE TRIAD) (FALL TRIAD)
    (ABSOLUTE
      (IOPATH a y (1.5:2.5:3.4) (2.5:3.6:4.7))
      (IOPATH b y (1.4:2.3:3.2) (2.3:3.4:4.3)))
    )
  )
// CELL 2
(CELL
  (CELLTYPE "DFF")
  (INSTANCE top/b/c)
  (DELAY
    (ABSOLUTE
      (IOPATH (posedge clk) q (2:3:4) (5:6:7))
      (PORT clr (2:3:4) (5:6:7)))
    )
  )
  (TIMINGCHECK
    (SETUPHOLD d (posedge clk) (3:4:5) (-1:-1:-1))
    (WIDTH clk (4.4:7.5:11.3))
  )
)
)
```

This page does not contain notes.

\$sdf_annotate System Task



```
$sdf_annotate ( "sdf_file" , "module_instance" , <other 5 options> );
```

\$sdf_annotate system task in the testbench



```
counter_test.v (~/rtl2gdsll_labs/counter_design_database_45nm)
File Edit Tools Syntax Buffers Window Help
[Icons]
`timescale 1ns/10ps
module counter_test;
  reg clk, rst;
  wire [7:0] count;
  wire scan_out;
  reg scan_in, SE;

  counter counter1(clk, rst, count, SE, scan_in, scan_out);

  `ifdef SDF_TEST
  initial
  begin
    $sdf_annotate("delays.sdf", counter_test.counter1, "sdf.log", "MAXIMUM");
  end
  `endif

  initial
  begin
    clk=0;
    rst=0;
    #10 rst=1;
    #1000 $stop;
  end

  always #5 clk=~clk;
endmodule
```

This page does not contain notes.

The `$sdf_annotate` System Task (continued)

`$sdf_annotate` system task syntax

<code>\$sdf_annotate ("sdf_file"</code>	The full or relative path of the SDF file.
<code>{ , module_instance }</code>	Optional: Specifies the scope in which the annotation takes place.
<code>{ , "config_file" }</code>	Optional: The name of the configuration file, specified in quotation marks, that the SDF Annotator reads before annotation begins.
<code>{ , "log_file" }</code>	Optional: The name of the log file, specified in quotation marks, that the SDF Annotator generates during annotation.
<code>{ , "mtm_spec" }</code>	Optional: One of the keywords, specified in in the table in the next slide, indicating the delay values that are annotated to the Verilog family tool.
<code>{ , "scale_factors" }</code>	Optional: The minimum, typical, and maximum timing data values, specified in quotation marks, expressed as a set of three positive real number multipliers (min_mult:typ_mult:max_mult). For example: 1.6:1.4:1.2.
<code>{ , "scale_type" });</code>	Optional: One of the following keywords, specified in quotation marks, to scale the timing specifications in SDF, which are annotated to the Verilog family tool.

340 © Cadence Design Systems, Inc. All rights reserved.

cadence

You must specify the arguments to the `$sdf_annotate` system task in the order shown in the syntax. You can skip an argument specification, but the number of comma separators must maintain the argument sequence. For example, to specify only the first and last arguments, use the following syntax:

```
$sdf_annotate ( "sdf_file",,,,,, "scale_type");
```

Values for the *mtm_spec* Option

Keyword	Description
MAXIMUM	Annotates the maximum delay value.
MINIMUM	Annotates the minimum delay value.
TOOL_CONTROL (default)	Annotates the delay value that is determined by the Verilog-XL and Verifault-XL command-line options (+mindelays, +typdelays or +maxdelays); minimum, typical and maximum values are always annotated to Veritime. If none of the TOOL_CONTROL command-line options is specified, then the default keyword is TYPICAL.
TYPICAL	Annotates the typical delay value.



mtm stands for Minimum, Typical, Maximum

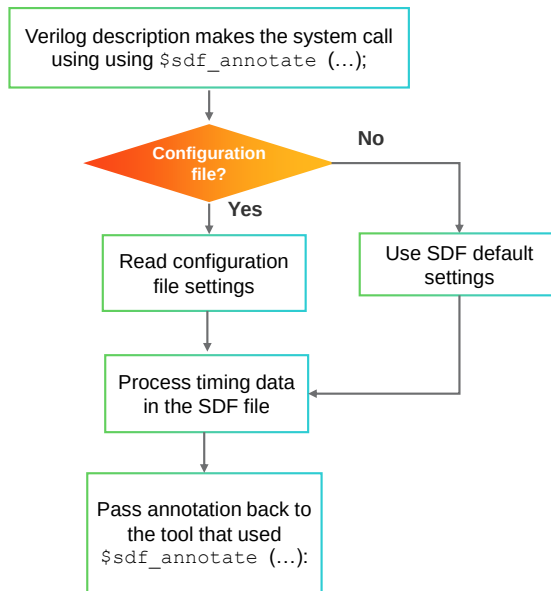
Values for the *scale-factor* Option

Keyword	Description
FROM_MAXIMUM	Scales from the maximum timing specification.
MINIMUM	Scales from the minimum timing specification.
TOOL_CONTROL (default)	Scales from the minimum, typical and maximum timing specifications. This is the default.
TYPICAL	Scales from the typical timing specification.



This page does not contain notes.

Flowchart: Standard Delay Format Annotation Process



A Verilog family tool like Xcelium responds to the `$sdf_annotate` system task, which calls the SDF Annotator. This task is called from the testbench.

The SDF Annotator then reads the configuration file, if one exists. The configuration file filters timing data before it is annotated to a Verilog Family tool.

The SDF Annotator reads the timing data from the SDF file, which is an ASCII text file that stores the timing data generated by the Verilog family tool.

The SDF Annotator processes the timing data according to the configuration file commands or the SDF Annotator's settings.

The processed data is annotated to the Verilog family tool.

This page does not contain notes.

Invoking the Xcelium Tool to Run Gate-Level Simulations



```
xrun -timescale <unit/precision> <netlist> <testbench> -v <library file in .v format> -access +rwc -define <SDF definition in testbench> -mess -gui
```

Given a simple counter design in `counter.v` and the relevant testbench with SDF annotation information in it, the delay file, and the `library.v` file, let us execute the above command to perform GLS:

- `-timescale`: Used to mention the time unit and time precision
- `-access`: Passed to the elaborator to provide read access to simulation objects
- `-gui`: Used to invoke the `xrun` in gui mode
- `-mess`: Used to display all the messages in detail
- `-sdf_verbose`: Used to create instance-level annotation details in the `sdf.log`
- `-define`: Used to provide SDF definition present in the testbench
- `-v`: Used to provide a library in `.v` format

```
gate_level_simulation : csh
File Edit View Scrollback Bookmarks Settings Help
[prabha@noivl-prabha gate_level_simulation]$ ll
total 348
-rwxrwxr-x 1 prabha cadence1 2031 Jun 19 2017 counter_netlist.v
-rwxrwxr-x 1 prabha cadence1 364 Jun 19 2017 counter_test.v
-rwxrwxr-x 1 prabha cadence1 10788 Jun 19 2017 delays.sdf
-rwxrwxr-x 1 prabha cadence1 329651 Jun 19 2017 slow_vddlv0_basicCells.v
[prabha@noivl-prabha gate_level_simulation]$ xrun -timescale 1ns/10ps counter_netlist.v counter_test.v -v slow_vddlv0_basicCells.v -access +rwc -define SDF_TES
T -mess -gui
```

344 © Cadence Design Systems, Inc. All rights reserved.

This slide and the next few slides in this section will explain in detail the exact commands and the options you provide to the `xrun` command using the Xcelium tool in order to run the gate-level simulations. So, the command would be `xrun -timescale`, you provide the time scale, `counter_netlist.v`, the netlist name, then your testbench `counter_test.v`, `-v` and the timing information file and the `-access +rwc` for read-write connectivity. Then you define the `SDF_TEST`; if not defined, then the verbosity with messages. And if you want to invoke the GUI then `-gui`.

Given a simple counter design in `counter.v` and testbench `counter_test.v` and the relevant testbench with SDF annotation information in it. The delay file and the `library.v` file, let us execute the above command to perform the gate-level simulation. Time scale is used to mention the time unit and time precision. In this case, `1ns/10ps`. That is a time unit by time precision.

`-access` is passed to the elaborator to provide the read-write and connectivity access to simulation objects. `-gui` is used to invoke the `xrun` in GUI mode. `-messages` or `-mess` is used to display all the messages in detail for the verbosity. `sdf_verbose` is used to create instance-level annotation details in the `sdf.log`. `-define` is used to provide the SDF definition present in the testbench. `-v` is used to provide the library in `.v` format.

In the terminal, this is how we initiated the gate-level simulation by providing the command `xrun -timescale` followed by the netlist.v, the testbench `-v` for the library.v file, access with `+rwc`. Then `-define` is used to provide the SDF definition present in the testbench, then the verbosity and the GUI for invoking SimVision.

SDF Annotation Details from the Execution

```

Elaborating the design hierarchy:
Top level design units:
  counter_test
Reading SDF file from location "delays.sdf"
Writing compiled SDF file to "delays.sdf.X".
Annotating SDF timing data:
  Compiled SDF file:    delays.sdf.X
  Log file:            sdf.log
  Backannotation scope: counter_test.counter1
  Configuration file:
  MTM control:         MAXIMUM
  Scale factors:
  Scale type:

```

SDF Annotation – Statistics Output

Annotation completed successfully...

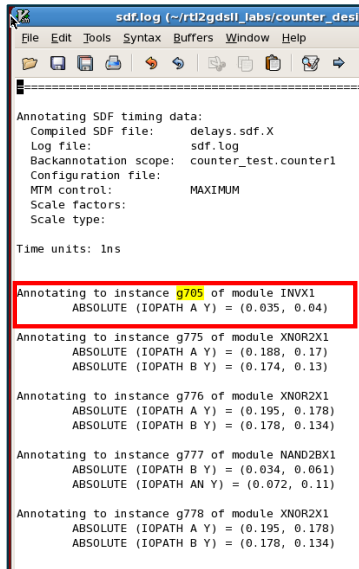
SDF statistics:

No. of Pathdelays = 57	No. of Disabled Pathdelays = 0	Annotated = 100.00% (57/57)
No. of Tchecks = 96	No. of Disabled Tchecks = 0	Annotated = 66.67% (64/96)
		Percentage(A/(T-D))
Path Delays	Total(T)	Disabled(D)
\$hold	57	0
\$removal	8	0
\$width	24	0
\$recovery	8	8
\$setuphold	48	0
		Annotated(A)
		57
		8
		48
		100.00
		100.00
		100.00
		100.00
		100.00

Building instance overlay tables: Done

Once you run the command shown in the previous slide, then the execution happens, and the annotation is completed. The screen displays the SDF statistics that are highlighted within this red rectangle, and this is the statistics output of the SDF annotation. Before the control is relinquished to SimVision™, this shows up on your screen.

Excerpts from the Log Files After Execution



```
sdf.log (~rt2gdsll_jabs/counter_desi

File Edit Tools Syntax Buffers Window Help

Annotating SDF timing data:
Compiled SDF file: delays.sdf.X
Log file: sdf.log
Backannotation scope: counter_test.counter1
Configuration file:
MTM control: MAXIMUM
Scale factors:
Scale type:

Time units: 1ns

Annotating to instance g705 of module INVX1
ABSOLUTE (IOPATH A Y) = (0.035, 0.04)

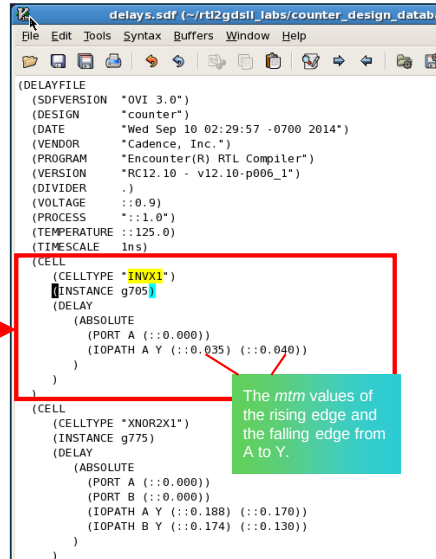
Annotating to instance g775 of module XNOR2X1
ABSOLUTE (IOPATH A Y) = (0.188, 0.17)
ABSOLUTE (IOPATH B Y) = (0.174, 0.13)

Annotating to instance g776 of module XNOR2X1
ABSOLUTE (IOPATH A Y) = (0.195, 0.178)
ABSOLUTE (IOPATH B Y) = (0.178, 0.134)

Annotating to instance g777 of module NAND2BX1
ABSOLUTE (IOPATH A Y) = (0.034, 0.061)
ABSOLUTE (IOPATH AN Y) = (0.072, 0.11)

Annotating to instance g778 of module XNOR2X1
ABSOLUTE (IOPATH A Y) = (0.195, 0.178)
ABSOLUTE (IOPATH B Y) = (0.178, 0.134)
```

The delays were introduced due to I/O paths shown in the *sdf.log* and *delays.sdf* files. This example shows an instance *g705* of inverter *INVX1* which has 2 signals *A* and *Y*, and the delay between the two.



```
delays.sdf (~rt2gdsll_jabs/counter_design_databa

File Edit Tools Syntax Buffers Window Help

(DELAYFILE
(SDFVERSION "0VI 3.0")
(DSIGN "counter")
(ATE "Wed Sep 10 02:29:57 -0700 2014")
(VENDOR "Cadence, Inc.")
(PROGRAM "Encounter(R) RTL Compiler")
(VERSION "RC12.10 - v12.10-p006_1")
(DIVIDER .)
(VOLTAGE ::0.9)
(PROCESS "::1.0")
(TEMPERATURE ::125.0)
(TIMESCALE 1ns)

(CELL
(CELTYPE "INVX1")
INSTANCE g705
(DELAY
(ABSOLUTE
(PORT A (::0.000))
(IOPATH A Y (::0.035) (::0.040))
)
)
)
(CELL
(CELTYPE "XNOR2X1")
(INSTANCE g775)
(DELAY
(ABSOLUTE
(PORT A (::0.000))
(PORT B (::0.000))
(IOPATH A Y (::0.188) (::0.170))
(IOPATH B Y (::0.174) (::0.130))
)
)
)
```

The *mtm* values of the rising edge and the falling edge from *A* to *Y*.

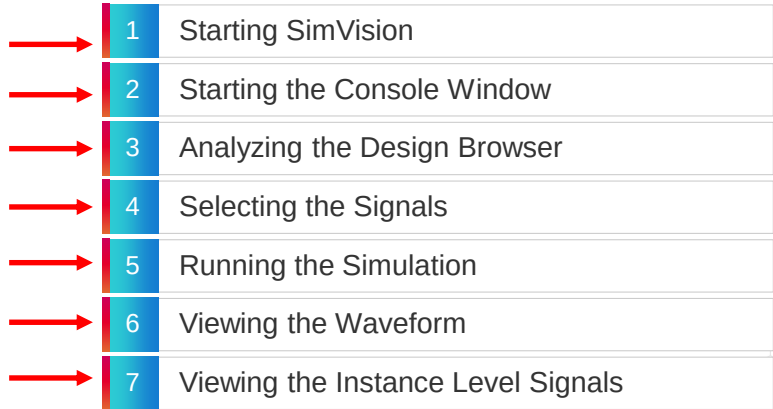
These are the excerpts from the log file after the GLS is completed. The delay is introduced due to the I/O paths; input-output paths are shown in the *sdf.log* and *delays.sdf* files. So *sdf.log* annotating to instance *g705* of the module name. Absolute I/O path *A* and *Y* and the delay introduced, 0.04, is shown here. We will be looking at this detail in the waveform in the next slide and in the *delays.sdf*, you can again see the I/O path *A* and *Y* and the delay introduced, the *mtm* values of the rising edge and the falling edge from *A* to *Y*. So, these are the areas highlighted in the red rectangle here. So, this is the example that shows an instance *g705* of inverter *INVX1*, which has two signals *A* and *Y*, and now the delay is introduced between these two.

How to Run the Gate-Level Simulation in the SimVision Tool



SimVision™ is the GUI-based tool used to run both RTL as well as gate-level simulations or GLS.

- The Console window of SimVision is used to run the simulation.
- The Design Browser is used to analyze the signals in the design and its hierarchy.
- The Waveform window is used to pull the required signals, view the waveform, and analyze the timing delay as a result of the SDF annotation.



This slide and the next few slides show how to run the gate-level simulations or the GLS using the SimVision tool. SimVision is a GUI-based tool used to run both RTL as well as gate-level simulations or GLS. The different windows that are used for this purpose are the Console window, which is used to run the simulation. The Design Browser window is used to analyze the signals in the design and also look at the hierarchy. The waveform we do is used to pull all the required signals from the Design Browser into the window, then view the waveform and analyze the timing delay as a result of the history of annotation.

The different steps that are involved in running the gate-level simulation using the SimVision tool are you start the same vision tool. Next, you start the console window; then you use the Design Browser window to analyze the different signals of the design and the hierarchy. You select the required signals from the design browser and pull them into the Waveform window. You run the simulation in the Console window, view the signals as well as the timing delay in the Waveform window, and also view the instance level signals in the Waveform window.

Running the Simulation in the Console Window and Performing Analysis in the Design Browser Window

Building instance overlay tables: Done
Generating native compiled code:
worklib.counter:v <0x3bcd23b>
streams: 0, words: 0
worklib.counter_test:v <0x5f0dbd66>
streams: 7, words: 1901
slow_vddlv0_basicCells.SDFFRHQX1:v <0x6a373da5>
streams: 0, words: 0
slow_vddlv0_basicCells.SDFFRHQX1:v <0x01133152>
streams: 0, words: 0
Building instance specific data structures.
Loading native compiled code: Done
Design hierarchy summary:

	Instances	Unique
Modules:	24	8
UDPs:	16	2
Primitives:	79	5
Timing outputs:	22	5
Registers:	12	7
Scalar wires:	43	1
Always blocks:	1	1
Initial blocks:	2	2
Cont. assignments:	0	1
Pseudo assignments:	4	4
Timing checks:	144	19
Interconnect:	67	-
Delayed tcheck signals:	40	18
Simulation timescale:	10ps	

Writing initial simulation snapshot: worklib.counter_test:v
..... Done
Relinquished control to SimVision...
xcclium> source /grid/avs/install/xcclium/2009/20-09-003/tools/xcclium/files/xmsimrc
xcclium>

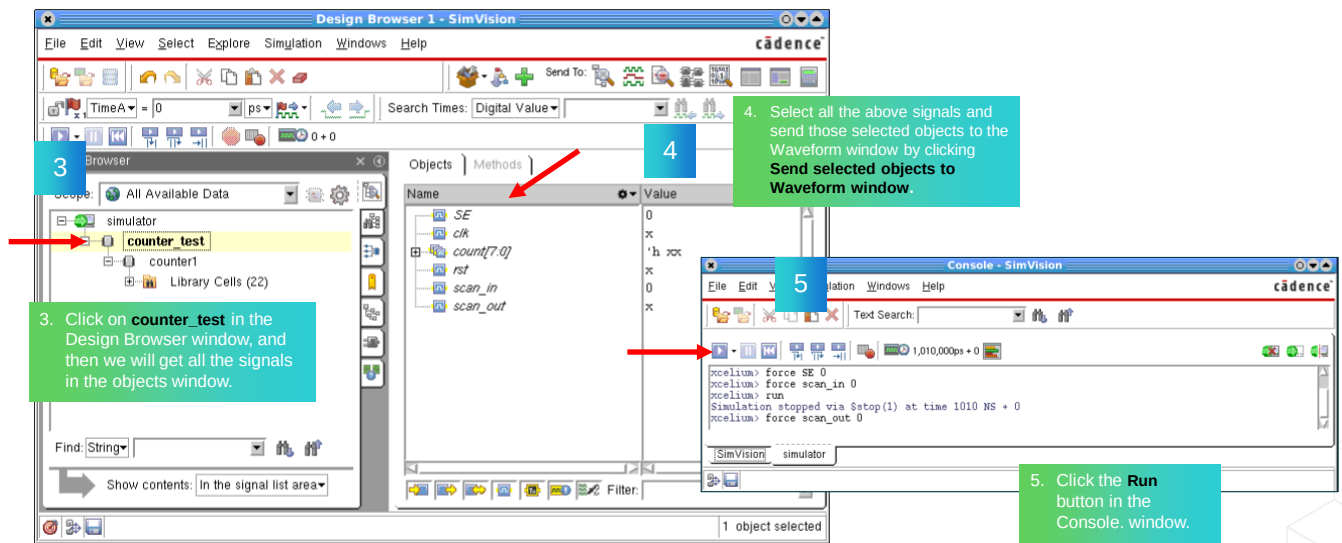
1. SimVision starts with Console and Design Browser windows, opening as shown below.

2. In the Console window, force DFT signals such as SE, scan_in and scan_out signals with value 0.

```
xcclium> source /grid/avs/install/xcclium/2009/20-09-003/tools/xcclium/files/xmsimrc
xcclium> force SE 0
xcclium> force scan_in 0
xcclium> force scan_out 0
```

More information is provided on your screen before SimVision opens. Timing checks, Interconnect, Delayed tchecks signals, and the Simulation Time Scale is provided under the Design Hierarchy Summary. Then the control is relinquished to the SimVision tool. Then the Design Browser opens, as well as the console in SimVision; the SimVision starts with these two windows. In the Console window, you need to provide the force signals, force DFT signals such as SE, scan_in and scan_out and give the values as 0 here to start your gate-level simulation process.

Select the Desired Signals and Pull Them to the Waveform Window

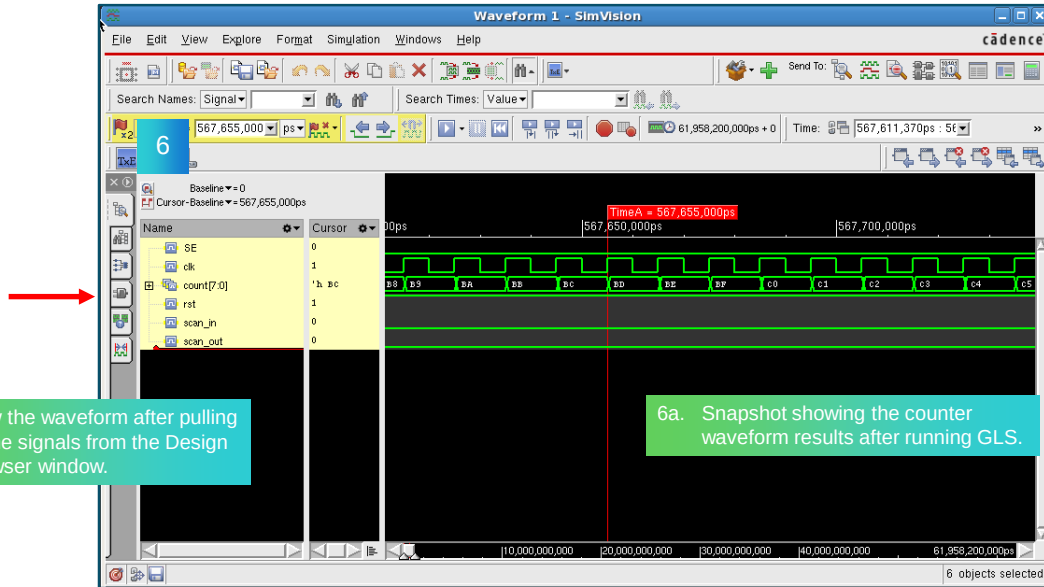


349 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the Design Browser window, you click on the counter_test in the hierarchy and then select all the signals in the Object window, and by clicking the waveform icon, you send all these signals to the Waveform window. Then in the console, once you have forced the scan enable (SE) scan_in and scan_out to 0, then you click on the run button, the play button here, which is the run button, and start the GLS, gate-level simulation.

Viewing the Waveform

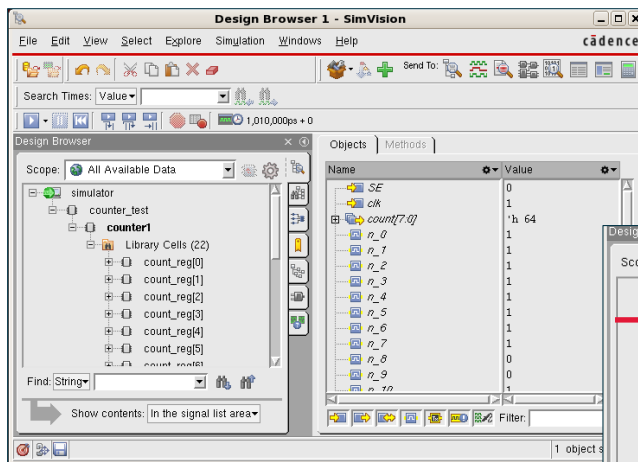


350 © Cadence Design Systems, Inc. All rights reserved.

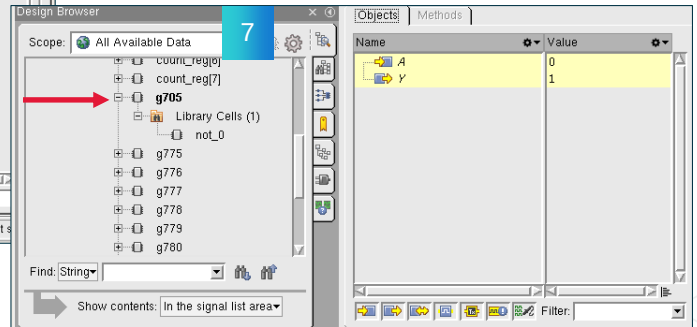
cadence

Once the gate-level simulation is ongoing and completed, you can go to the waveform and view. This is a snapshot showing the counter waveform results after running the GLS.

Viewing the Instance Level Signals and the Delays Introduced



7. View the Instance level signals by clicking on the counter signal to open up all the instances. Focus on our example instance g705 instance with 2 signals A and Y.

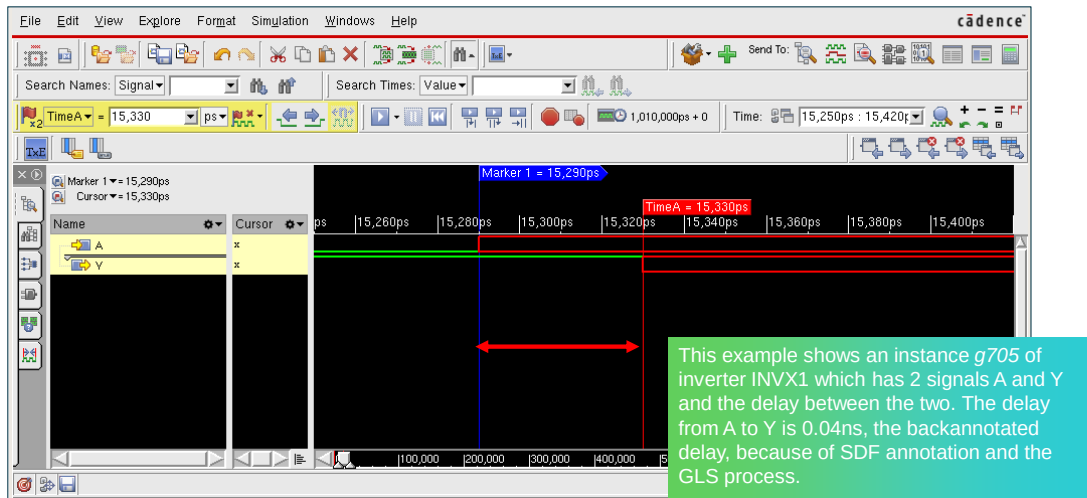


351 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the Design Browser, you can view the instance-level signals and the delays introduced. One of the examples here is the g705 Library Cells. The A and Y and the delay between these two will be shown in the next slide. Objects A and Y signals and what is the SDF annotated or the backannotated delay between these two, which was not present in the original logic simulation, is shown in the next slide in the SimVision waveform.

Backannotated Delay Display in Waveform



352 © Cadence Design Systems, Inc. All rights reserved.

cadence

This is how we see the backannotated delay display in the waveform. Signals A and Y are separately and you can see the delay here. Between the two markers, the delay from A to Y is 0.04ns, which is the backannotated delay because of the SDF annotation and the GLS process; you will be executing this example in the lab at the end of this module.

Module Summary

In this module, you

- Identified the process of gate-level simulation:
 - Described the reason for performing it in the design flow
- Identified the concept of SDF Annotation
- Invoked the Xcelium simulator and simulated the gate-level netlist of a simple Counter Design



This page does not contain notes.

References

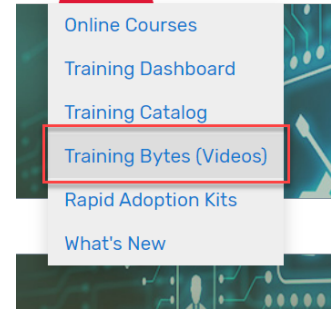


Gate-Level Simulation Training Bytes

- Click on the [link](#) and go to [Learning > Training Bytes \(Videos\)](#)
- Use the search engine to bring up the GLS Training Bytes



as Learning Software My!



Gate-Level Simulation One-Stop Page

- [Gate-Level Simulation One-Stop Page Reference](#)

Xcelium Tool User Guide

- [Xcelium User Guide 25.09](#)
- [Gate-Level Simulation Product Manuals](#)

Gate-Level Simulation Articles Reference

- [Gate-Level Simulation Articles and App Notes](#)



This page does not contain notes.

Lab



Lab 10-1 Running Gate-Level Simulations on a Simple Counter Design

- In this lab, you will invoke the Xcelium Simulator to perform the GLS on a simple counter design netlist.



This page does not contain notes.



Module 11

Course Conclusions

cādence®

This page does not contain notes.

Summary

In this course, you

- Implemented the RTL of a design from its specification
- Simulated a design using the Xcelium™ Simulator tool
- Verified Code Coverage using the Integrated Metrics Center
- Synthesized the design from RTL to Gates using Genus™ Synthesis Solution
- Inserted test structures to be able to test the design using the Genus Synthesis Solution and verify the test coverage using the Encounter® test
- Compared the design against the RTL using Conformal® Equivalence Checker
- Ran the digital implementation flow with the Innovus™ Implementation System:
 - Created a floorplan
 - Implemented power structures and clock trees
 - Performed Place and Route on the design
 - Verified the design
- Ran signoff checks to make sure that the design chip can be fabricated

357 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

References

Other Courses References:

- [Xcelium Simulator](#)
- [Metric Driven Verification Using Cadence vManager](#)
- [Innovus Block Implementation with Stylus Common UI](#)
- [Conformal Equivalence Checking](#)
- [Genus Synthesis Solution with Stylus Common UI](#)
- [Advanced Synthesis with Genus Stylus Common UI](#)
- [Tempus Signoff Timing Analysis and Closure with Stylus Common UI](#)



This page does not contain notes.



Module 12

Next Steps

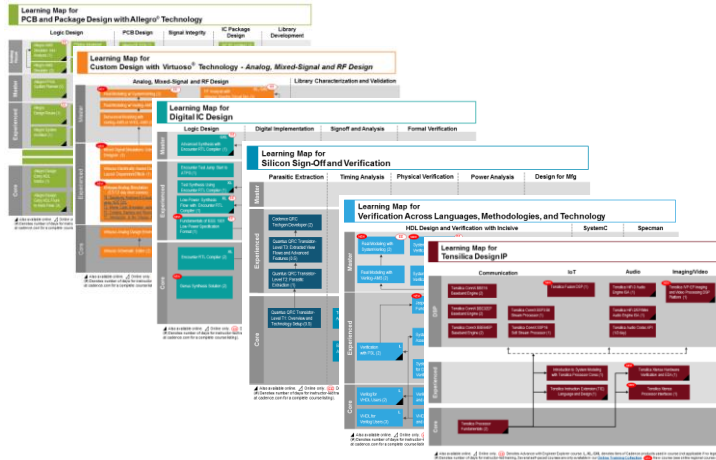
cādence[®]

This page does not contain notes.

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click [here](#) to see all our courses in each technology area and the recommended order in which to take them.



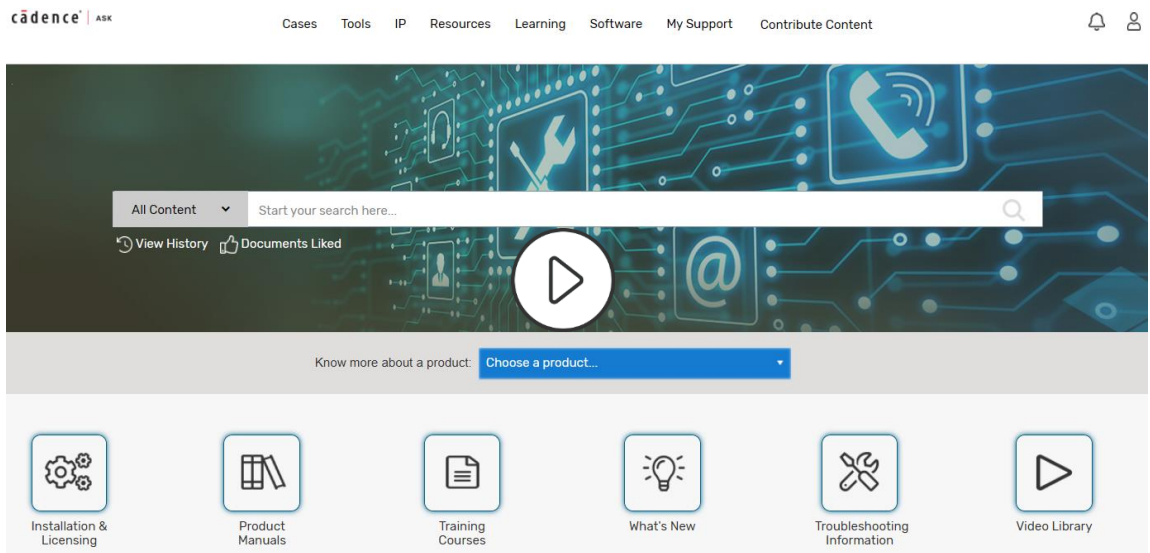
360 © Cadence Design Systems, Inc. All rights reserved.



Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

Cadence ASK – Learning and Support Portal



[Cadence Ask](#) now includes over 2000 product/language/methodology videos (“Training Bytes”)!

361 © Cadence Design Systems, Inc. All rights reserved.



Click [here](#) to view a demo of Welcome to Cadence ASK – Learning and Support Portal.

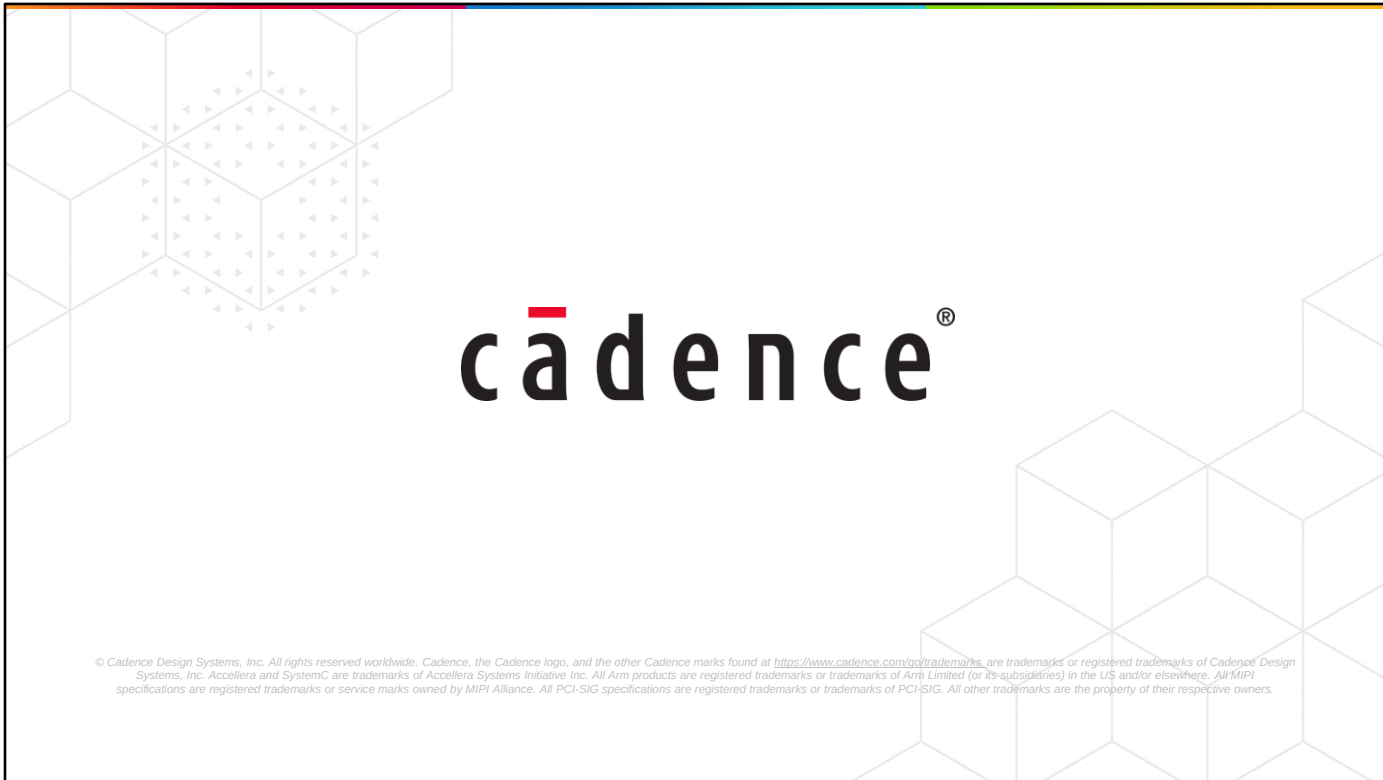
Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion

Thank you!



This page does not contain notes.



© Cadence Design Systems, Inc. All rights reserved worldwide. Cadence, the Cadence logo, and the other Cadence marks found at <https://www.cadence.com/oc/trademarks> are trademarks or registered trademarks of Cadence Design Systems, Inc. Accellera and SystemC are trademarks of Accellera Systems Initiative Inc. All Arm products are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All MIPI specifications are registered trademarks or service marks owned by MIPI Alliance. All PCI-SIG specifications are registered trademarks or trademarks of PCI-SIG. All other trademarks are the property of their respective owners.

This page does not contain notes.